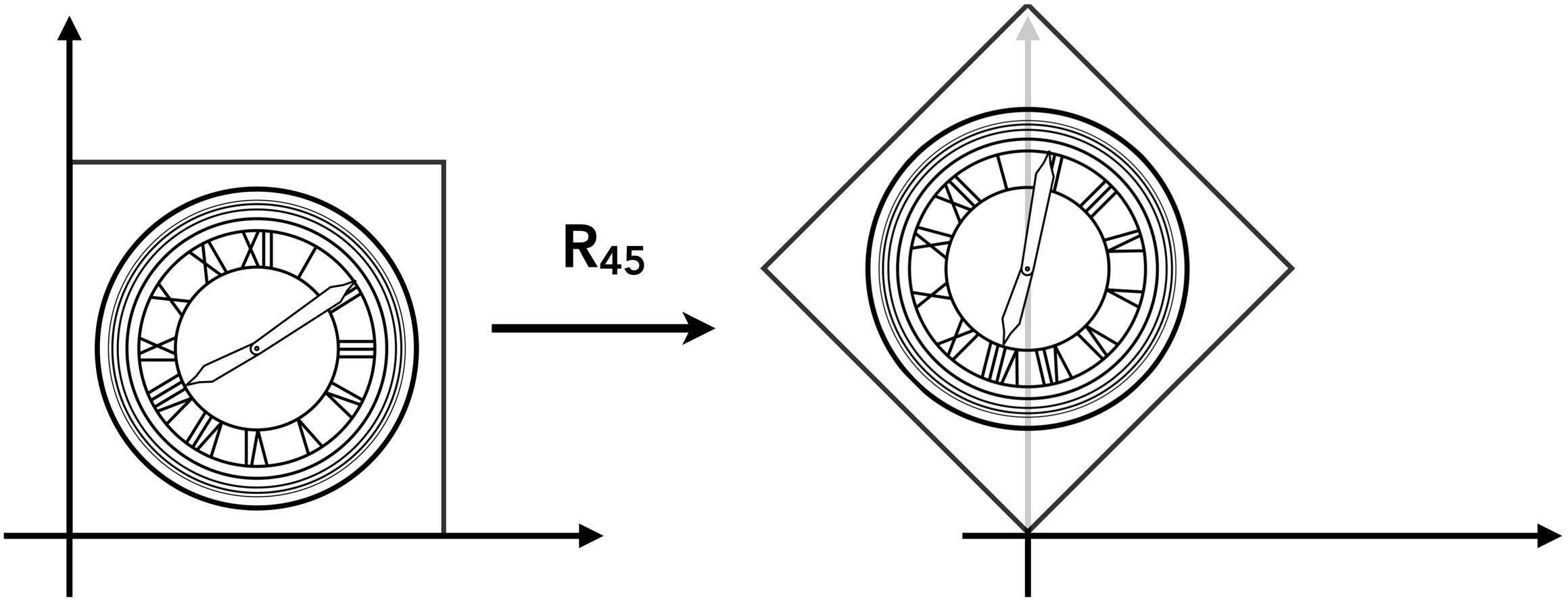
Section 4:

Transforms

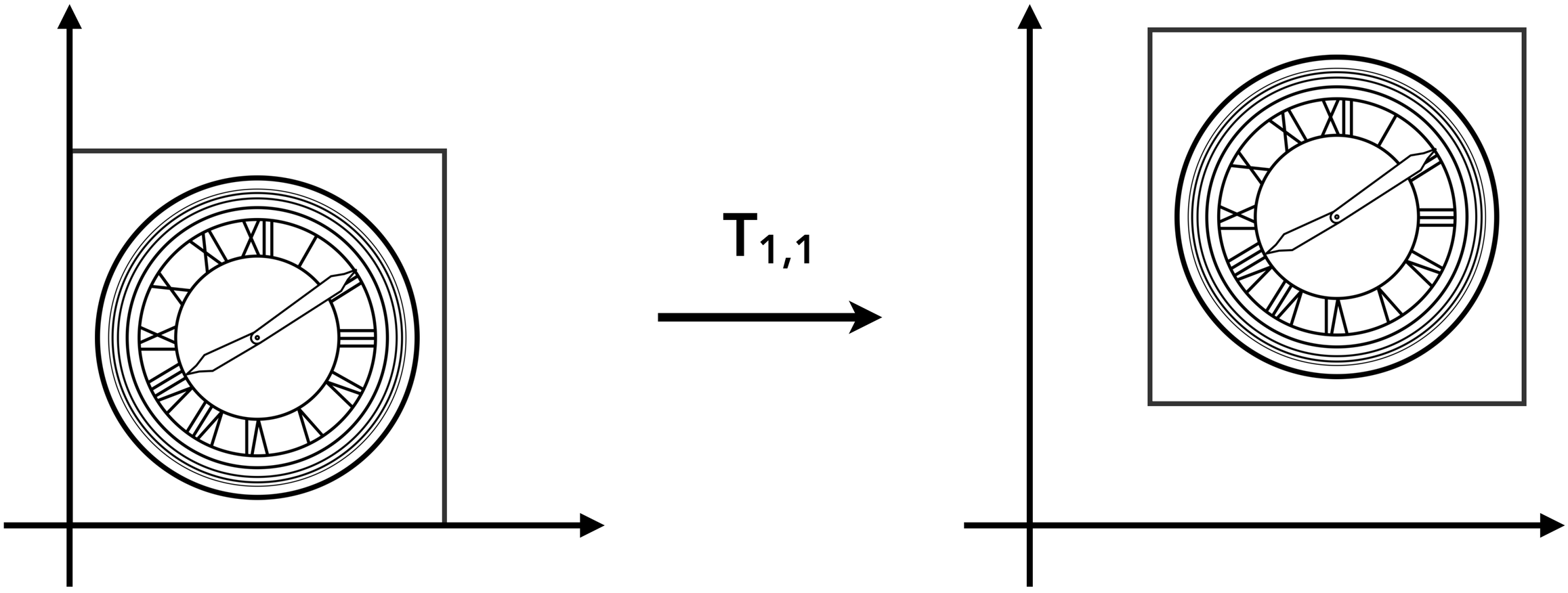
Computer Graphics and Imaging
UC Berkeley CS184/284A

Basic Transforms

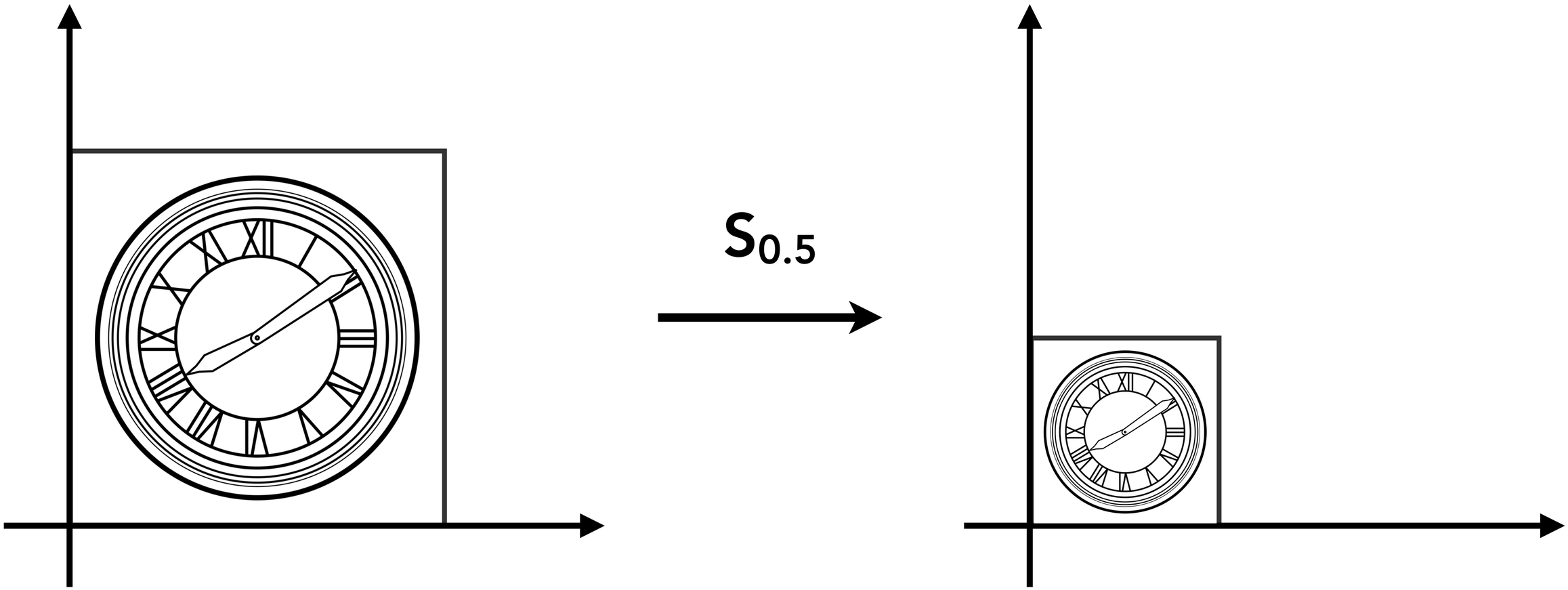
Rotate



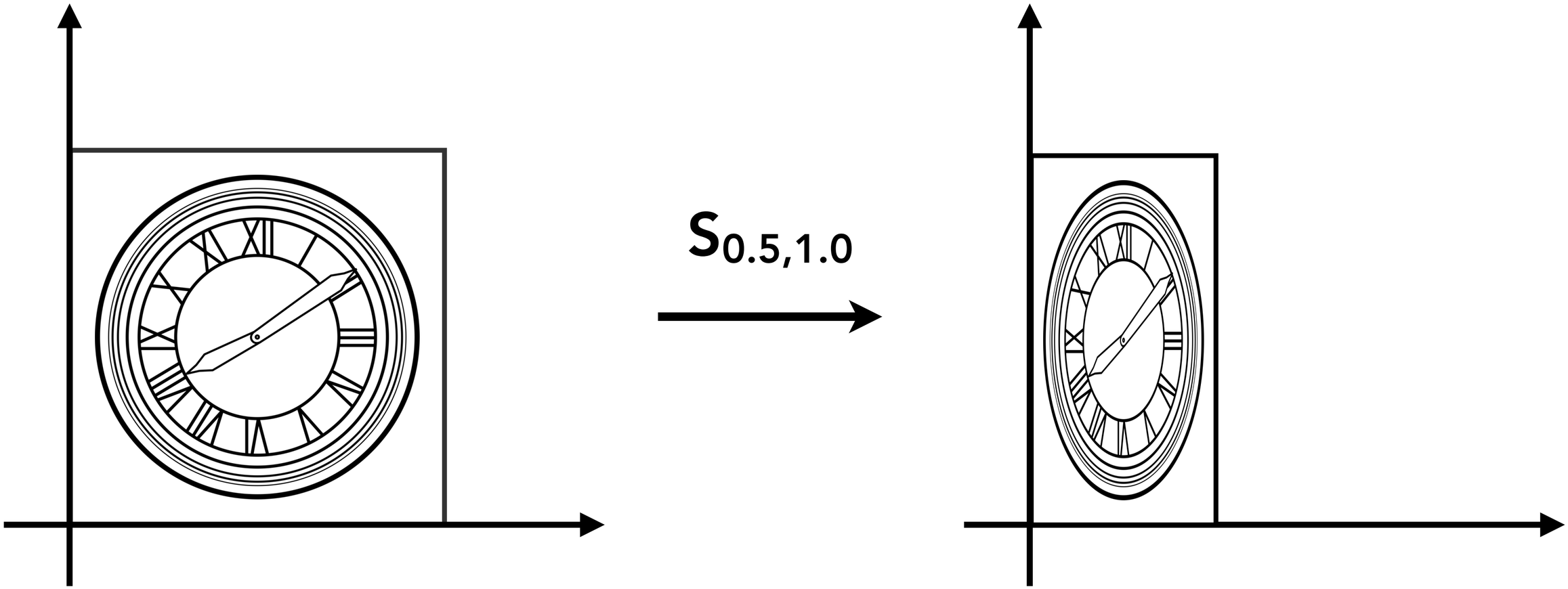
Translate



Scale



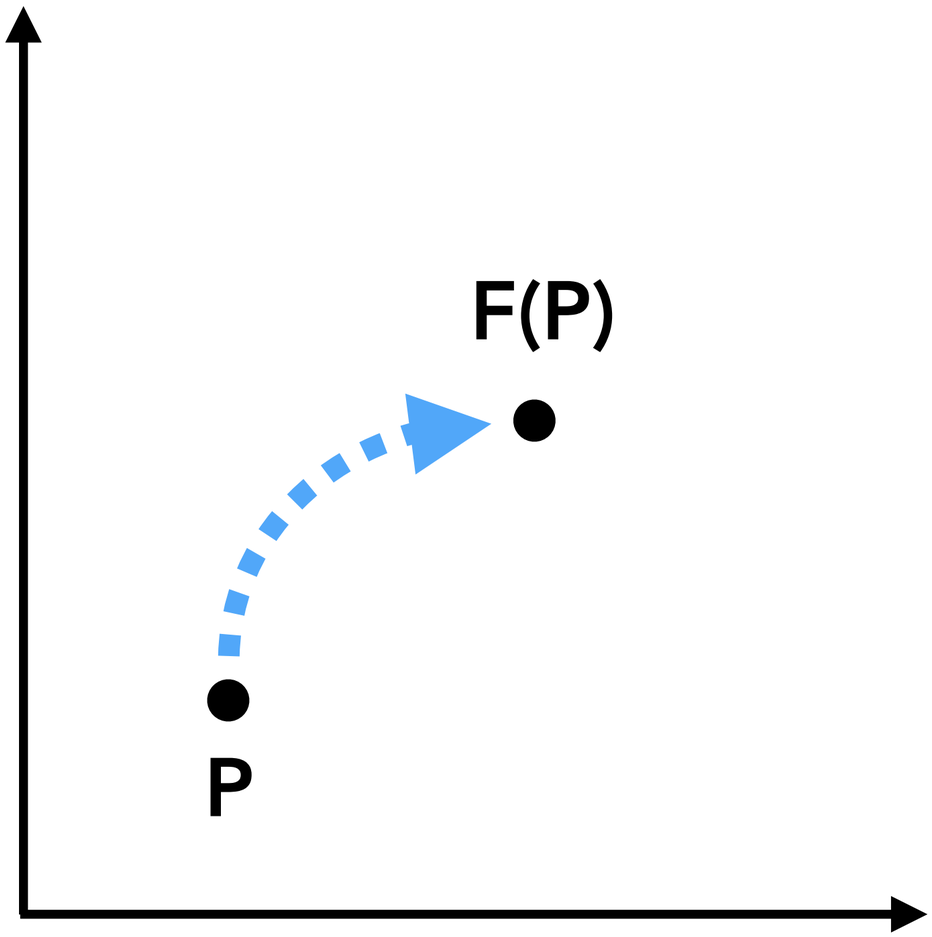
Scale (Non-Uniform)



What Are Transforms?

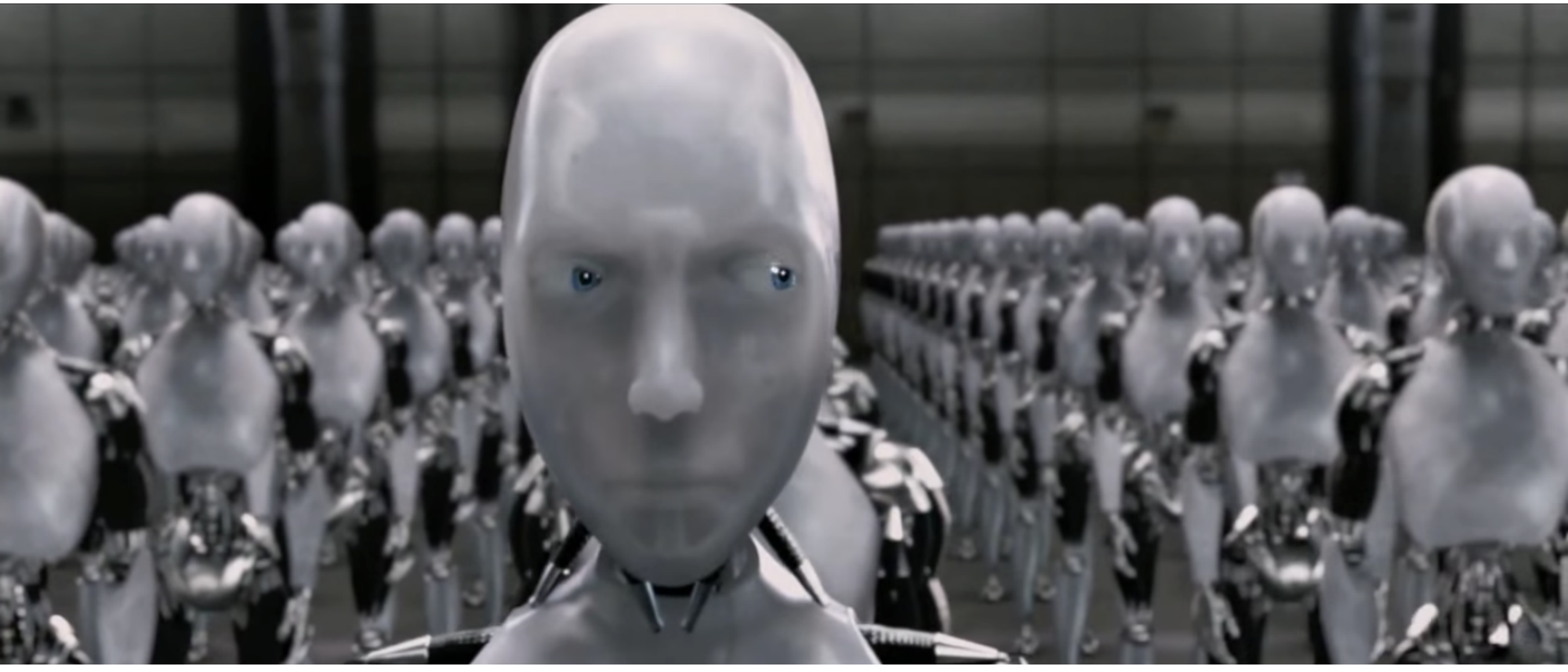
Just functions acting on points

- (x′,y′,z′) = F(x,y,z)
- P′ = F(P)



Why Study Transforms?

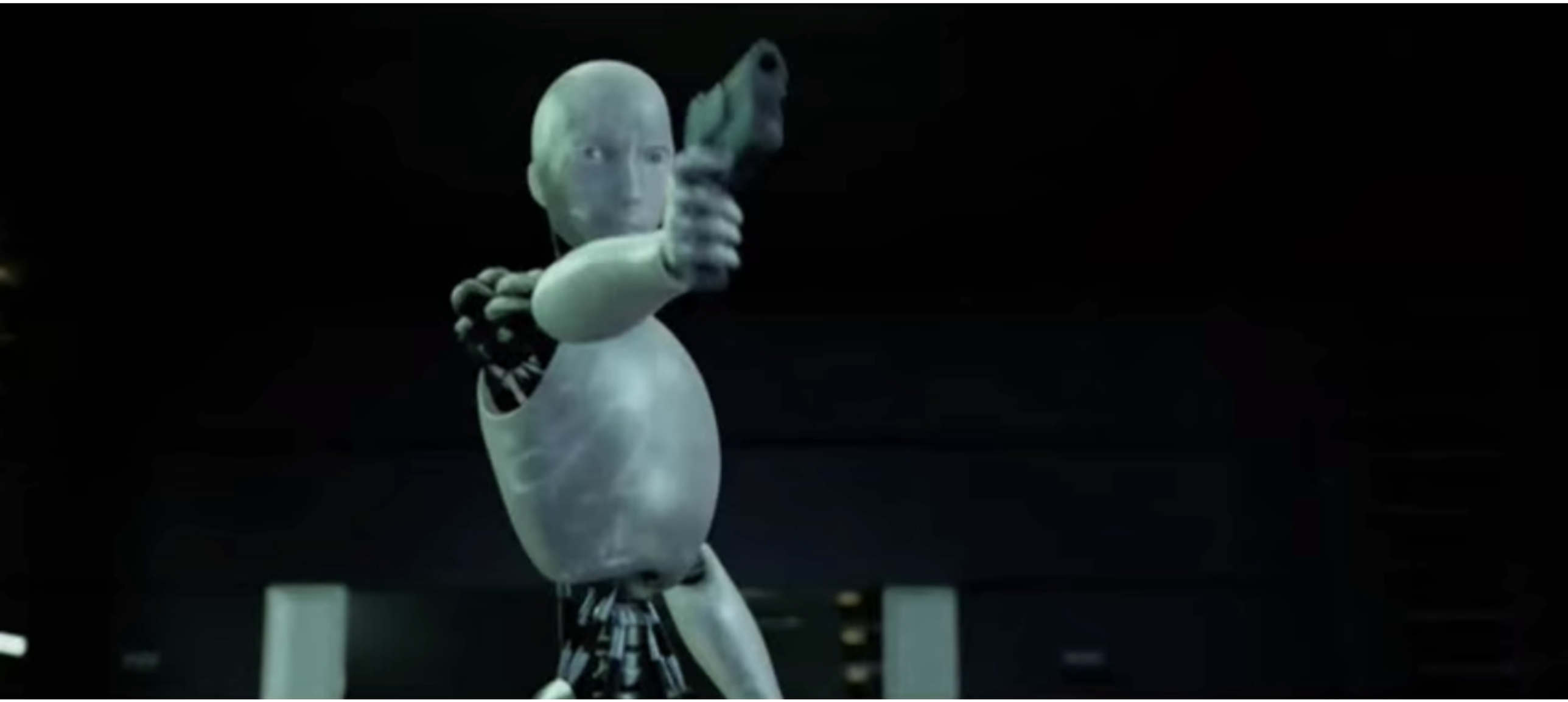
Modeling A Robot Army



iRobot movie

Transforms can describe position of object instances

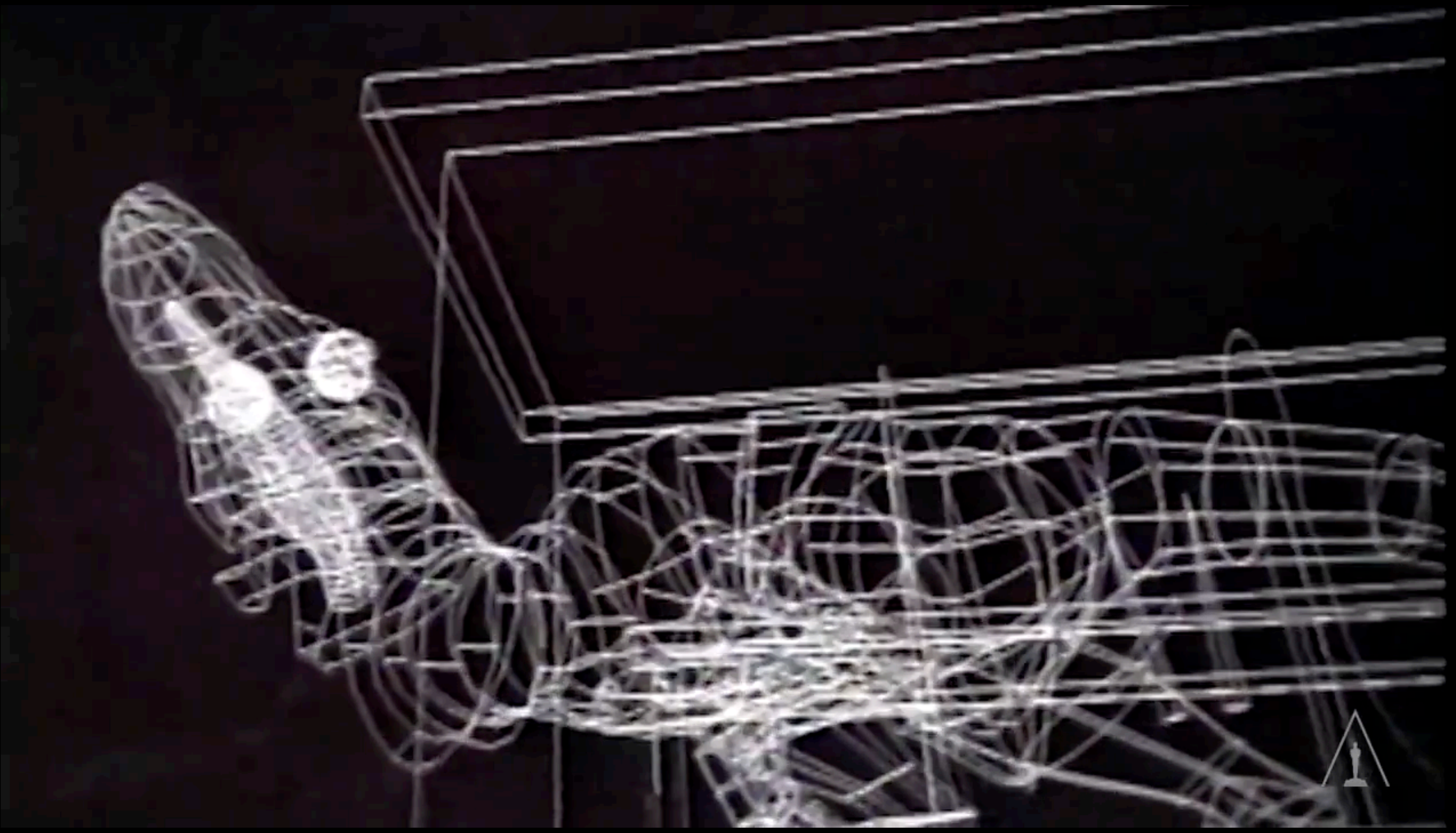
Posing a Character's Skeleton



iRobot movie

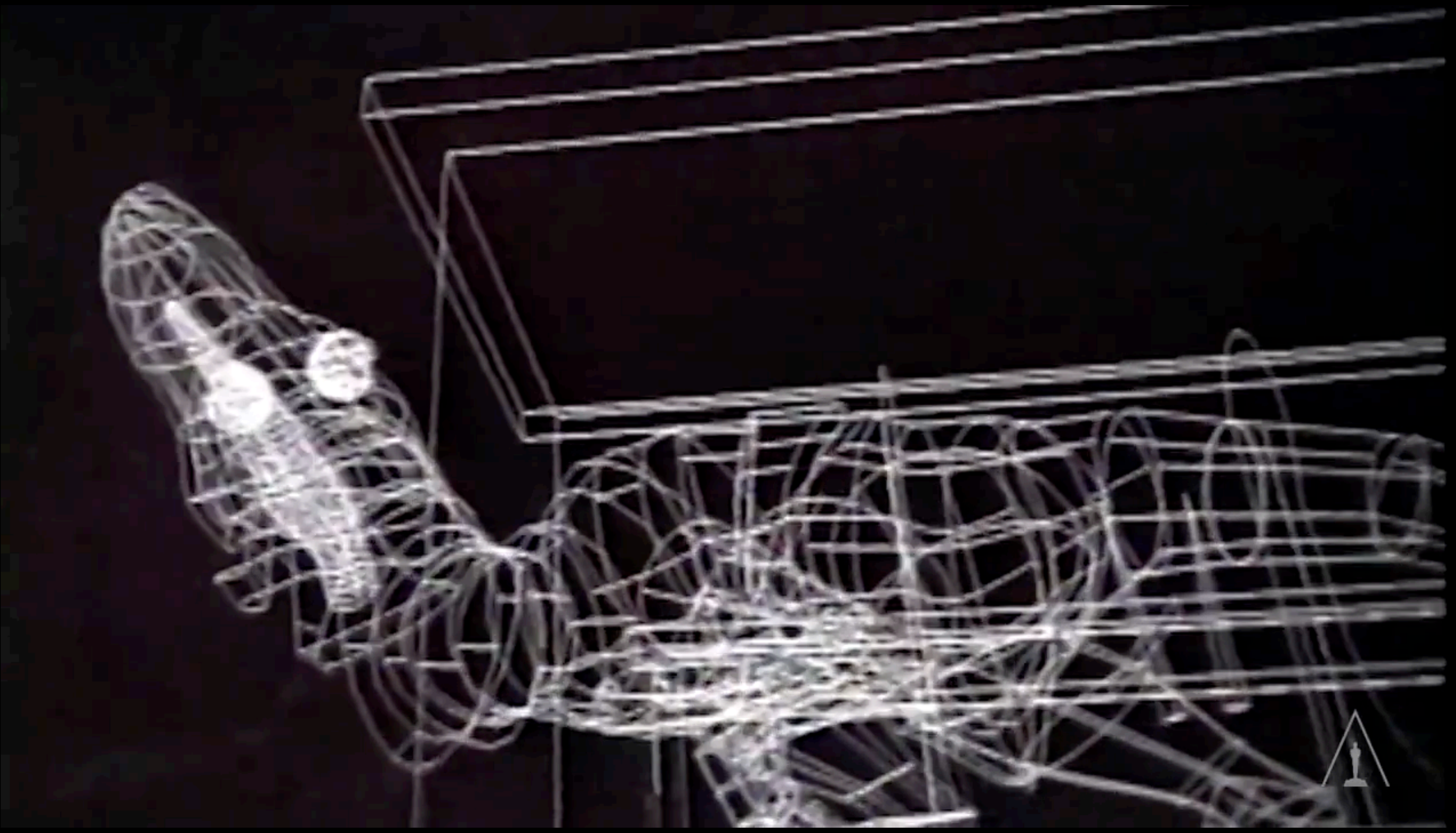
Transforms can describe relative position of connected body parts

Project Polygons in 3D to 2D Screen



Moments That Changed The Movies: Jurassic Park
https://www.youtube.com/watch?v=KWsbcBvYqN8

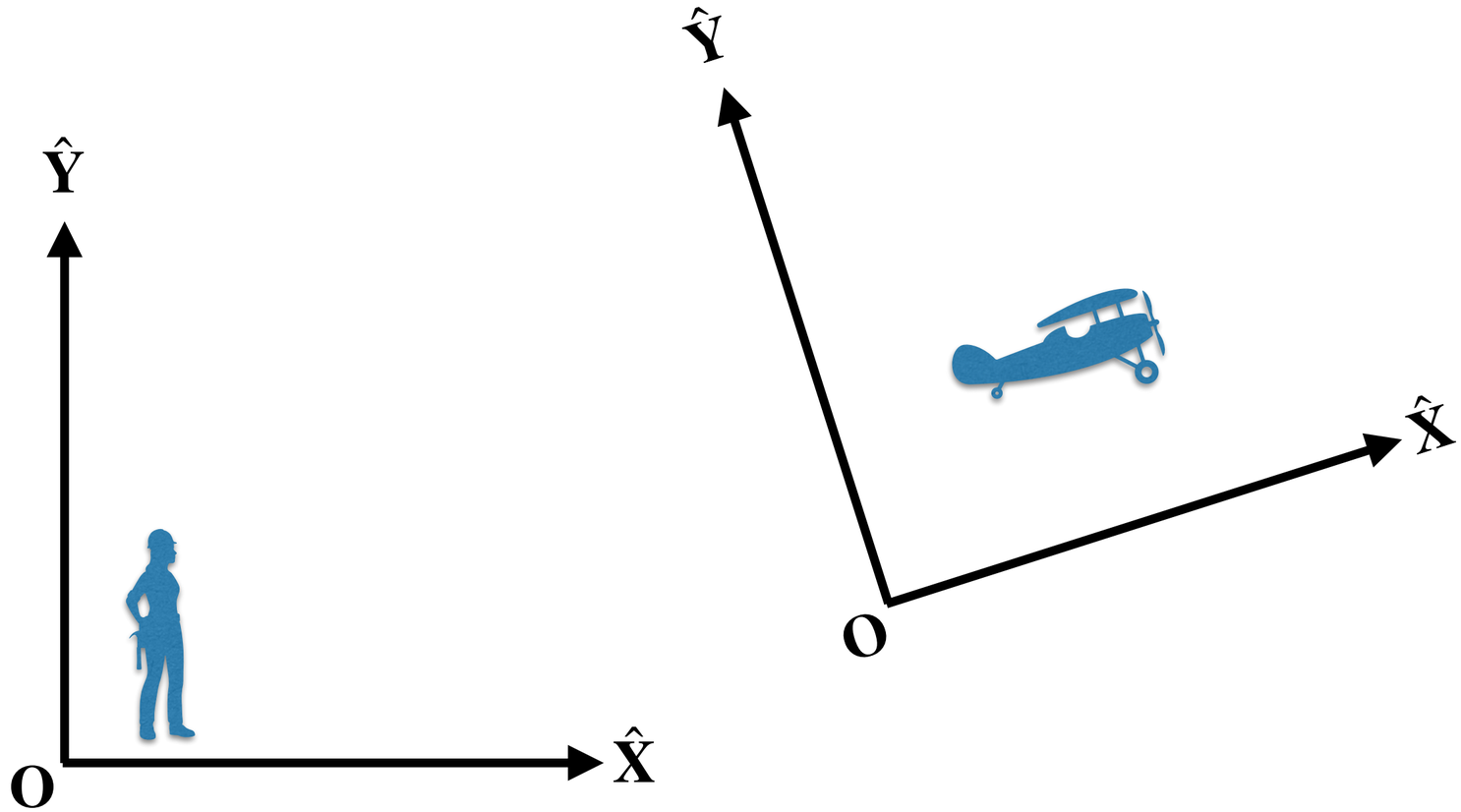
Project Polygons in 3D to 2D Screen



Moments That Changed The Movies: Jurassic Park
https://www.youtube.com/watch?v=KWsbcBvYqN8

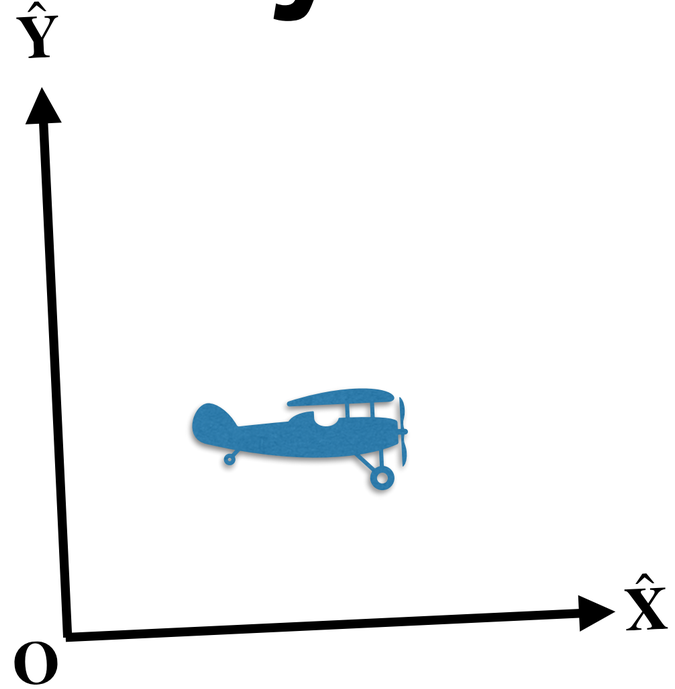
Coordinate Systems are Arbitrary

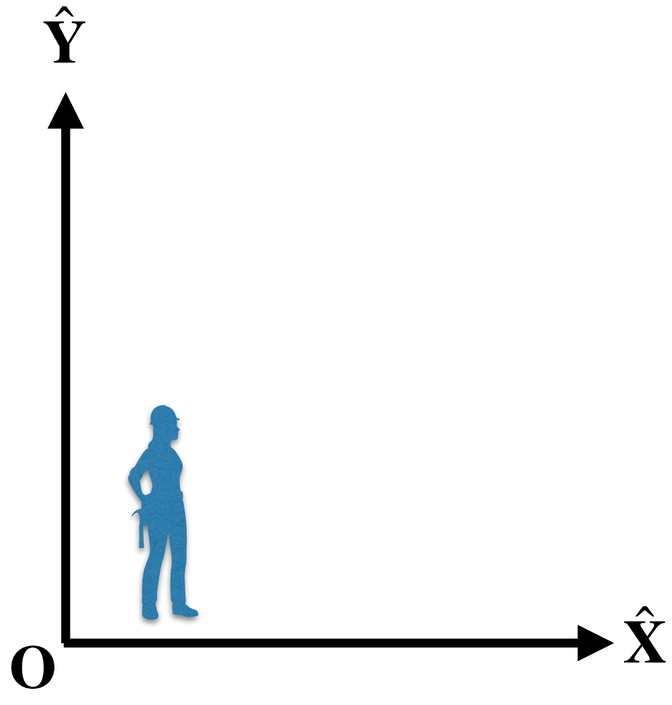
So feel free to pick one you like!



Coordinate Systems are Arbitrary

So feel free to pick one you like!





Why Study Transforms?

Modeling

- Define shapes in convenient coordinates
- Enable multiple copies of the same object
- Efficiently represent hierarchical scenes

Viewing

- World coordinates to camera coordinates
- Parallel / perspective projections from 3D to 2D

Other

- Switching between coordinate systems

Lecture Outline

How to think about and use transformations

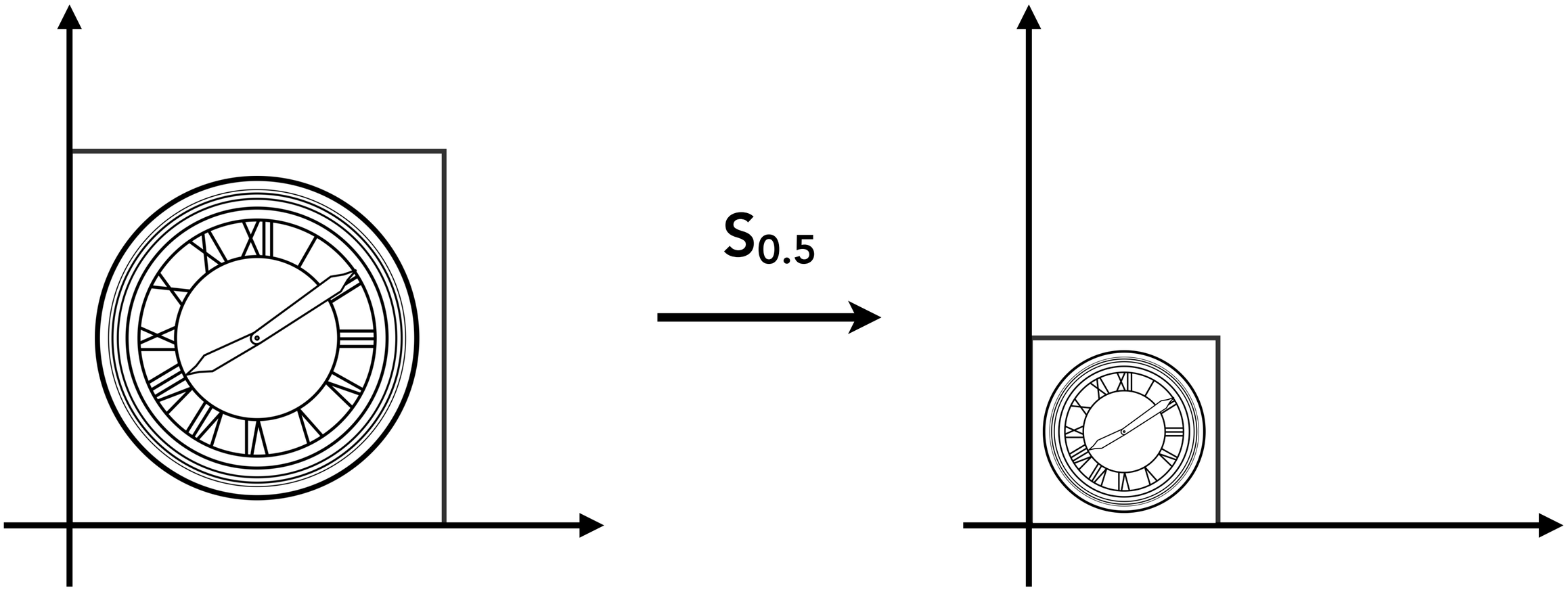
- Types: rotate, translate, scale, …
- Coordinate frames
- Composing multiple transformations
- Hierarchical transforms
- Perspective projection

How to implement?

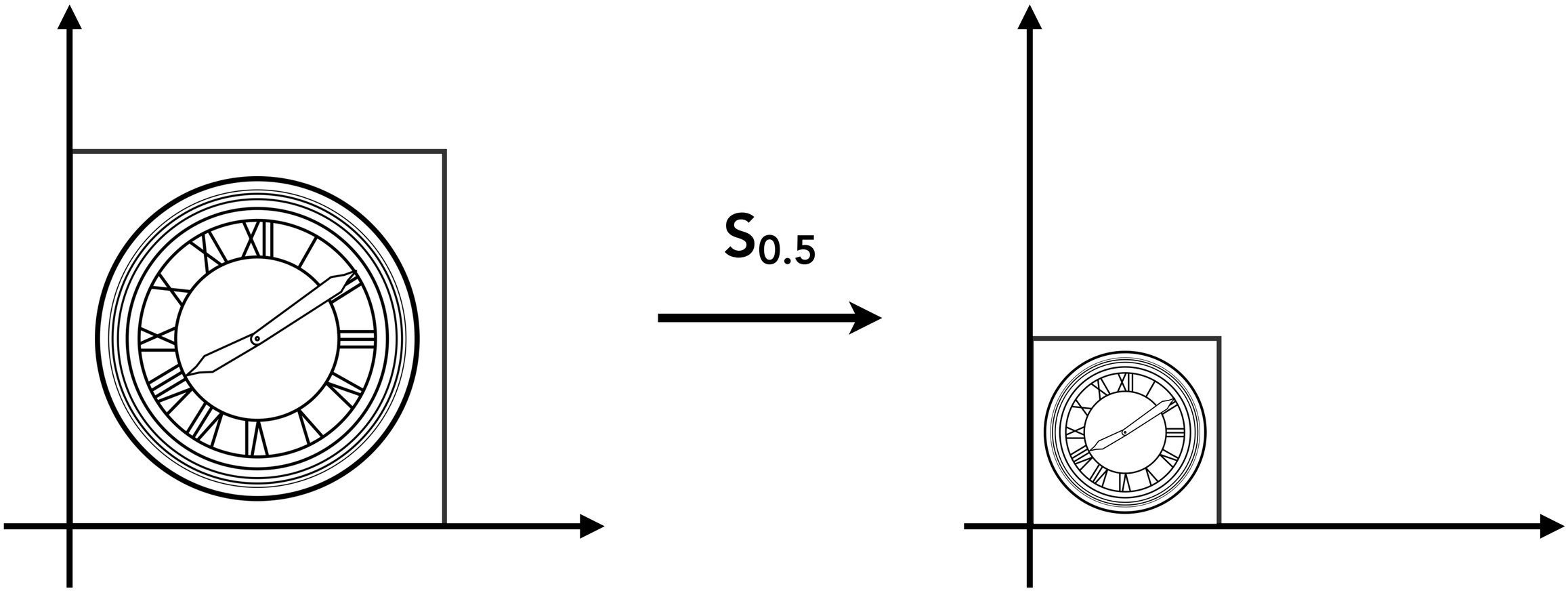
- Represent transforms as matrices
- Homogeneous coordinates

Linear Transforms = Matrices

Scale Transform

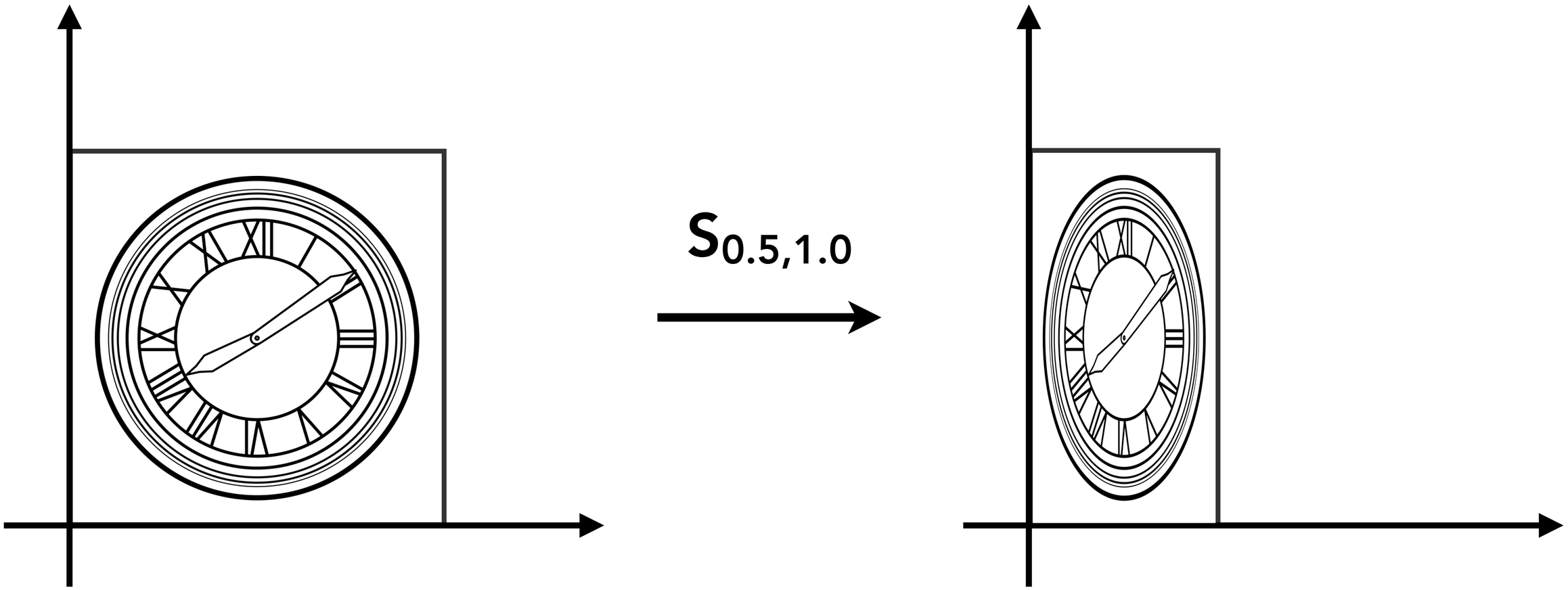


Scale Transform

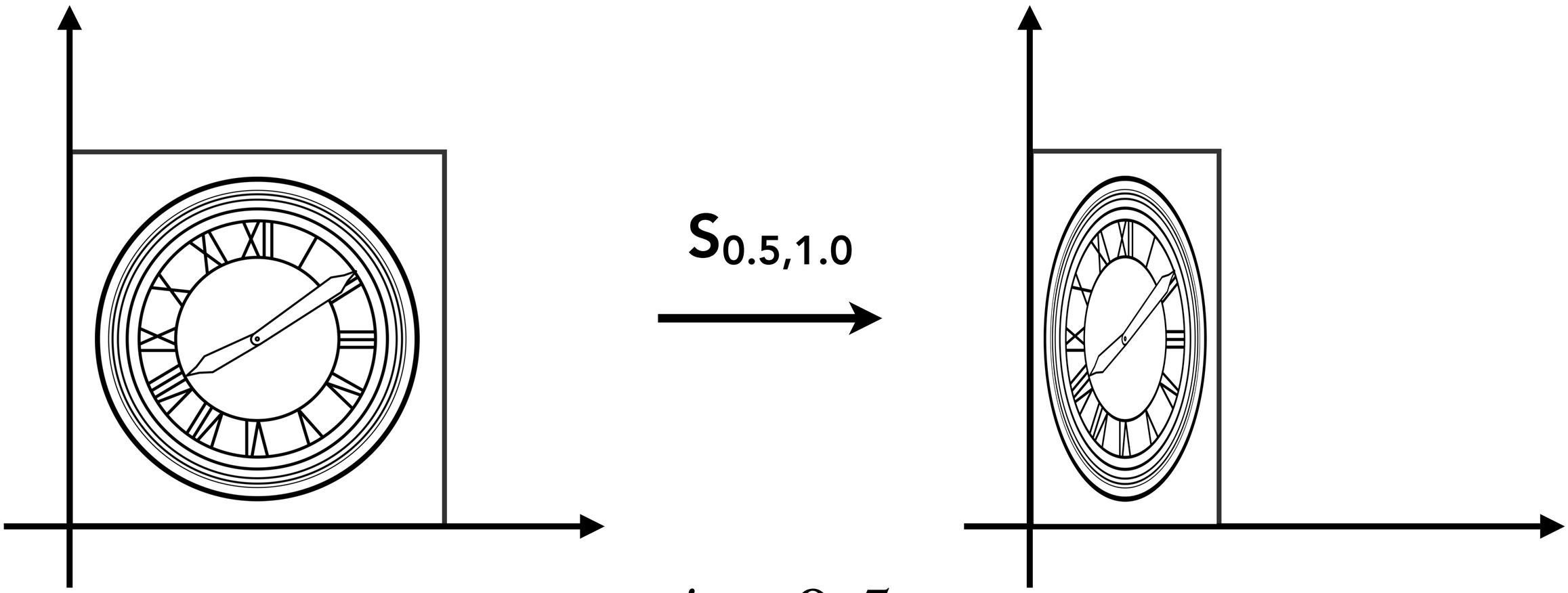


$$x' = 0.5x$$
$$y' = 0.5y$$

Scale Transform (non-uniform a.k.a anisotropic)



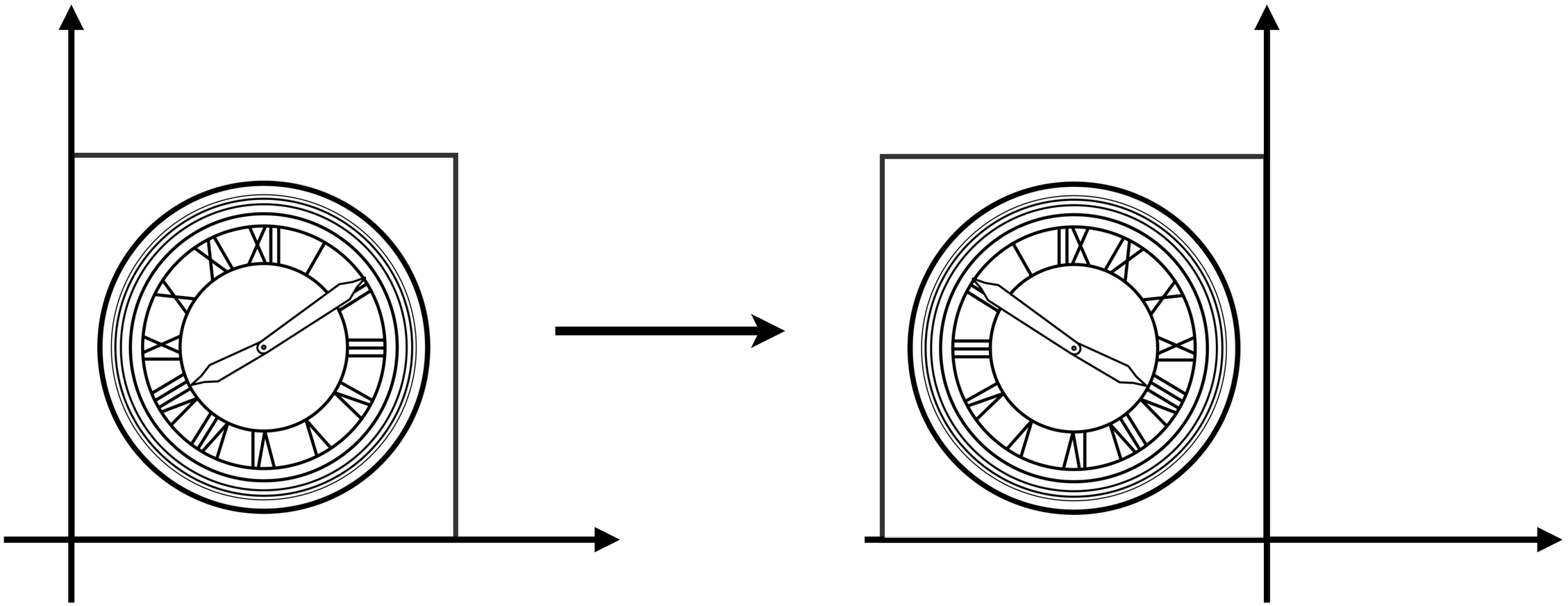
Scale Transform (non-uniform a.k.a anisotropic)



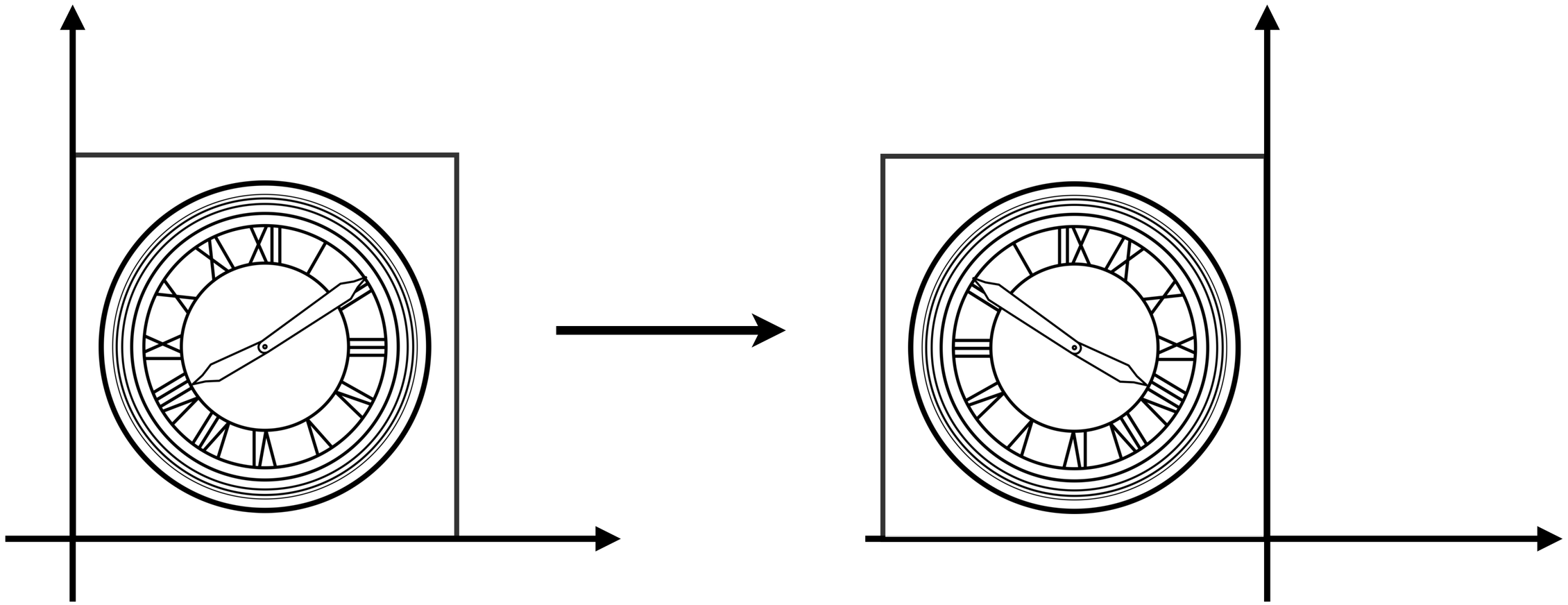
$$x' = 0.5x$$

$$y' = y$$

Reflection Transform



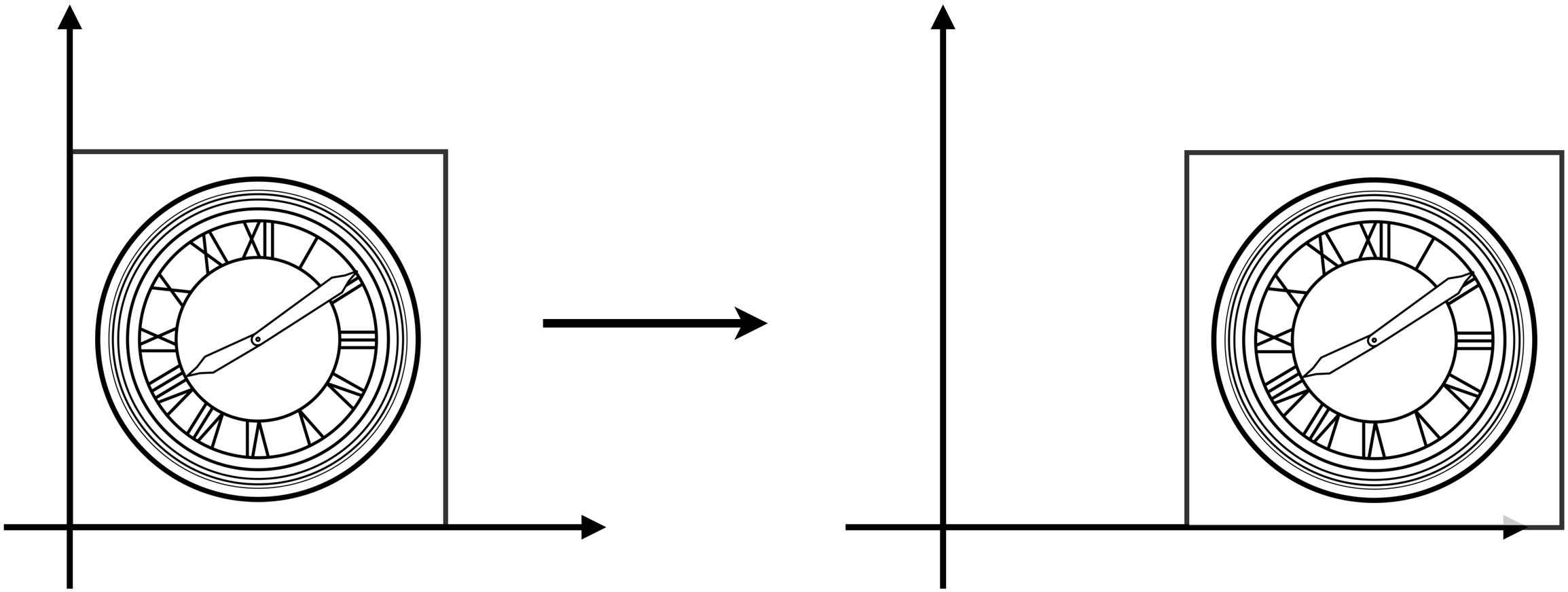
Reflection Transform



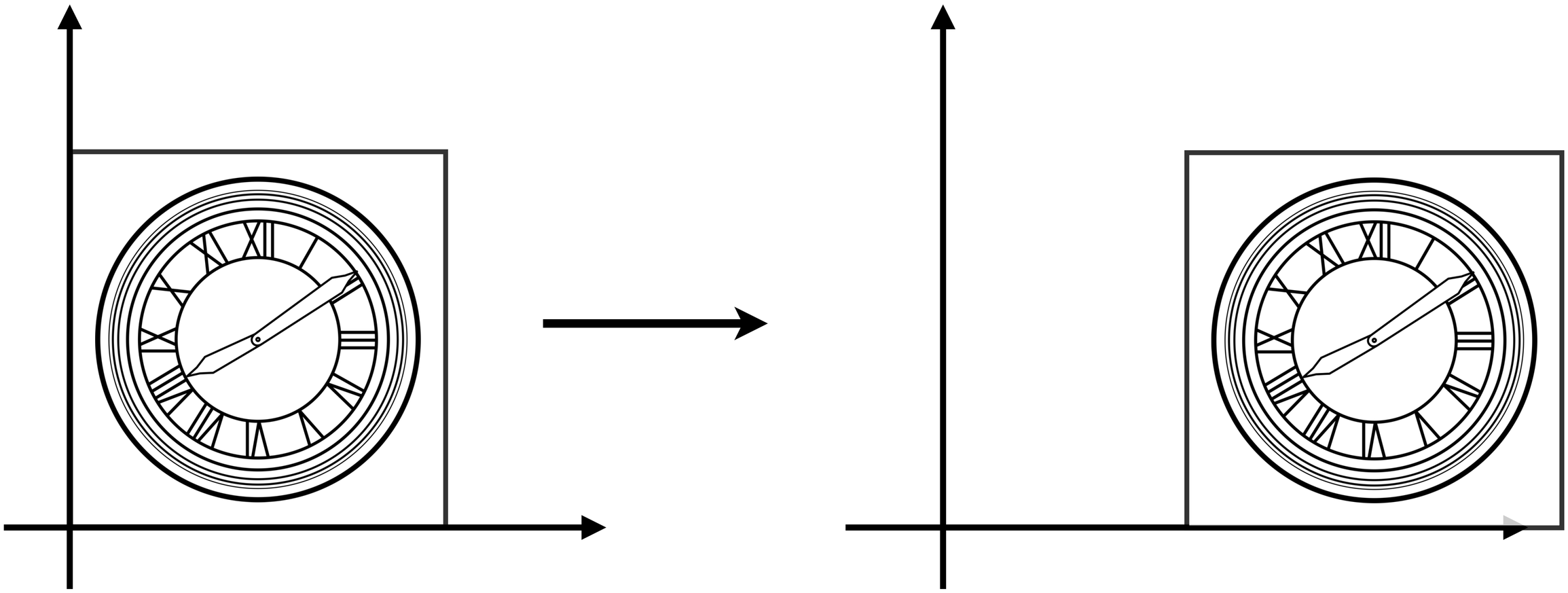
$$x' = -1.0x$$

$$y' = y$$

Translation Transform



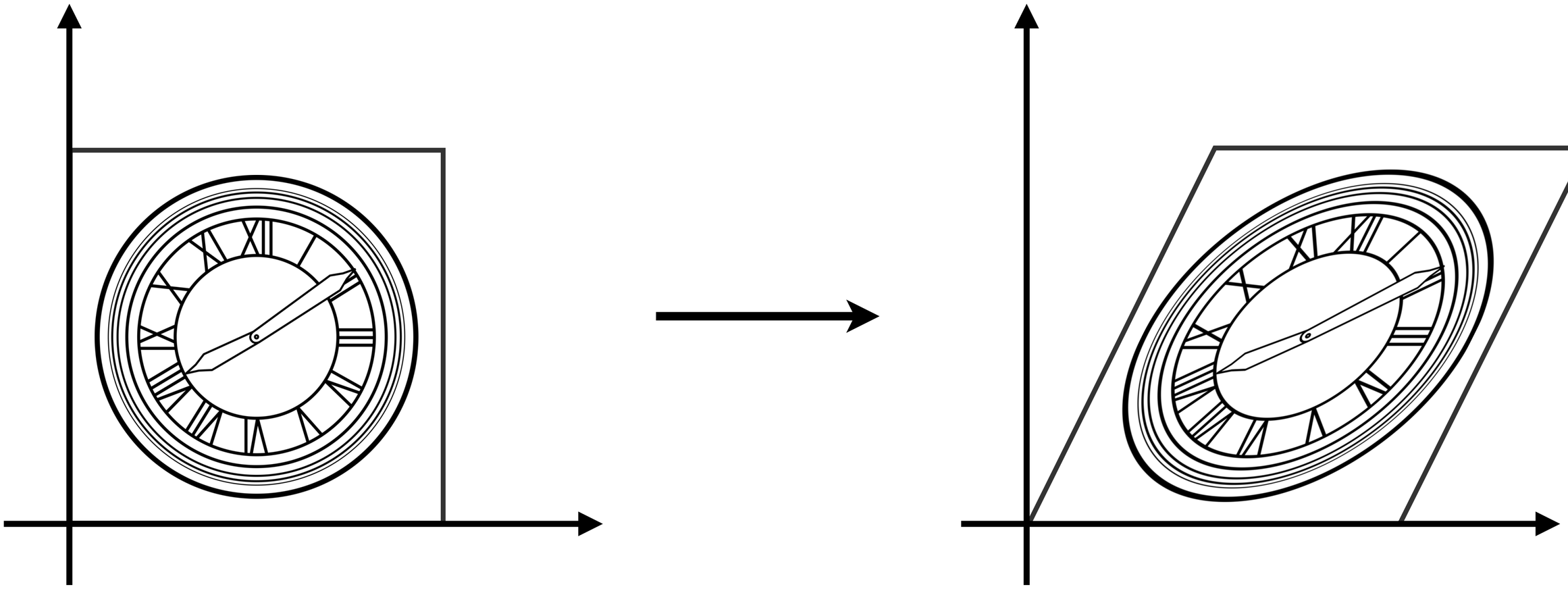
Translation Transform



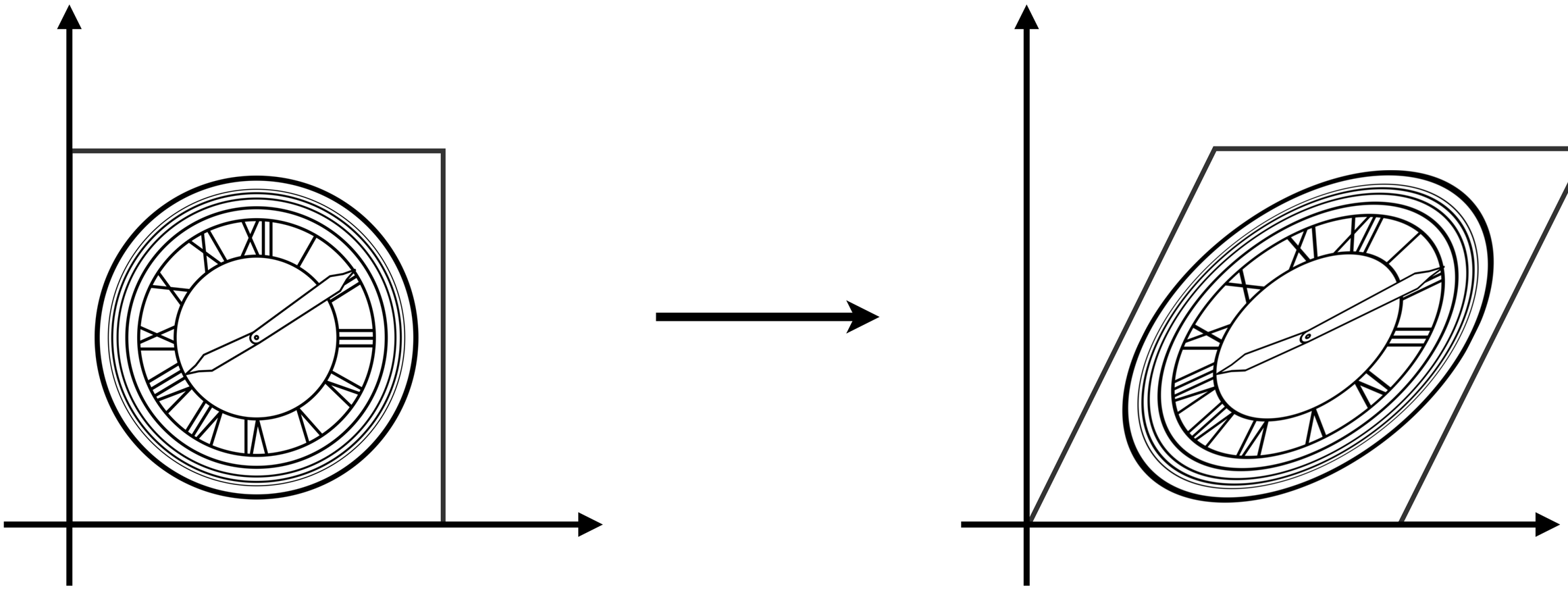
$$x' = x + 5$$

$$y' = y$$

Shear Transform



Shear Transform



$$x' = x + 0.5y$$

$$y' = y$$

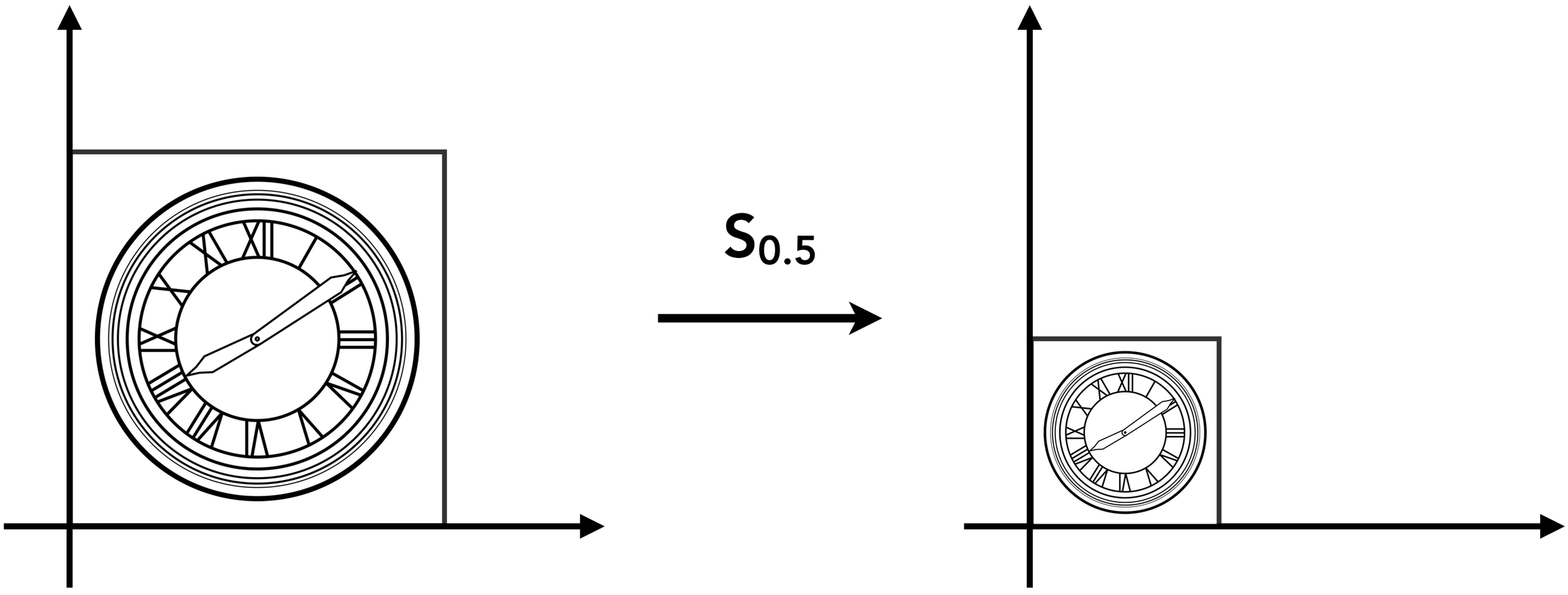
Linear Transforms = Matrices

$$x' = a\,x + b\,y$$
$$y' = c\,x + d\,y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

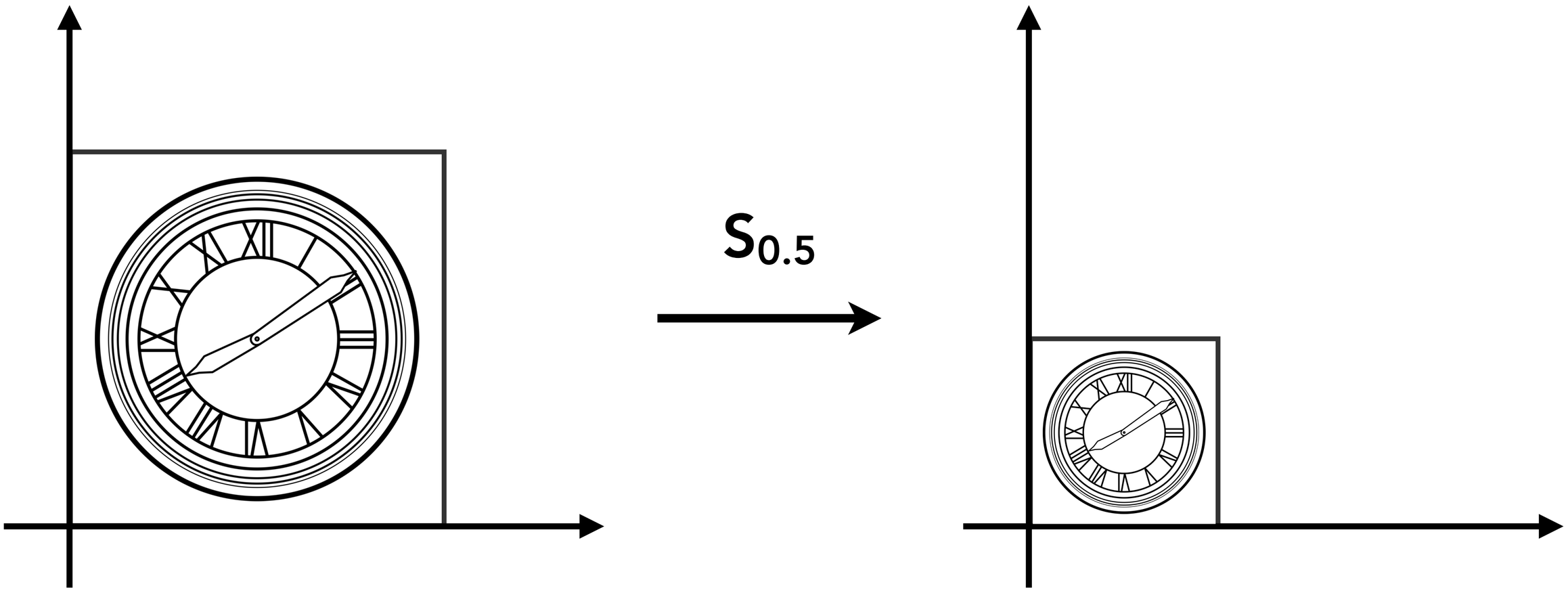
$$\mathbf{x}' = \mathbf{M}\,\mathbf{x}$$

Scale Matrix



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Scale Matrix



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Linear Transforms = Matrices

$$x' = a\,x + b\,y$$
$$y' = c\,x + d\,y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

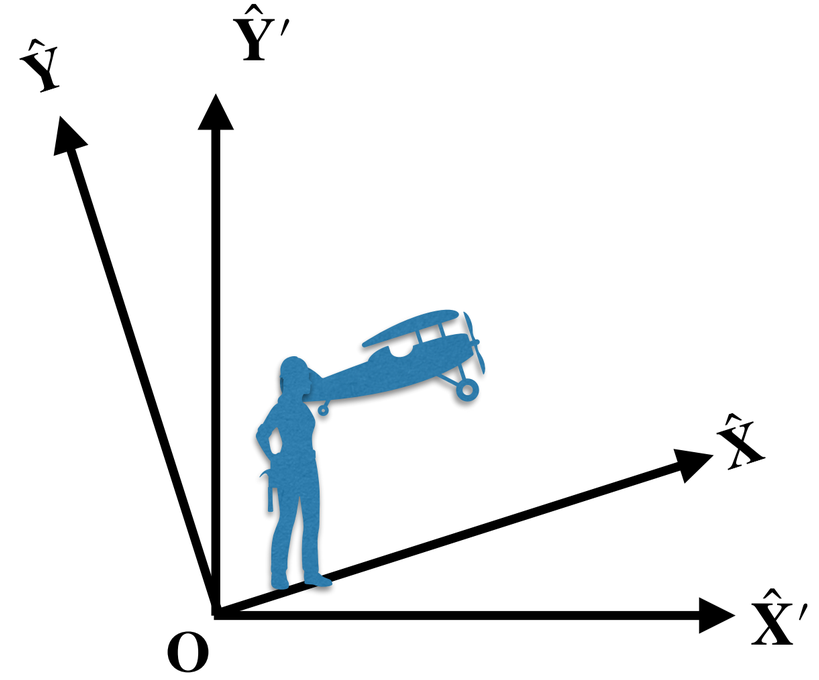
$$\mathbf{x}' = \mathbf{M}\ \mathbf{x}$$

Note: no translation (addition) yet. Will come later…

$$\mathbf{b} = \mathbf{M_1 a}$$
$$\mathbf{c} = \mathbf{M_2 b}$$
$$\mathbf{d} = \mathbf{M_3 c}$$
$$\mathbf{d} = \mathbf{M_3 M_2 M_1 a}$$
$$\mathbf{d} = (\mathbf{M_3 M_2 M_1})\mathbf{a}$$
$$\mathbf{d} = \mathbf{M a}$$

2D Coordinate Systems

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



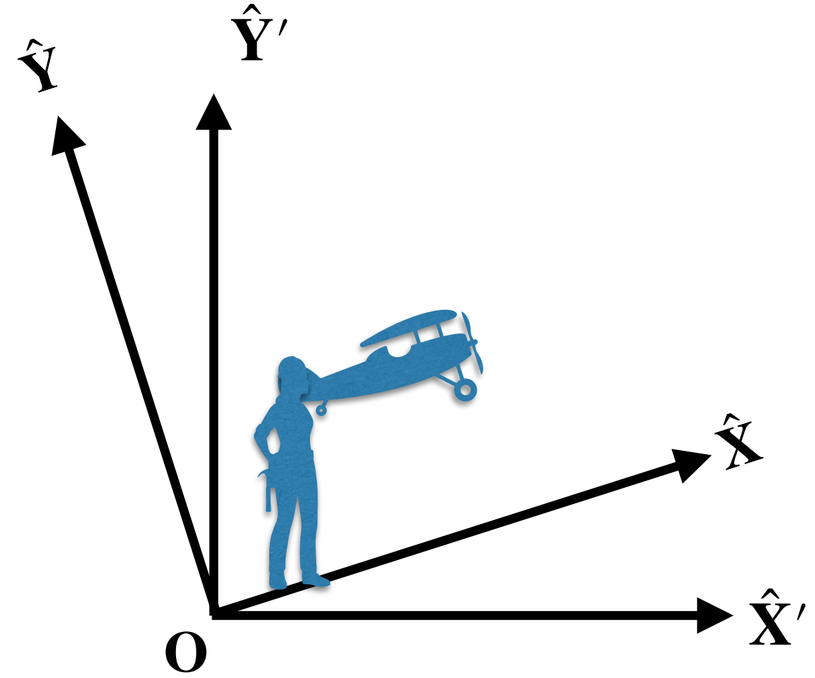
"The x-axis of the plane's coordinate system expressed in the person's coordinate system." $\begin{bmatrix} a \\ c \end{bmatrix}$

Can interpret the columns of the matrix as the x and y axes of the coordinate frame

2D Coordinate Systems

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} a \\ c \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$



"The x-axis of the plane's coordinate system expressed in the person's coordinate system." $\begin{bmatrix} a \\ c \end{bmatrix}$

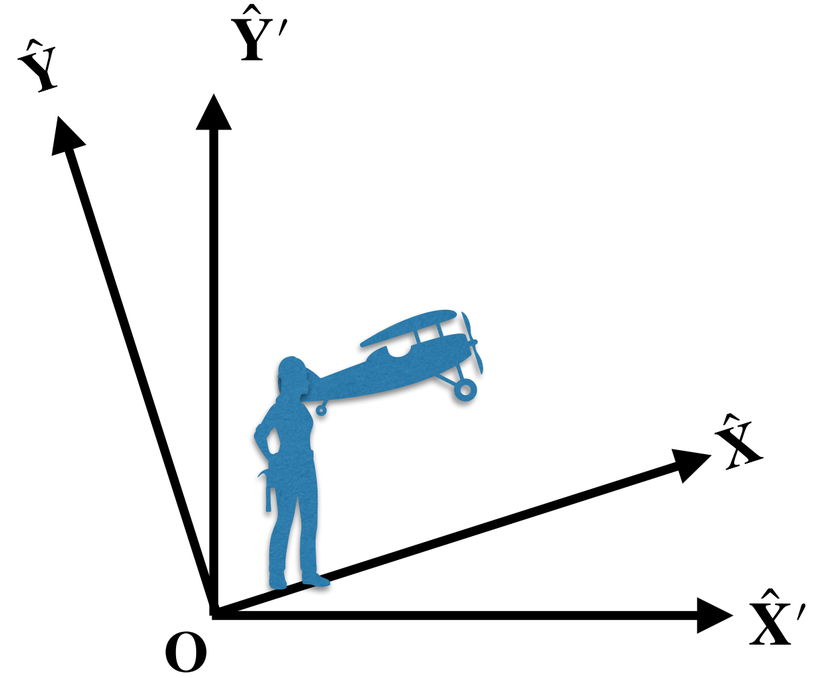
Can interpret the columns of the matrix as the x and y axes of the coordinate frame

2D Coordinate Systems

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} a \\ c \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} b \\ d \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



"The x-axis of the plane's coordinate system expressed in the person's coordinate system." $\begin{bmatrix} a \\ c \end{bmatrix}$

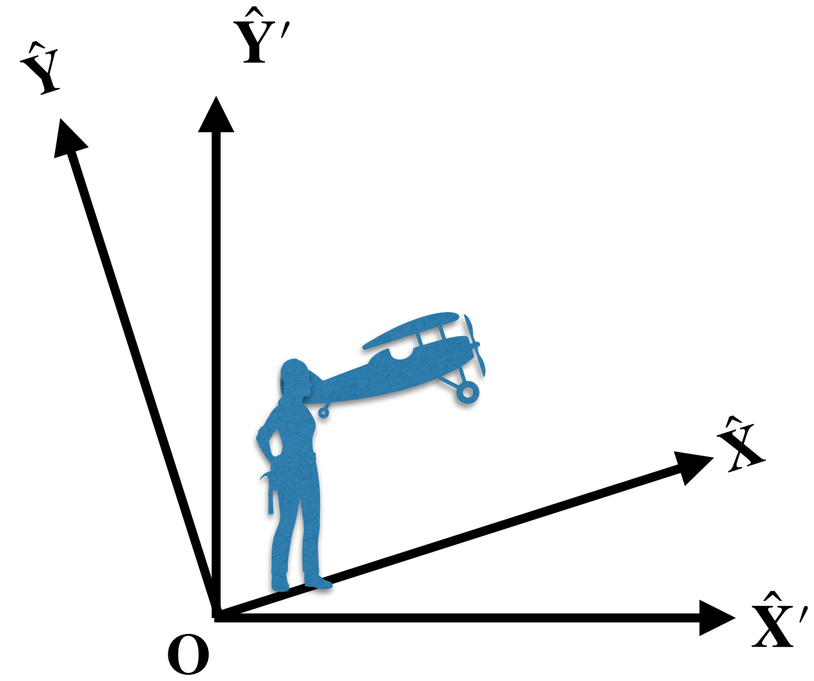
Can interpret the columns of the matrix as the x and y axes of the coordinate frame

2D Coordinate Systems

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

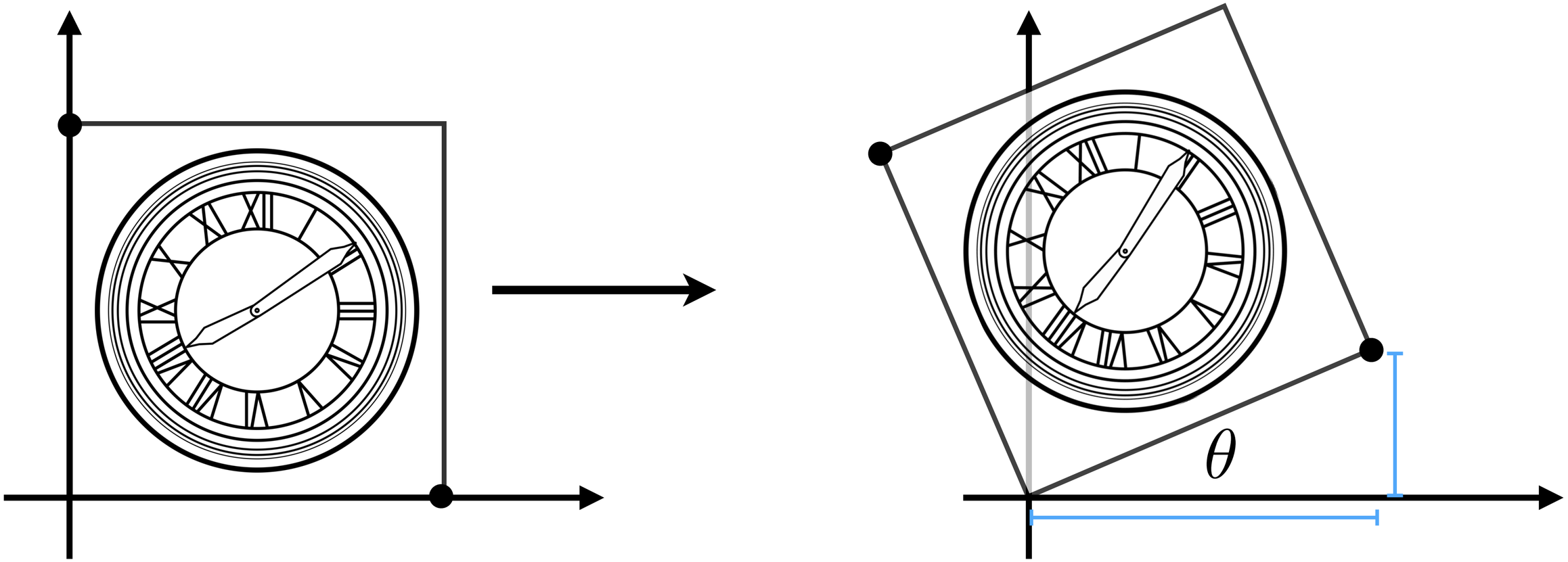
$$\begin{bmatrix} a \\ c \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} b \\ d \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

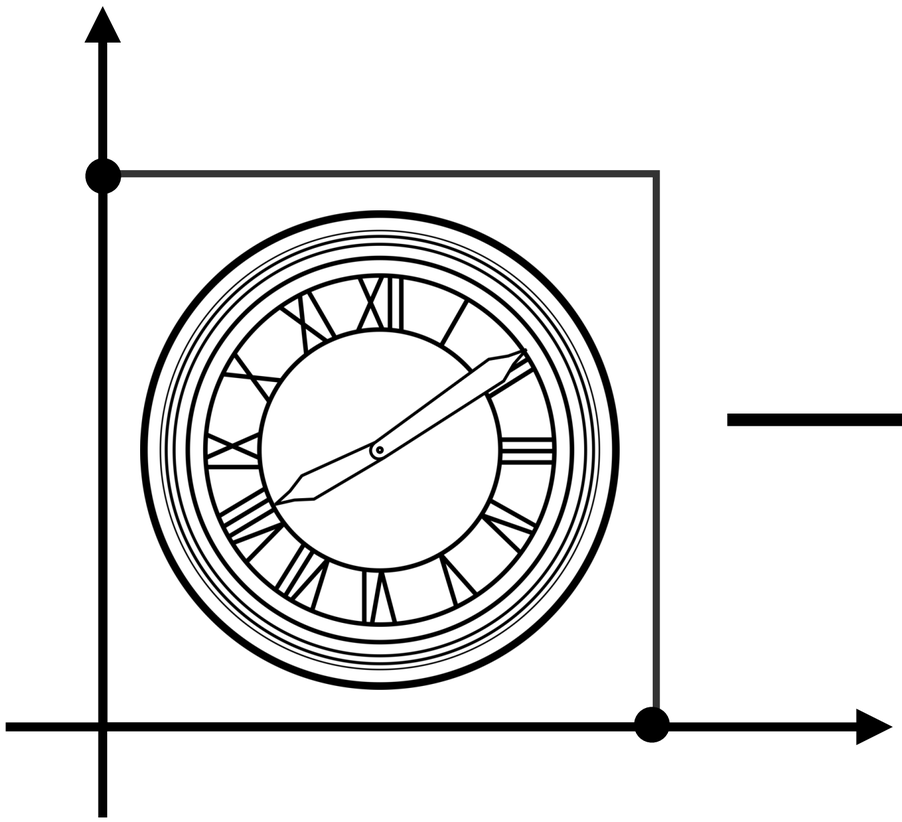


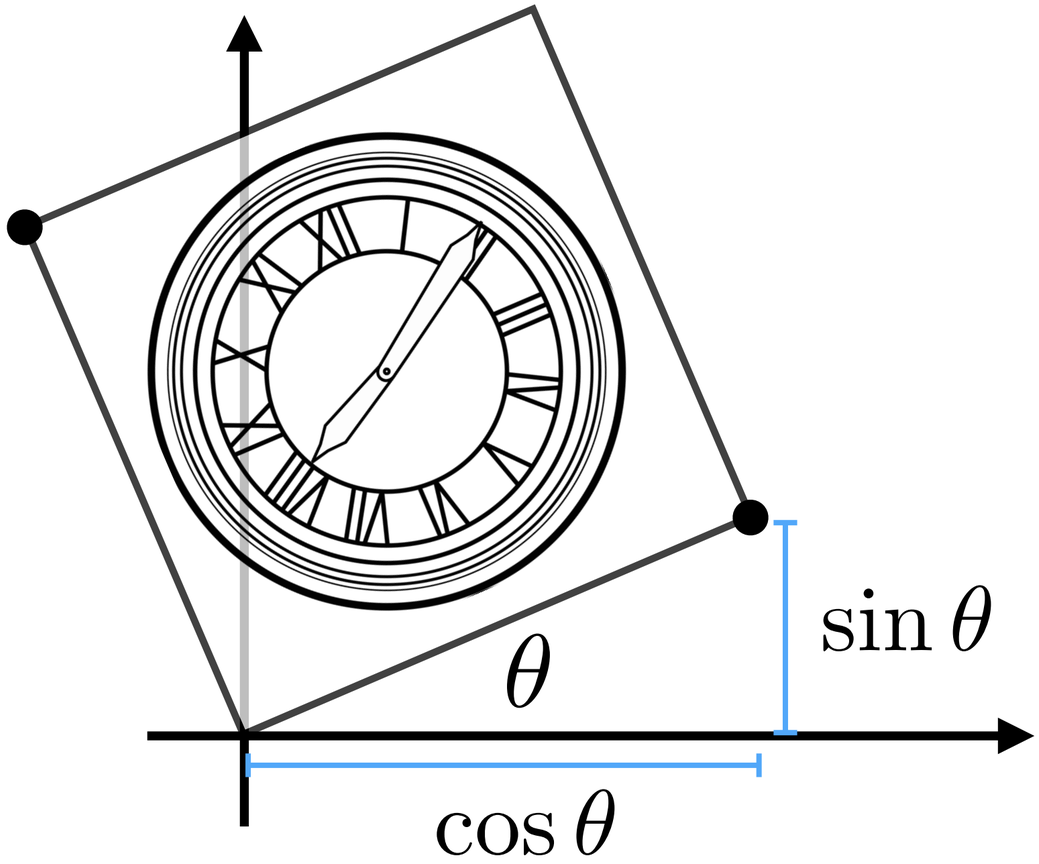
Consider the inverse matrix…

Rotation Matrix

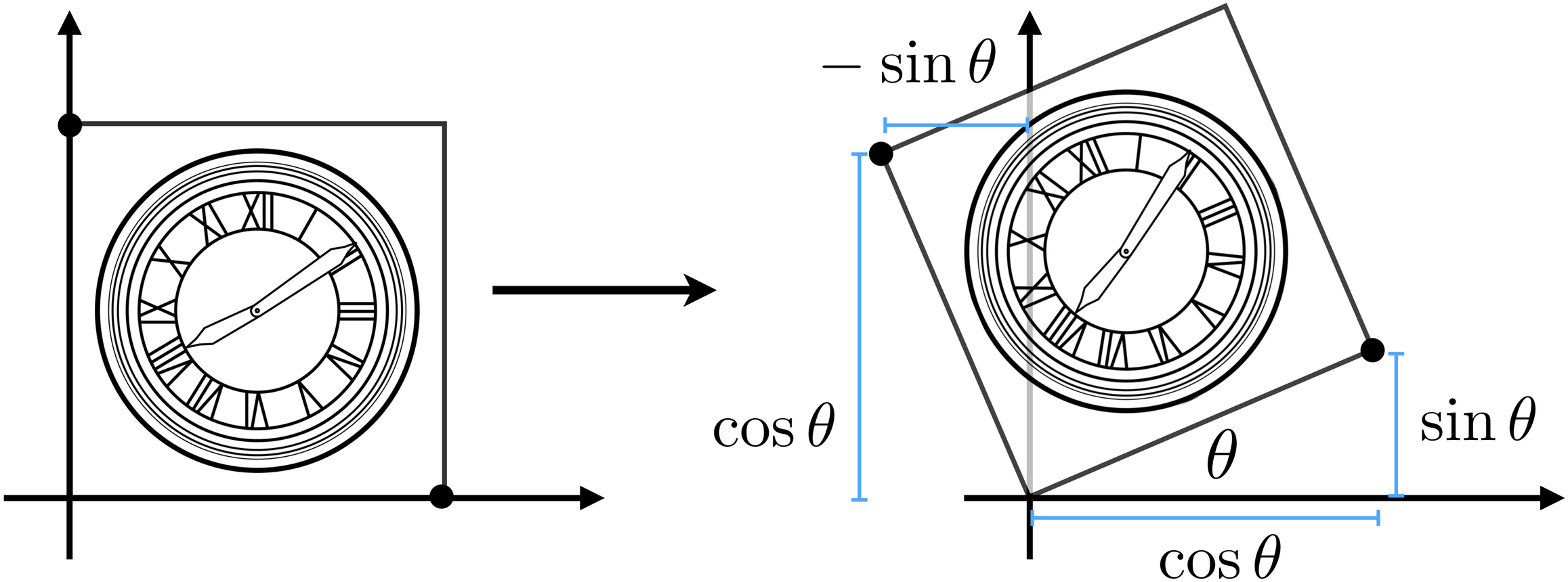


Rotation Matrix

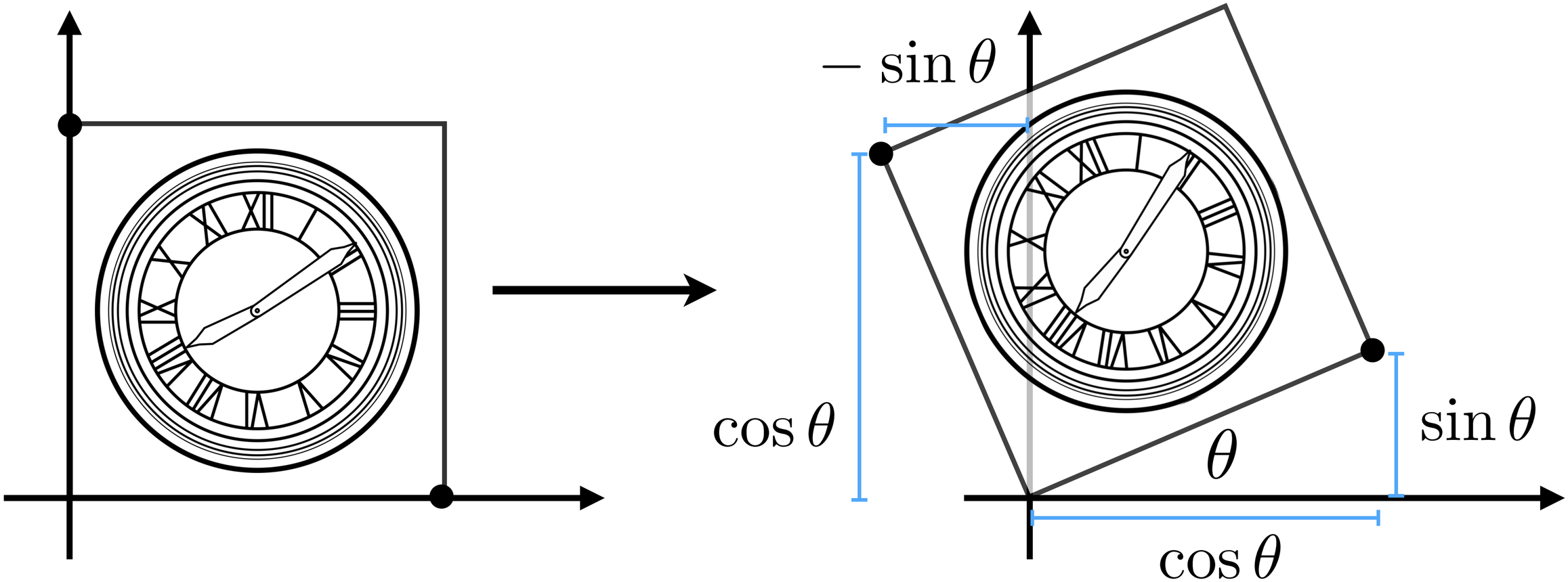




Rotation Matrix

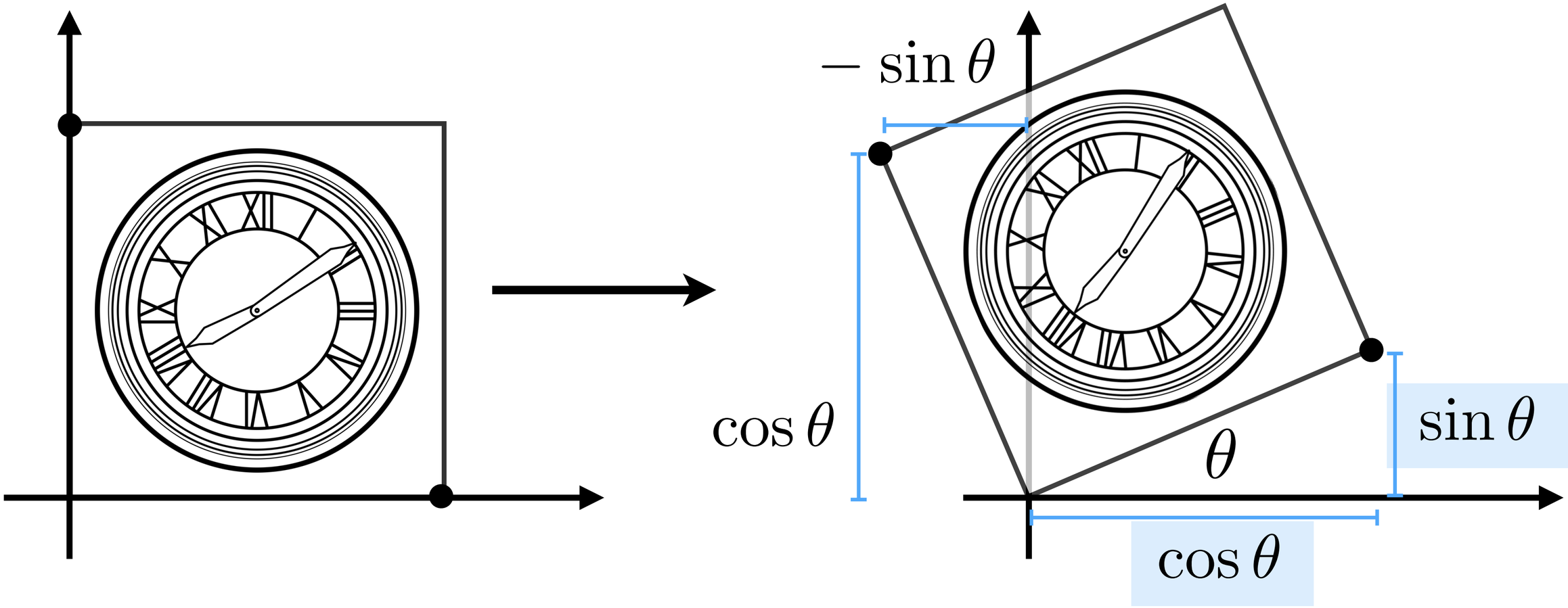


Rotation Matrix



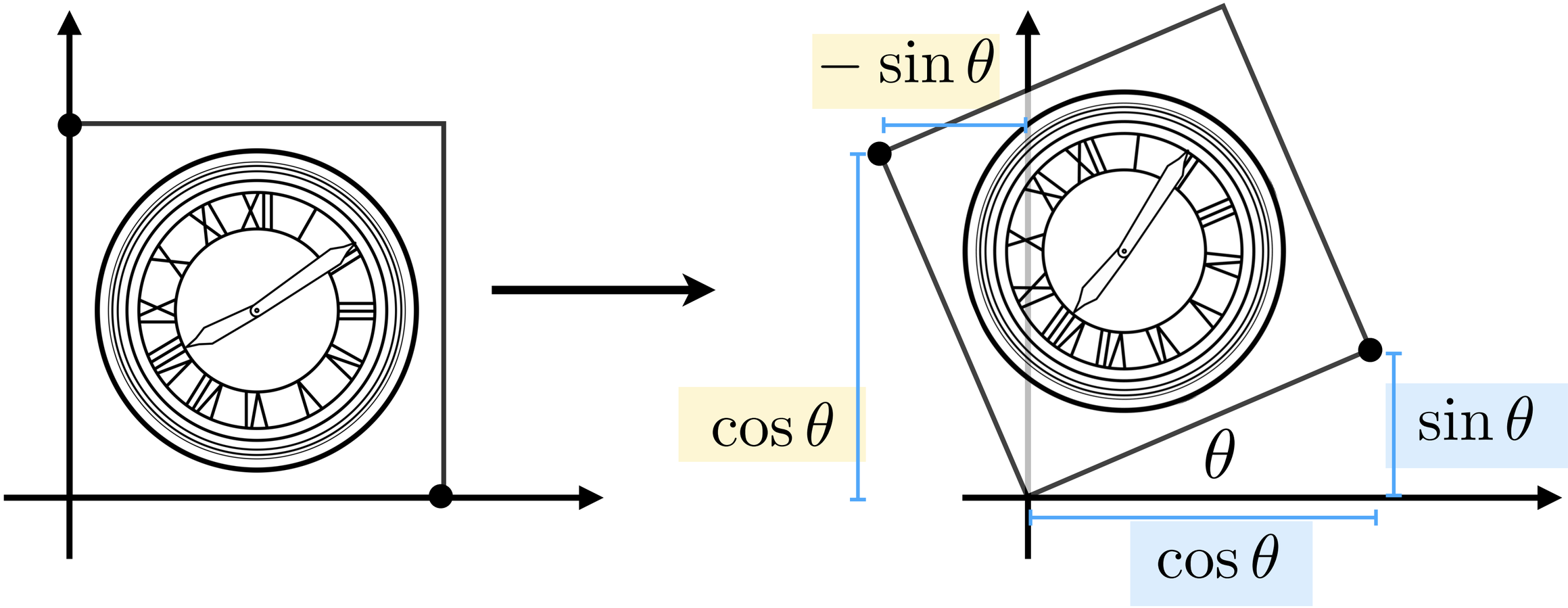
$$\mathbf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Rotation Matrix



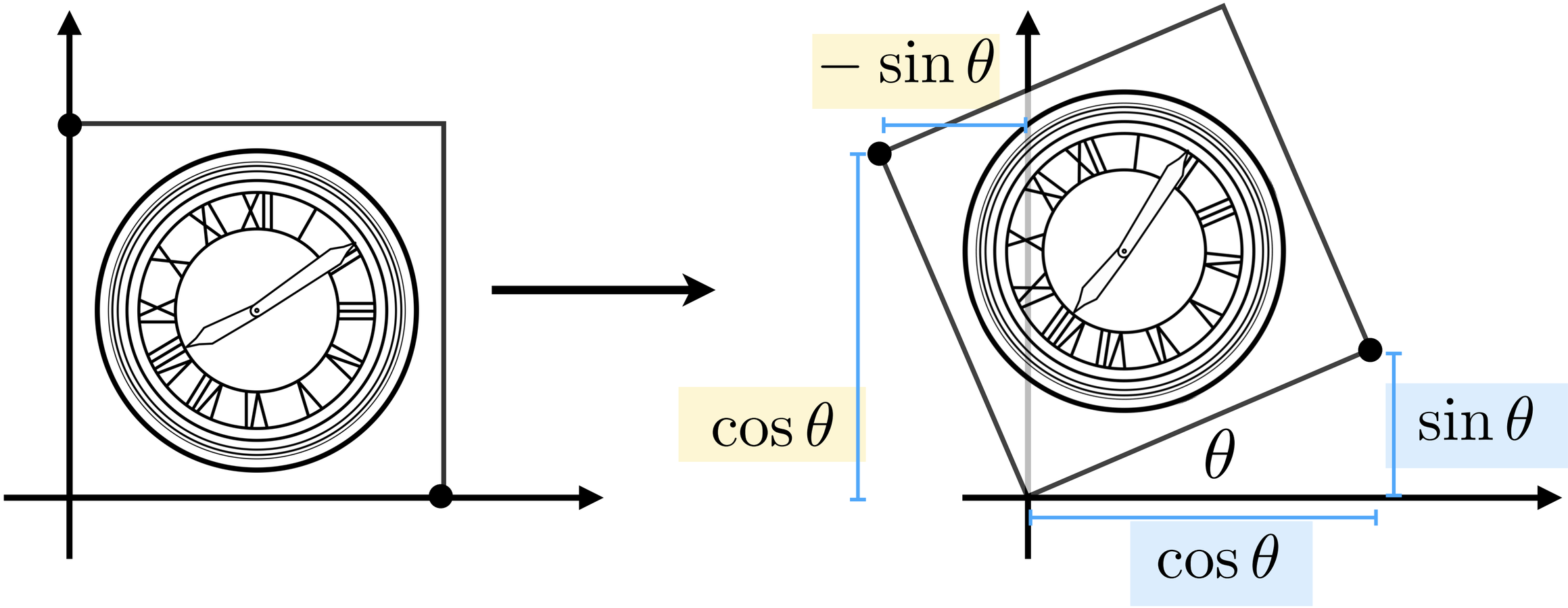
$$\mathbf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Rotation Matrix



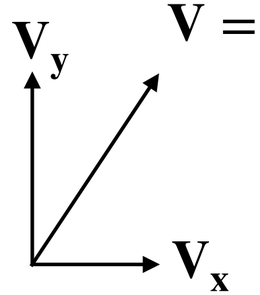
$$\mathbf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Rotation Matrix



$$\mathbf{R}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Where did this come from?



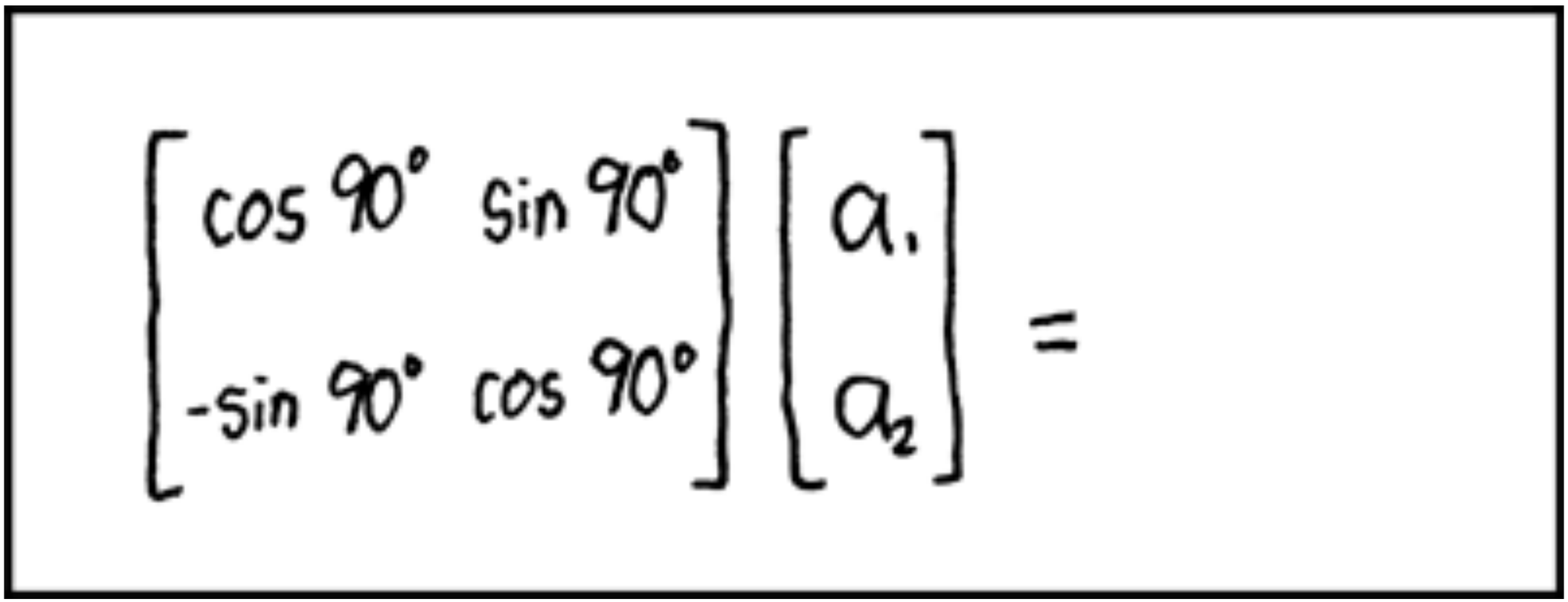
$$\mathbf{V} = \mathbf{V}_x + \mathbf{V}_y$$

If you can figure out horizontal and vertical then you can do any vector!

2D Rotation Matrix: Another Way

xkcd

A WEBCOMIC OF ROMANCE, SARCASM, MATH, AND LANGUAGE.

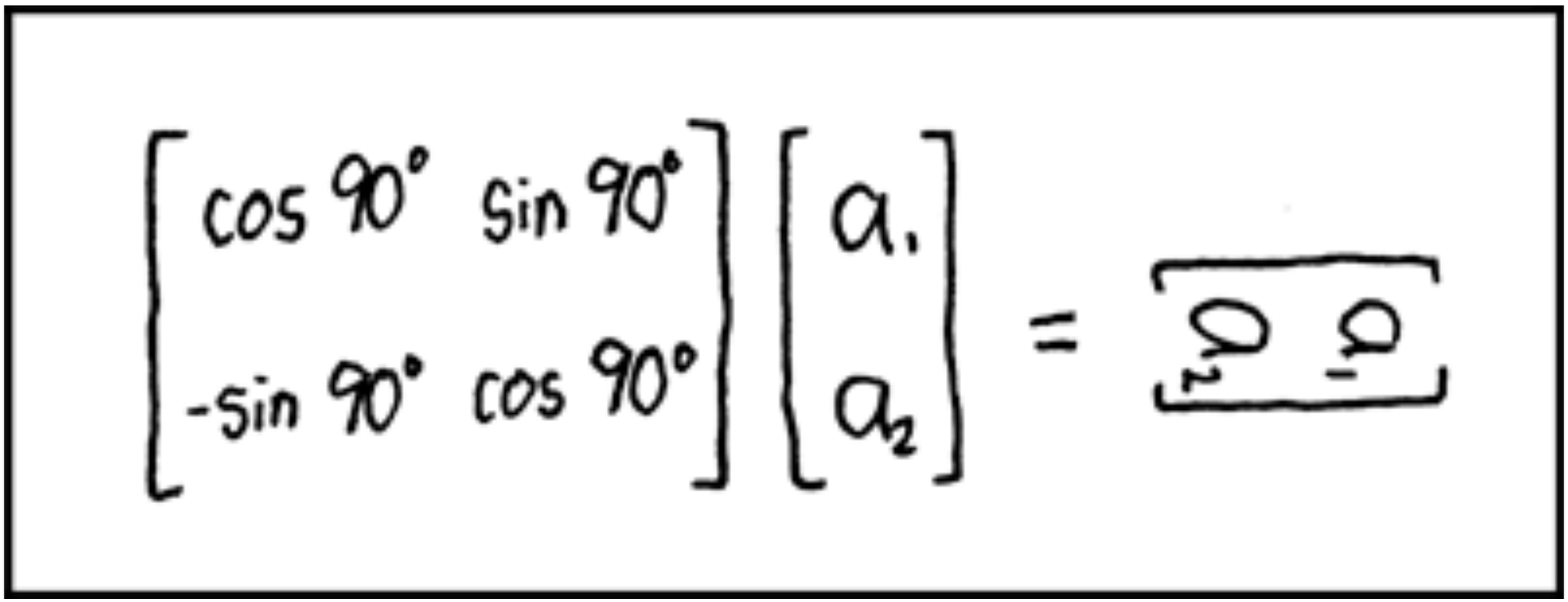


http://xkcd.com/184/

2D Rotation Matrix: Another Way

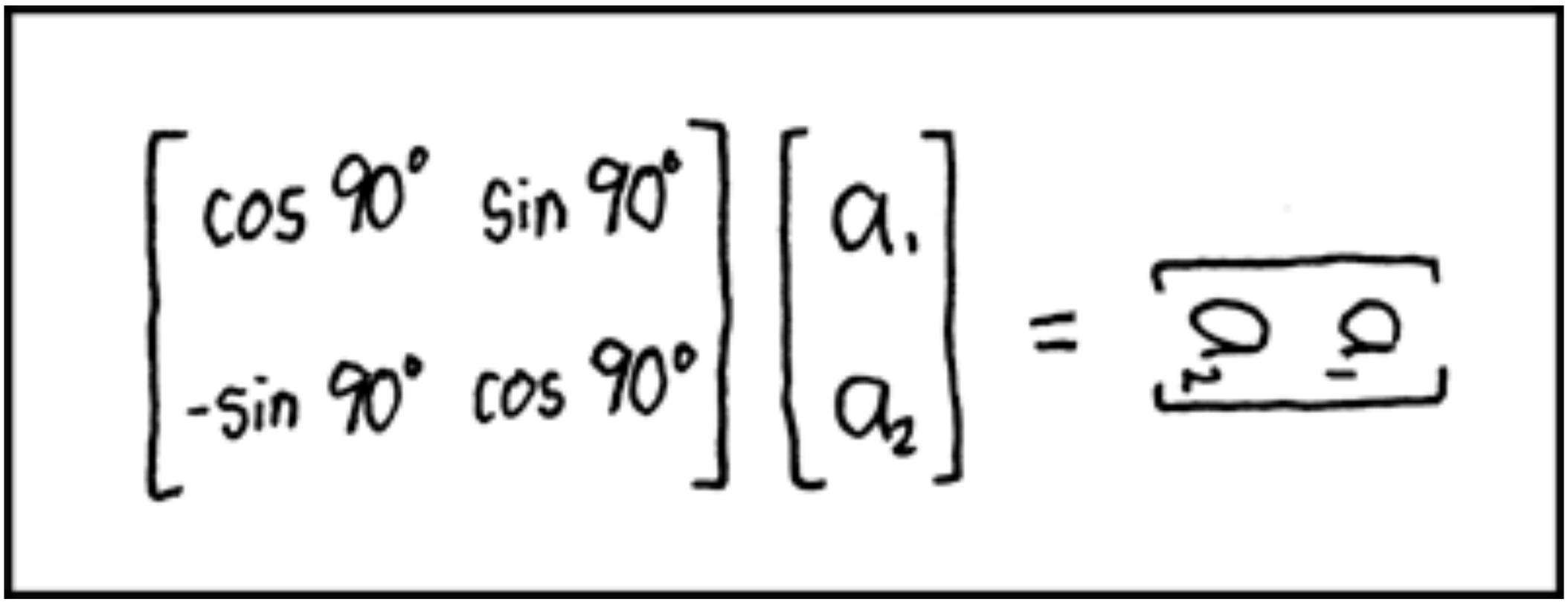
xkcd

A WEBCOMIC OF ROMANCE, SARCASM, MATH, AND LANGUAGE.



http://xkcd.com/184/

2D Rotation Matrix: Another Way



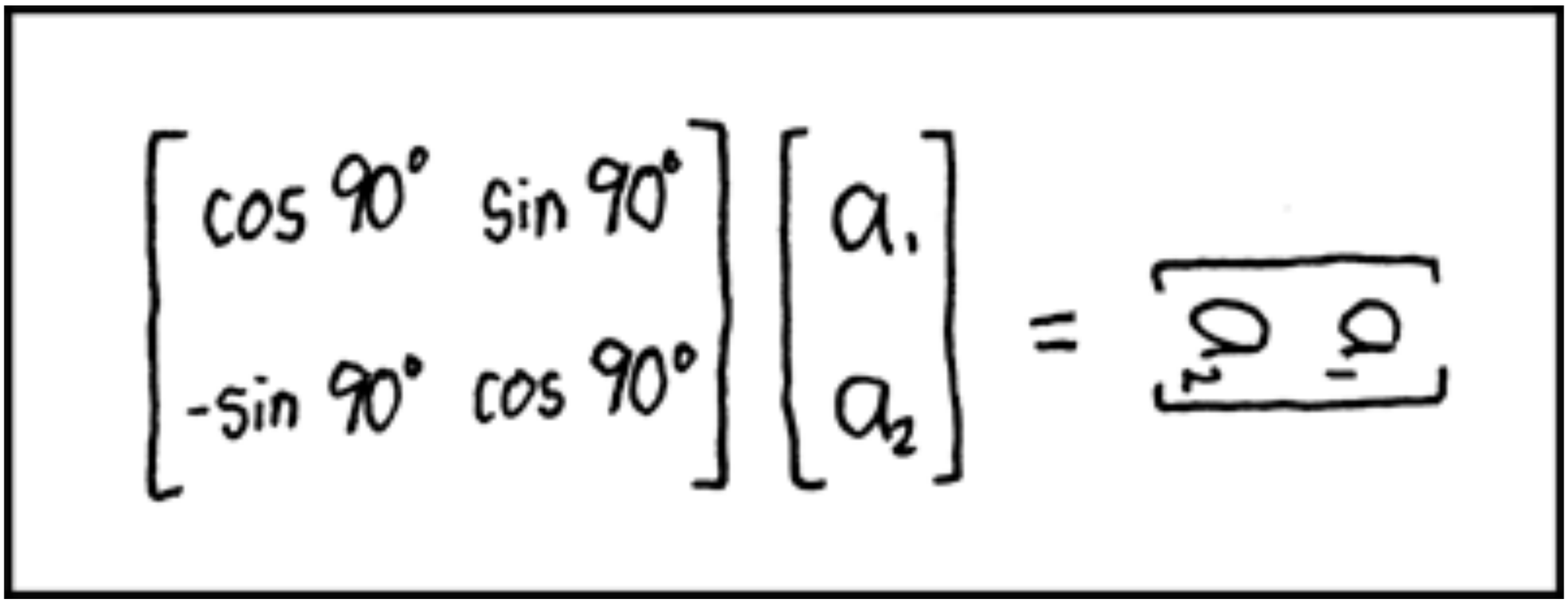
http://xkcd.com/184/

(What's wrong with this picture??)

2D Rotation Matrix: Another Way

xkcd

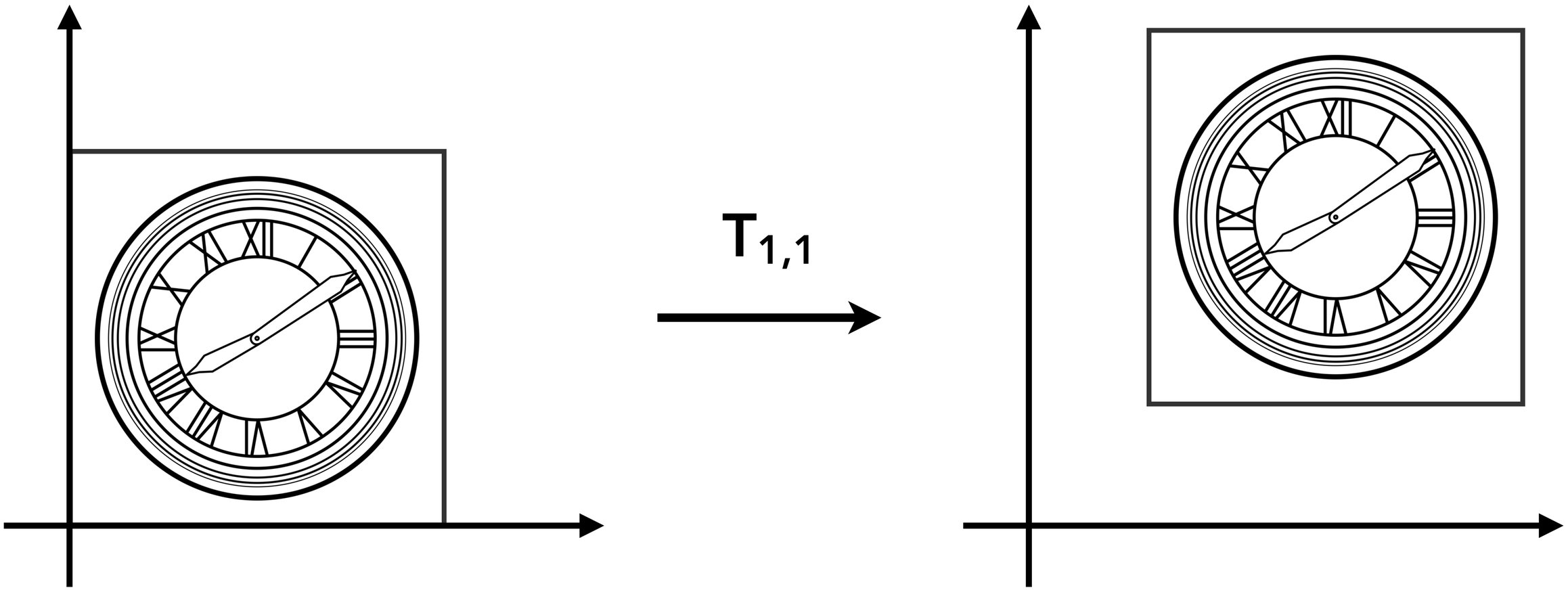
A WEBCOMIC OF ROMANCE, SARCASM, MATH, AND LANGUAGE.



http://xkcd.com/184/

(What's wrong with this picture??)
(Left handed matrix!)

Translation??



$$x' = x + t_x$$

$$y' = y + t_y$$

Solution: Homogenous Coordinates

Add a third coordinate (*w-coordinate*)

- 2D point = $(x, y, 1)^T$
- 2D vector = $(x, y, 0)^T$

Solution: Homogenous Coordinates

Add a third coordinate (*w-coordinate*)

- 2D point = $(x, y, 1)^T$
- 2D vector = $(x, y, 0)^T$

Now you can express translation as a matrix!!

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

Homogenous Coordinates

If w is not zero but not one either

- Divide entire vector by w
- Generally you can do it later

Homogenous Coordinates

If w is not zero but not one either

- Divide entire vector by w
- Generally you can do it later

Valid operation if w-coordinate of result is 1 or 0

Homogenous Coordinates

If w is not zero but not one either

- Divide entire vector by w
- Generally you can do it later

Valid operation if w-coordinate of result is 1 or 0

- **vector + vector = vector**

Homogenous Coordinates

If w is not zero but not one either

- Divide entire vector by w
- Generally you can do it later

Valid operation if w-coordinate of result is 1 or 0

- **vector + vector = vector**
- **point – point = vector**

Homogenous Coordinates

If w is not zero but not one either

- Divide entire vector by w
- Generally you can do it later

Valid operation if w-coordinate of result is 1 or 0

- **vector + vector = vector**
- **point – point = vector**
- **point + vector = point**

Homogenous Coordinates

If w is not zero but not one either

- Divide entire vector by w
- Generally you can do it later

Valid operation if w-coordinate of result is 1 or 0

- **vector + vector = vector**
- **point – point = vector**
- **point + vector = point**
- **point + point = ??**

Affine Transformations

Affine map = linear map + translation

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

Using homogenous coordinates:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

2D Transformations
(homogenous coordinates)

Scale

$$\mathbf{S}(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Translation

$$\mathbf{T}(t_x, t_y) = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

2D Transformations
(homogenous coordinates)

Scale

$$\mathbf{S}(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Translation

$$\mathbf{T}(t_x, t_y) = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

"Rigid Transform"

2D Transformations
(homogenous coordinates)

"Projective Transform"
(any matrix)

"Affine Transform"

"Similarity Transform if Sx=Sy"

"Rigid Transform"

Scale

$$\mathbf{S}(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Translation

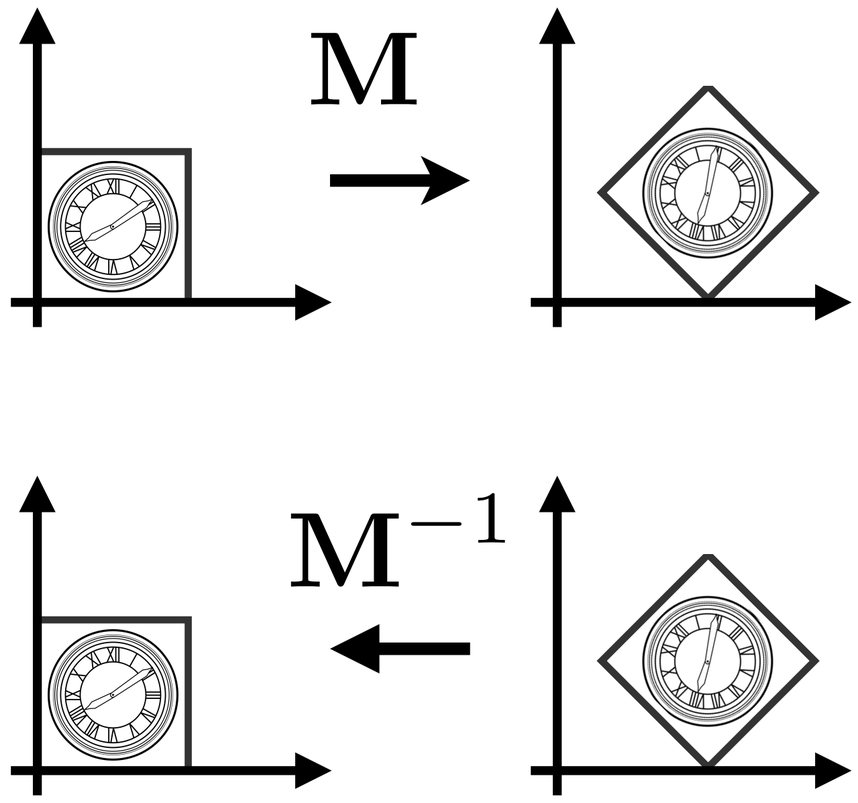
$$\mathbf{T}(t_x, t_y) = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

Inverse Transform

$$\mathbf{M}^{-1}$$

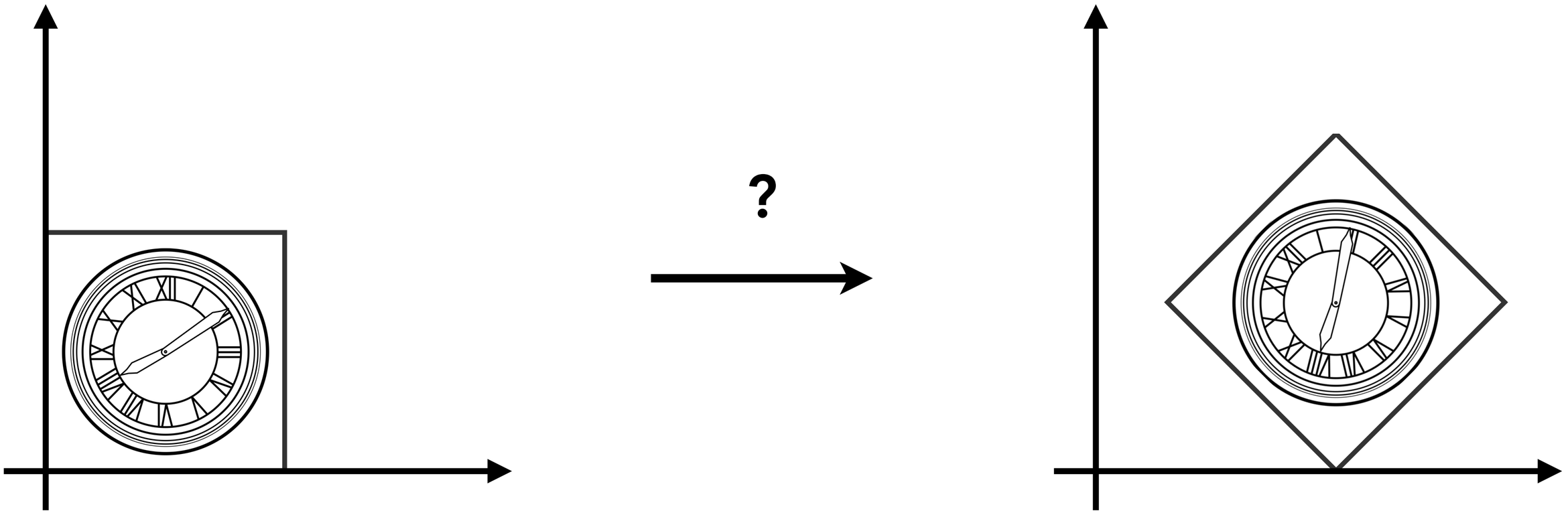
$\mathbf{M}^{-1}$ is the inverse of transform $\mathbf{M}$ in both a matrix and geometric sense

- Special cases:
  - Translation: negate t_x and t_y
  - Rotation: transpose
  - Scale: invert diagonal (axis-aligned scales)
- Others:
  - Invert matrix
  - Invert SVD matrices

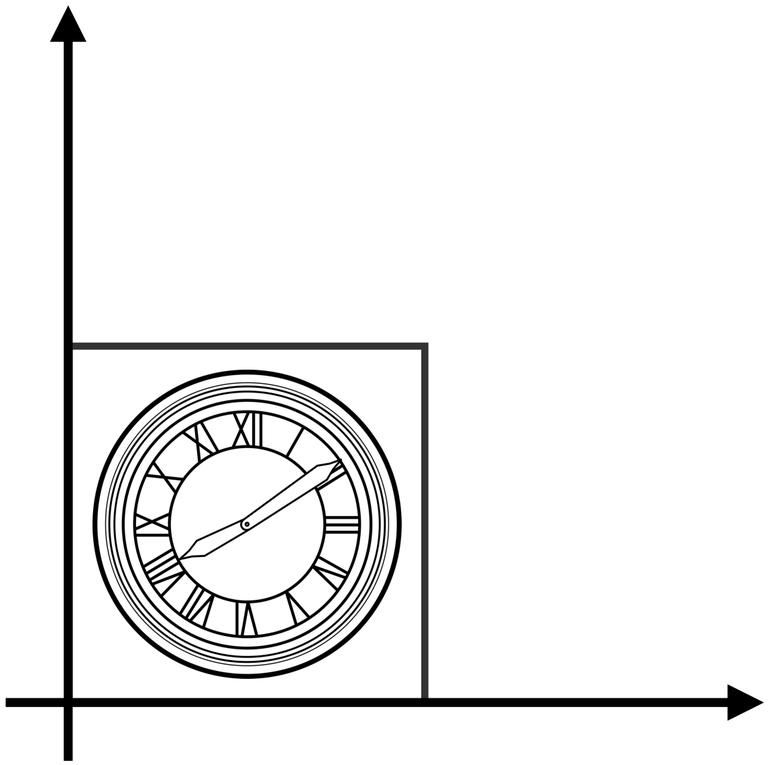


Composing Transforms

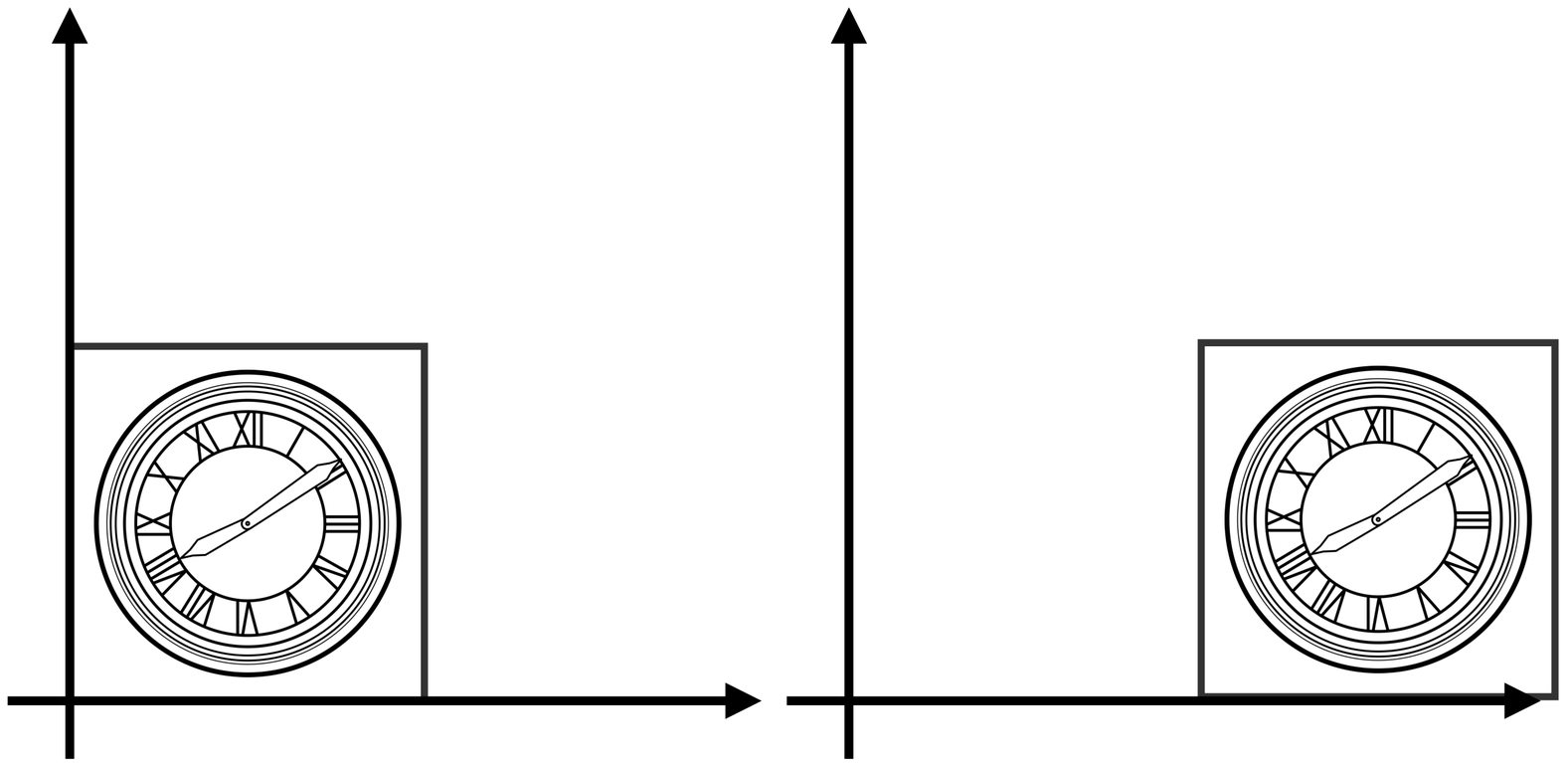
Composite Transform



Translate Then Rotate?

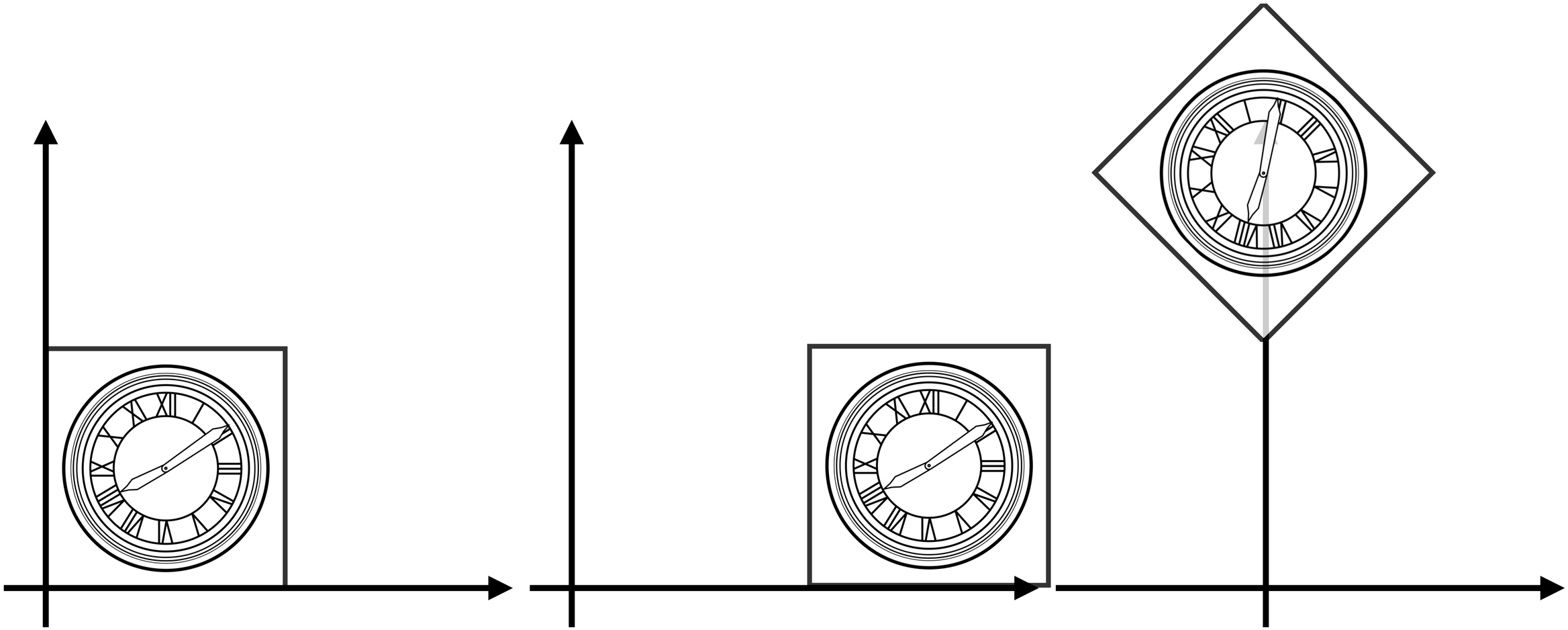


Translate Then Rotate?



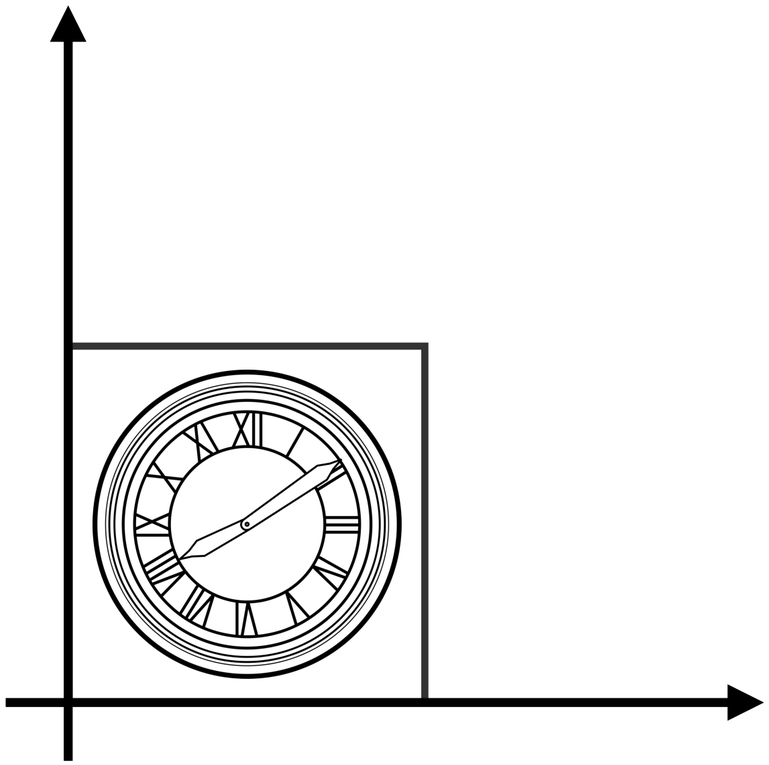
$$T_{(1,0)}$$

Translate Then Rotate?

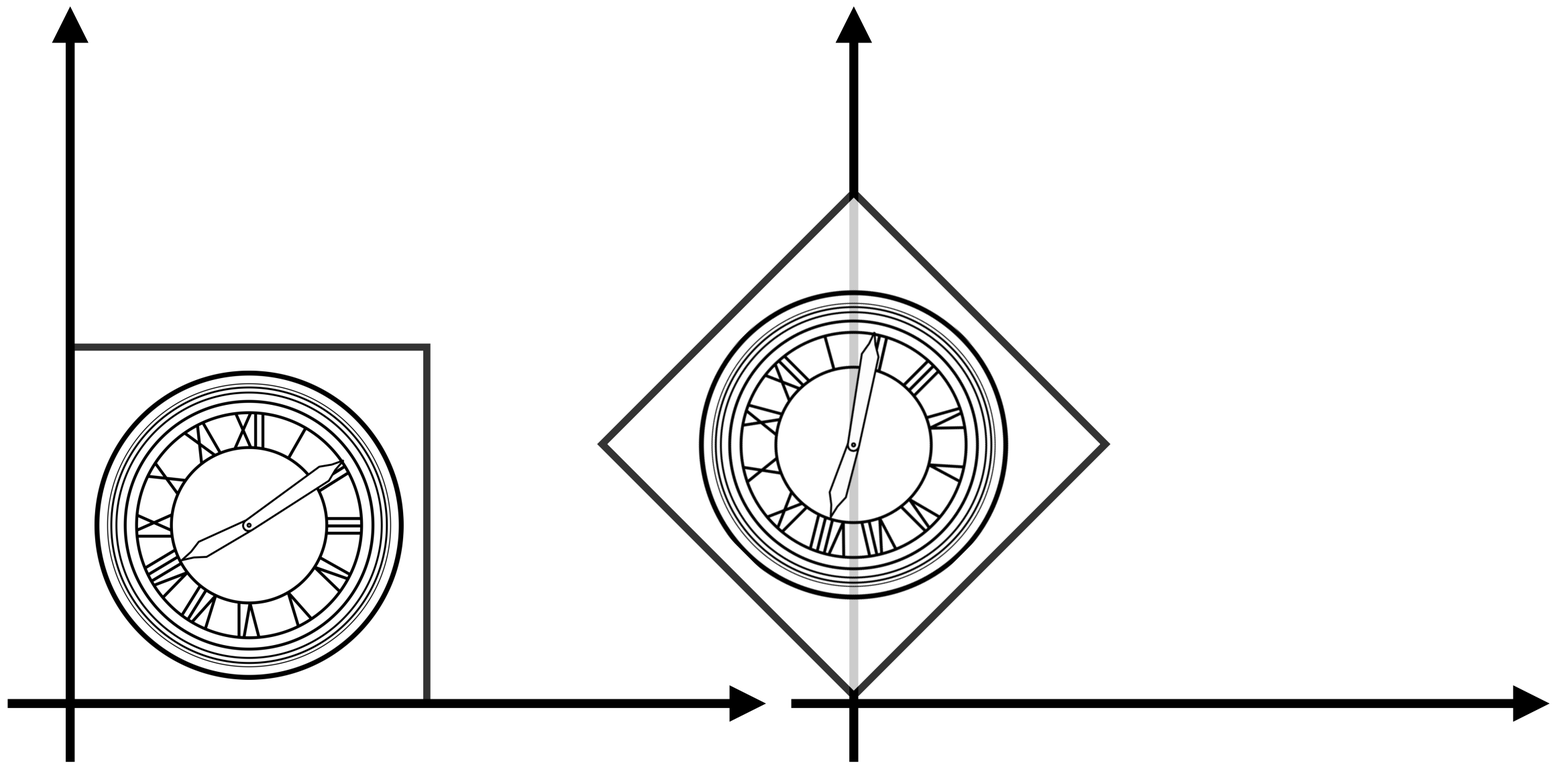


$T_{(1,0)}$ $R_{45} \cdot T_{(1,0)}$

Rotate Then Translate

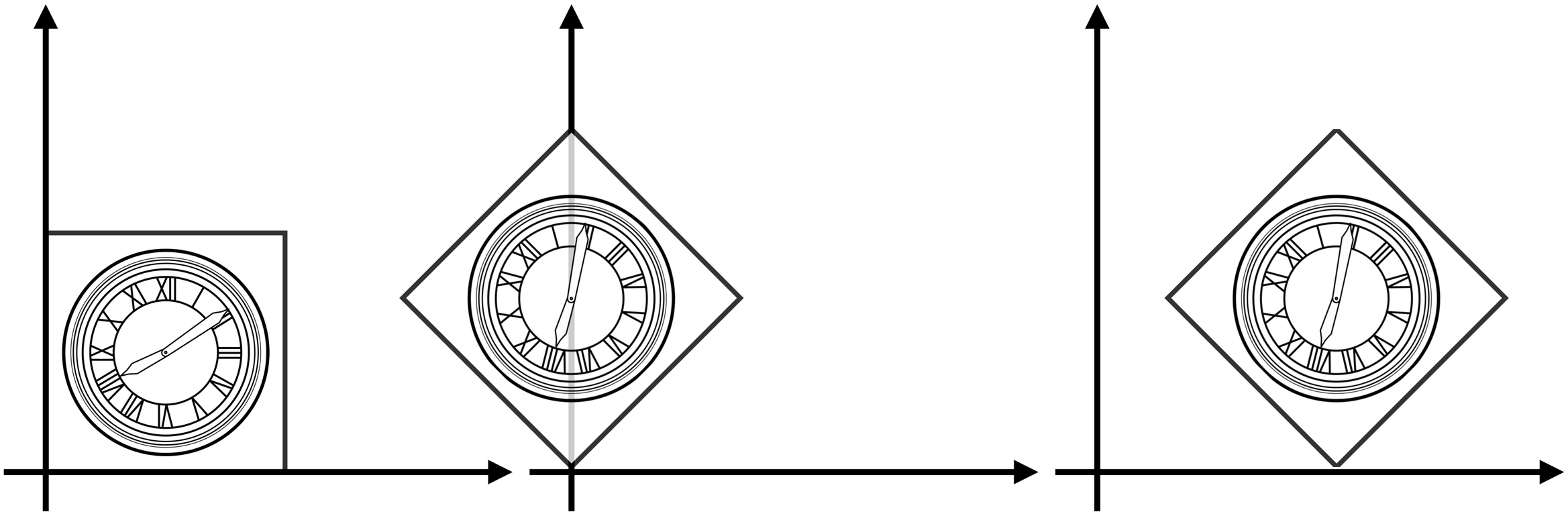


Rotate Then Translate



R_{45}

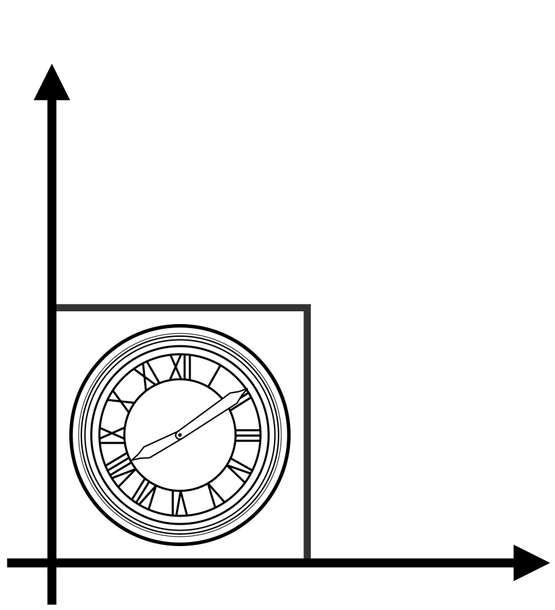
Rotate Then Translate

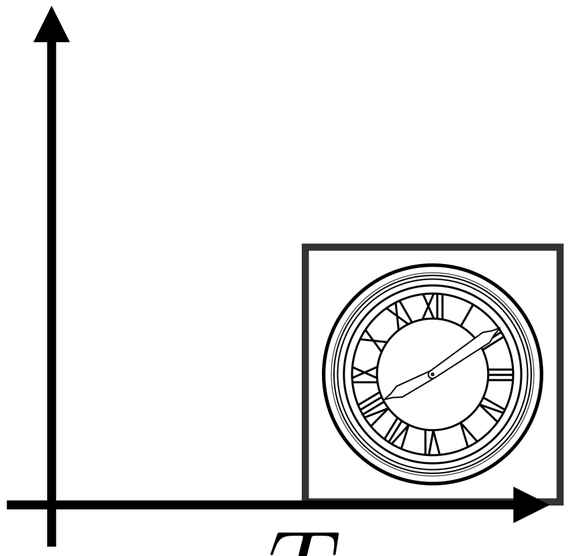


R_{45}

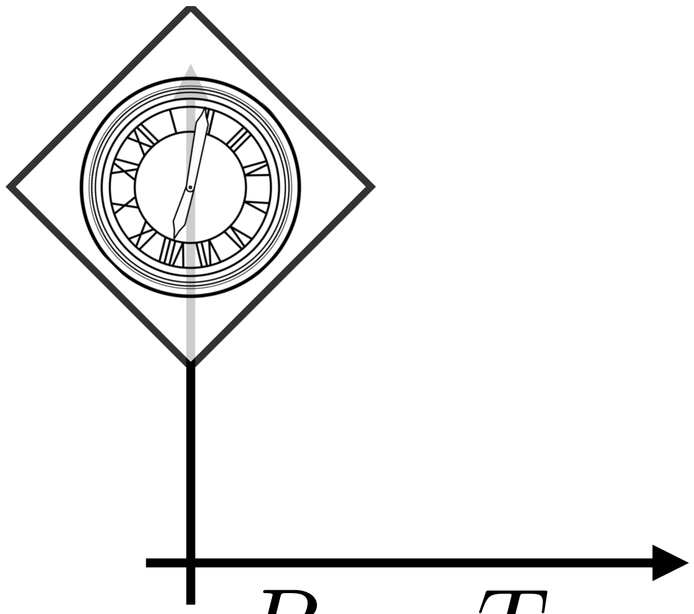
$T_{(1,0)} \cdot R_{45}$

Transform Ordering Matters!

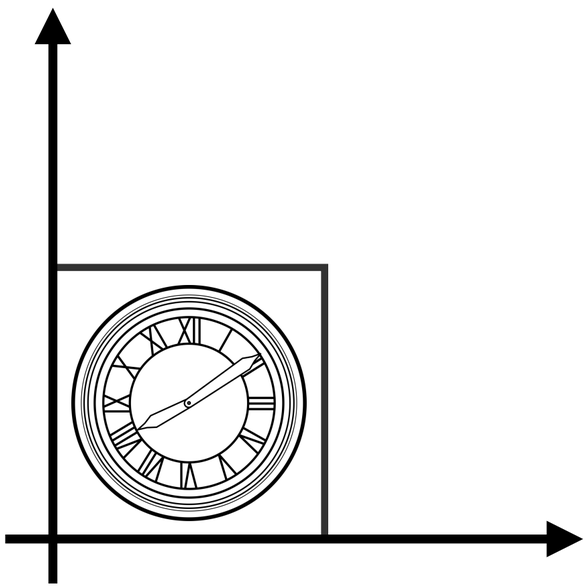


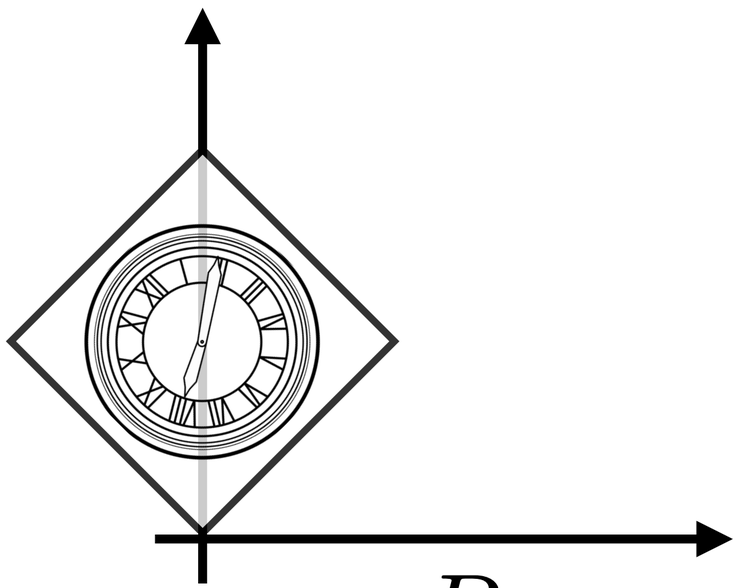


$T_{(1,0)}$

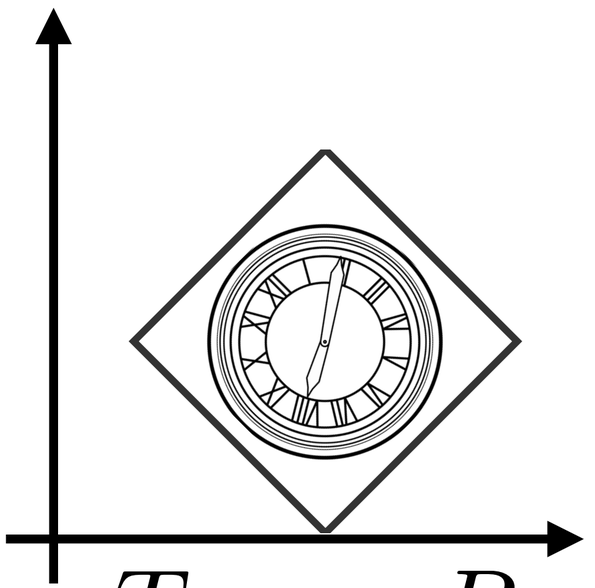


$R_{45} \cdot T_{(1,0)}$





R_{45}



$T_{(1,0)} \cdot R_{45}$

Transform Ordering Matters!

Matrix multiplication is <u>not</u> commutative

$$R_{45} \cdot T_{(1,0)} \neq T_{(1,0)} \cdot R_{45}$$

Transform Ordering Matters!

Matrix multiplication is <u>not</u> commutative

$$R_{45} \cdot T_{(1,0)} \neq T_{(1,0)} \cdot R_{45}$$

Recall the matrix math represented by these symbols:

$$\begin{bmatrix} \cos 45° & -\sin 45° & 0 \\ \sin 45° & \cos 45° & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \neq \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 45° & -\sin 45° & 0 \\ \sin 45° & \cos 45° & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Transform Ordering Matters!

Matrix multiplication is <u>not</u> commutative

$$R_{45} \cdot T_{(1,0)} \neq T_{(1,0)} \cdot R_{45}$$

Recall the matrix math represented by these symbols:

$$\begin{bmatrix} \cos 45° & -\sin 45° & 0 \\ \sin 45° & \cos 45° & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \neq \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 45° & -\sin 45° & 0 \\ \sin 45° & \cos 45° & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Note that matrices are applied right to left:

$$T_{(1,0)} \cdot R_{45} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos 45° & -\sin 45° & 0 \\ \sin 45° & \cos 45° & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Composing Transforms

Sequence of transforms A_1, A_2, A_3, ...

- Compose by matrix multiplication
  - Very important for performance!
  - Very important for complexity!

Composing Transforms

Sequence of transforms A_1, A_2, A_3, ...

- Compose by matrix multiplication
  - Very important for performance!
  - Very important for complexity!

$$A_n(\ldots A_2(A_1(\mathbf{x}))) \;=\; \mathbf{A}_n \cdots \mathbf{A}_2 \cdot \mathbf{A}_1 \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Composing Transforms

Sequence of transforms A_1, A_2, A_3, ...

- Compose by matrix multiplication
  - Very important for performance!
  - Very important for complexity!

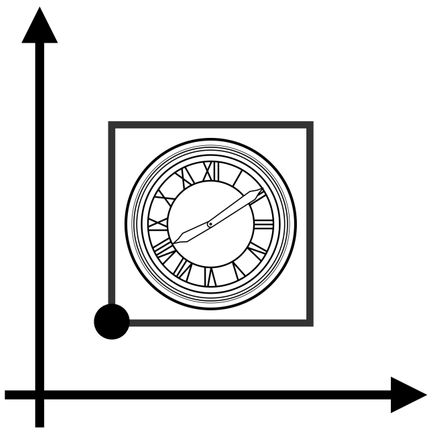
$$A_n(\ldots A_2(A_1(\mathbf{x}))) \;=\; \mathbf{A}_n \cdots \mathbf{A}_2 \cdot \mathbf{A}_1 \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Pre-multiply *n* matrices to obtain a single matrix representing combined transform

Building Complex Transforms

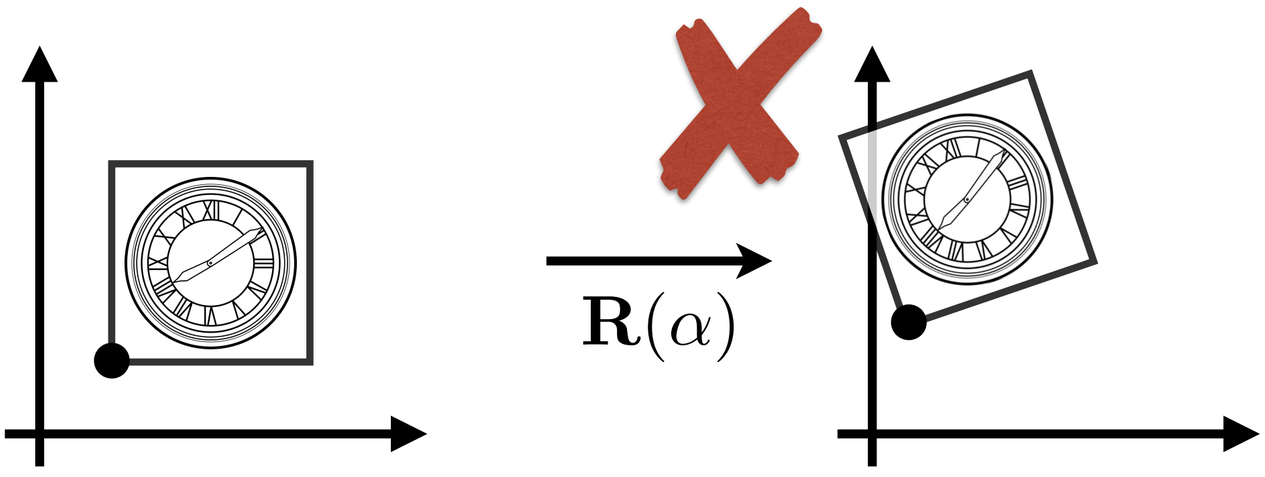
Building Complex Transforms

How to rotate around a given point c?



Building Complex Transforms

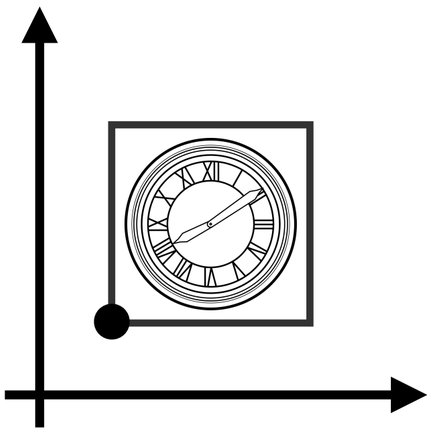
How to rotate around a given point c?



Building Complex Transforms

Building Complex Transforms

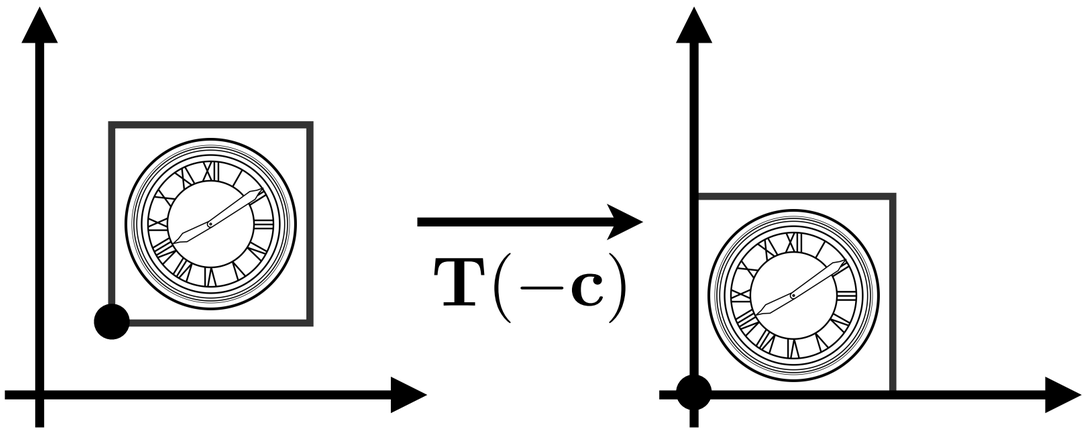
How to rotate around a given point c?



Building Complex Transforms

How to rotate around a given point c?

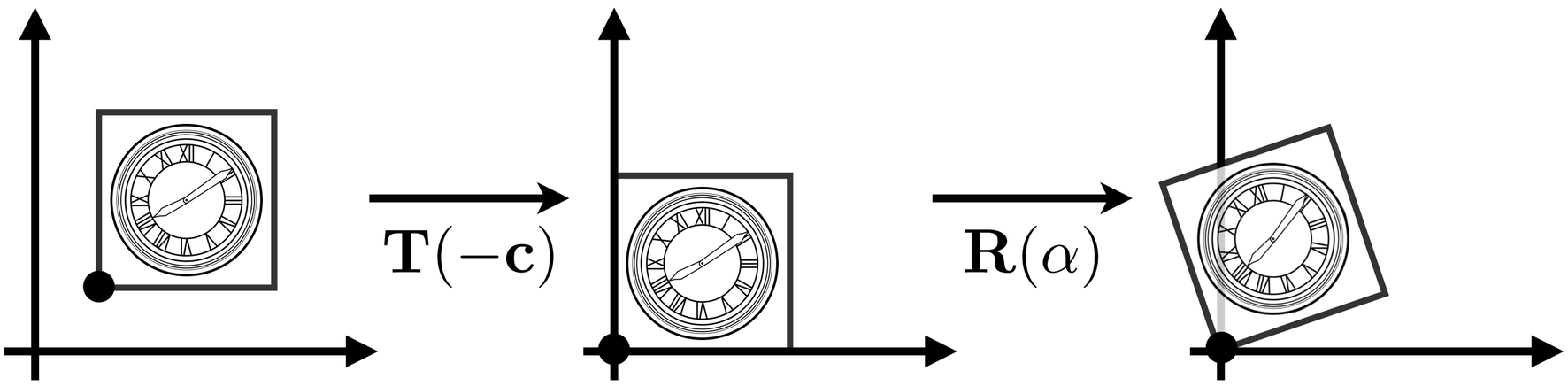
1. Translate center to origin



Building Complex Transforms

How to rotate around a given point c?

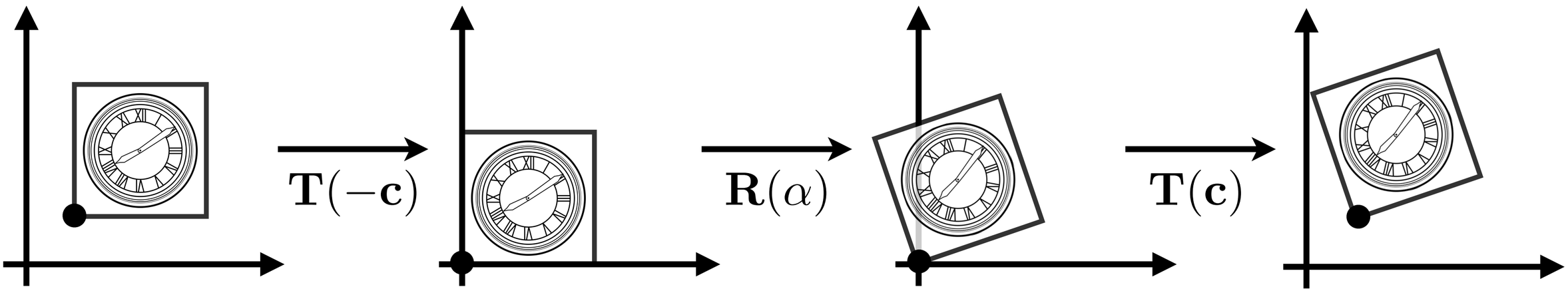
1. Translate center to origin
2. Rotate



Building Complex Transforms

How to rotate around a given point c?

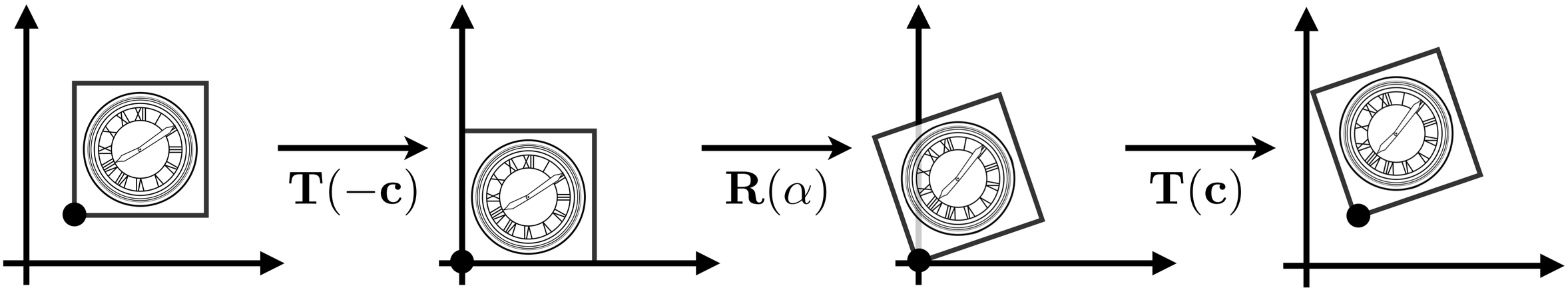
1. Translate center to origin
2. Rotate
3. Translate back



Building Complex Transforms

How to rotate around a given point c?

1. Translate center to origin
2. Rotate
3. Translate back



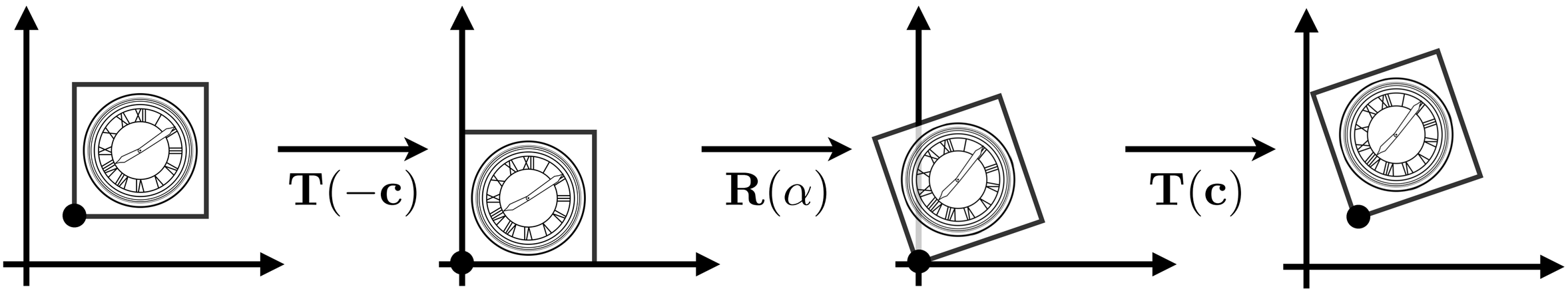
Matrix representation

$$\mathbf{T}(\mathbf{c}) \cdot \mathbf{R}(\alpha) \cdot \mathbf{T}(-\mathbf{c})$$

Building Complex Transforms

How to rotate around a given point c?

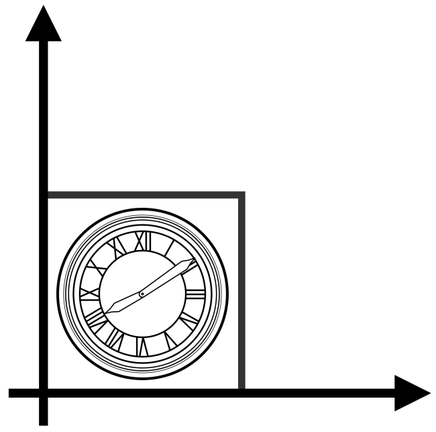
1. Translate center to origin
2. Rotate
3. Translate back

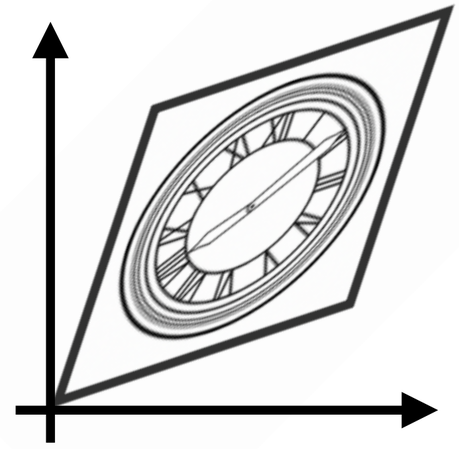


Matrix representation

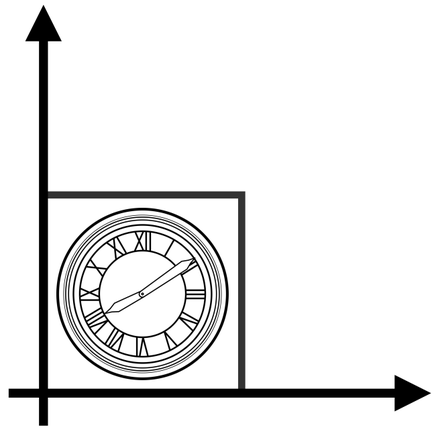
$$\mathbf{M}_{\text{composite}} = \quad \mathbf{T}(\mathbf{c}) \cdot \mathbf{R}(\alpha) \cdot \mathbf{T}(-\mathbf{c})$$

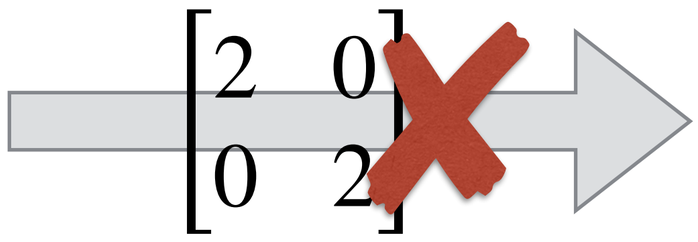
Building Not-Axis-Aligned Scale Matrix

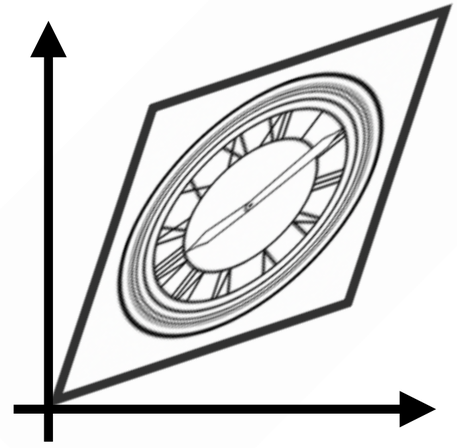




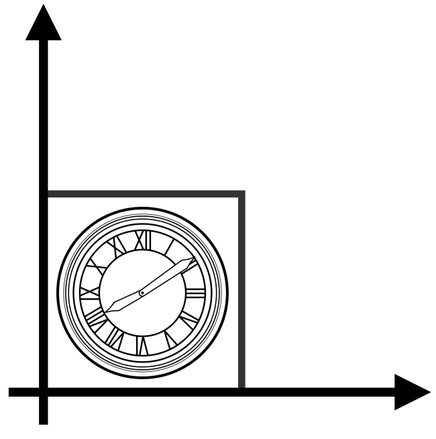
Building Not-Axis-Aligned Scale Matrix

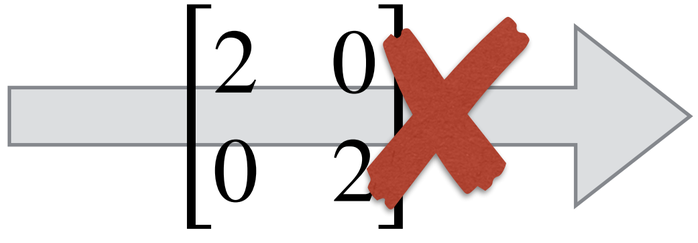


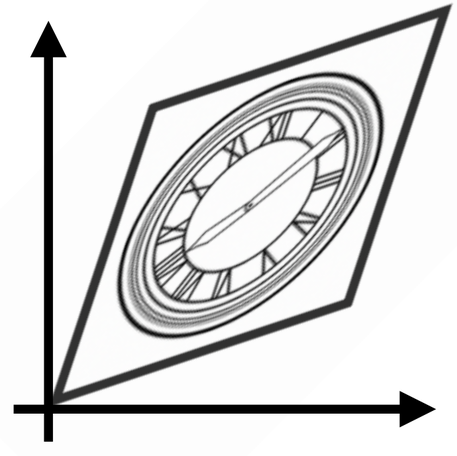


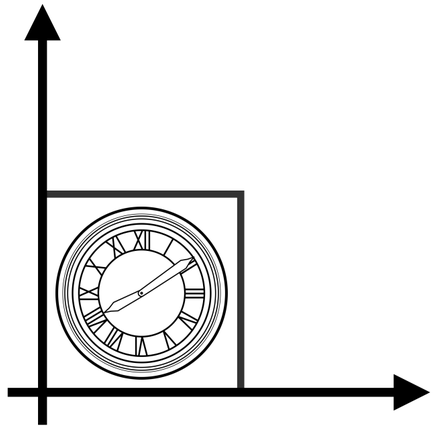


Building Not-Axis-Aligned Scale Matrix

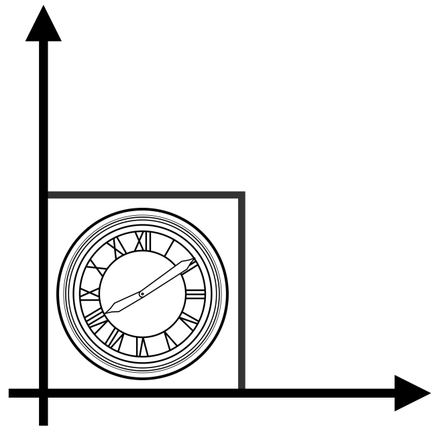


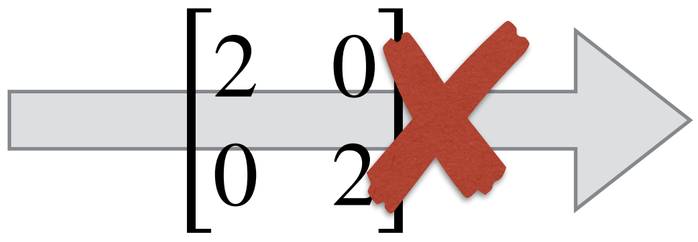


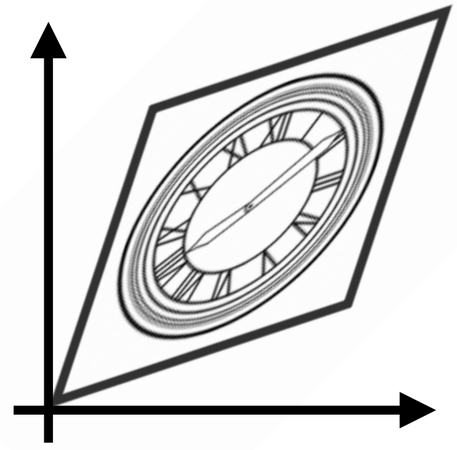


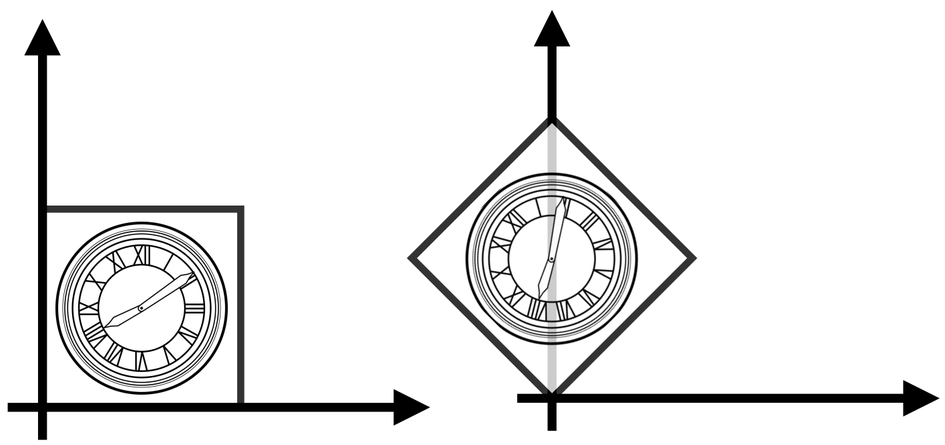


Building Not-Axis-Aligned Scale Matrix

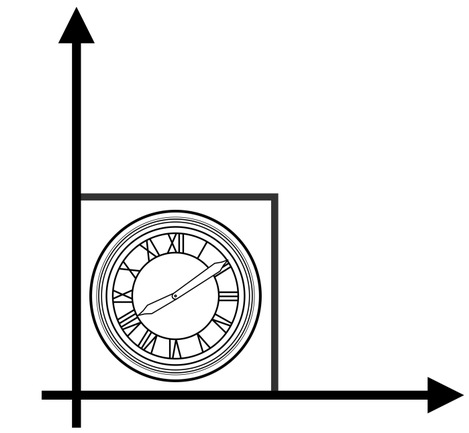




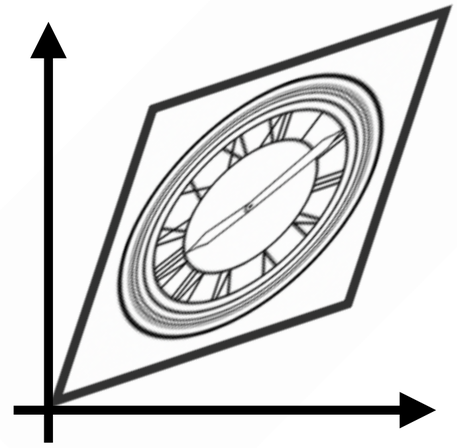


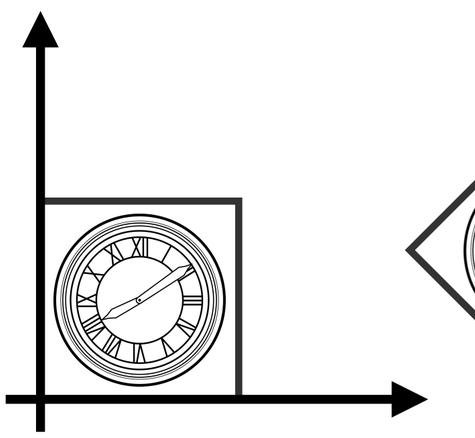


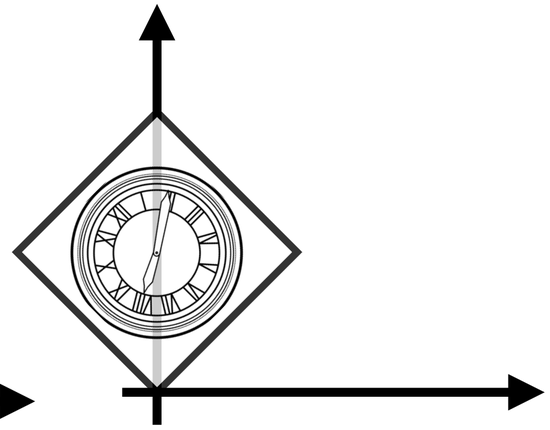
Building Not-Axis-Aligned Scale Matrix

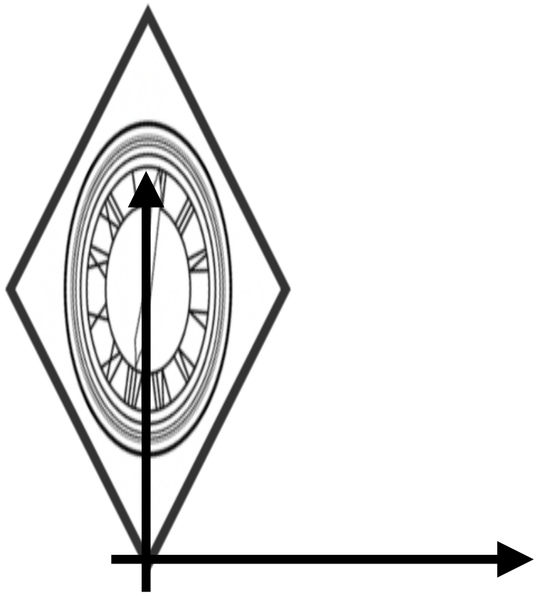


$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

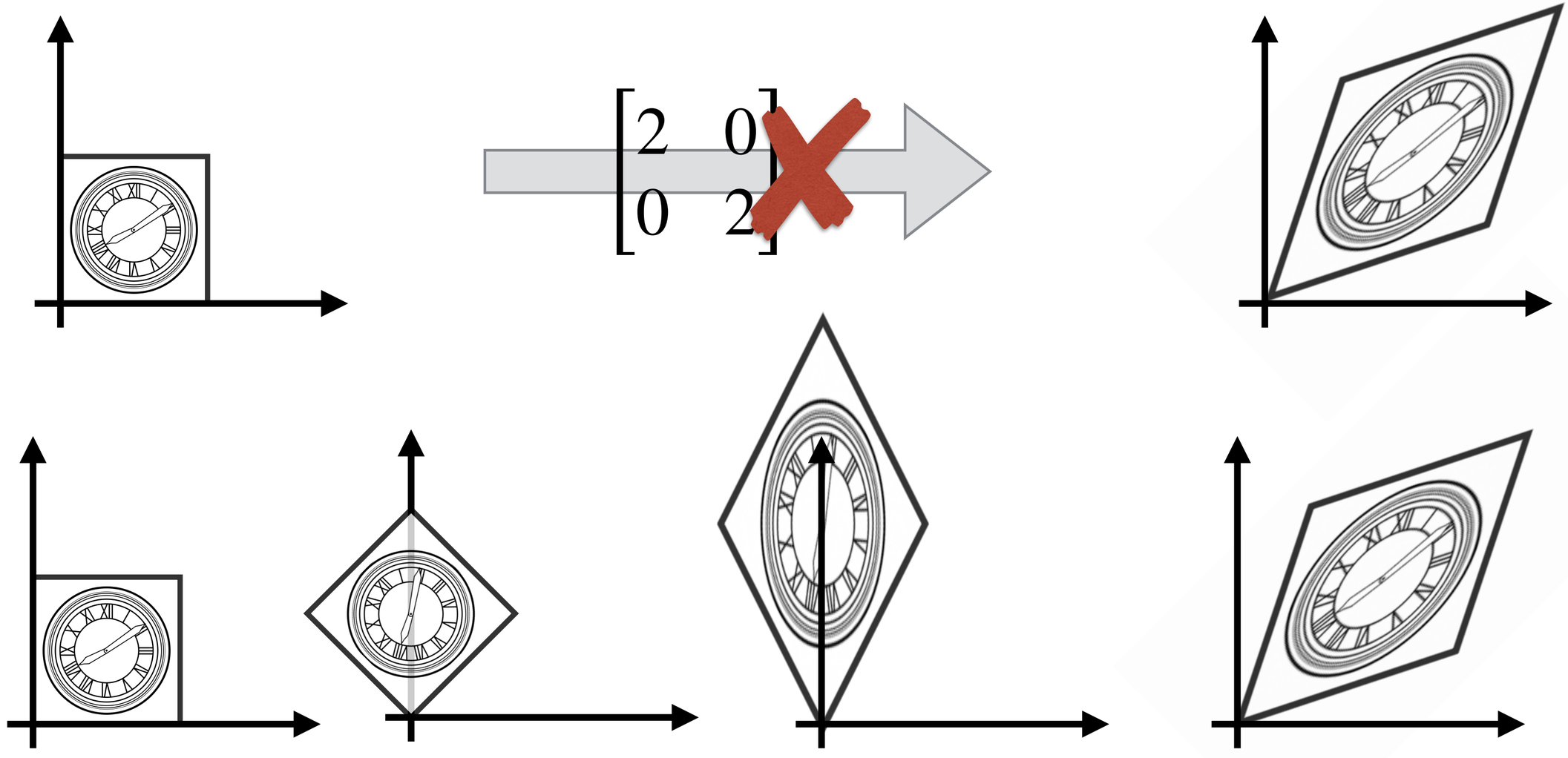




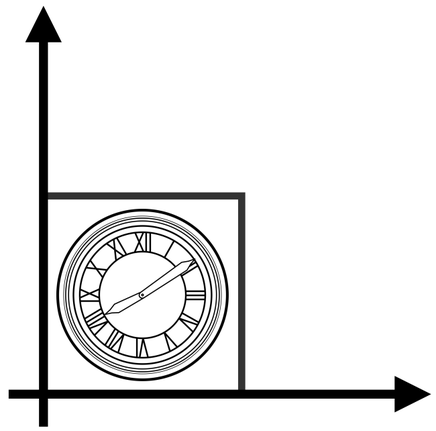




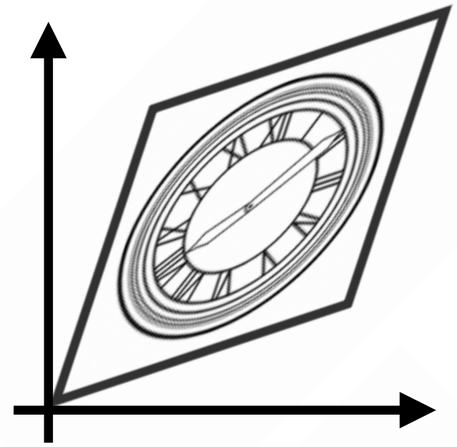
Building Not-Axis-Aligned Scale Matrix

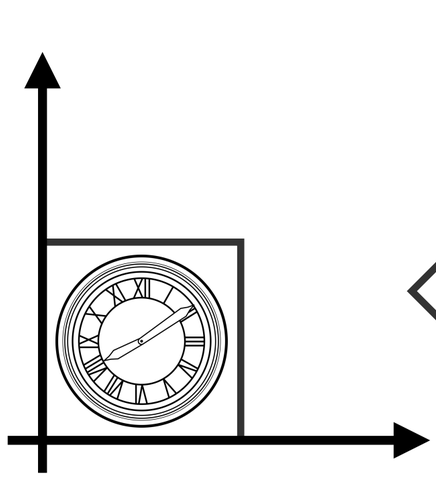


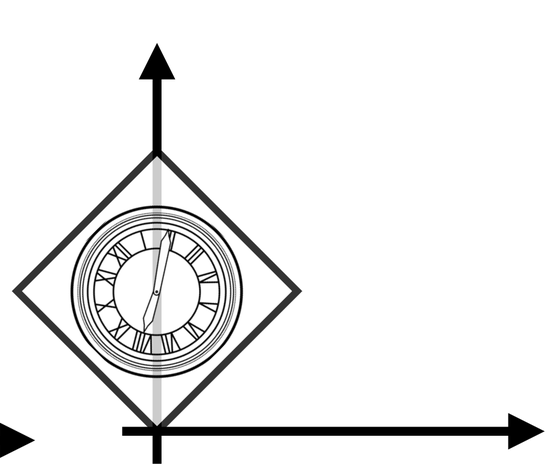
Building Not-Axis-Aligned Scale Matrix

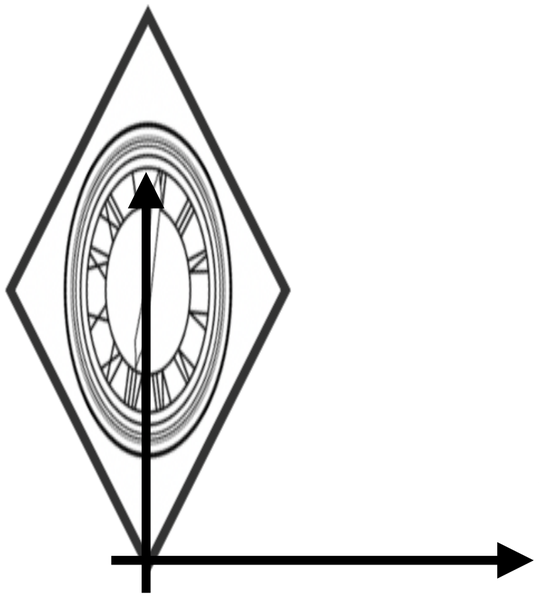


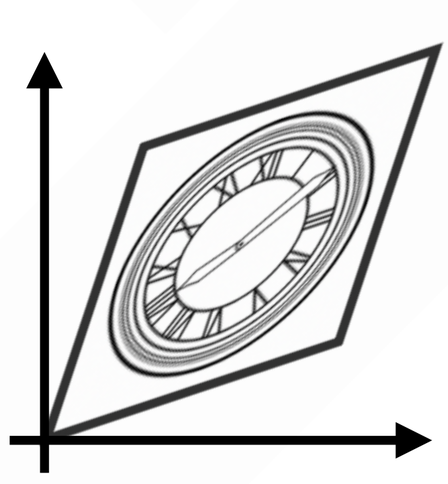
$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$







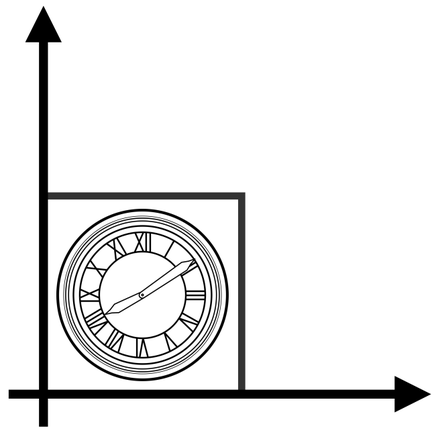




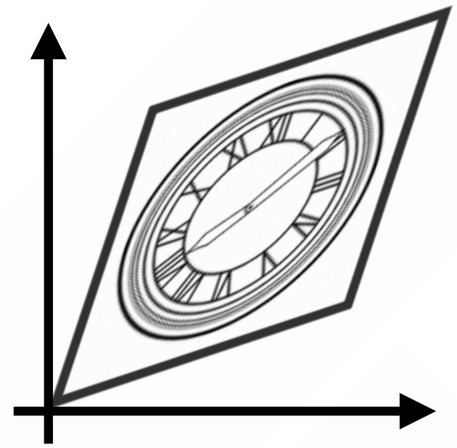
$$\begin{bmatrix} \cos(45) & -\sin(45) \\ \sin(45) & \cos(45) \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \cdot \begin{bmatrix} \cos(-45) & -\sin(-45) \\ \sin(-45) & \cos(-45) \end{bmatrix} = \begin{bmatrix} \frac{3}{2} & -\frac{1}{2} \\ -\frac{1}{2} & \frac{3}{2} \end{bmatrix}$$

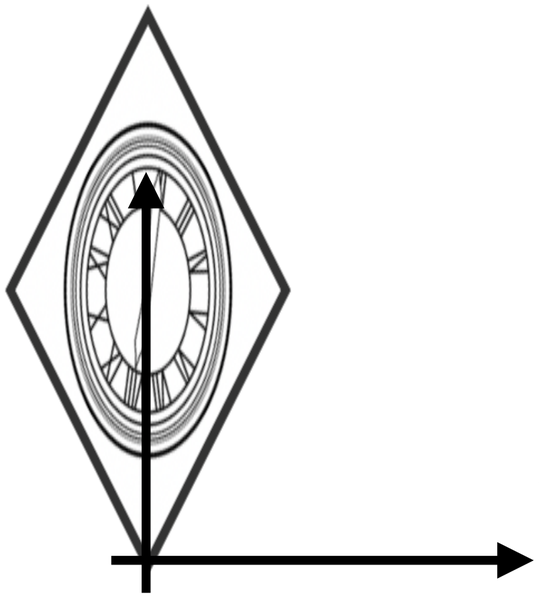
R(45) **s(1,2)** **R(-45)**

Building Not-Axis-Aligned Scale Matrix



$$\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$





$$\begin{bmatrix} \cos(45) & -\sin(45) \\ \sin(45) & \cos(45) \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \cdot \begin{bmatrix} \cos(-45) & -\sin(-45) \\ \sin(-45) & \cos(-45) \end{bmatrix} = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 1.5 \end{bmatrix}$$

R(45) **s(1,2)** **R(-45)**

Useful Matrix Decompositions

Eigen system
Lambda is diagonal

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$$

Singular Value Decomposition (SVD)
S is diagonal
Q and R are orthonormal

$$\mathbf{A} = \mathbf{Q}\mathbf{S}\mathbf{R}^{\mathsf{T}}$$

Polar Decomposition
S is diagonal
P, W and R are orthonormal
$\mathbf{PR} = \mathbf{Q}$

$$\mathbf{A} = \mathbf{P}\mathbf{R}\mathbf{S}\mathbf{R}^{\mathsf{T}}$$

Note: V=Q=R if A symmetric and real

Useful Matrix Decompositions

Eigen system
Lambda is diagonal

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$$

Singular Value Decomposition (SVD)
S is diagonal
Q and R are orthonormal

$$\mathbf{A} = \mathbf{Q}\mathbf{S}\mathbf{R}^{\mathsf{T}}$$

Polar Decomposition
S is diagonal
P, W and R are orthonormal

$$\mathbf{PR} = \mathbf{Q}$$

$$\mathbf{A} = \mathbf{P}\mathbf{R}\mathbf{S}\mathbf{R}^{\mathsf{T}}$$

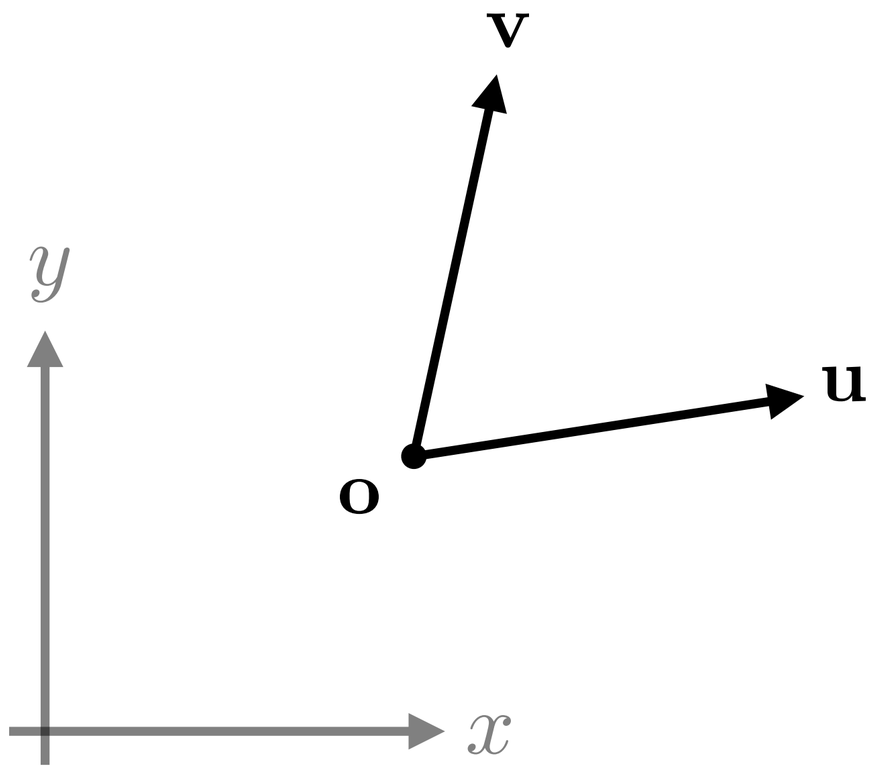
Note: V=Q=R if A symmetric and real

Rotation Matrix ≅ Orthonormal (det+1)
Axis-aligned Scale Matrix ≅ Diagonal
Scale Matrix ≅ OrthonormalTranspose . Diagonal . Orthonormal

Coordinate Systems

Coordinate System Transform

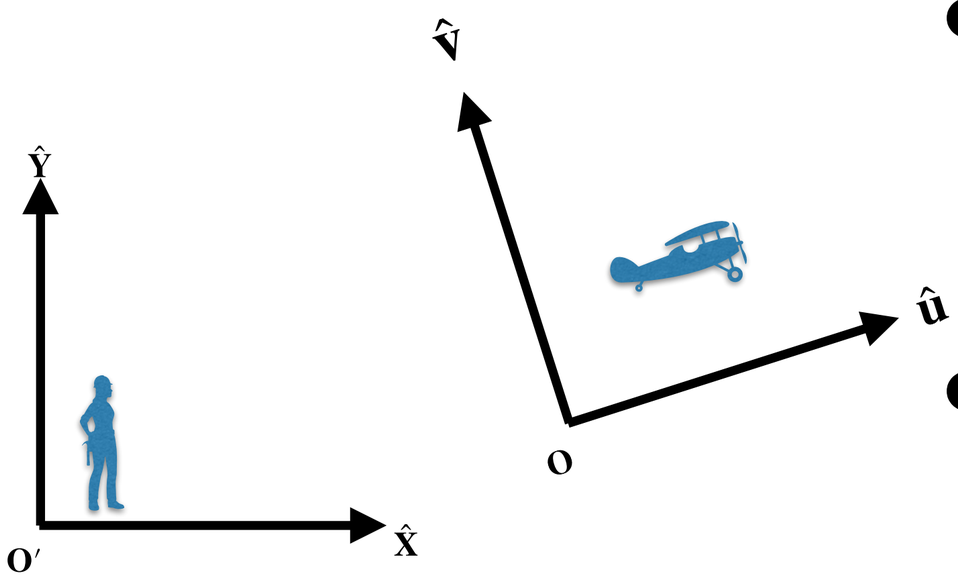
In general, a new coordinate frame is defined by an origin (point) and two unit axes (vectors)



Given coordinates in the (o,u,v) reference frame, what is the transform to coordinates in the (x,y) frame?

Coordinate System Transform Matrix

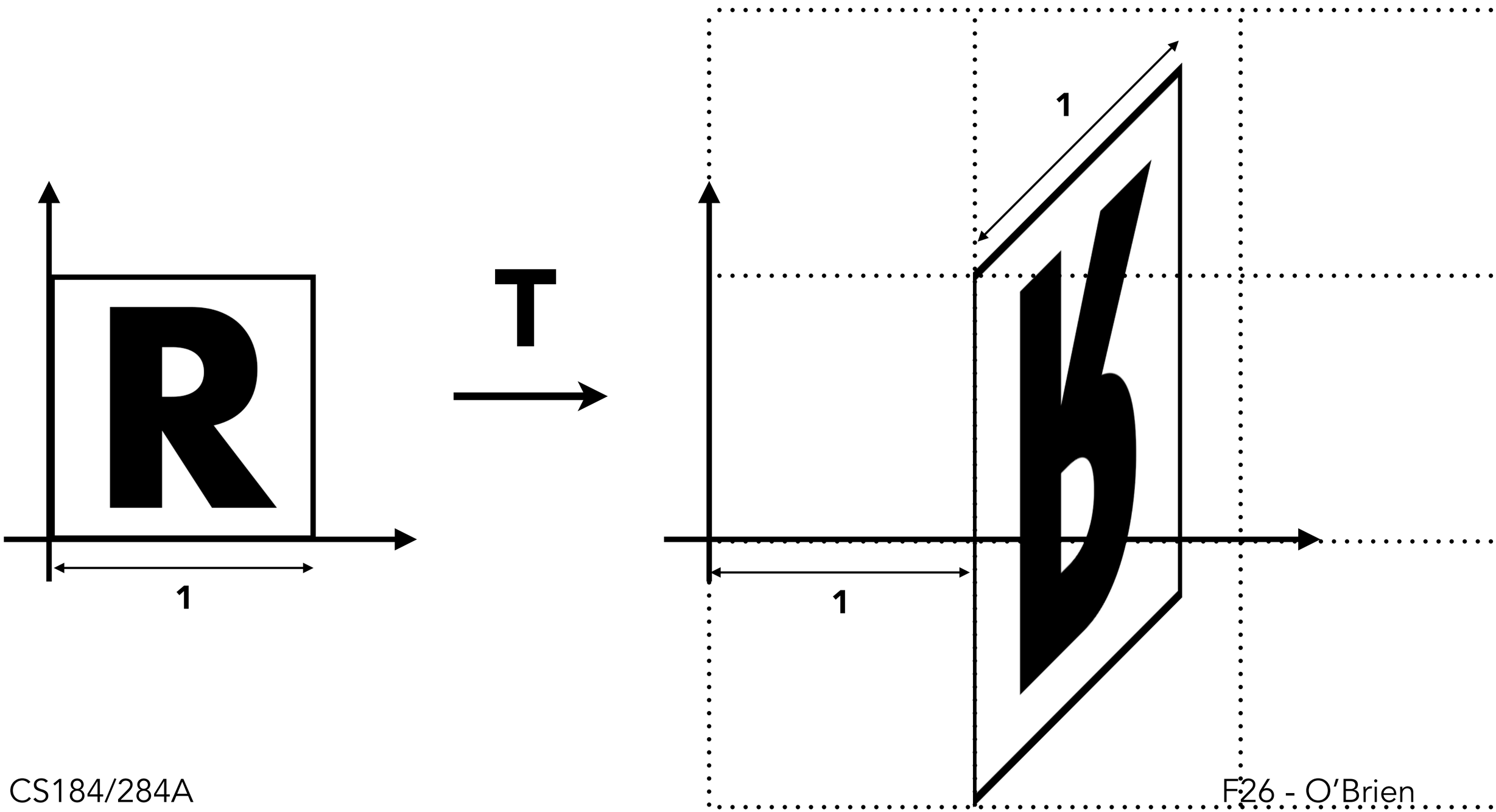
$$F = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{o} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} u_x & v_x & o_x \\ u_y & v_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$



- Columns of matrix are defined by the reference frame's coordinates in the world
- Gives a new way to "read off" columns of matrix

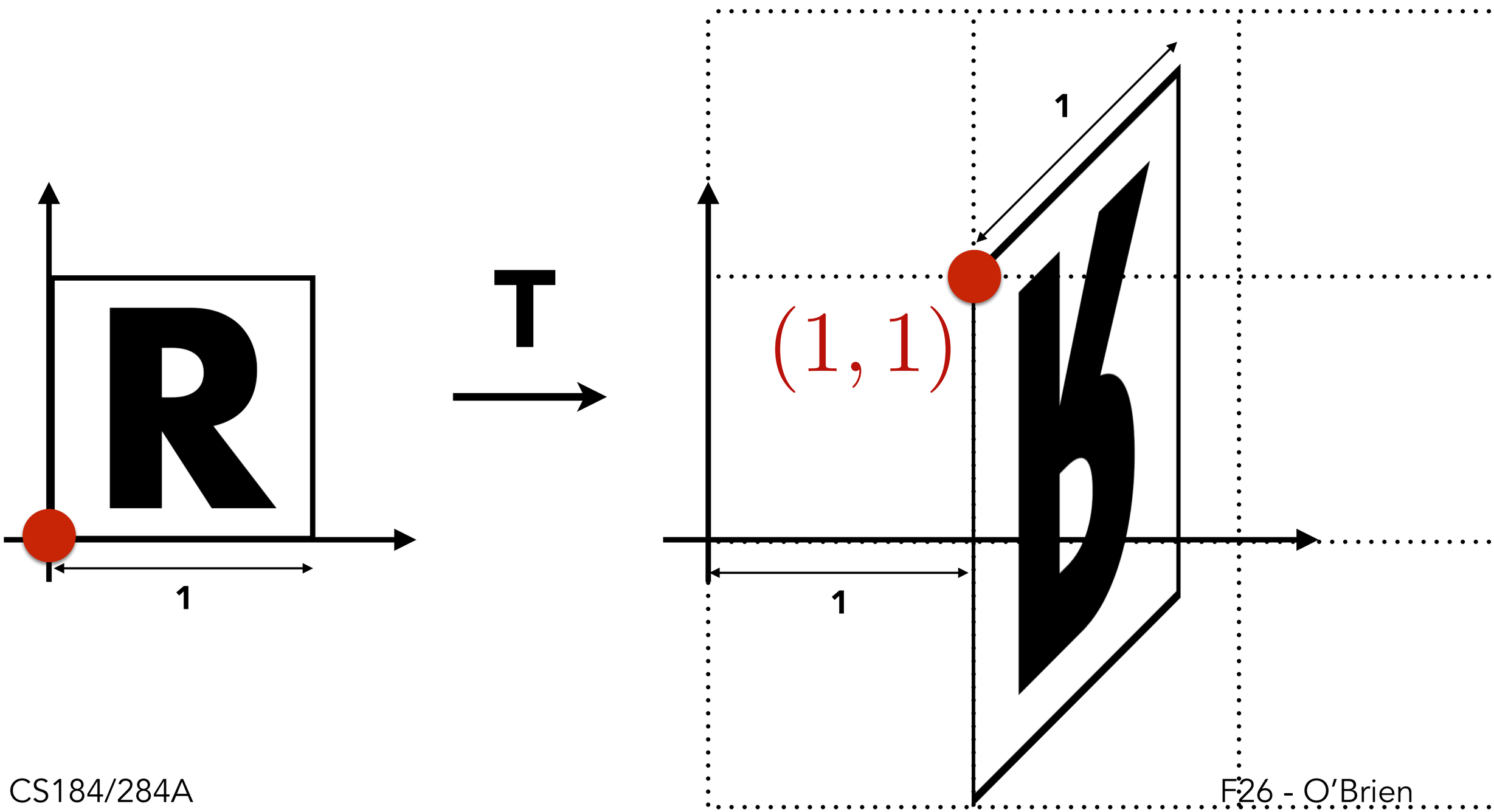
Coordinate System Transform - Example

Write down a matrix T representing this transform:



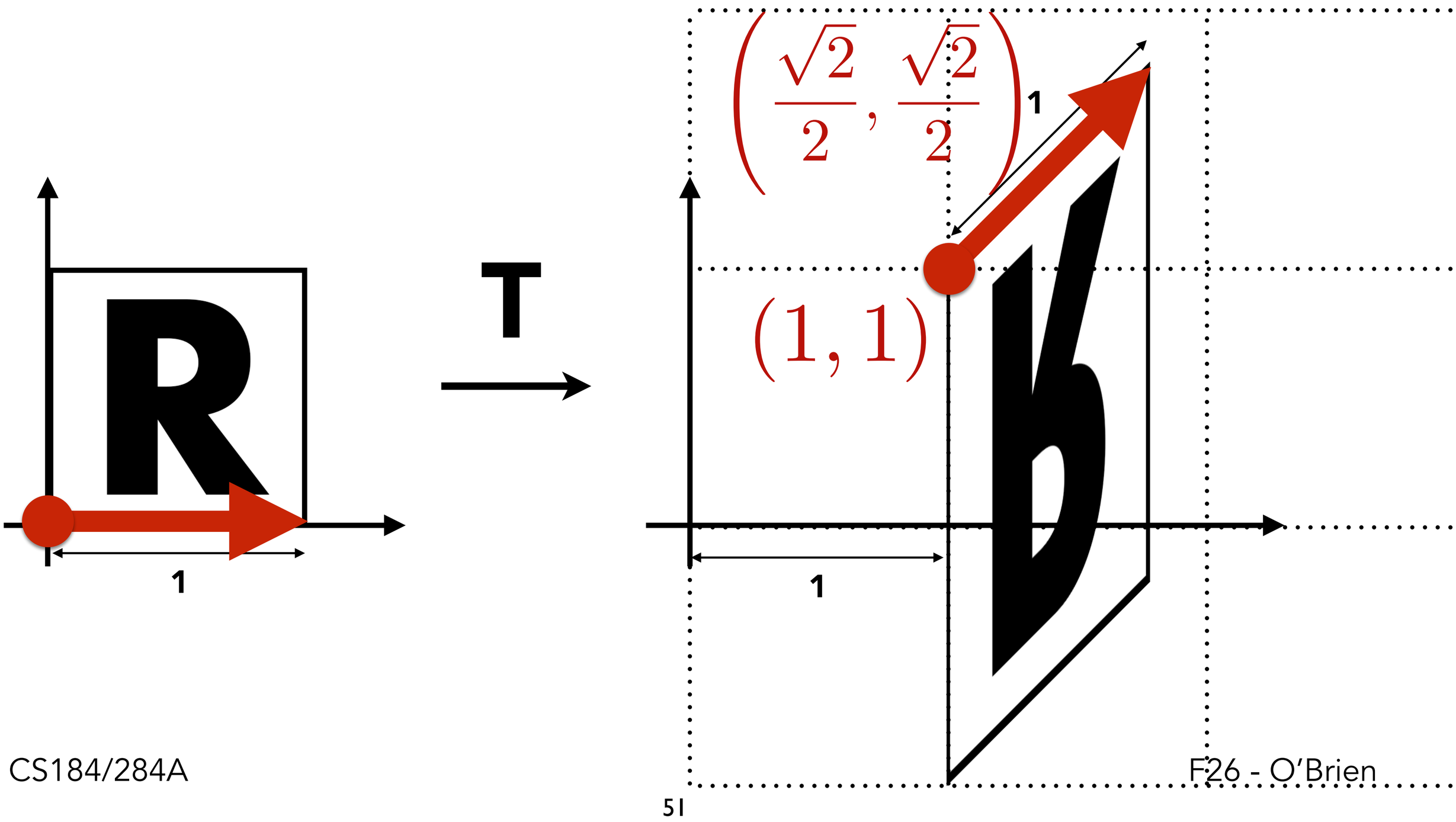
Coordinate System Transform - Example

Write down a matrix T representing this transform:



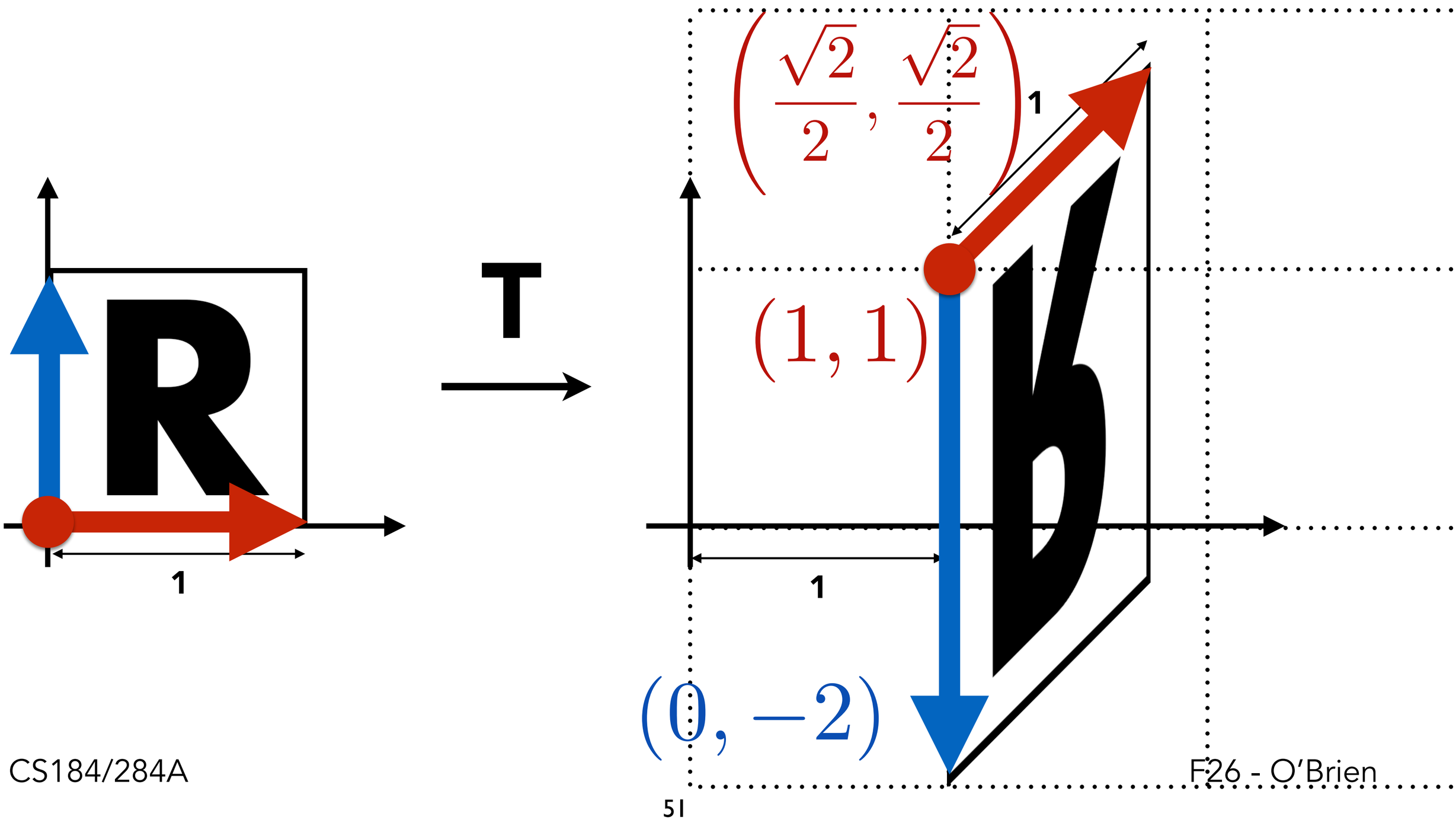
Coordinate System Transform - Example

Write down a matrix T representing this transform:



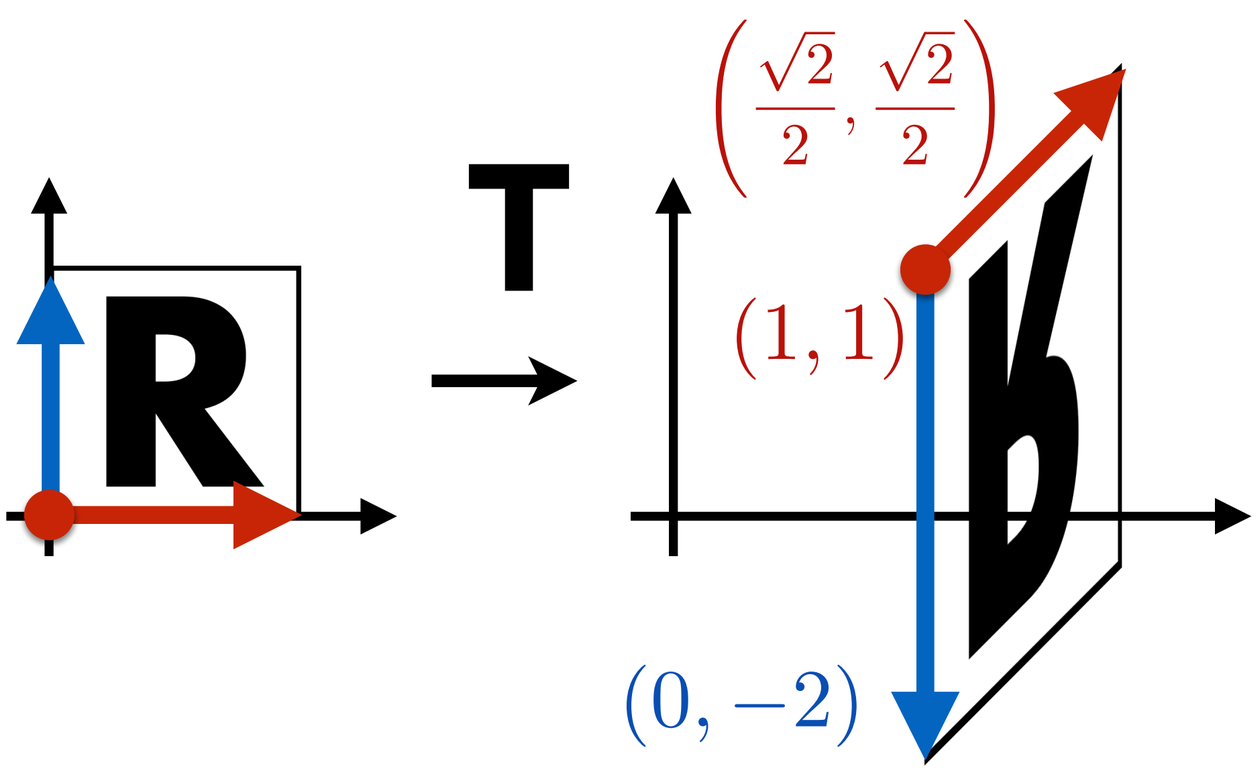
Coordinate System Transform - Example

Write down a matrix T representing this transform:



Coordinate System Transform - Example

Write down a matrix T representing this transform:



$$\begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{o} \\ 0 & 0 & 1 \end{bmatrix}$$

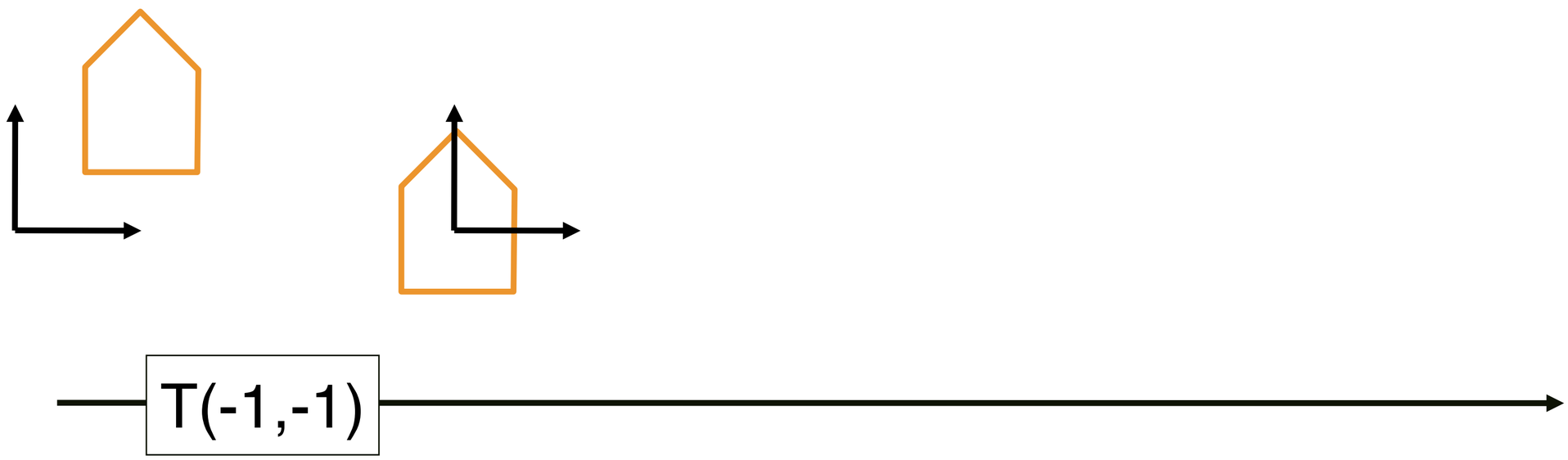
$$\begin{bmatrix} \frac{\sqrt{2}}{2} & 0 & 1 \\ \frac{\sqrt{2}}{2} & -2 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Two Interpretations of A Transform

Interpretation 1: Transforms object points

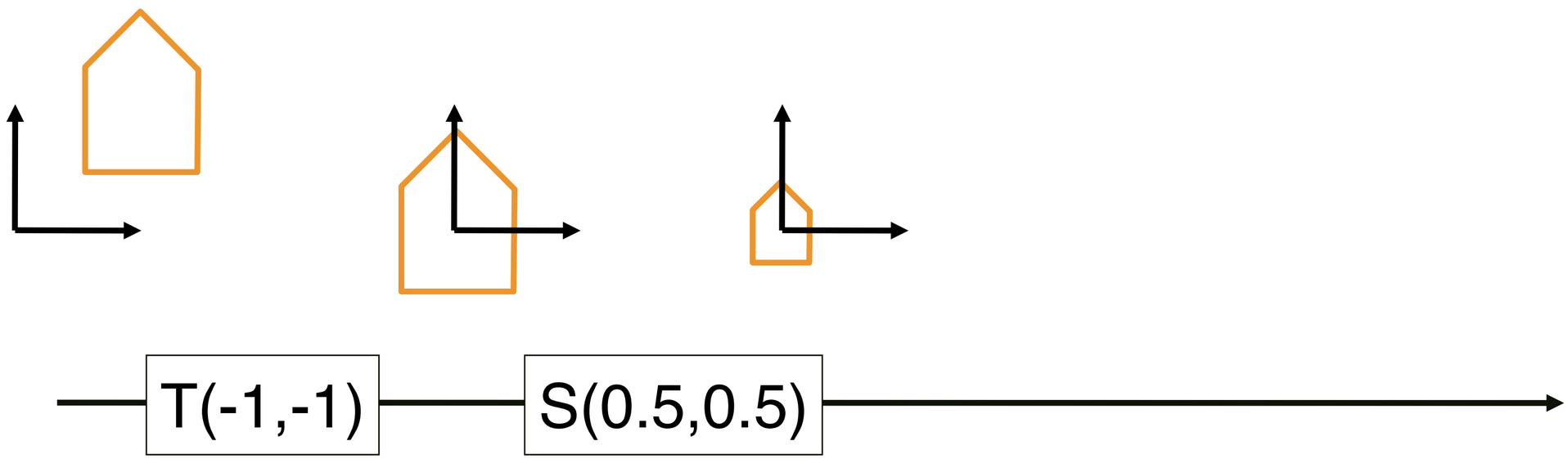
Two Interpretations of A Transform

Interpretation 1: Transforms object points



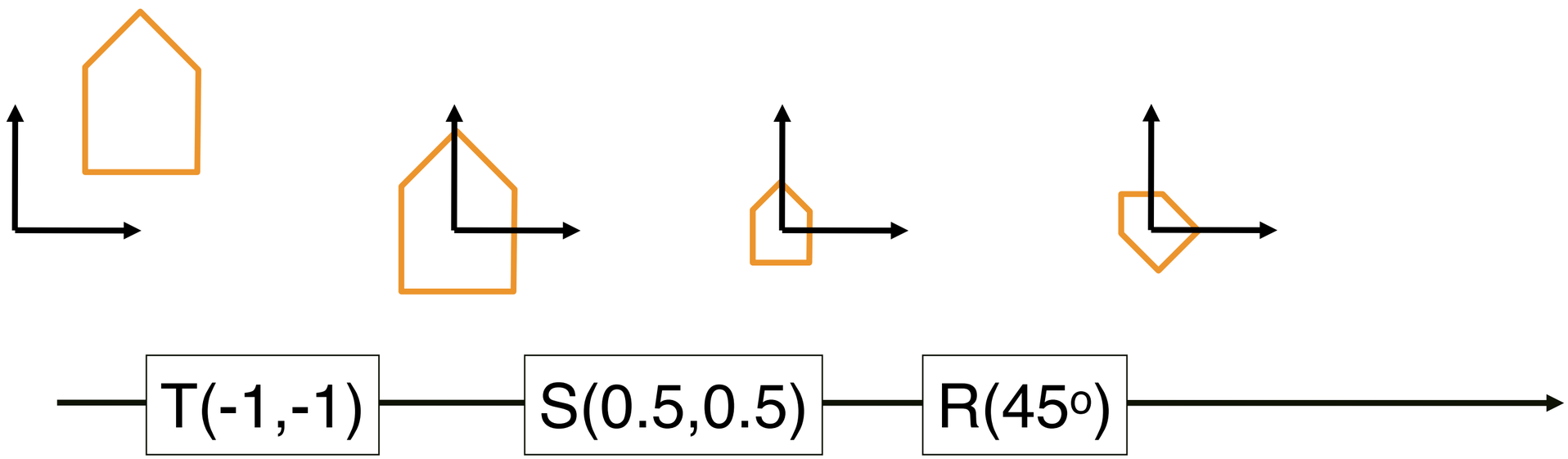
Two Interpretations of A Transform

Interpretation 1: Transforms object points



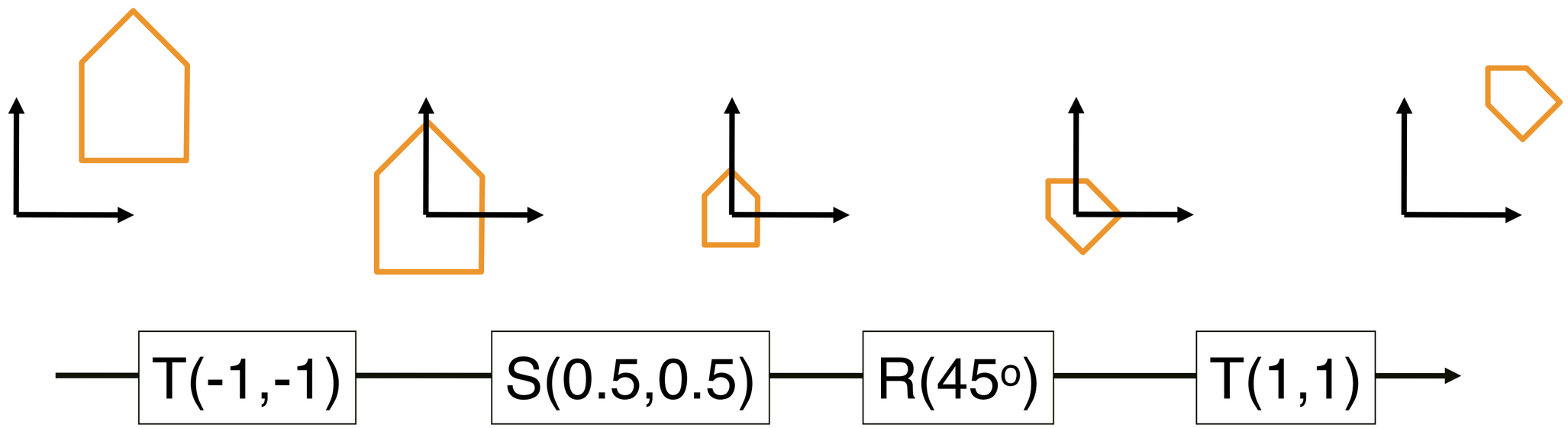
Two Interpretations of A Transform

Interpretation 1: Transforms object points



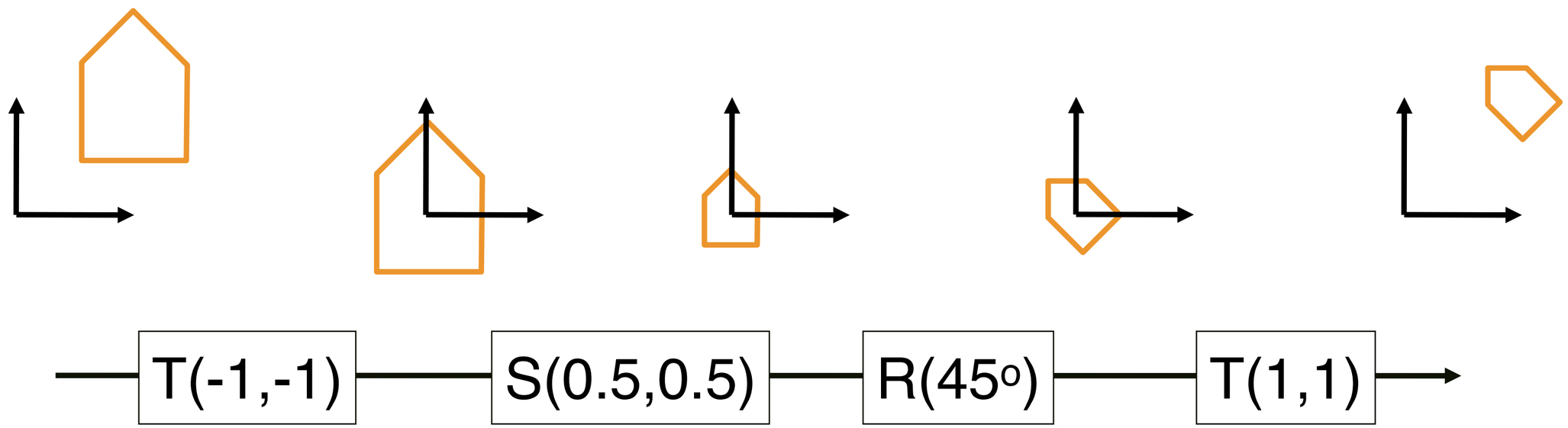
Two Interpretations of A Transform

Interpretation 1: Transforms object points



Two Interpretations of A Transform

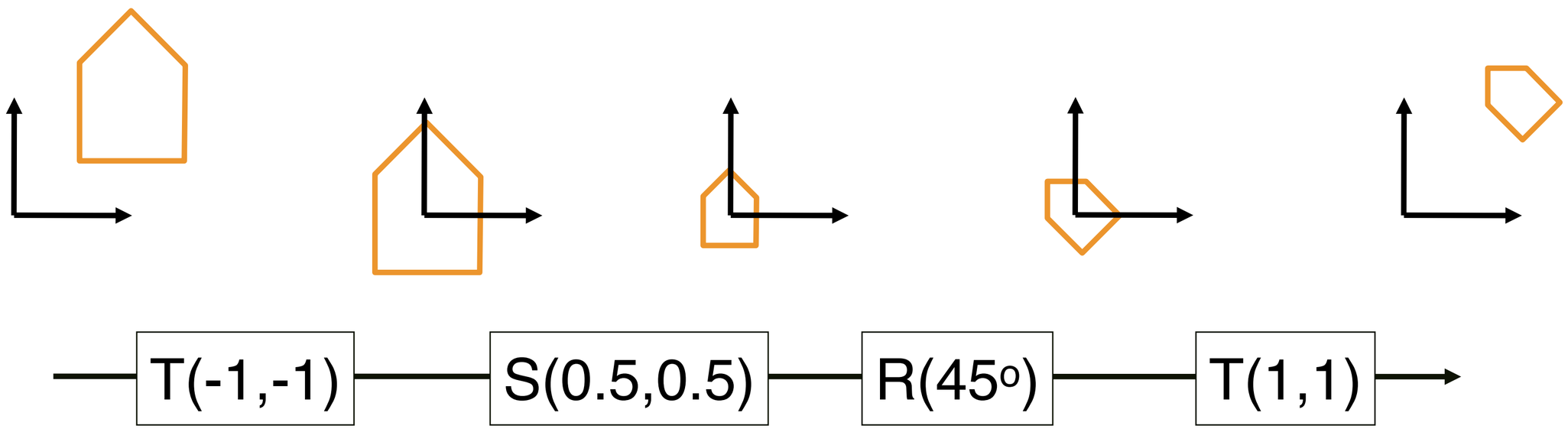
Interpretation 1: Transforms object points



Interpretation 2: Transforms coordinate system

Two Interpretations of A Transform

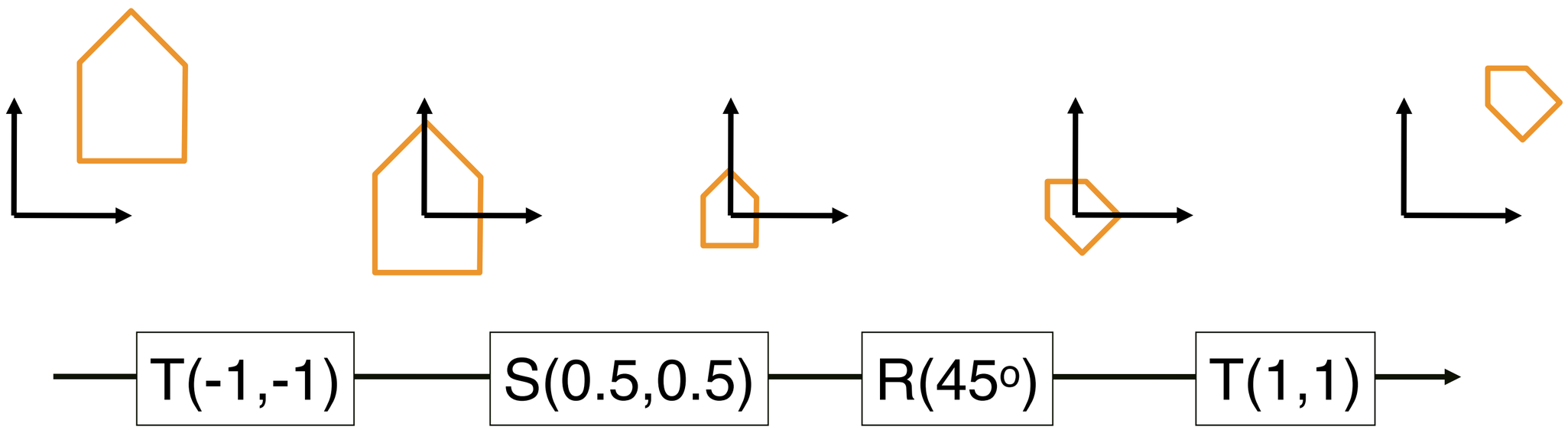
Interpretation 1: Transforms object points



Interpretation 2: Transforms coordinate system

Two Interpretations of A Transform

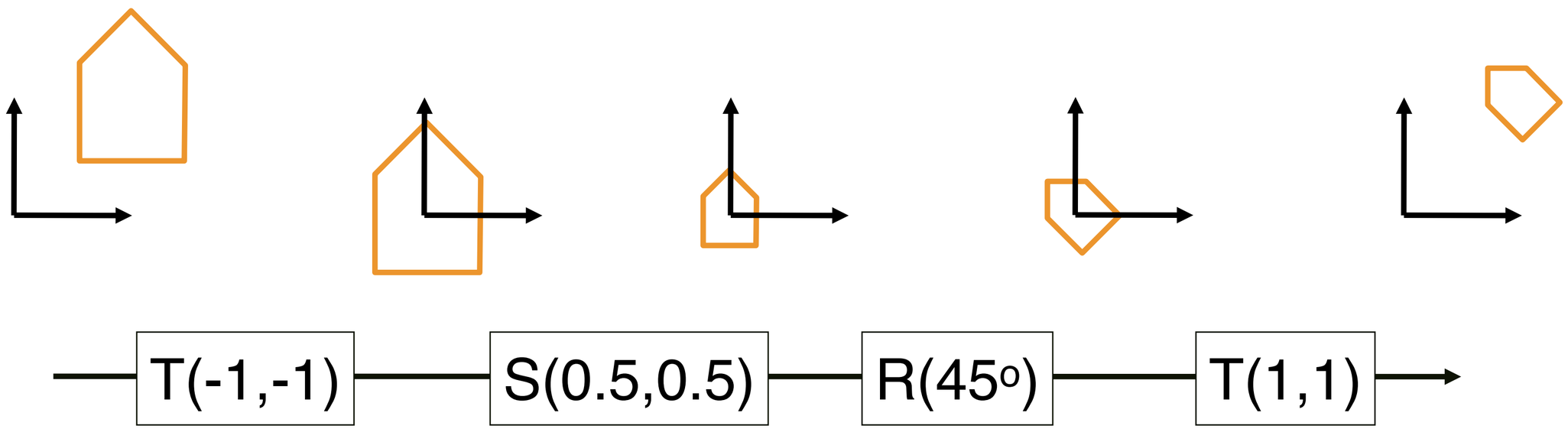
Interpretation 1: Transforms object points



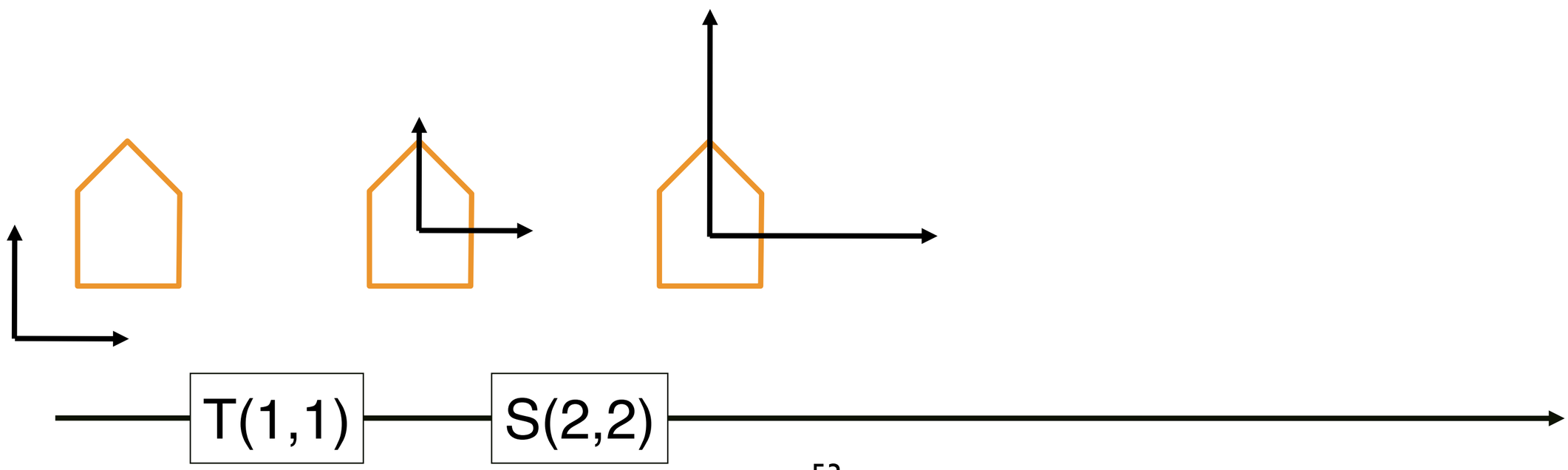
Interpretation 2: Transforms coordinate system

Two Interpretations of A Transform

Interpretation 1: Transforms object points

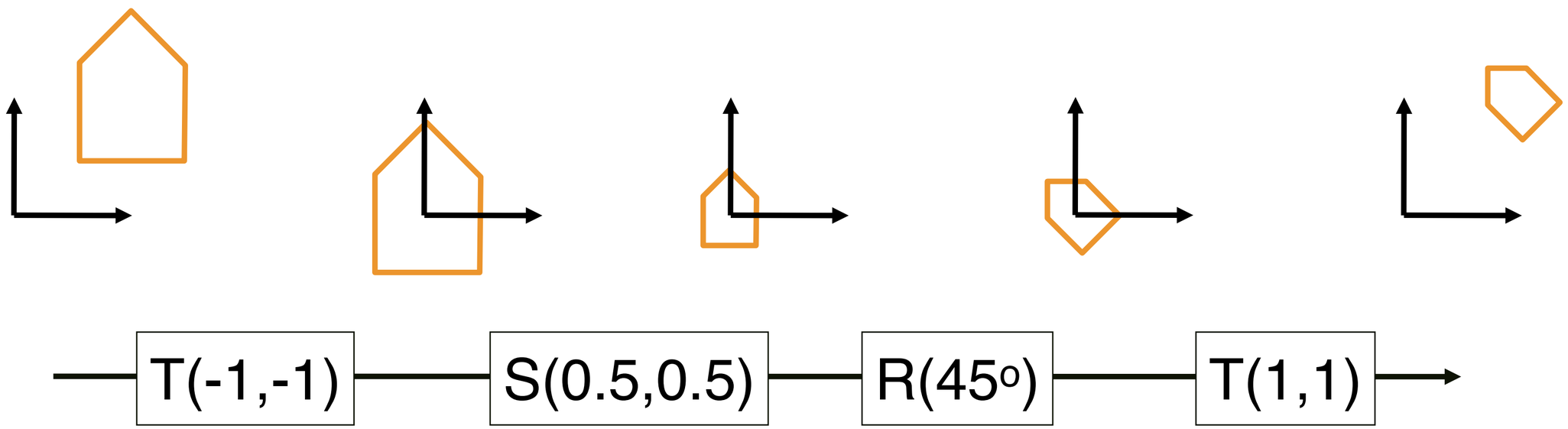


Interpretation 2: Transforms coordinate system

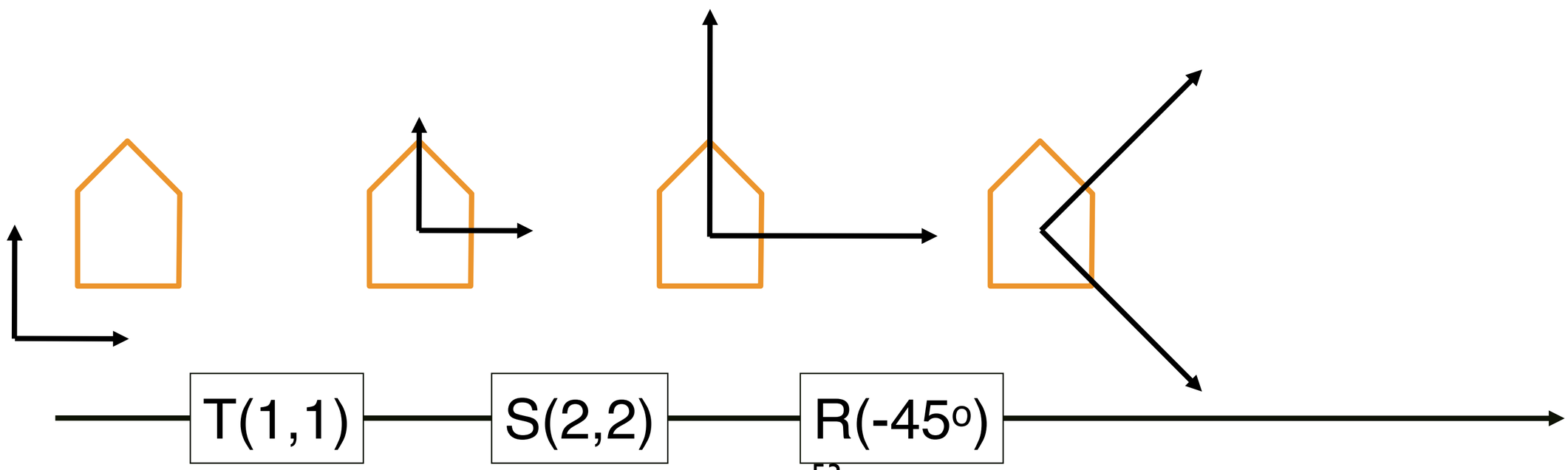


Two Interpretations of A Transform

Interpretation 1: Transforms object points

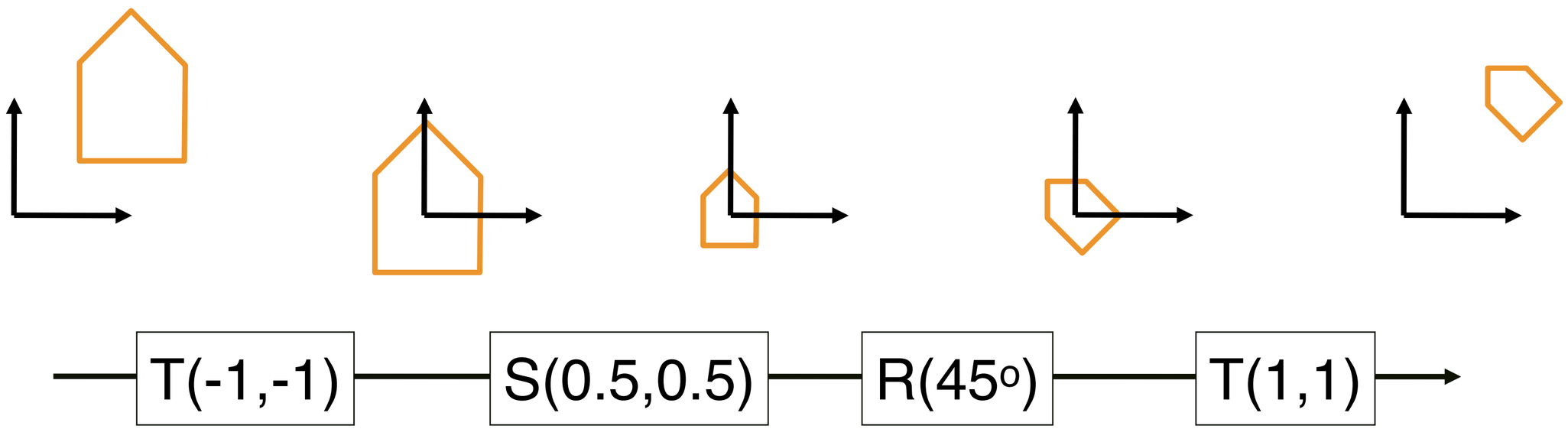


Interpretation 2: Transforms coordinate system

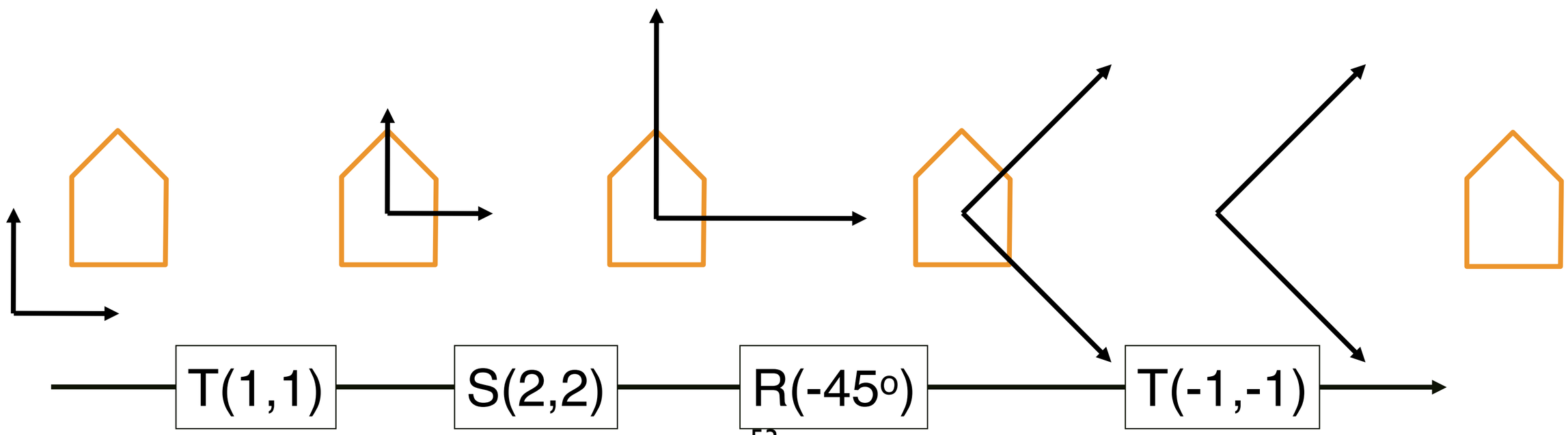


Two Interpretations of A Transform

Interpretation 1: Transforms object points



Interpretation 2: Transforms coordinate system



3D Transforms

3D Transformations

Scale

$$\mathbf{S}(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Translation

$$\mathbf{T}(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Coordinate Change (Frame-to-world)

$$\mathbf{F}(\mathbf{u}, \mathbf{v}, \mathbf{w}, \mathbf{o}) = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{o} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3D Transformations

Use homogeneous coordinates again:

- 3D point = $(x, y, z, 1)^T$
- 3D vector = $(x, y, z, 0)^T$

Use 4×4 matrices for affine transformations

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c & t_x \\ d & e & f & t_y \\ g & h & i & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Rotations

Rotations still orthonormal

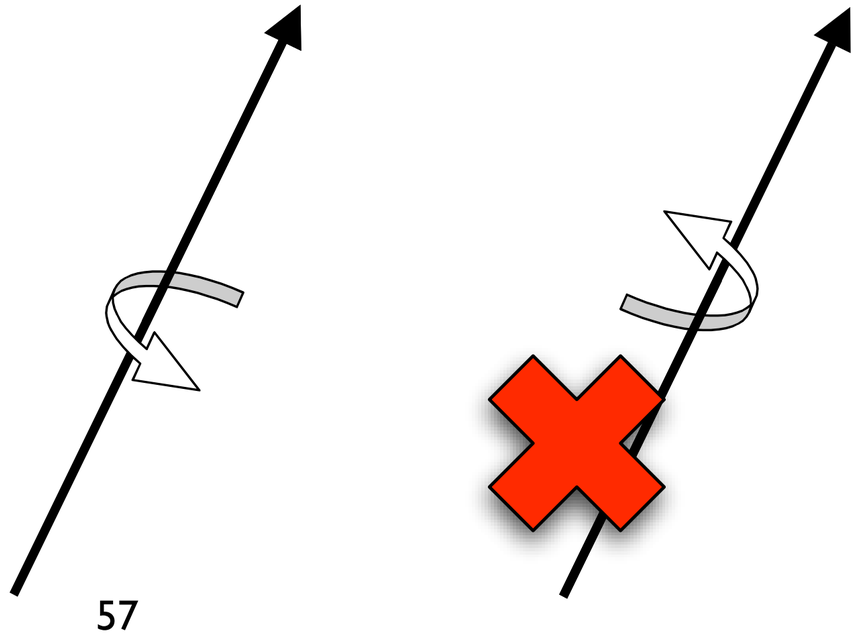
$$\mathrm{Det}(\mathbf{R}) = 1 \neq -1$$

Preserve lengths and distance to origin

3D rotations DO NOT COMMUTE!

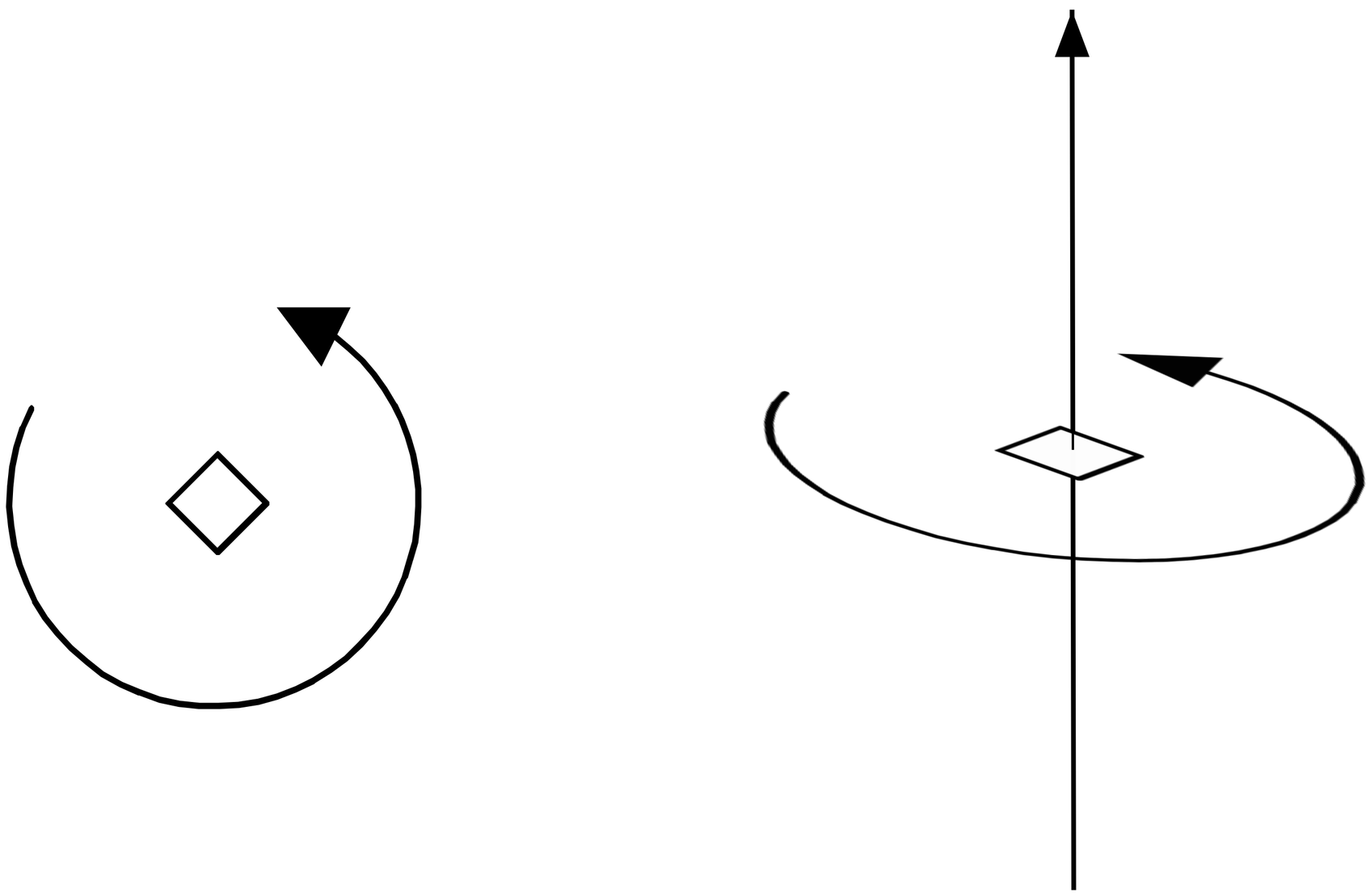
Right-hand rule

Unique matrices



2D and 3D Rotations

2D rotations implicitly rotate about a third out-of-plane axis



3D Transformations

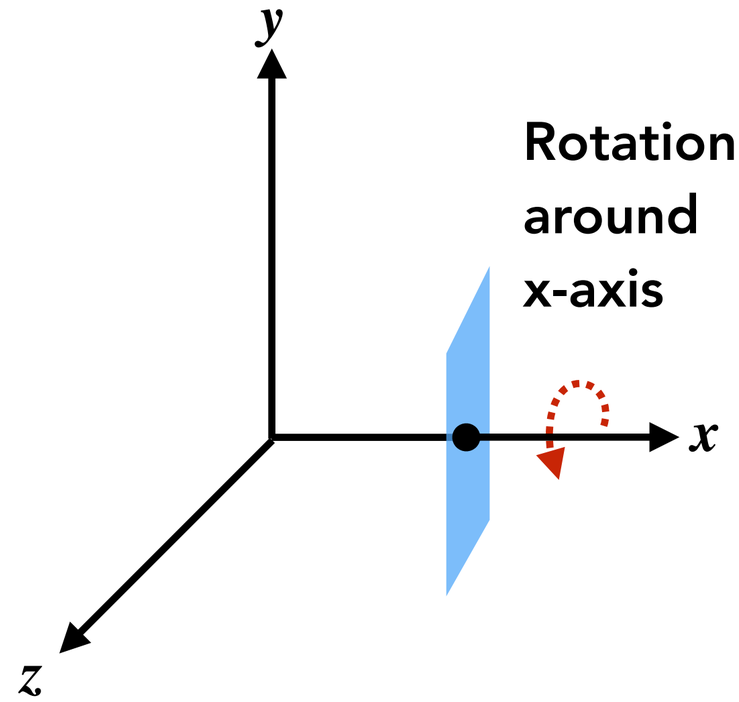
Rotation around x-, y-, or z-axis

- Also known as "direction-cosine" matrices

$$\mathbf{R}_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{R}_y(\alpha) = \begin{pmatrix} \cos\alpha & 0 & \sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\alpha & 0 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{R}_z(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 & 0 \\ \sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



3D Rotations

Compose any 3D rotation from $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$?

$$\mathbf{R}_{xyz}(\alpha, \beta, \gamma) \;=\; \mathbf{R}_x(\alpha)\,\mathbf{R}_y(\beta)\,\mathbf{R}_z(\gamma)$$

A list of the Euler angles is NOT a vector!

3D Rotations

Compose any 3D rotation from $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$?

$$\mathbf{R}_{xyz}(\alpha, \beta, \gamma) = \mathbf{R}_x(\alpha)\,\mathbf{R}_y(\beta)\,\mathbf{R}_z(\gamma)$$

- So-called *Euler angles*

A list of the Euler angles is NOT a vector!

3D Rotations

Compose any 3D rotation from $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$?

$$\mathbf{R}_{xyz}(\alpha, \beta, \gamma) \;=\; \mathbf{R}_x(\alpha)\,\mathbf{R}_y(\beta)\,\mathbf{R}_z(\gamma)$$

- So-called *Euler angles*
- Often used in flight simulators: roll, pitch, yaw

A list of the Euler angles is NOT a vector!

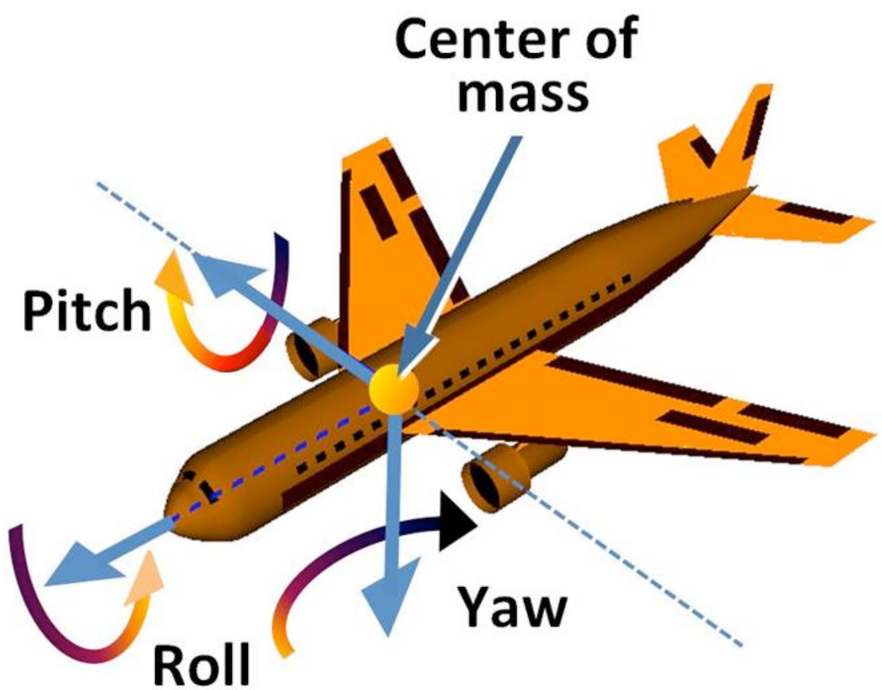
3D Rotations

Compose any 3D rotation from $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$?

$$\mathbf{R}_{xyz}(\alpha, \beta, \gamma) = \mathbf{R}_x(\alpha)\,\mathbf{R}_y(\beta)\,\mathbf{R}_z(\gamma)$$

- So-called *Euler angles*
- Often used in flight simulators: roll, pitch, yaw

A list of the Euler angles is NOT a vector!



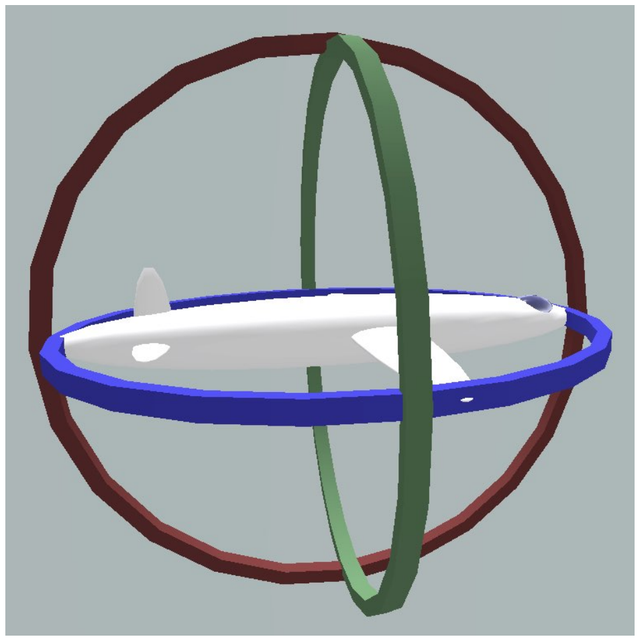
3D Rotations

Compose any 3D rotation from $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$?

$$\mathbf{R}_{xyz}(\alpha, \beta, \gamma) = \mathbf{R}_x(\alpha)\,\mathbf{R}_y(\beta)\,\mathbf{R}_z(\gamma)$$

- So-called *Euler angles*
- Often used in flight simulators: roll, pitch, yaw

- Problems:
  Gimbal Lock!
  Not unique!
  Ugly manifold!



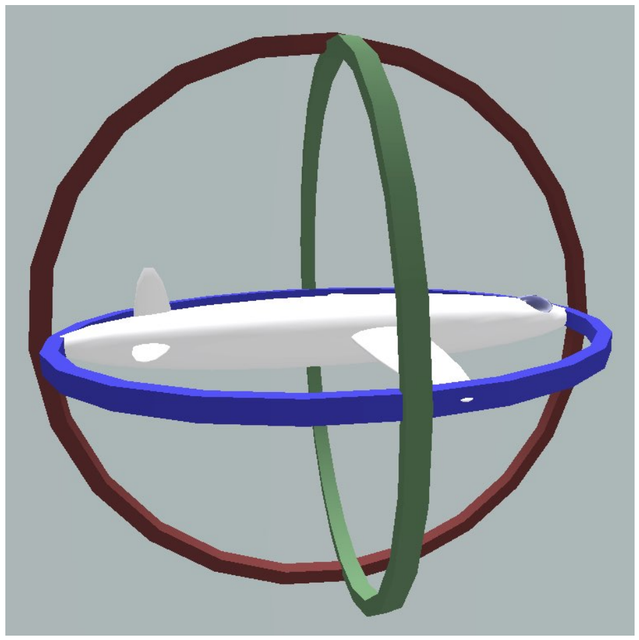
3D Rotations

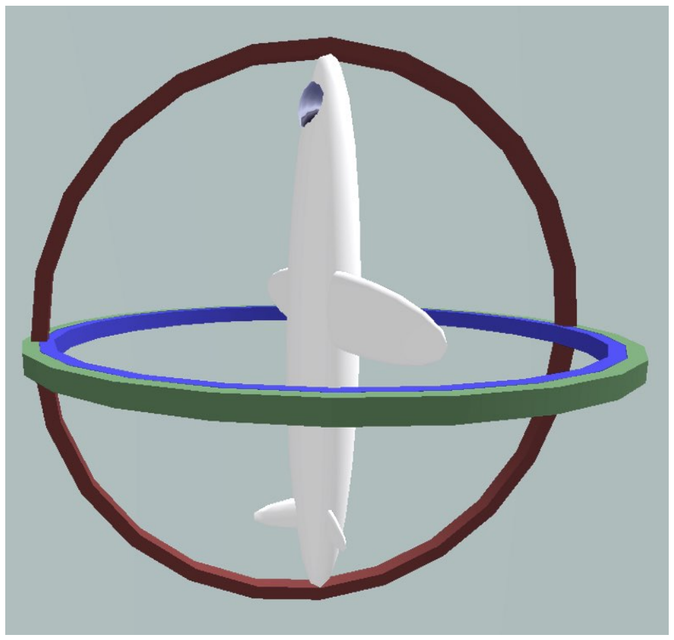
Compose any 3D rotation from $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$?

$$\mathbf{R}_{xyz}(\alpha, \beta, \gamma) = \mathbf{R}_x(\alpha)\,\mathbf{R}_y(\beta)\,\mathbf{R}_z(\gamma)$$

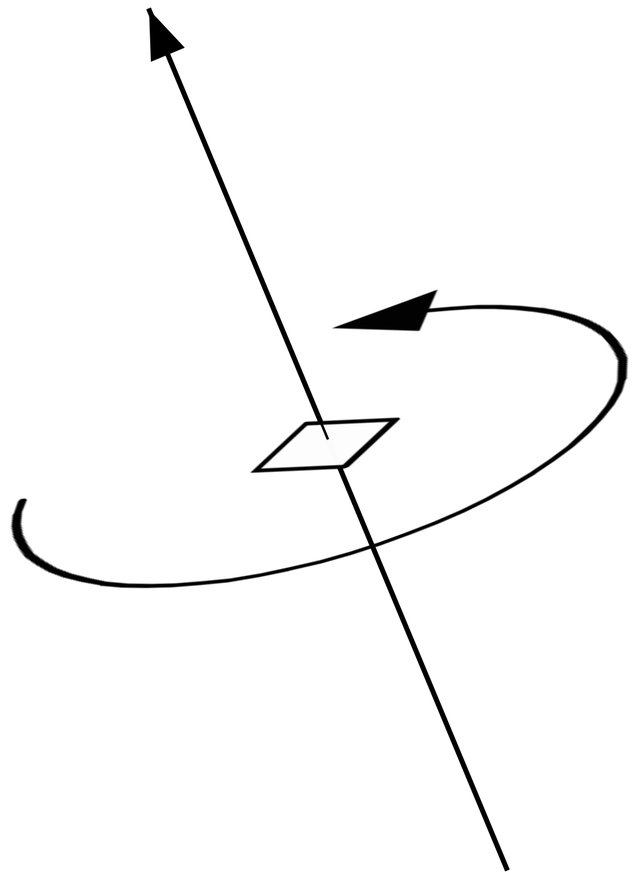
- So-called *Euler angles*
- Often used in flight simulators: roll, pitch, yaw

- Problems:
  Gimbal Lock!
  Not unique!
  Ugly manifold!





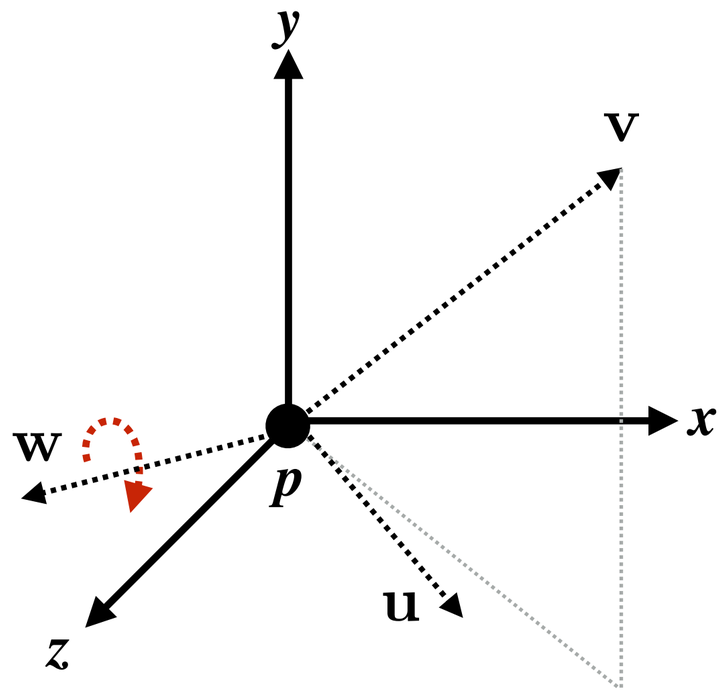
3D Rotation Around Arbitrary Axis



3D Rotation Around Arbitrary Axis

- **Easy to understand but less useful method:**
  1. **rotate about x axis to put $\mathbf{w}$ into the x-y plane**
  2. **rotate about z axis align $\mathbf{w}$ with the x axis**
  3. **rotate θ degrees about x axis**
  4. **undo #2 and then #1**
  5. **composite together**

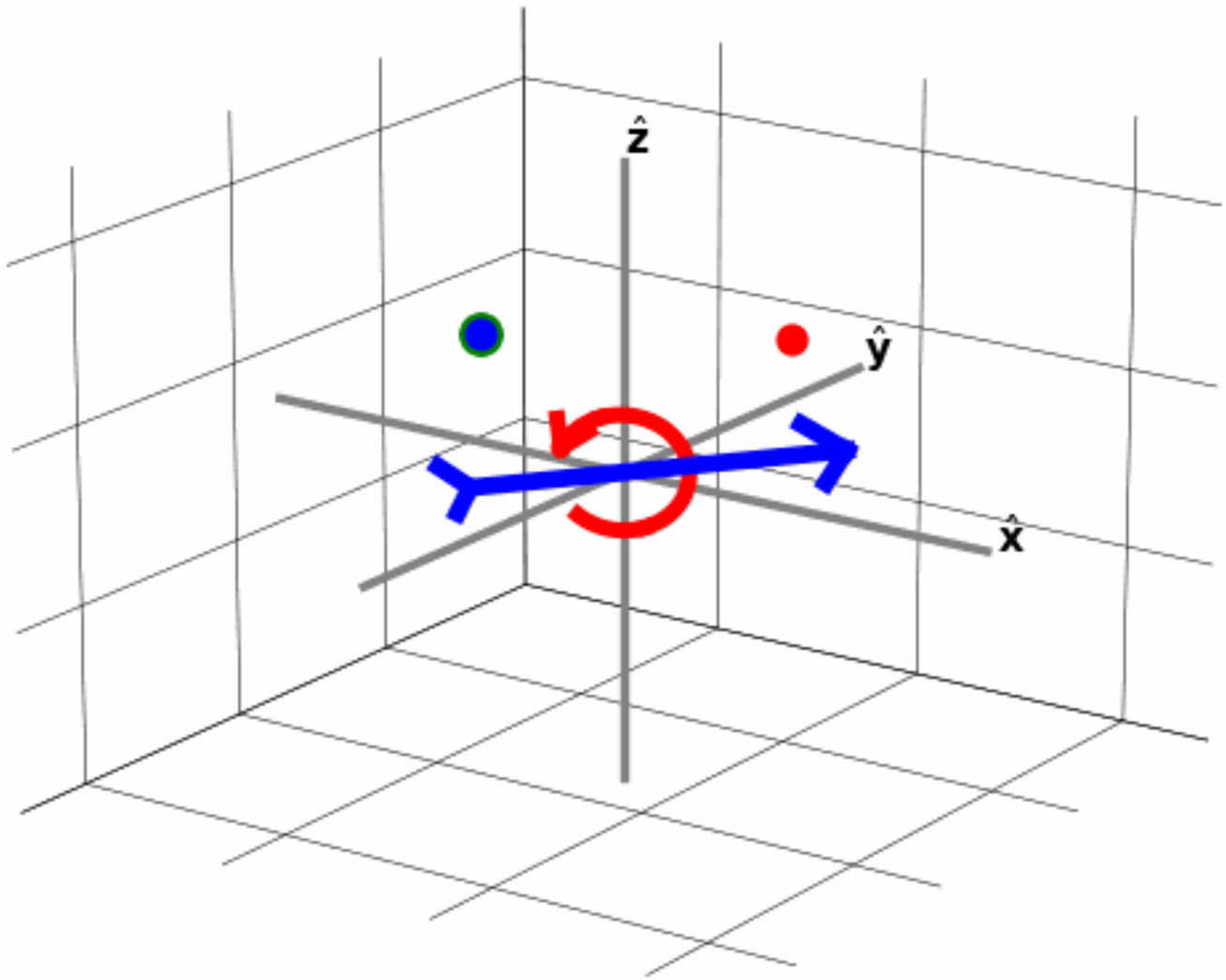
How would you take a derivative w.r.t. w?



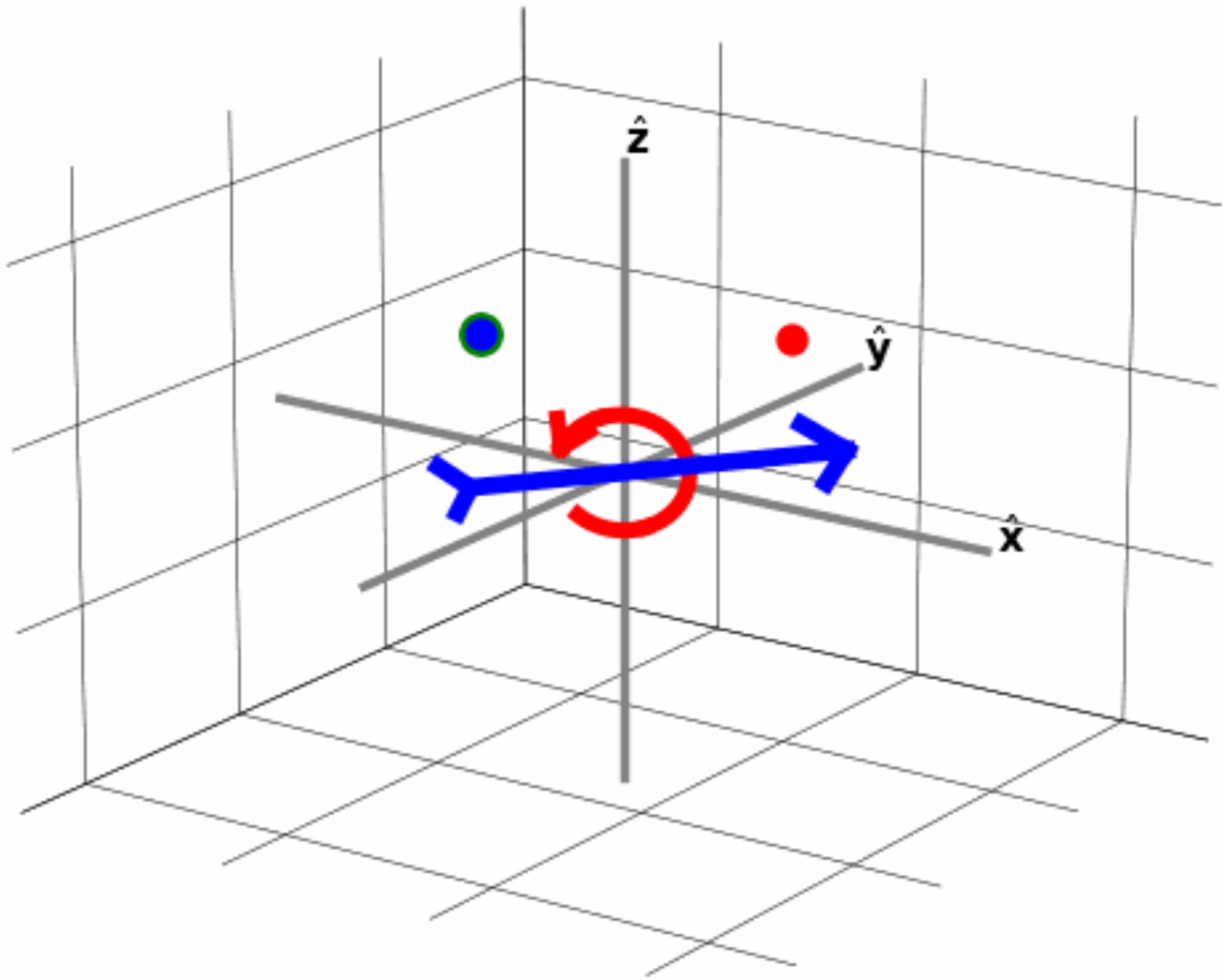
3D Rotation Around Arbitrary Axis

3D Rotation Around Arbitrary Axis

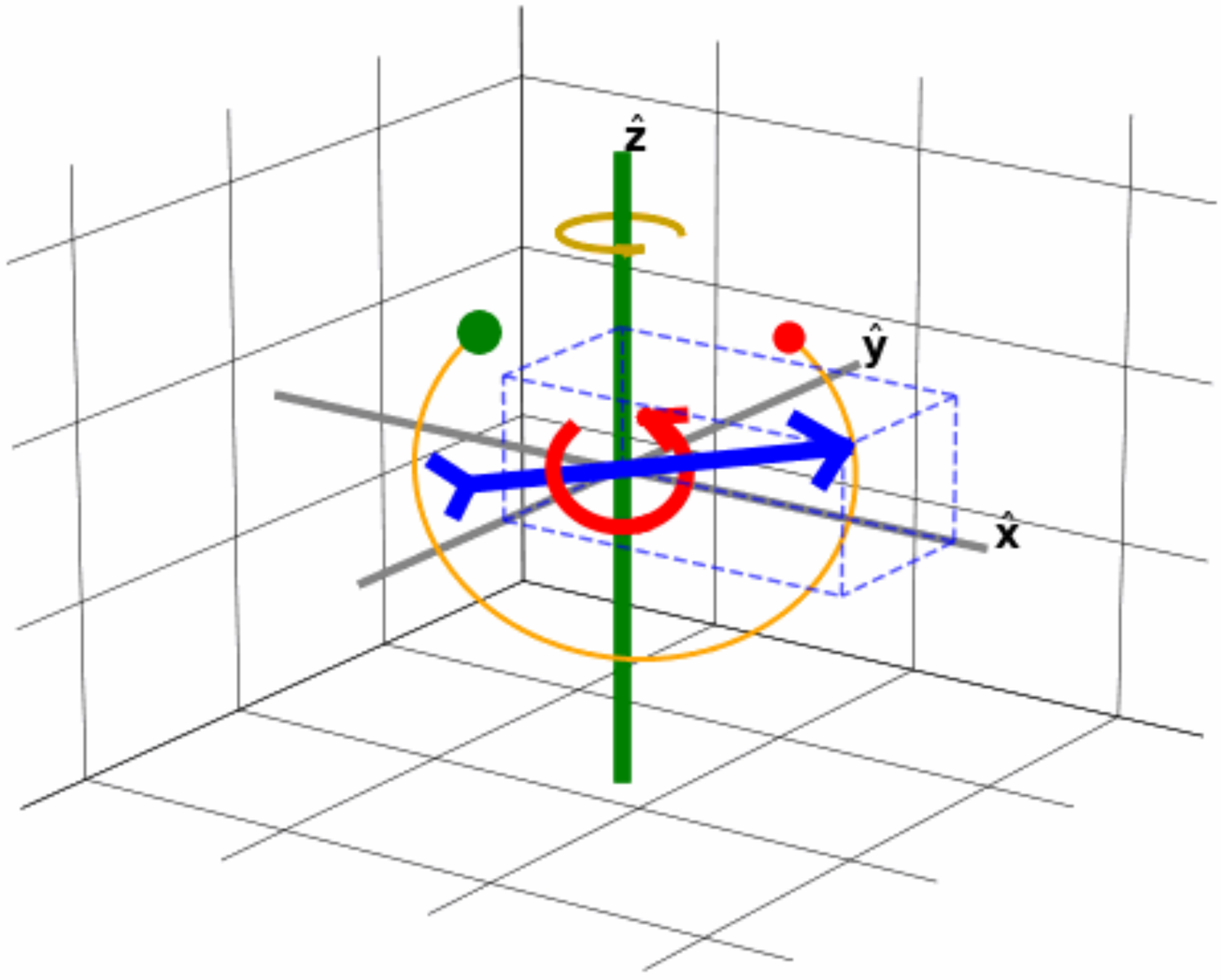
3D Rotation Around Arbitrary Axis



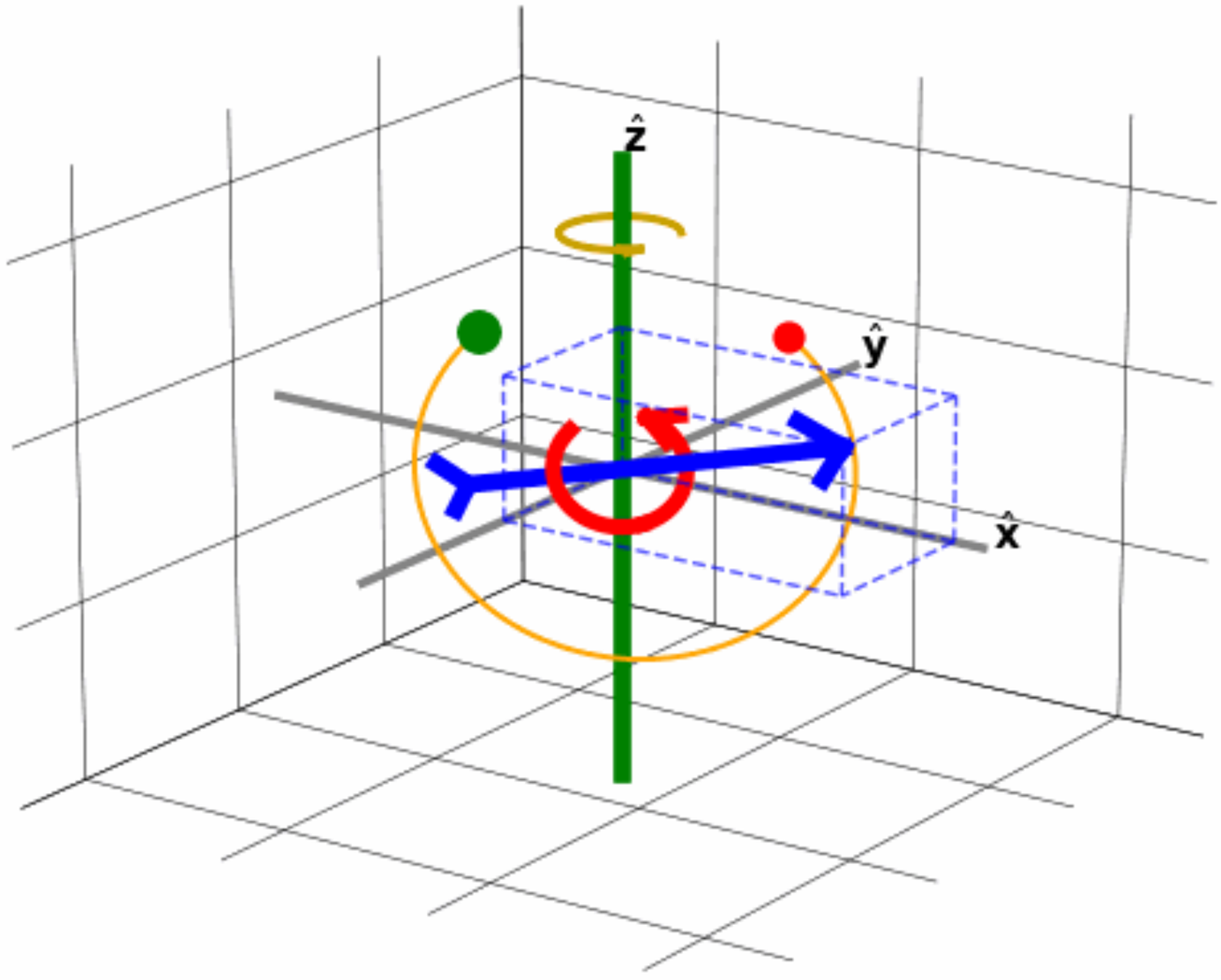
3D Rotation Around Arbitrary Axis



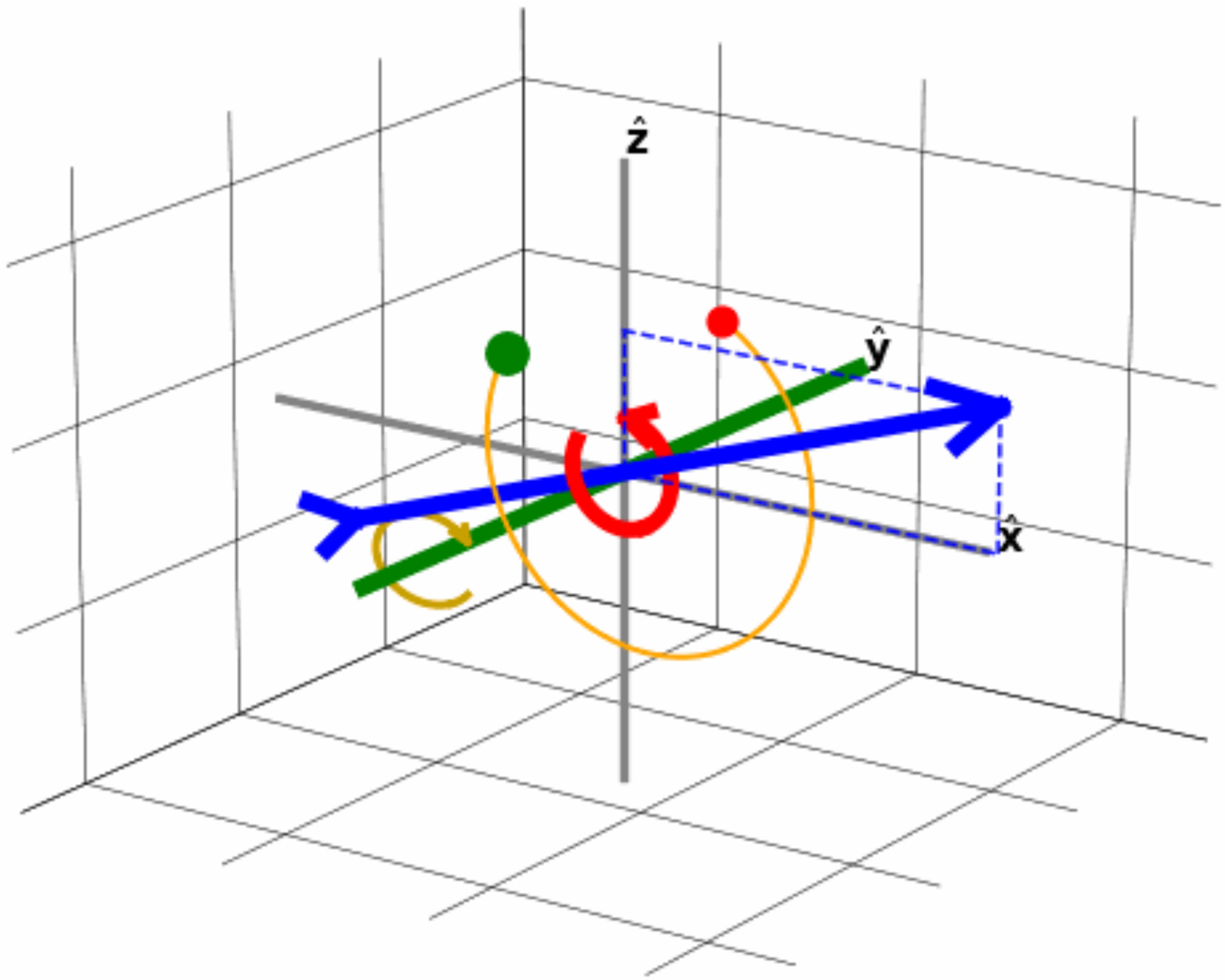
3D Rotation Around Arbitrary Axis



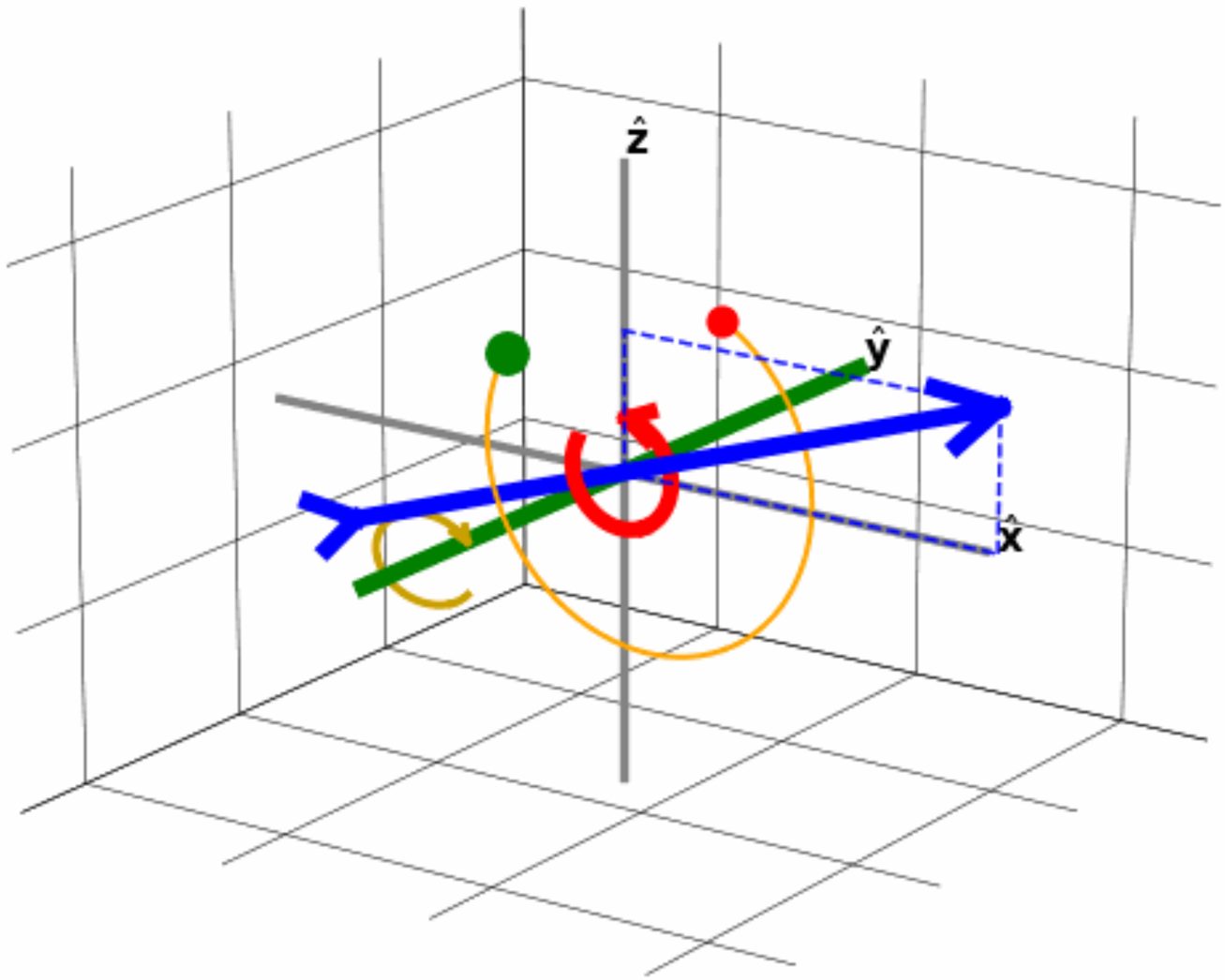
3D Rotation Around Arbitrary Axis



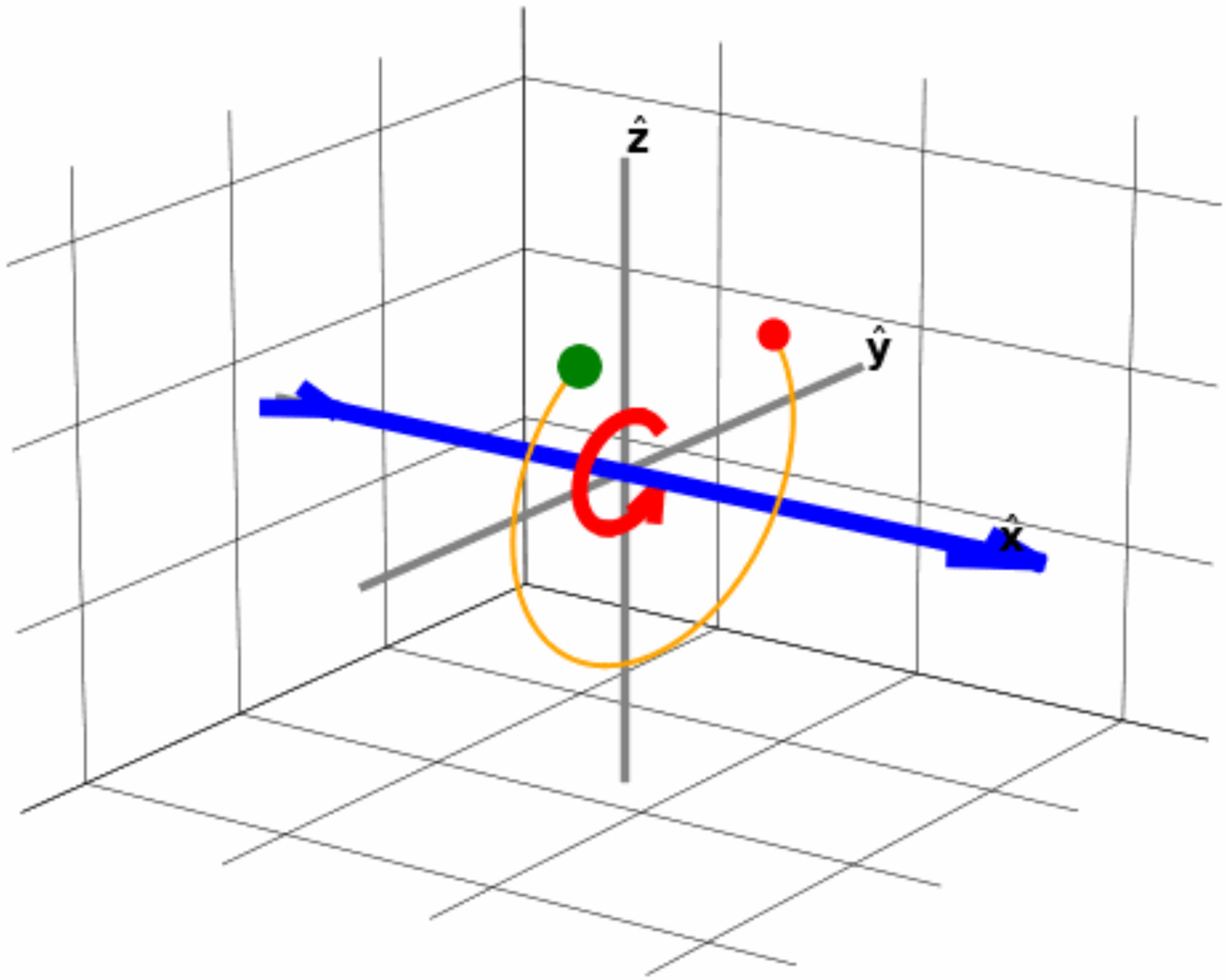
3D Rotation Around Arbitrary Axis



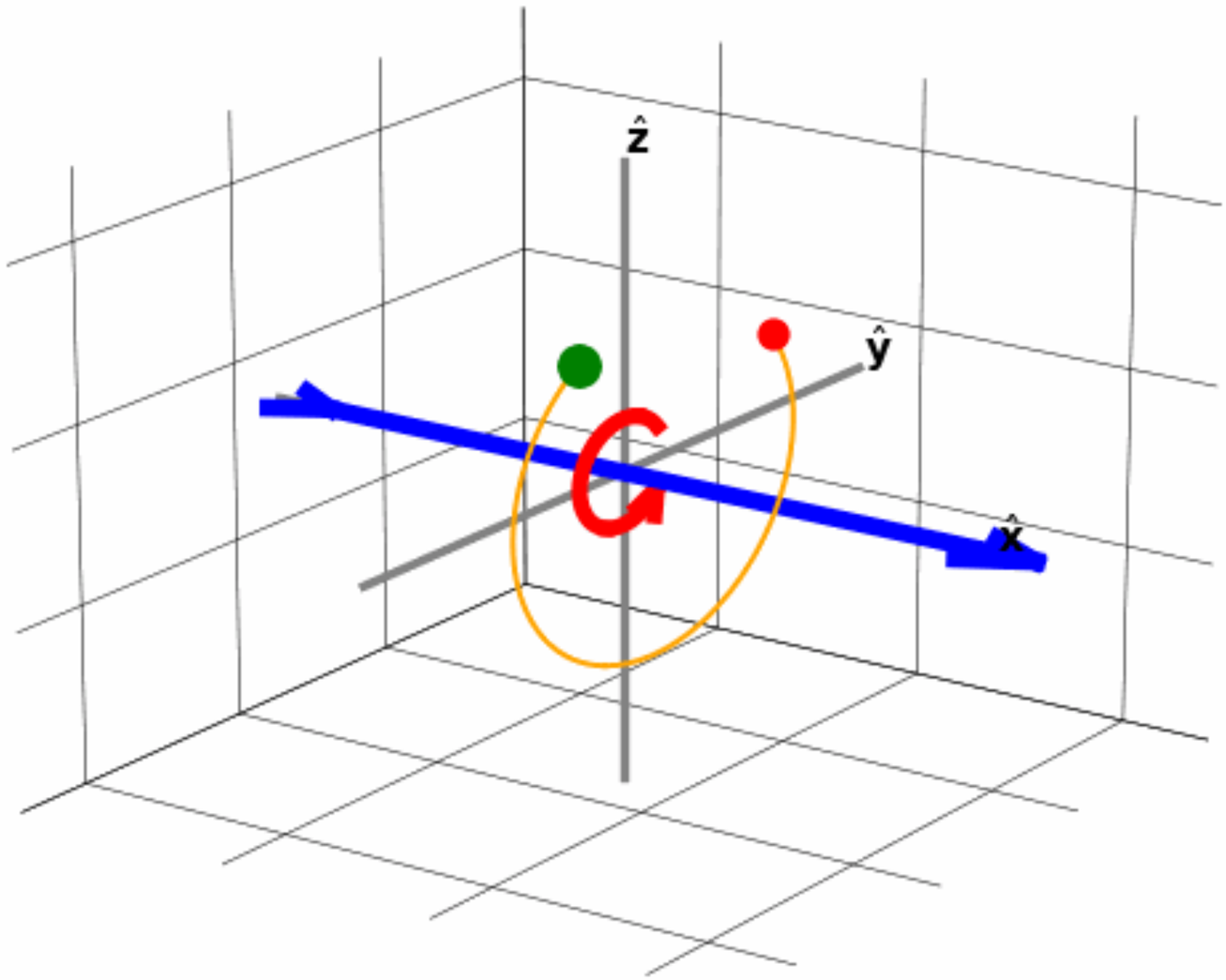
3D Rotation Around Arbitrary Axis



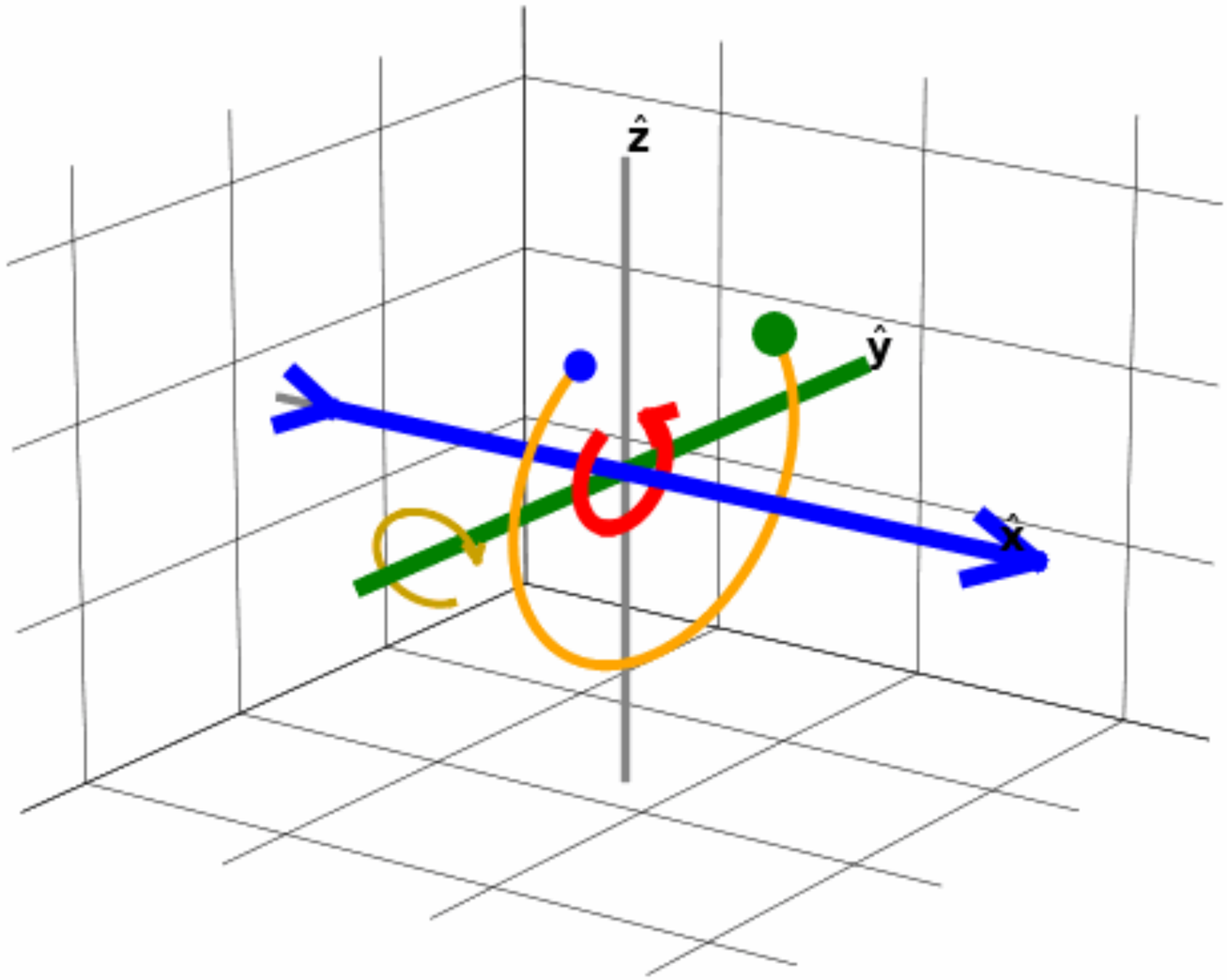
3D Rotation Around Arbitrary Axis



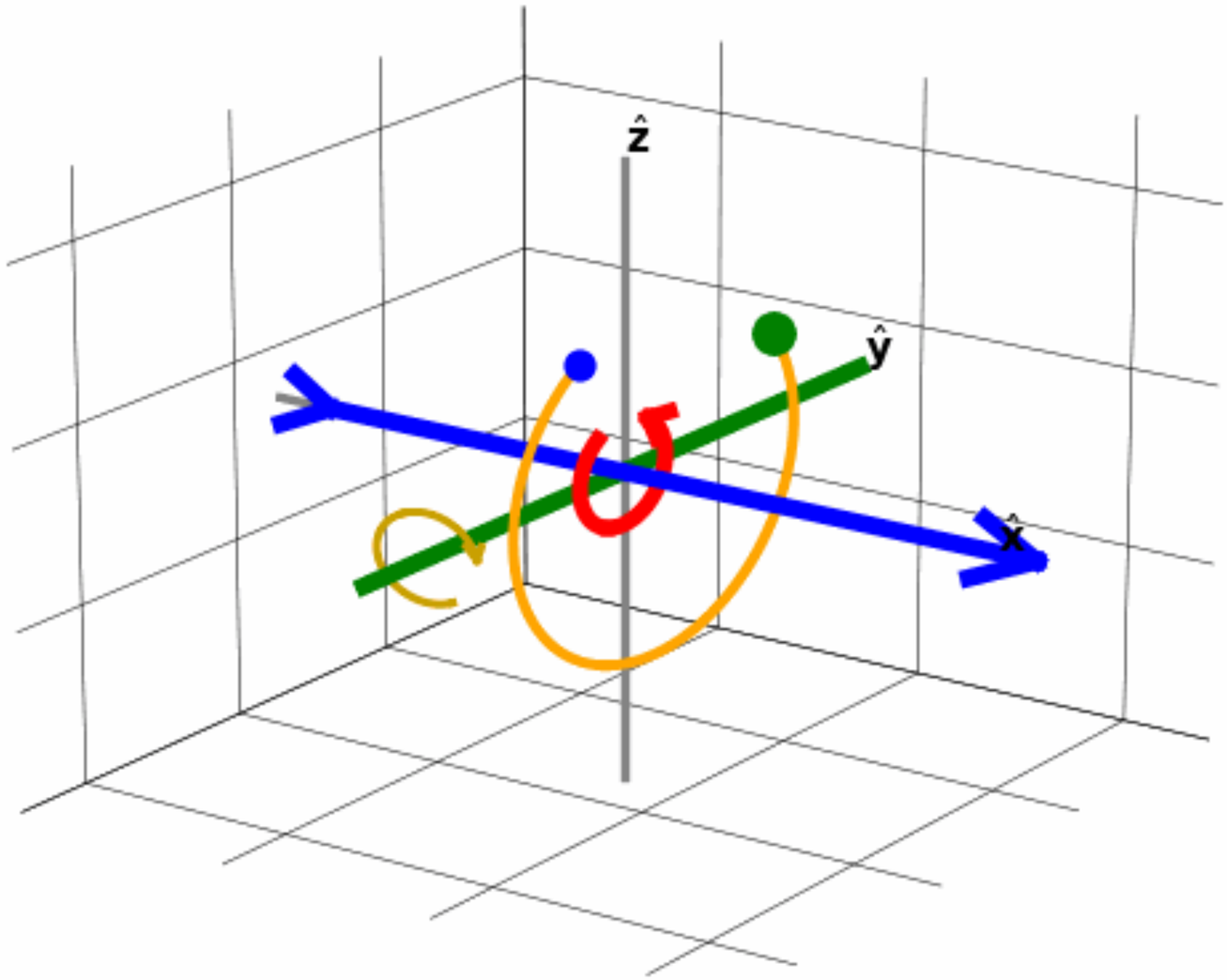
3D Rotation Around Arbitrary Axis



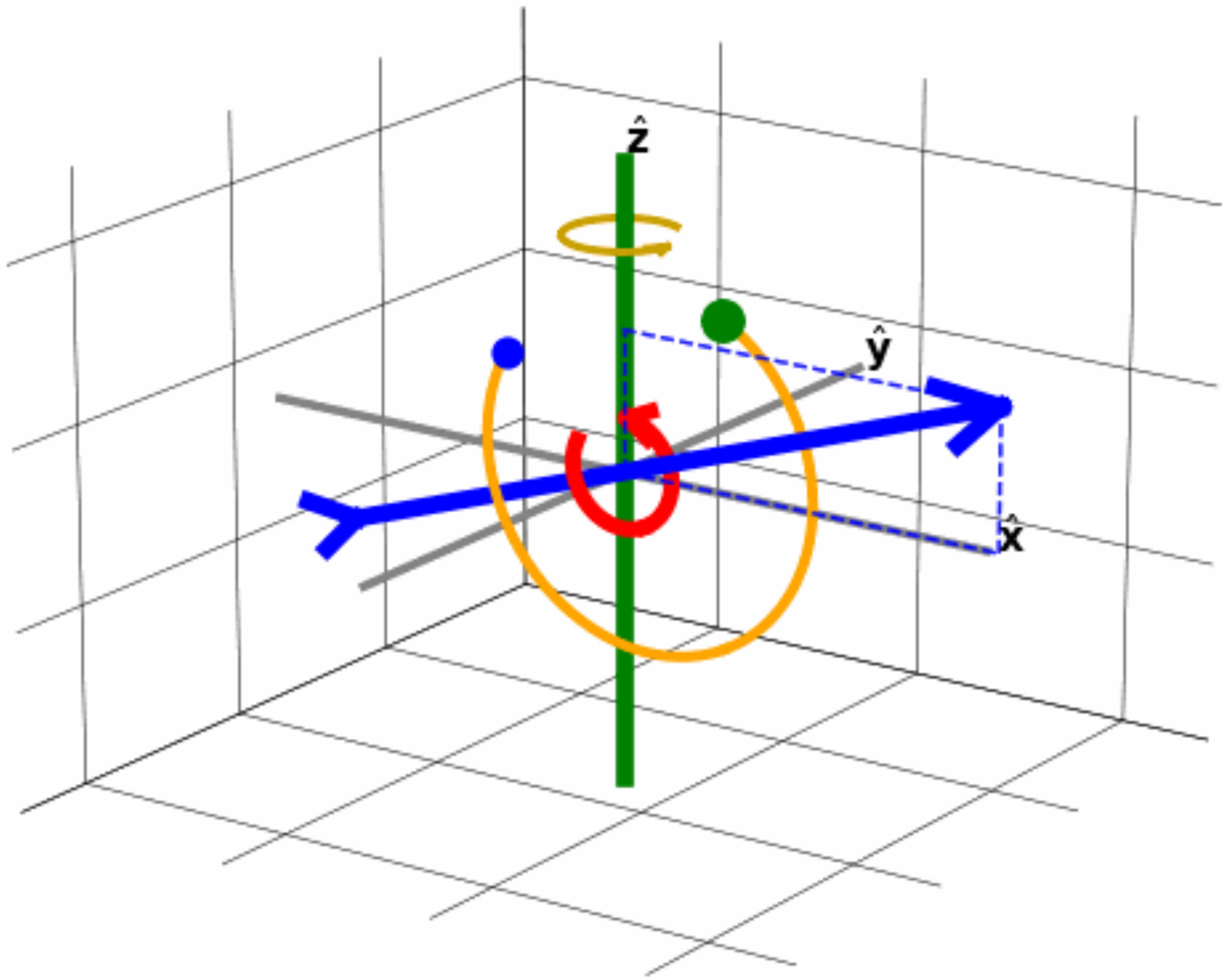
3D Rotation Around Arbitrary Axis



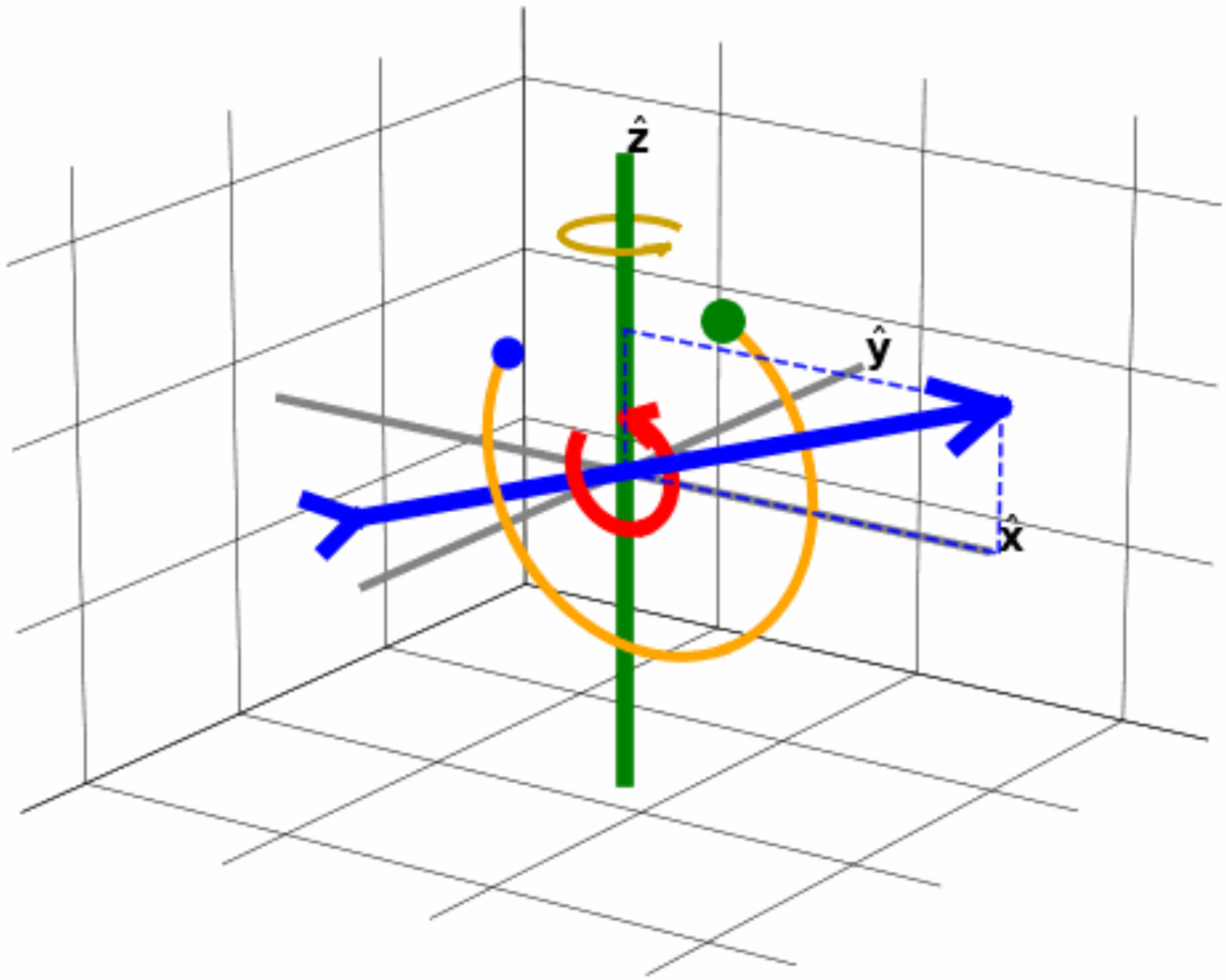
3D Rotation Around Arbitrary Axis



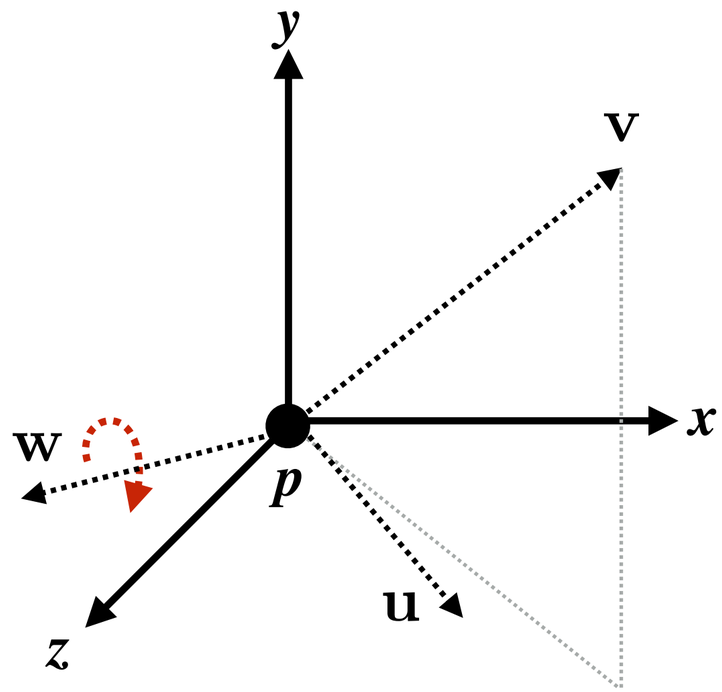
3D Rotation Around Arbitrary Axis



3D Rotation Around Arbitrary Axis

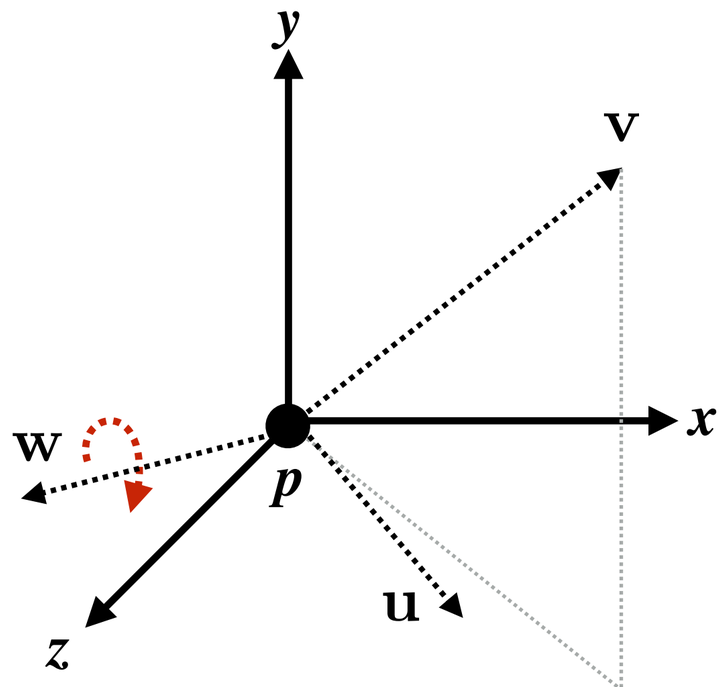


3D Rotation Around Arbitrary Axis



3D Rotation Around Arbitrary Axis

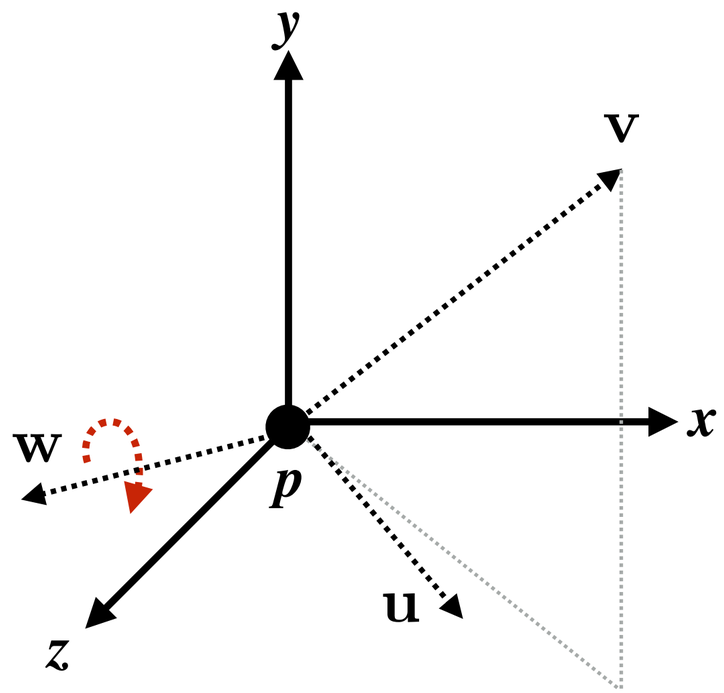
Construct orthonormal frame transformation F with p, u, v, w, where p and w match the rotation axis



3D Rotation Around Arbitrary Axis

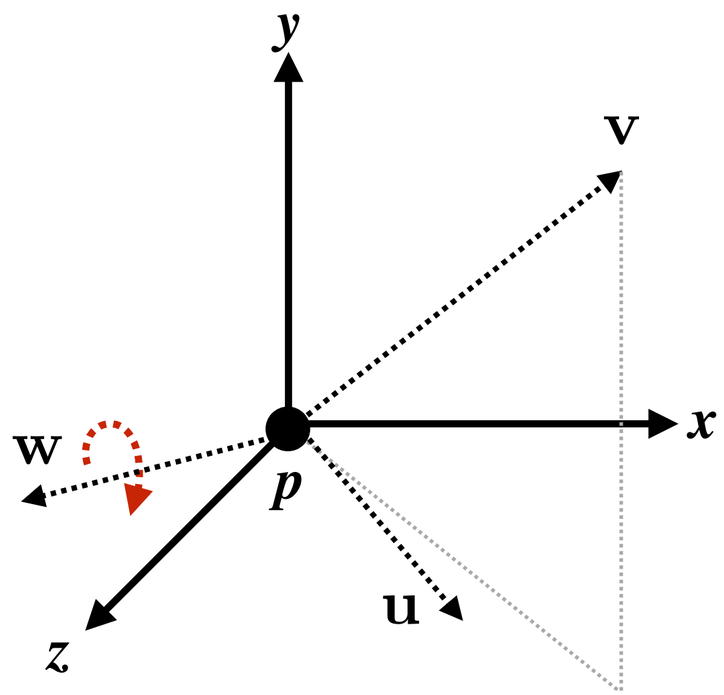
Construct orthonormal frame transformation F with p, u, v, w, where p and w match the rotation axis

Apply the transform ($F\ R_z(\theta)\ F^{-1}$)



3D Rotation Around Arbitrary Axis

Construct orthonormal frame transformation F with p, u, v, w, where p and w match the rotation axis

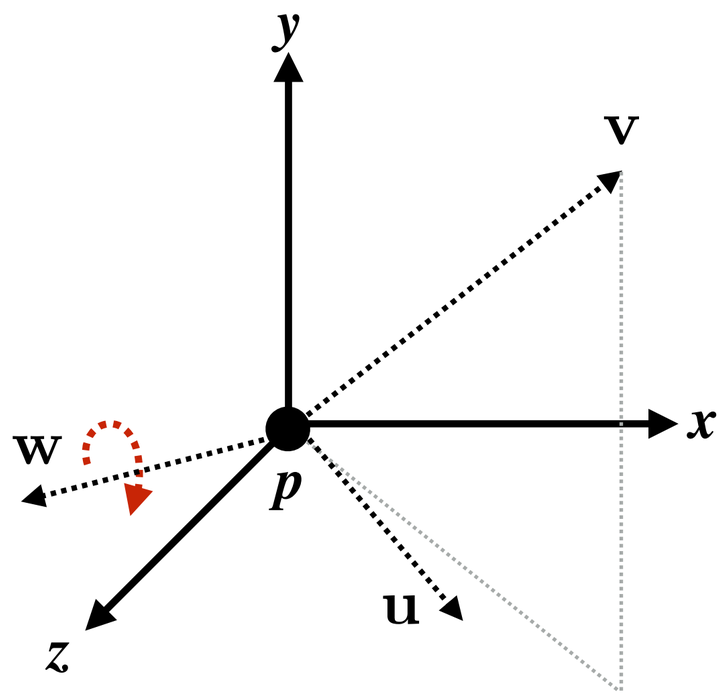


Apply the transform ($F\ R_z(\theta)\ F^{-1}$)

Interpretation:

3D Rotation Around Arbitrary Axis

Construct orthonormal frame transformation F with p, u, v, w, where p and w match the rotation axis



Apply the transform ($F\ R_z(\theta)\ F^{-1}$)

Interpretation:

- Rotate to Z axis, rotate, then move back

Rodrigues' Rotation Formula

Rotation by angle α around axis $\mathbf{n}$

$$\mathbf{R}(\mathbf{n},\alpha) \;=\; \cos(\alpha)\,\mathbf{I} \;+\; (1-\cos(\alpha))\,\mathbf{n}\mathbf{n}^T \;+\; \sin(\alpha)\underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_{\mathbf{N}}$$

Turn cross-product into matrix multiplication!

$$\mathbf{n}\times\mathbf{v} = \mathbf{N}\mathbf{v}$$

Alternative notation

$$\mathbf{n}\times\mathbf{v} = [\mathbf{n}]\mathbf{v}$$

$$\mathbf{n}\times\mathbf{v} = (\mathbf{n}\times)\mathbf{v}$$

Rodrigues' Rotation Formula

Rotation by angle α around axis $\mathbf{n}$

$$\mathbf{R}(\mathbf{n},\alpha) \;=\; \cos(\alpha)\,\mathbf{I} \;+\; (1-\cos(\alpha))\,\mathbf{n}\mathbf{n}^T \;+\; \sin(\alpha)\underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_{\mathbf{N}}$$

How to prove this magic formula?

Turn cross-product into matrix multiplication!

$$\mathbf{n}\times\mathbf{v} = \mathbf{N}\mathbf{v}$$

Alternative notation

$$\mathbf{n}\times\mathbf{v} = [\mathbf{n}]\mathbf{v}$$

$$\mathbf{n}\times\mathbf{v} = (\mathbf{n}\times)\mathbf{v}$$

Rodrigues' Rotation Formula

Rotation by angle α around axis $\mathbf{n}$

$$\mathbf{R}(\mathbf{n},\alpha) \;=\; \cos(\alpha)\,\mathbf{I} \;+\; (1-\cos(\alpha))\,\mathbf{n}\mathbf{n}^T \;+\; \sin(\alpha)\underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_{\mathbf{N}}$$

How to prove this magic formula?

- Matrix $\mathbf{N}$ computes a cross-product:

Turn cross-product into matrix multiplication!

$$\mathbf{n} \times \mathbf{v} = \mathbf{N}\mathbf{v}$$

Alternative notation

$$\mathbf{n} \times \mathbf{v} = [\mathbf{n}]\mathbf{v}$$

$$\mathbf{n} \times \mathbf{v} = (\mathbf{n}\times)\mathbf{v}$$

Rodrigues' Rotation Formula

Rotation by angle α around axis $\mathbf{n}$

$$\mathbf{R}(\mathbf{n},\alpha) \;=\; \cos(\alpha)\,\mathbf{I} \;+\; (1-\cos(\alpha))\,\mathbf{n}\mathbf{n}^T \;+\; \sin(\alpha)\underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_{\mathbf{N}}$$

How to prove this magic formula?

- Matrix $\mathbf{N}$ computes a cross-product:
- Assume orthonormal system $\mathbf{e}_1, \mathbf{e}_2, \mathbf{n}$

Turn cross-product into matrix multiplication!

$$\mathbf{n} \times \mathbf{v} = \mathbf{N}\mathbf{v}$$

Alternative notation

$$\mathbf{n} \times \mathbf{v} = [\mathbf{n}]\mathbf{v}$$

$$\mathbf{n} \times \mathbf{v} = (\mathbf{n}\times)\mathbf{v}$$

Rodrigues' Rotation Formula

Rotation by angle α around axis $\mathbf{n}$

$$\mathbf{R}(\mathbf{n},\alpha) \;=\; \cos(\alpha)\,\mathbf{I} \;+\; (1-\cos(\alpha))\,\mathbf{n}\mathbf{n}^T \;+\; \sin(\alpha)\underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_{\mathbf{N}}$$

How to prove this magic formula?

- Matrix $\mathbf{N}$ computes a cross-product:
- Assume orthonormal system $\mathbf{e}_1, \mathbf{e}_2, \mathbf{n}$

$$\begin{aligned} \mathbf{R}\mathbf{n} &= \mathbf{n} \\ \mathbf{R}\mathbf{e}_1 &= \cos\alpha\,\mathbf{e}_1 + \sin\alpha\,\mathbf{e}_2 \\ \mathbf{R}\mathbf{e}_2 &= -\sin\alpha\,\mathbf{e}_1 + \cos\alpha\,\mathbf{e}_2 \end{aligned}$$

Turn cross-product into matrix multiplication!

$$\mathbf{n}\times\mathbf{v} = \mathbf{N}\mathbf{v}$$

Alternative notation

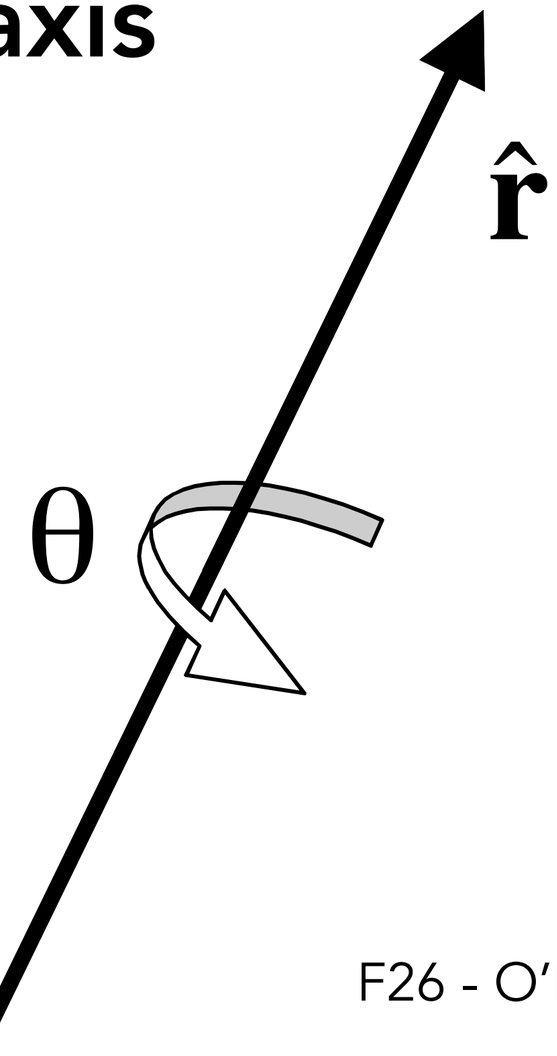
$$\mathbf{n}\times\mathbf{v} = [\mathbf{n}]\mathbf{v}$$

$$\mathbf{n}\times\mathbf{v} = (\mathbf{n}\times)\mathbf{v}$$

Exponential Maps

- Direct representation of arbitrary rotation
- AKA: *axis-angle*, angular displacement vector
- Rotate θ degrees about some axis
- Encode θ by length of vector

$$\theta = |\mathbf{r}|$$

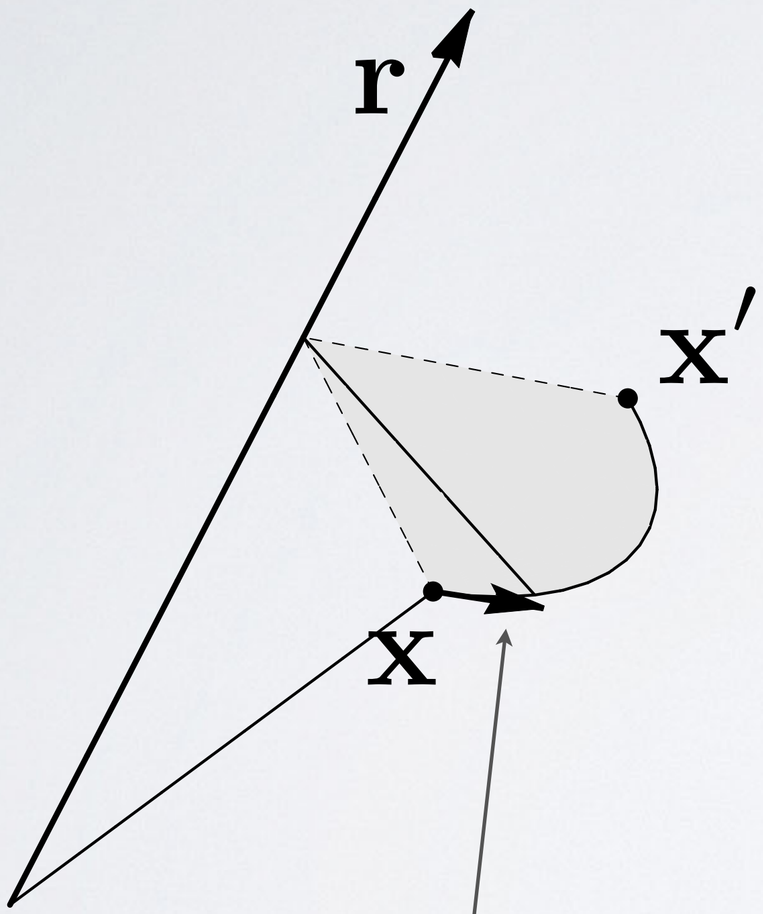


Exponential Maps

- Allows tumbling
- No gimbal-lock!
- Orientations are space within π-radius ball
- Nearly unique representation
- Singularities on shells at 2π
- Nice for interpolation

Exponential Maps

- Why exponential?



$$(\theta/n)\hat{\mathbf{r}} \times \mathbf{x}$$

- Instead of rotating once by θ, let's do n small rotations of θ/n
- Now the angle is small, so the rotated $\mathbf{x}$ is approximately

$$\mathbf{x} + (\theta/n)\hat{\mathbf{r}} \times \mathbf{x}$$

$$= \left(\mathbf{I} + \frac{(\hat{\mathbf{r}}\times)\theta}{n}\right)\mathbf{x}$$

- Do it n times and you get

$$\mathbf{x}' = \left(\mathbf{I} + \frac{(\hat{\mathbf{r}}\times)\theta}{n}\right)^n \mathbf{x}$$

Note: Material on gray slides is optional for cs184.

Exponential Maps

$$\mathbf{x}' = \lim_{n\to\infty} \left(\mathbf{I} + \frac{(\hat{\mathbf{r}}\times)\theta}{n} \right)^n \mathbf{x}$$

- Remind you of anything?

$$\lim_{n\to\infty} \left(1 + \frac{a}{n}\right)^n \text{ is a } \textit{definition} \text{ of } e^a$$

- So the rotation we want is the exponential of $(\hat{\mathbf{r}}\times)\theta$!

$$\mathbf{x}' = \underbrace{e^{(\hat{\mathbf{r}}\times)\theta}}_{\textbf{Rotation matrix!}} \mathbf{x}$$

- In fact you can just plug it into the infinite series...

Note: Material on gray slides is optional for cs184.

Exponential Maps

- Why exponential?
- Recall series expansion of e^x

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

Exponential Maps

- Why exponential?
- Recall series expansion of e^x
- Euler: what happens if you put in $i\theta$ for x

$$e^{i\theta} = 1 + \frac{i\theta}{1!} + \frac{-\theta^2}{2!} + \frac{-i\theta^3}{3!} + \frac{\theta^4}{4!} + \cdots$$

$$= \left(1 + \frac{-\theta^2}{2!} + \frac{\theta^4}{4!} + \cdots\right) + i\left(\frac{\theta}{1!} + \frac{-\theta^3}{3!} + \cdots\right)$$

$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

Exponential Maps

- Do same trick with matrices!

$$e^{(\hat{\mathbf{r}}\times)\theta} = \mathbf{I} + \frac{(\hat{\mathbf{r}}\times)\theta}{1!} + \frac{(\hat{\mathbf{r}}\times)^2\theta^2}{2!} + \frac{(\hat{\mathbf{r}}\times)^3\theta^3}{3!} + \frac{(\hat{\mathbf{r}}\times)^4\theta^4}{4!} + \cdots$$

But notice that: $(\hat{\mathbf{r}}\times)^3 = -(\hat{\mathbf{r}}\times)$

$$e^{(\hat{\mathbf{r}}\times)\theta} = \mathbf{I} + \frac{(\hat{\mathbf{r}}\times)\theta}{1!} + \frac{(\hat{\mathbf{r}}\times)^2\theta^2}{2!} + \frac{-(\hat{\mathbf{r}}\times)\theta^3}{3!} + \frac{-(\hat{\mathbf{r}}\times)^2\theta^4}{4!} + \cdots$$

Exponential Maps

$$e^{(\hat{\mathbf{r}}\times)\theta} = \mathbf{I} + \frac{(\hat{\mathbf{r}}\times)\theta}{1!} + \frac{(\hat{\mathbf{r}}\times)^2\theta^2}{2!} + \frac{-(\hat{\mathbf{r}}\times)\theta^3}{3!} + \frac{-(\hat{\mathbf{r}}\times)^2\theta^4}{4!} + \cdots$$

$$e^{(\hat{\mathbf{r}}\times)\theta} = (\hat{\mathbf{r}}\times)\left(\frac{\theta}{1!} - \frac{\theta^3}{3!} + \cdots\right) + \mathbf{I} + (\hat{\mathbf{r}}\times)^2\left(+\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \cdots\right)$$

$$e^{(\hat{\mathbf{r}}\times)\theta} = (\hat{\mathbf{r}}\times)\sin(\theta) + \mathbf{I} + (\hat{\mathbf{r}}\times)^2(1 - \cos(\theta))$$

Compare to Rodrigue's Formula

Note: Material on gray slides is optional for cs184.

Rotation Matrices

- Eigen system
  - One real eigenvalue
  - Real axis is axis of rotation
  - Imaginary values are 2D rotation as complex number

- Logarithmic formula

$$(\hat{\mathbf{r}}\times) = \ln(\mathbf{R}) = \frac{\theta}{2\sin\theta}(\mathbf{R} - \mathbf{R}^{\mathsf{T}})$$

$$\theta = \cos^{-1}\left(\frac{\mathrm{Tr}(\mathbf{R}) - 1}{2}\right)$$

Similar formulae as for exponential...

Note: Material on gray slides is optional for cs184.

Quaternions

- More popular than exponential maps
- Natural extension of $e^{i\theta} = \cos(\theta) + i\sin(\theta)$
- Due to Hamilton (1843)
  - Interesting bit of drama….

Quaternions

- Three-Times-As-Complex Numbers

$$\mathrm{q} = (z_1, z_2, z_3, s) = (\mathbf{z}, s)$$

$$\mathrm{q} = iz_1 + jz_2 + kz_3 + s$$

$$i^2 = j^2 = k^2 = -1$$

$$\begin{aligned} ij &= k & ji &= -k \\ jk &= i & kj &= -i \\ ki &= j & ik &= -j \end{aligned}$$

Quaternions

- Multiplication natural consequence of defn.

$$\mathsf{q} \cdot \mathsf{p} = \left(\mathbf{z}_q s_p + \mathbf{z}_p s_q + \mathbf{z}_p \times \mathbf{z}_q \;\;,\;\; s_p s_q - \mathbf{z}_p \cdot \mathbf{z}_q \right)$$

- Conjugate

$$\mathsf{q}^* = \left(-\mathbf{z}, s \right)$$

- Magnitude

$$||\mathsf{q}||^2 = \mathbf{z} \cdot \mathbf{z} + s^2 = \mathsf{q} \cdot \mathsf{q}^*$$

Note: Material on gray slides is optional for cs184.

Quaternions

- Vectors as quaternions

$$\mathsf{v} = (\mathbf{v}, 0)$$

- Rotations as quaternions

$$\mathsf{r} = (\hat{\mathbf{r}} \sin\frac{\theta}{2}, \cos\frac{\theta}{2})$$

- Rotating a vector

$$\mathsf{x}' = \mathsf{r} \cdot \mathsf{x} \cdot \mathsf{r}^*$$

- Composing rotations

$$\mathsf{r} = \mathsf{r}_1 \cdot \mathsf{r}_2$$ ← Compare to Exp. Map

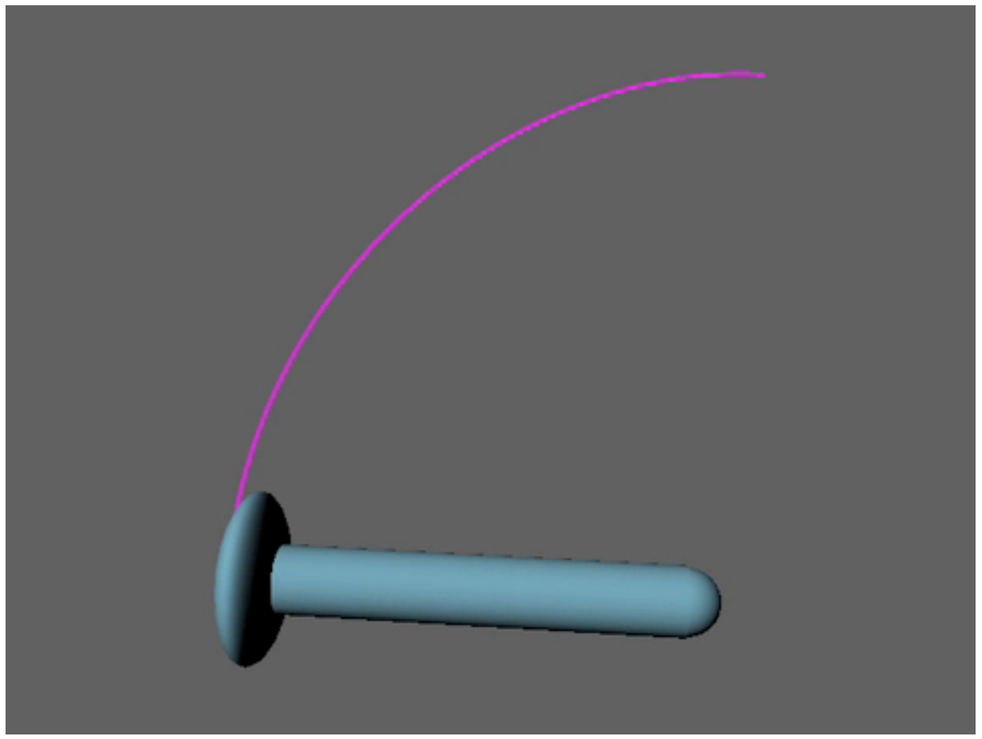
Note: Material on gray slides is optional for cs184.

Quaternions

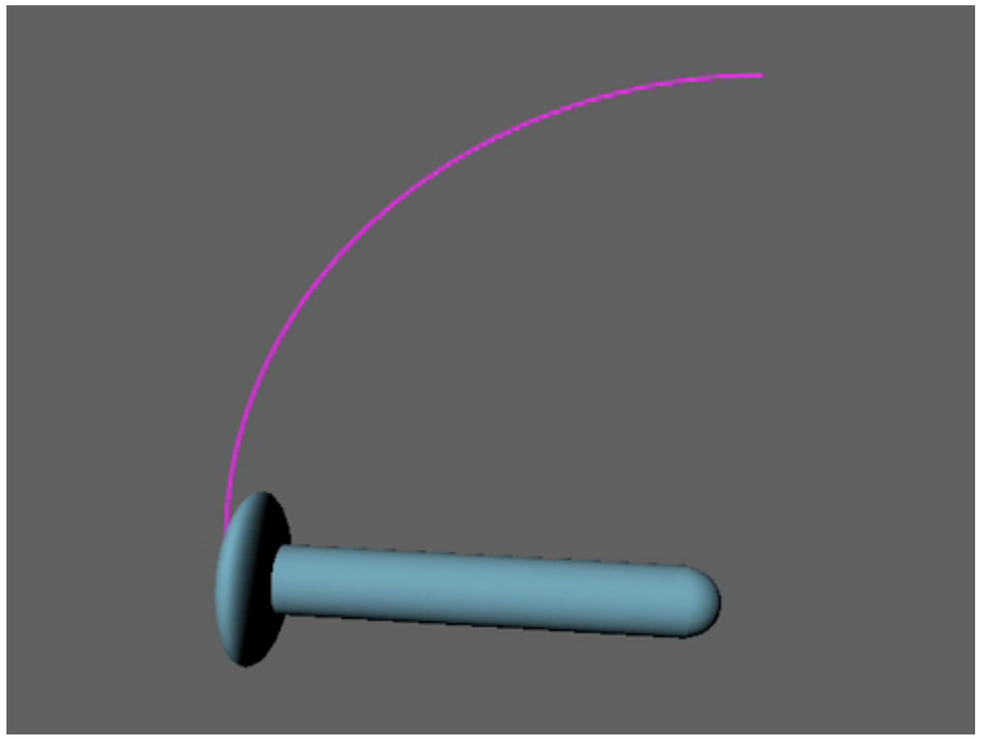
- No tumbling
- No gimbal-lock
- Orientations are "double unique"
- Surface of a 3-sphere in 4D $||\mathbf{r}|| = 1$
- Nice for interpolation

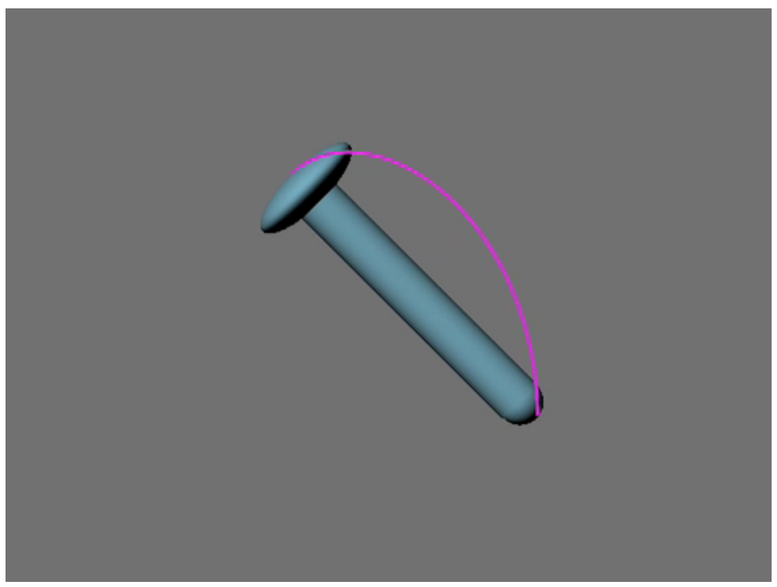
Interpolation

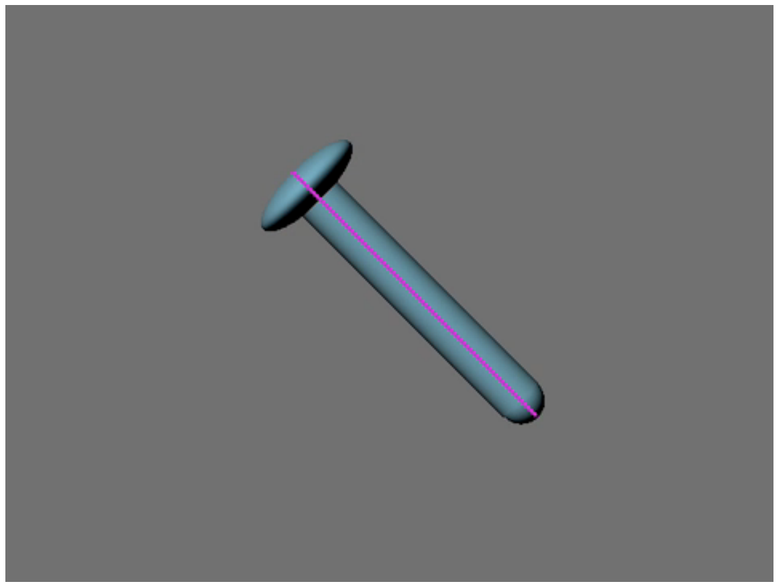
Euler angles



Exponential maps or quaternions

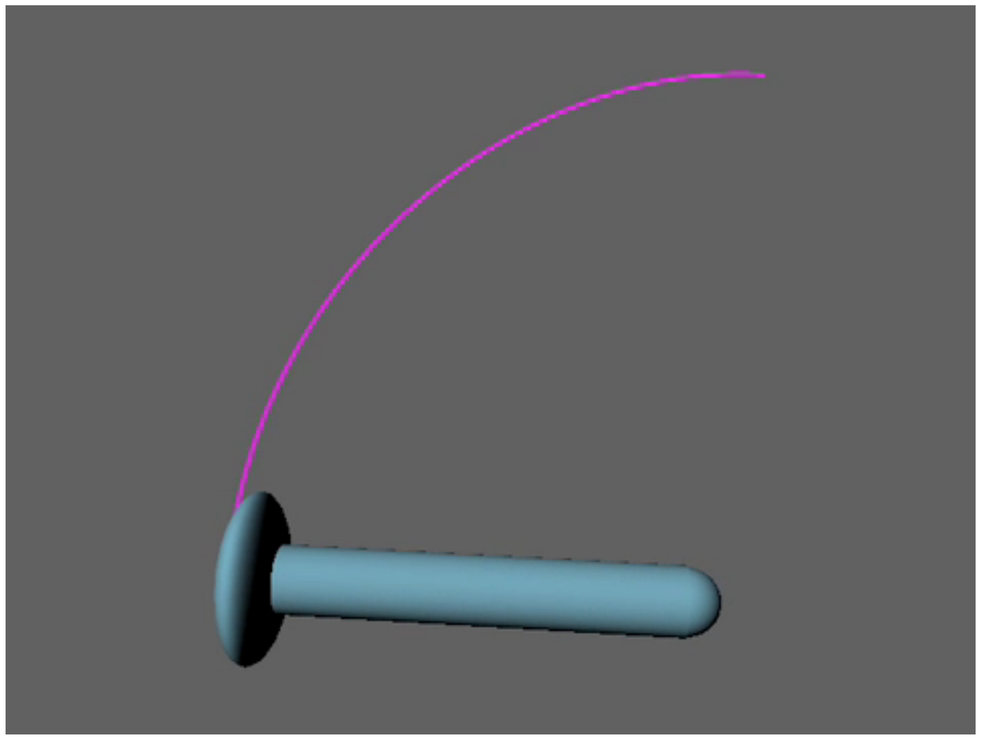




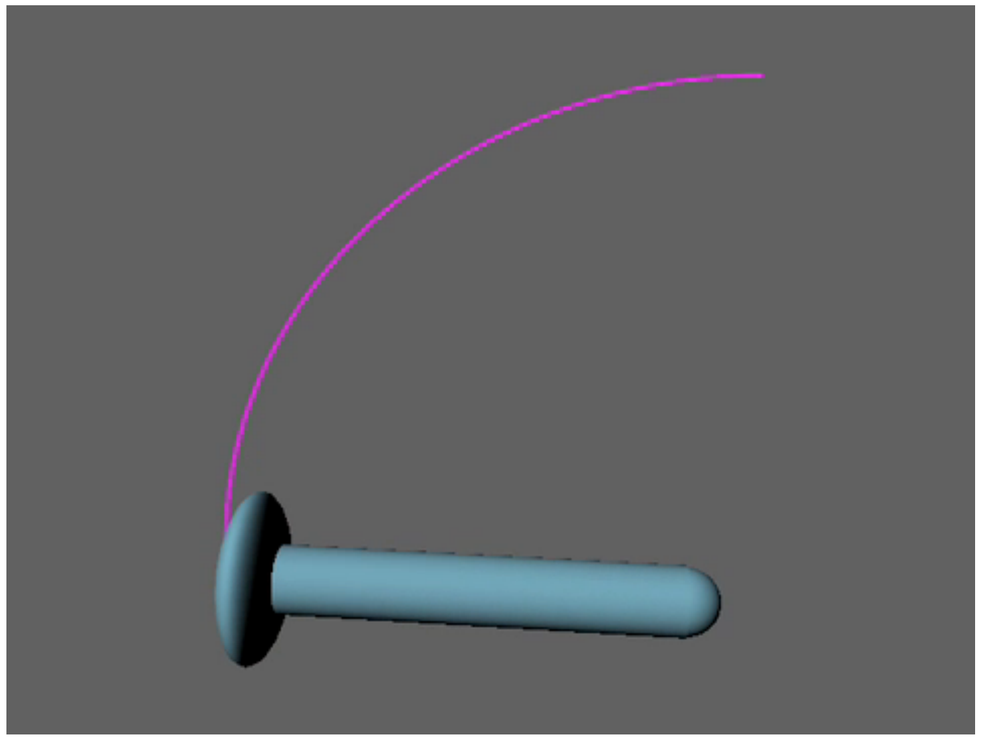


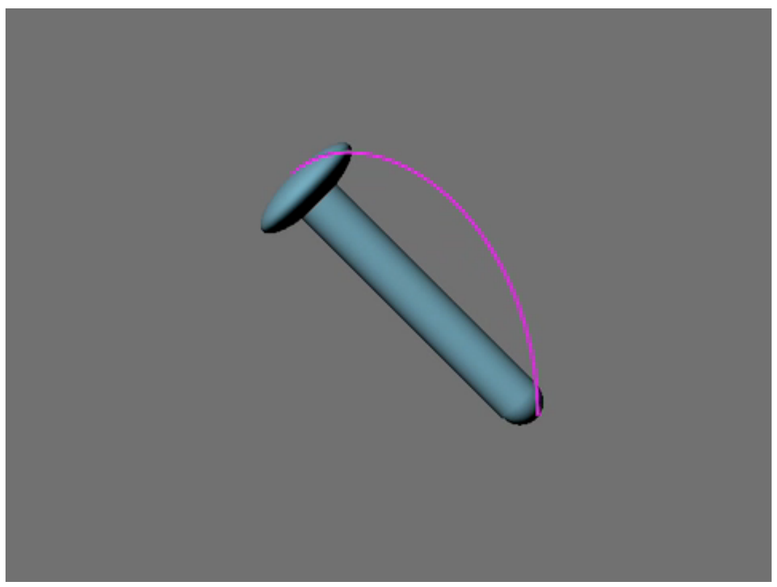
Interpolation

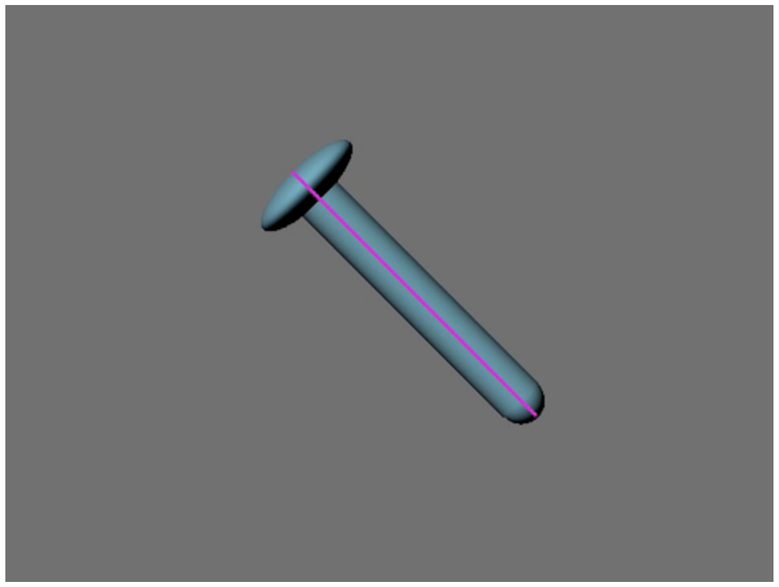
Euler angles



Exponential maps or quaternions



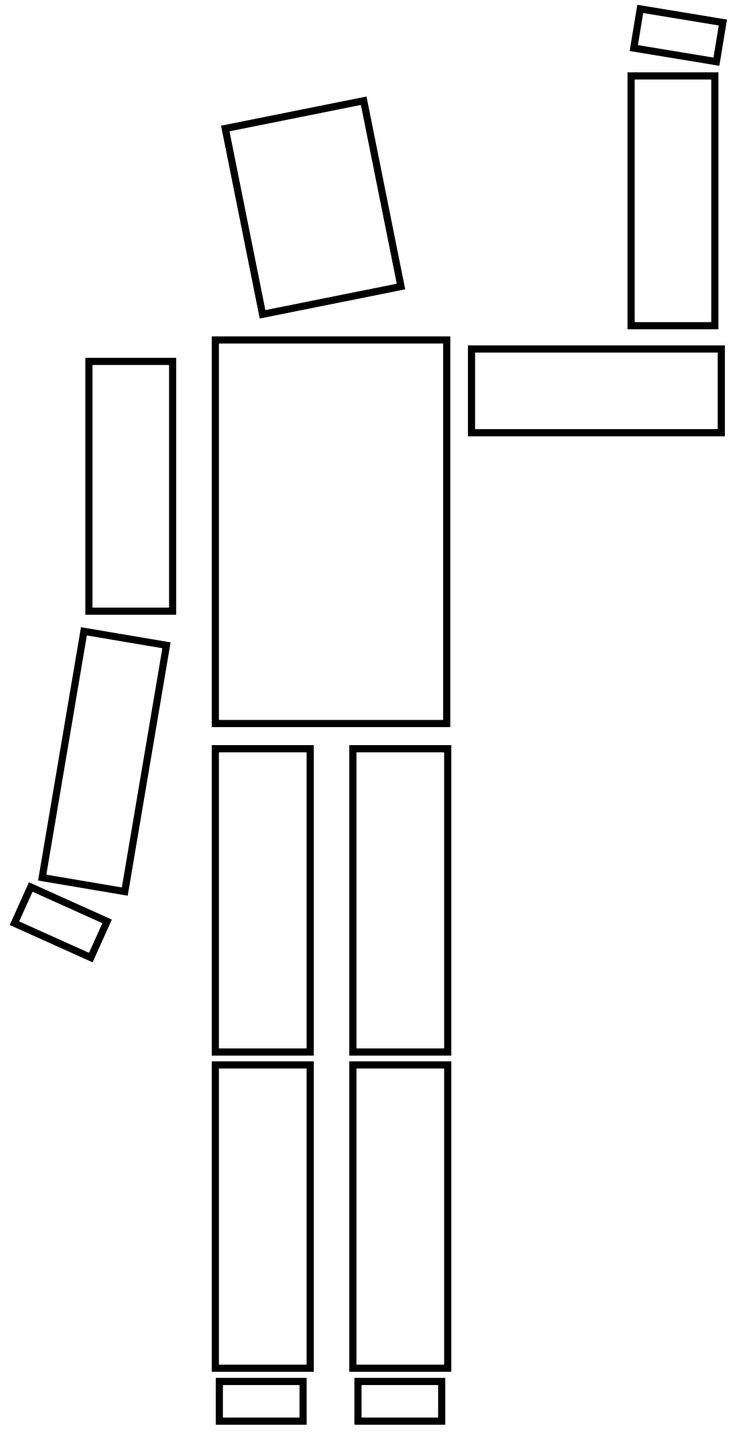




Hierarchical Transforms

Skeleton - Linear Representation

head
torso
right upper arm
right lower arm
right hand
left upper arm
left lower arm
left hand
right upper leg
right lower leg
right foot
left upper leg
left lower leg
left foot



Linear Representation

Each shape associated with its own transform

A single edit can require updating many transforms

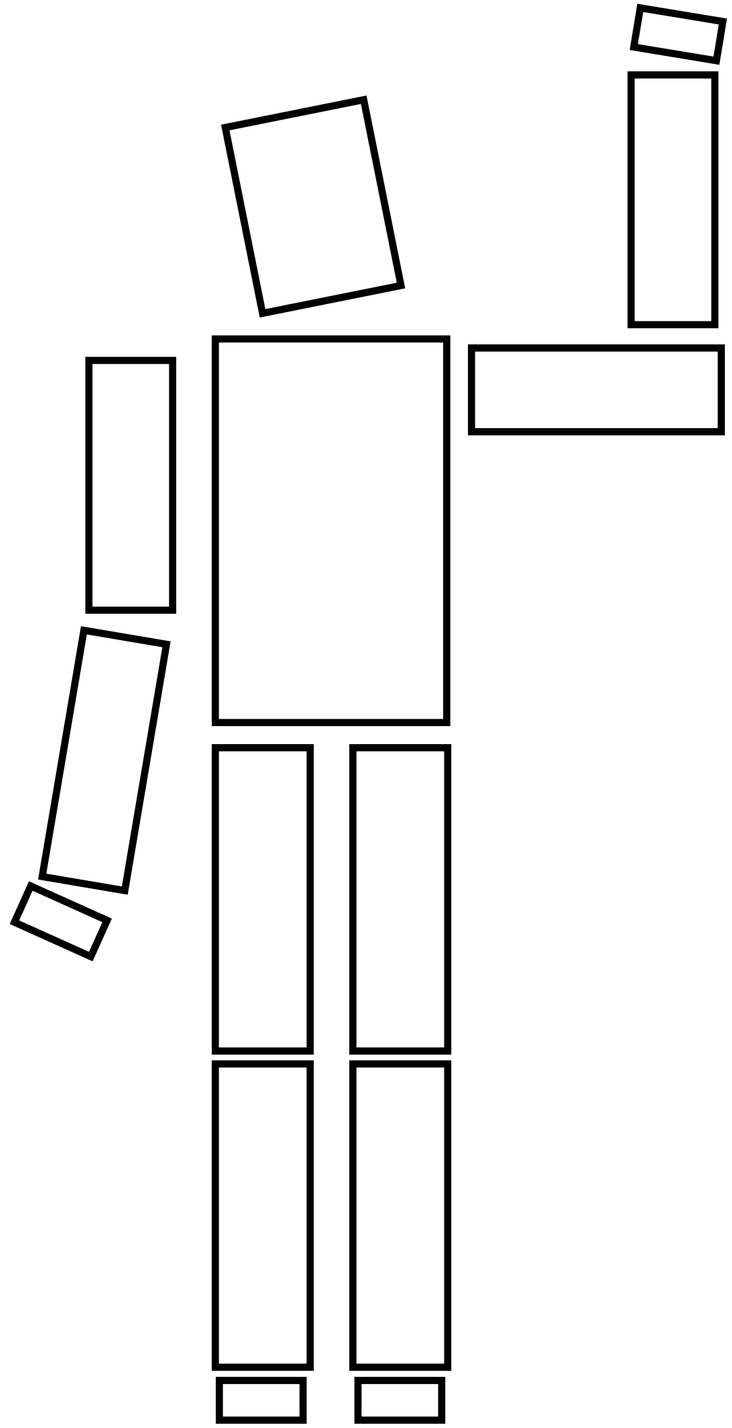
- E.g. raising arm requires updating transforms for all arm parts

Skeleton - Hierarchical Representation

torso
- head
- right arm
  - upper arm
    - lower arm
      - hand
- left arm
  - upper arm
    - lower arm
      - hand

right leg
- upper leg
  - lower leg
    - foot

left leg
- upper leg
  - lower leg
    - foot

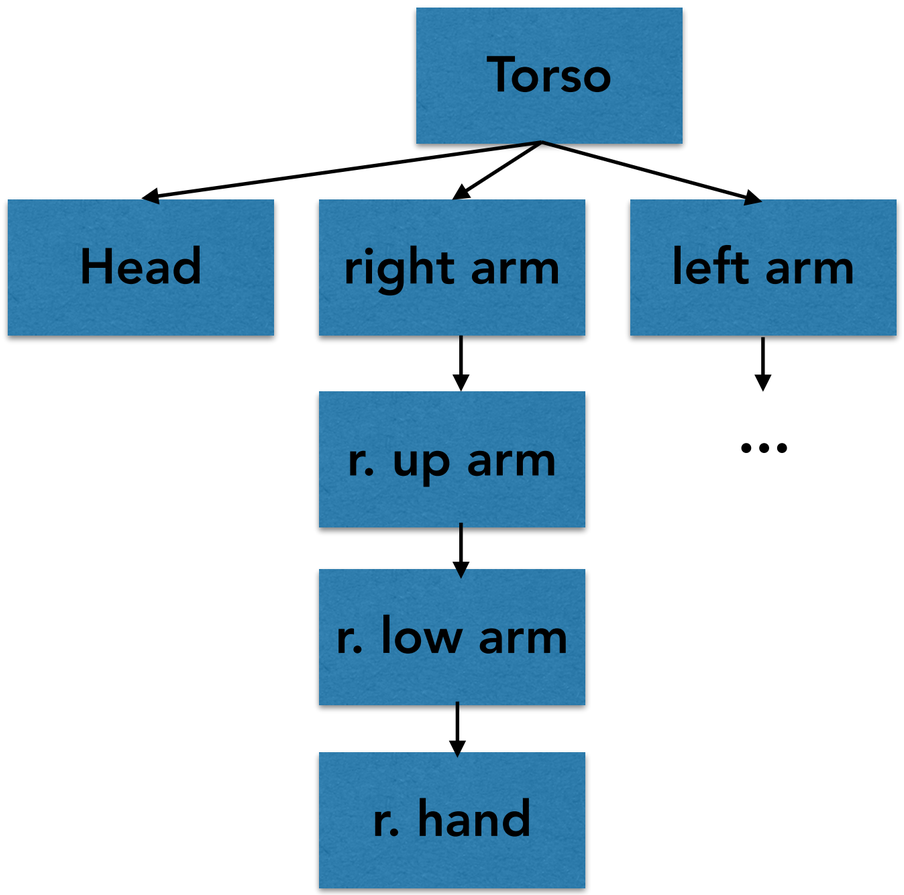


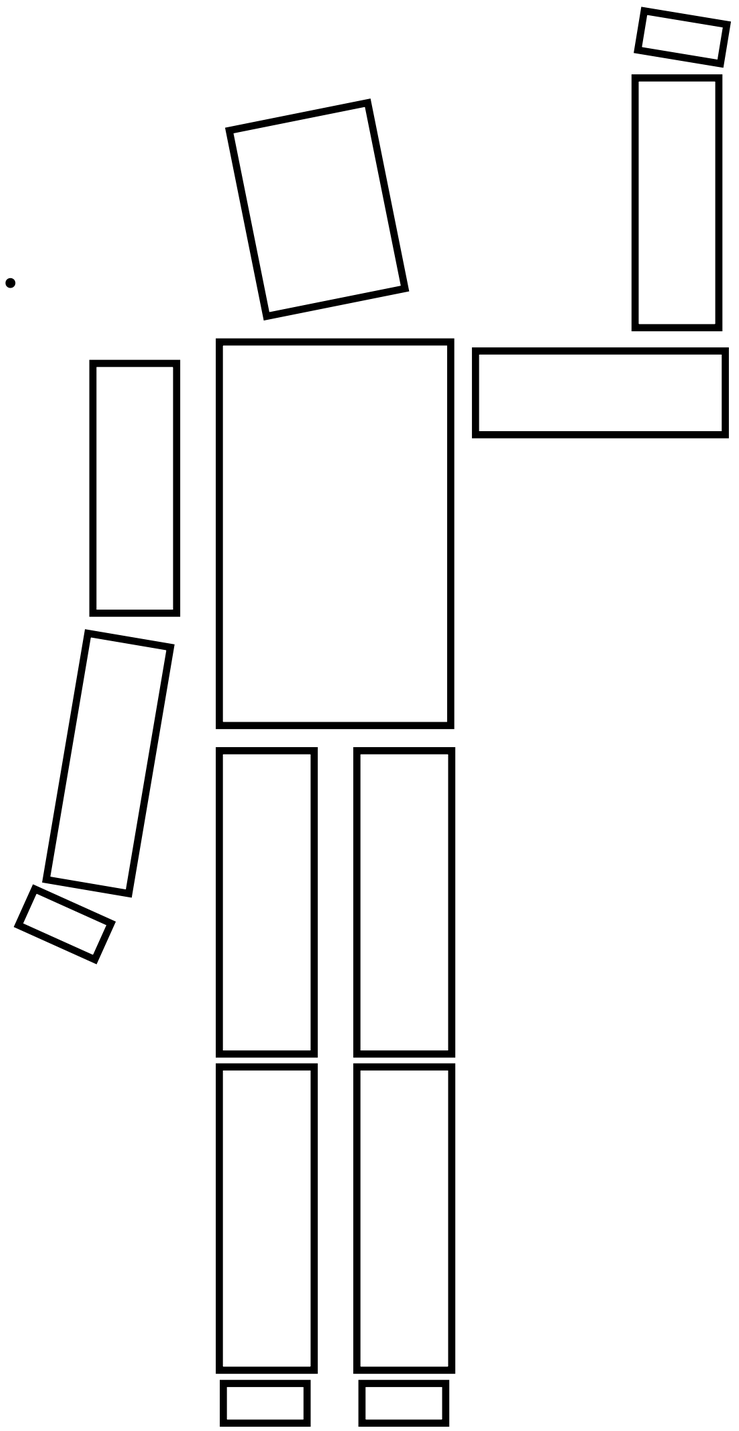
Skeleton - Hierarchical Representation

torso
- head
- right arm
  - upper arm
    - lower arm
      - hand
- left arm
  - upper arm
    - lower arm
      - hand

right leg
- upper leg
  - lower leg
    - foot

left leg
- upper leg
  - lower leg
    - foot





Hierarchical Representation

Hierarchical Representation

Grouped representation (tree)

Hierarchical Representation

Grouped representation (tree)

- Each group contains subgroups and/or shapes

Hierarchical Representation

Grouped representation (tree)

- Each group contains subgroups and/or shapes
- Each group is associated with a transform relative to parent group

Hierarchical Representation

Grouped representation (tree)

- Each group contains subgroups and/or shapes
- Each group is associated with a transform relative to parent group
- Transform on leaf-node shape is concatenation of all transforms on path from root node to leaf

Hierarchical Representation

Grouped representation (tree)

- Each group contains subgroups and/or shapes
- Each group is associated with a transform relative to parent group
- Transform on leaf-node shape is concatenation of all transforms on path from root node to leaf
- Changing a group's transform affects all parts

Hierarchical Representation

Grouped representation (tree)

- Each group contains subgroups and/or shapes
- Each group is associated with a transform relative to parent group
- Transform on leaf-node shape is concatenation of all transforms on path from root node to leaf
- Changing a group's transform affects all parts
  - Allows high level editing by changing only one node

Hierarchical Representation

Grouped representation (tree)

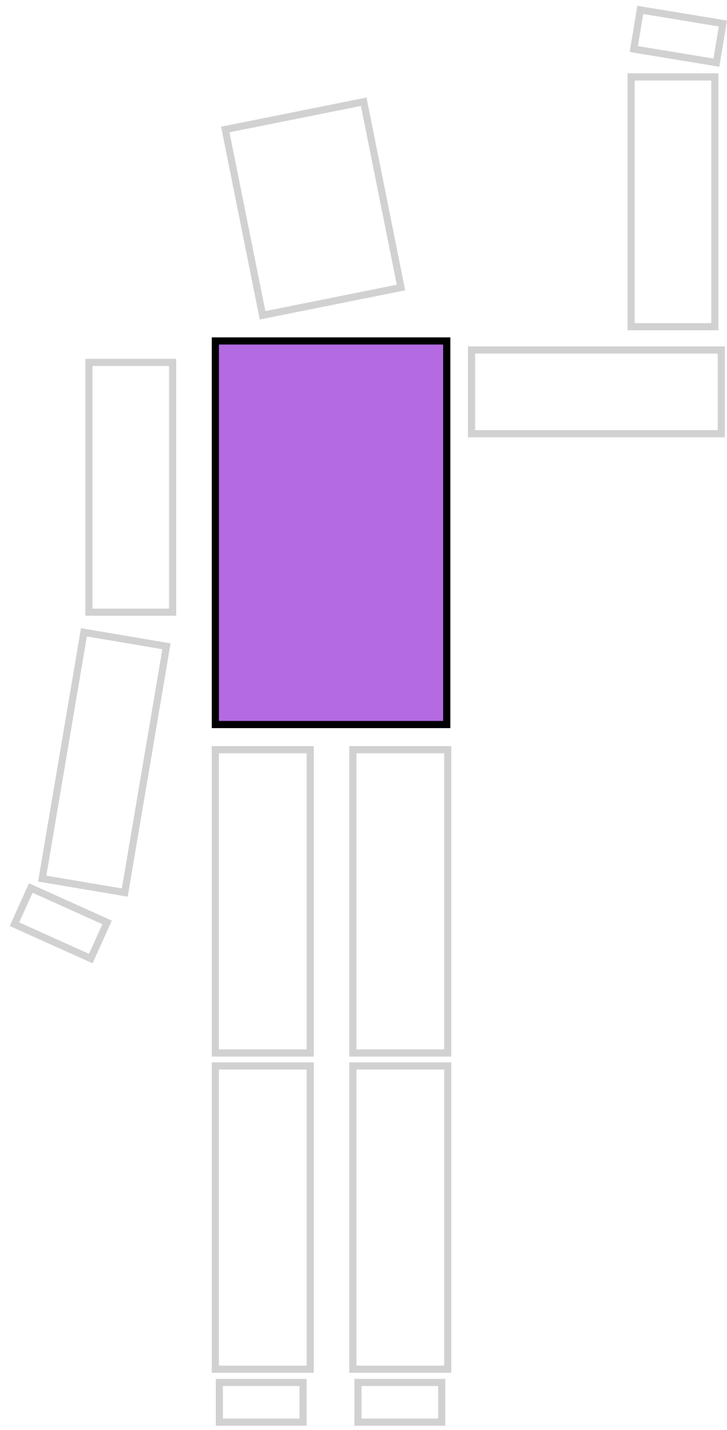
- Each group contains subgroups and/or shapes
- Each group is associated with a transform relative to parent group
- Transform on leaf-node shape is concatenation of all transforms on path from root node to leaf
- Changing a group's transform affects all parts
  - Allows high level editing by changing only one node
  - E.g. raising left arm requires changing only one transform for that group

Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
```

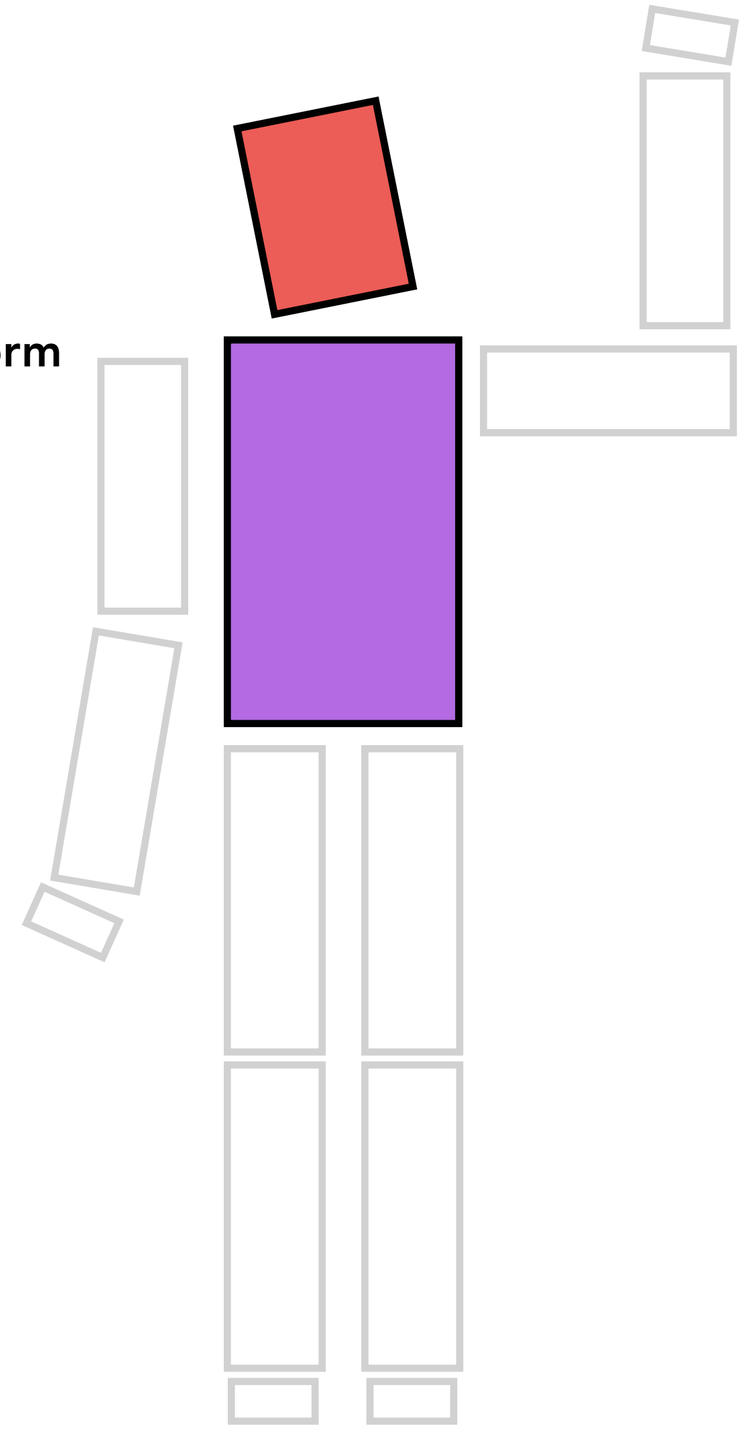
Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
```



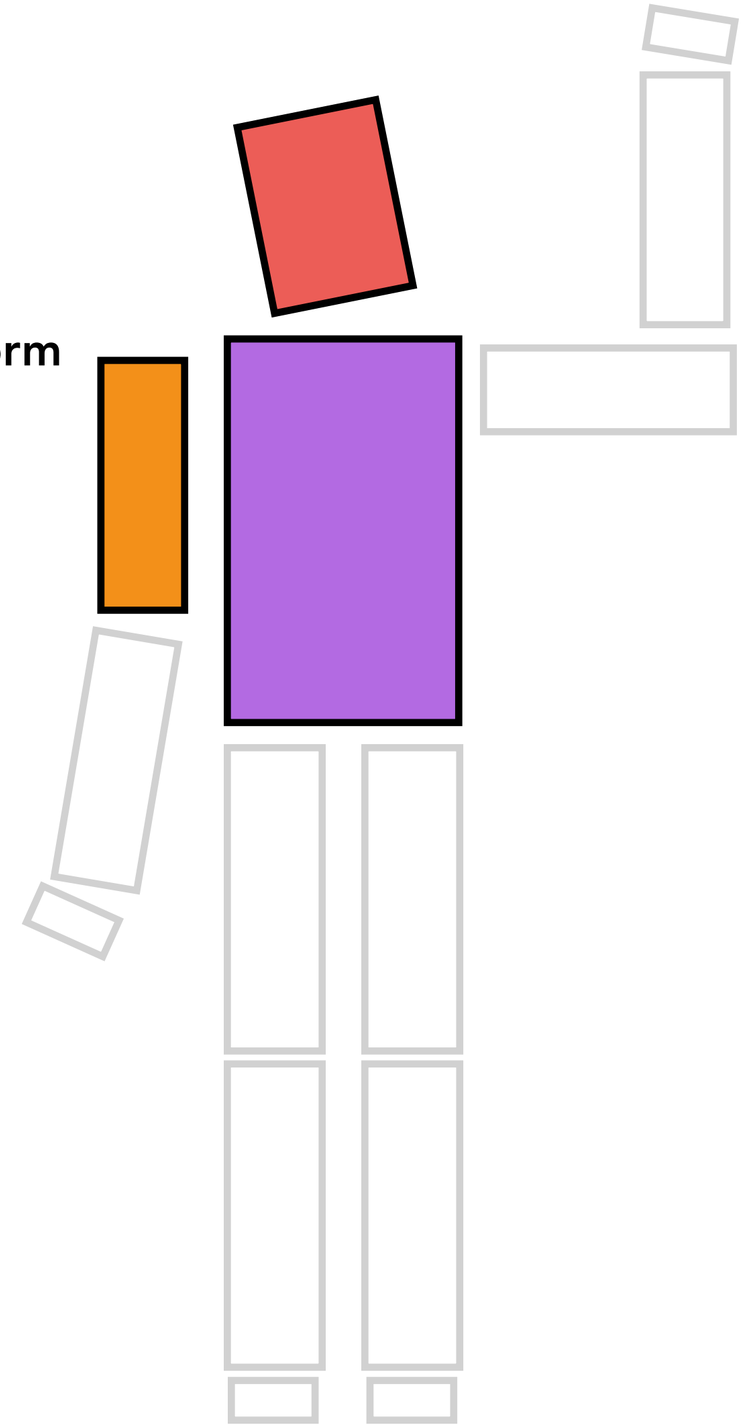
Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
```



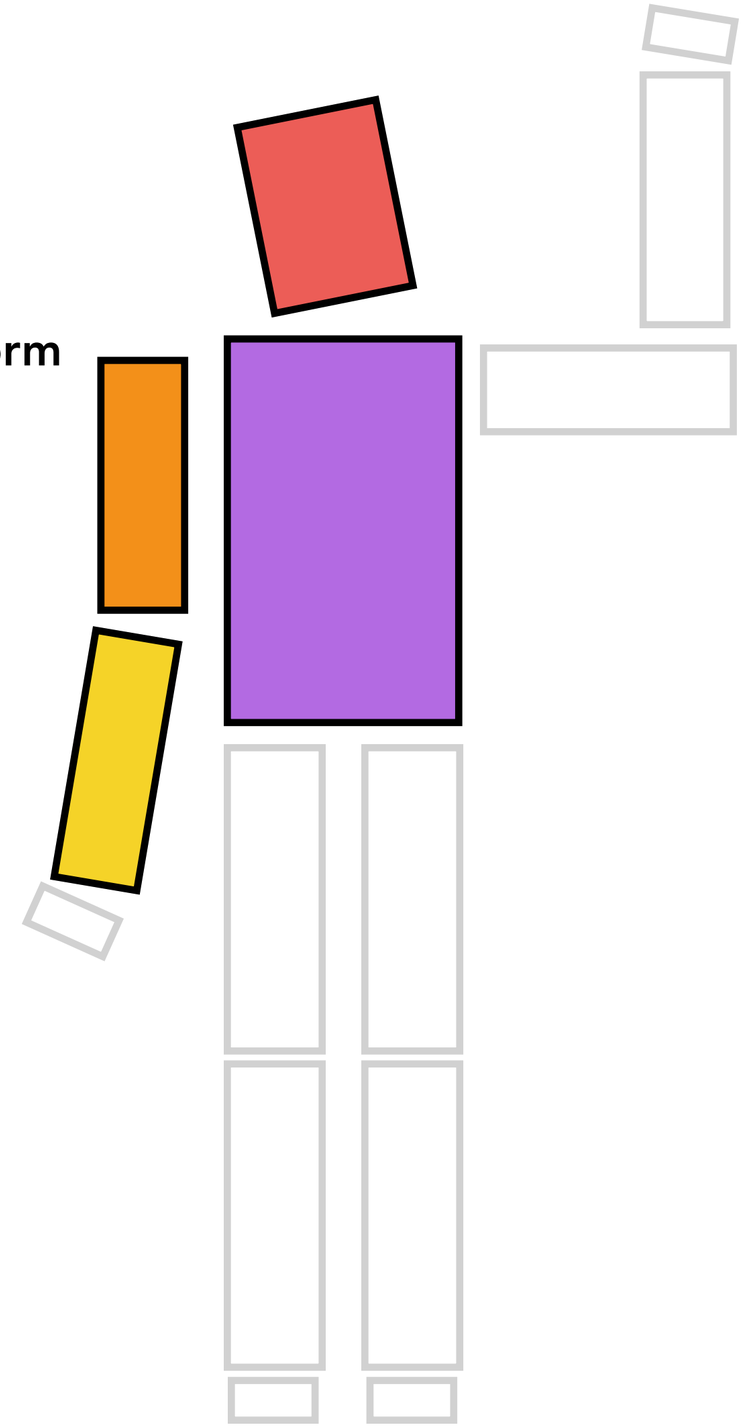
Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
```



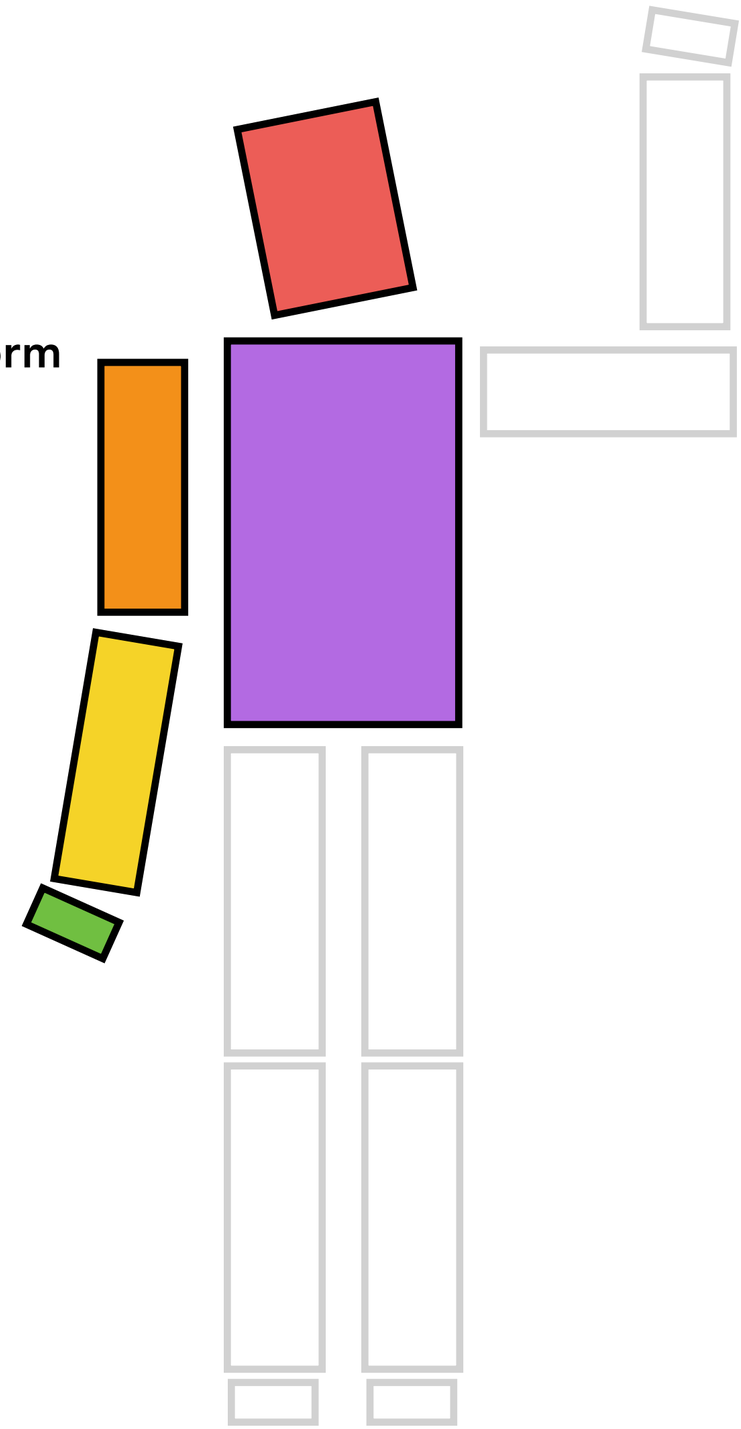
Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
        pushmatrix();
            translate(0, -3);
            rotate(elbowRotation);
            drawLowerArm();
```



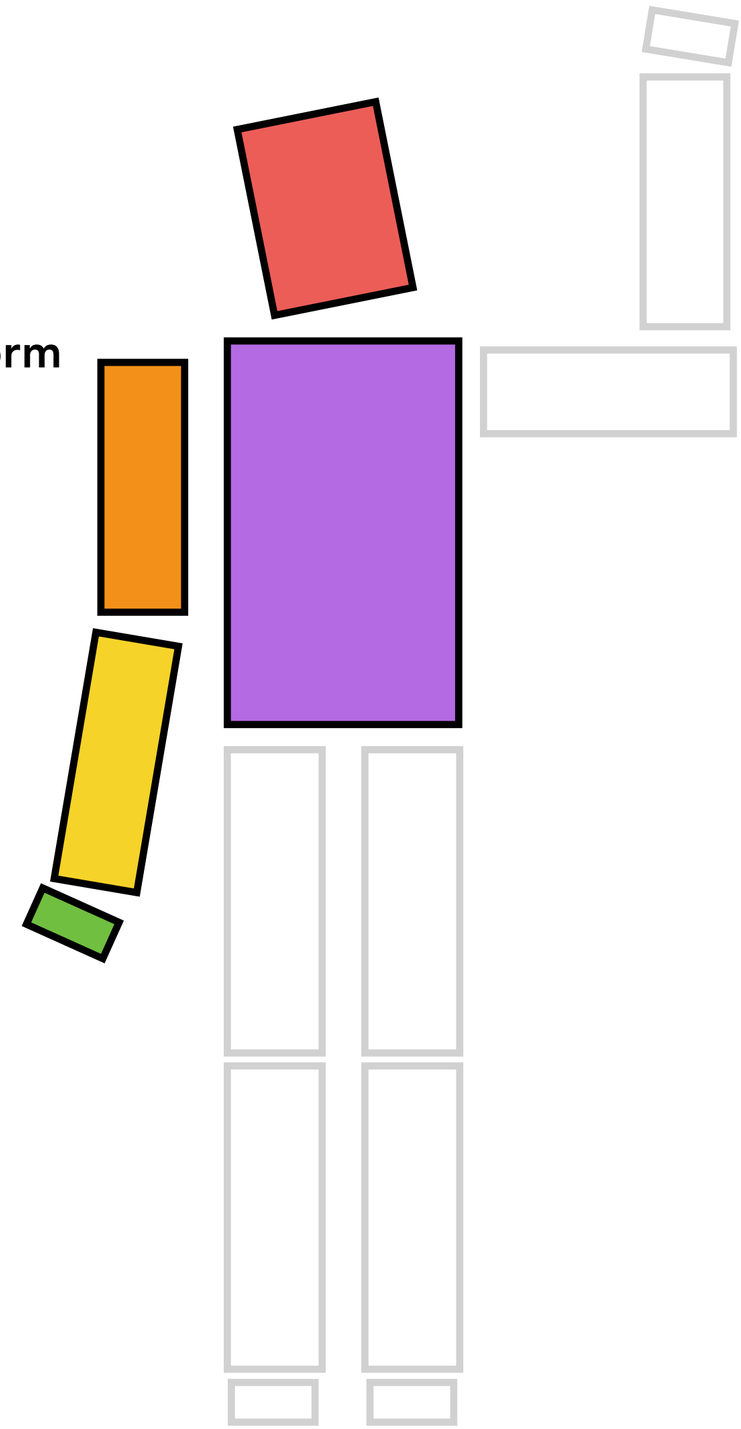
Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
        pushmatrix();
            translate(0, -3);
            rotate(elbowRotation);
            drawLowerArm();
            pushmatrix();
                translate(0, -3);
                rotate(wristRotation);
                drawHand();
            popmatrix();
```



Skeleton - Hierarchical Representation

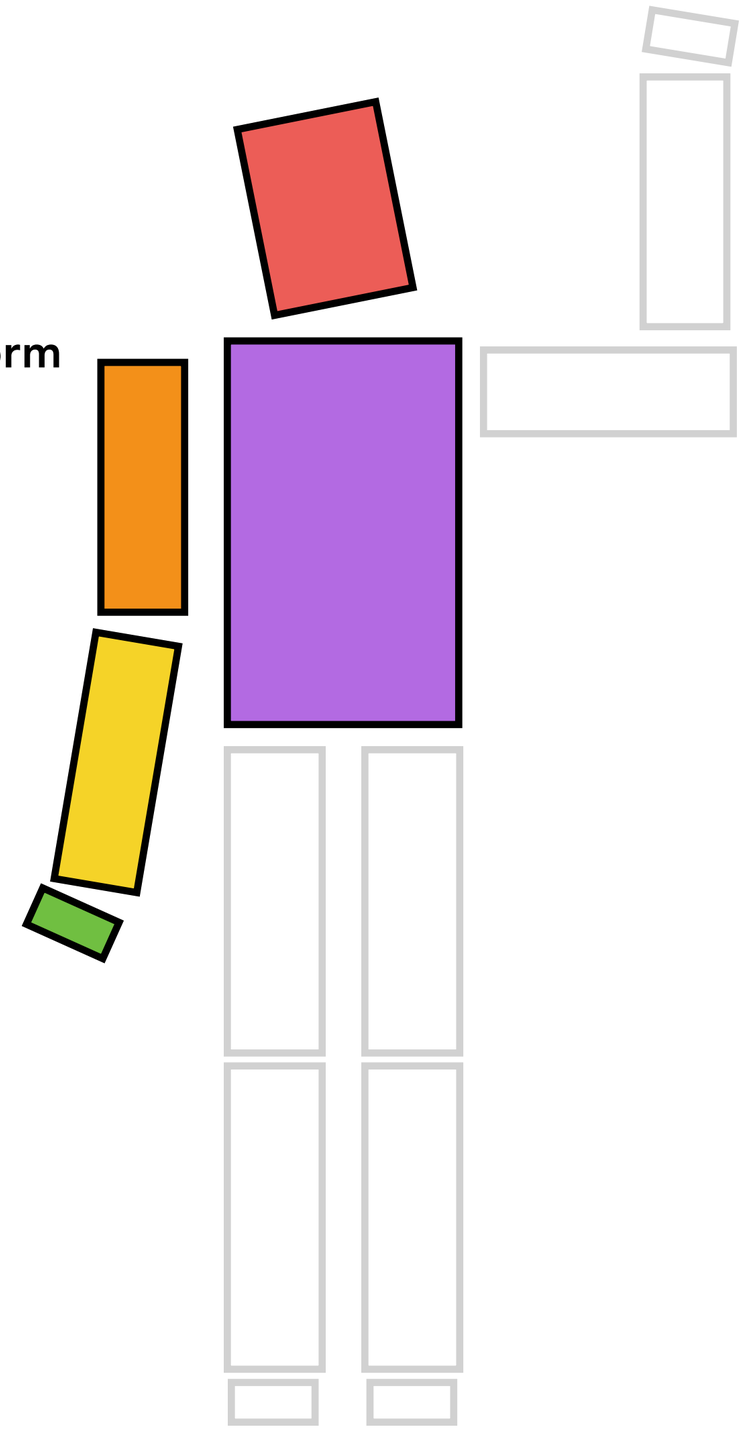
```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
        pushmatrix();
            translate(0, -3);
            rotate(elbowRotation);
            drawLowerArm();
            pushmatrix();
                translate(0, -3);
                rotate(wristRotation);
                drawHand();
            popmatrix();
        popmatrix();
    popmatrix();
    ....
```



Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
        pushmatrix();
            translate(0, -3);
            rotate(elbowRotation);
            drawLowerArm();
            pushmatrix();
                translate(0, -3);
                rotate(wristRotation);
                drawHand();
            popmatrix();
        popmatrix();
popmatrix();
....
```

right hand

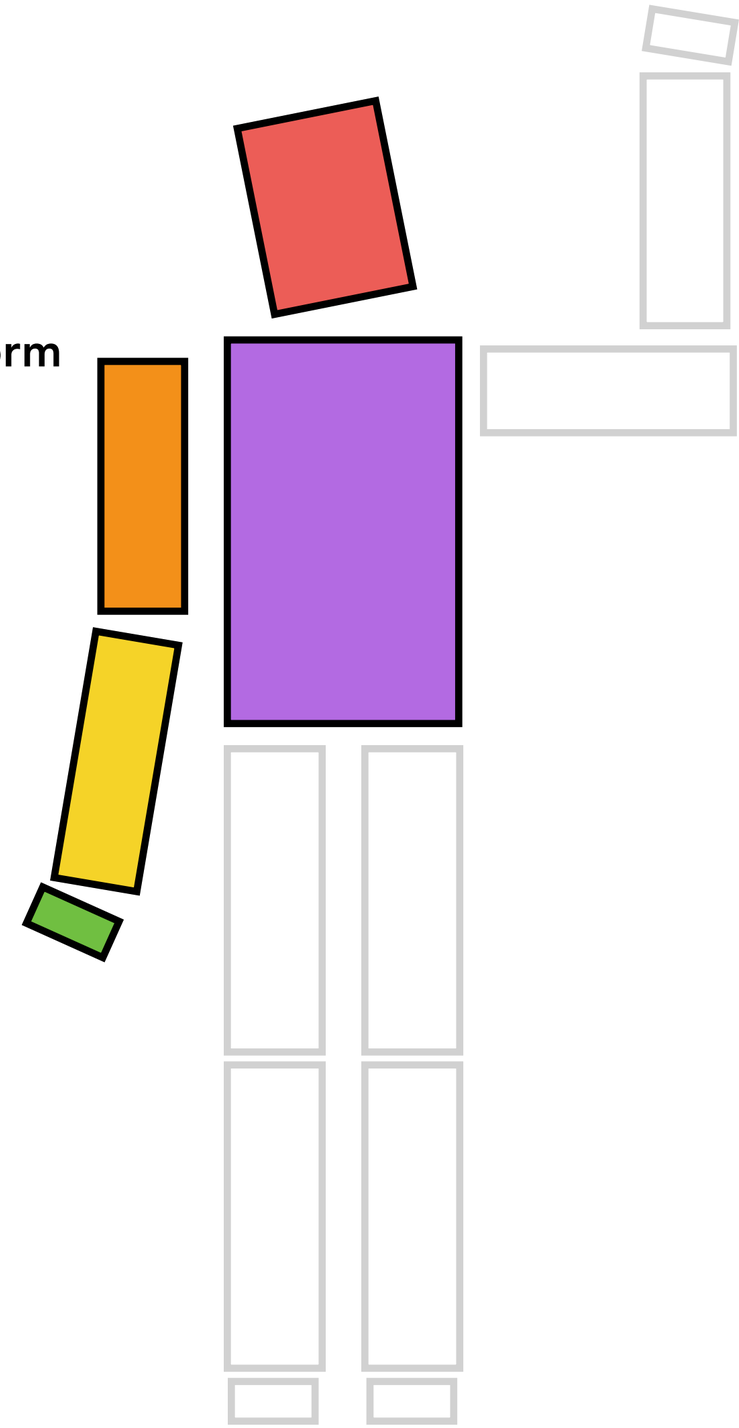


Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
        pushmatrix();  ----------------------
            translate(0, -3);
            rotate(elbowRotation);
            drawLowerArm();
            pushmatrix();  ---------
               translate(0, -3);
               rotate(wristRotation);
               drawHand();
            popmatrix();  ---------
        popmatrix();  ----------------------
popmatrix();
....
```

right lower arm group

right hand



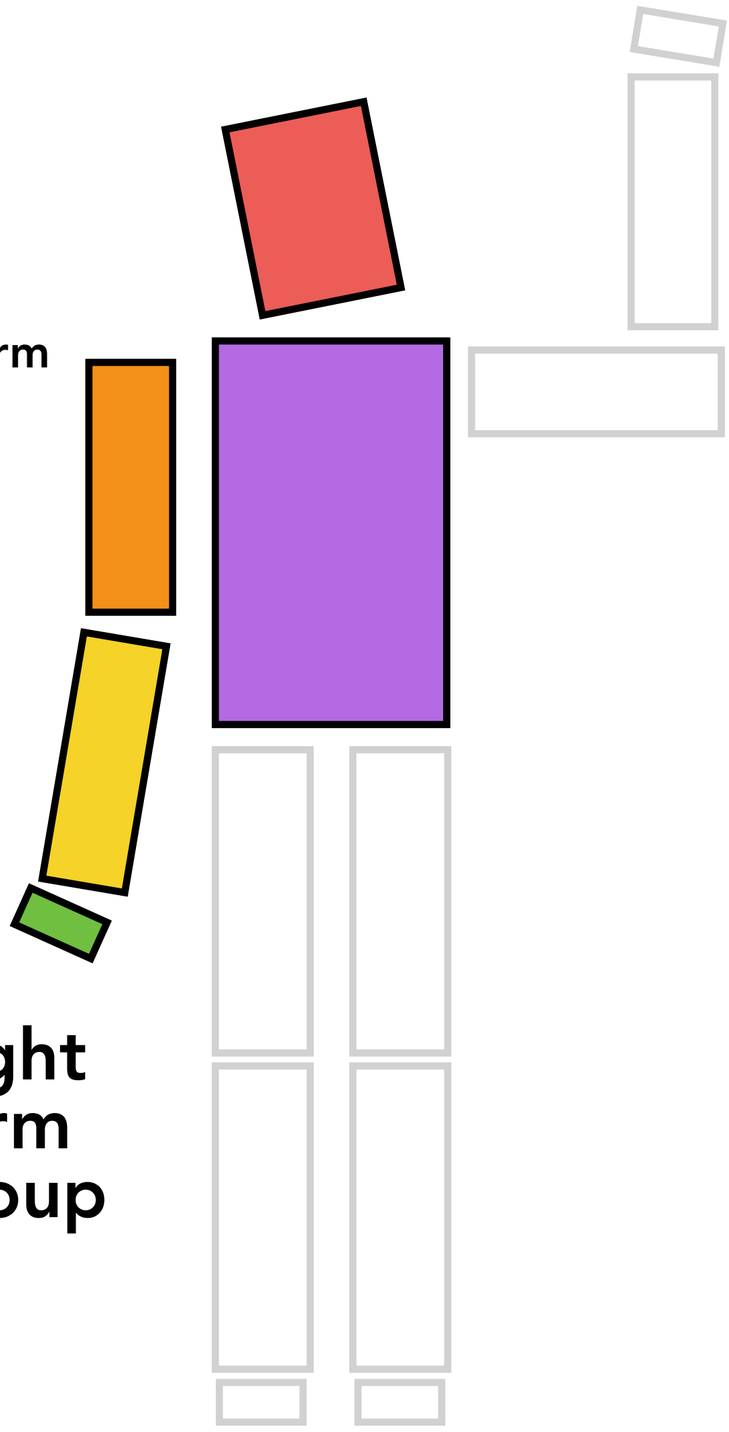
Skeleton - Hierarchical Representation

```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
            pushmatrix();
                translate(0, -3);
                rotate(elbowRotation);
                drawLowerArm();
                pushmatrix();
                    translate(0, -3);
                    rotate(wristRotation);
                    drawHand();
                popmatrix();
            popmatrix();
        popmatrix();
    ....
```

right hand

right lower arm group

right arm group



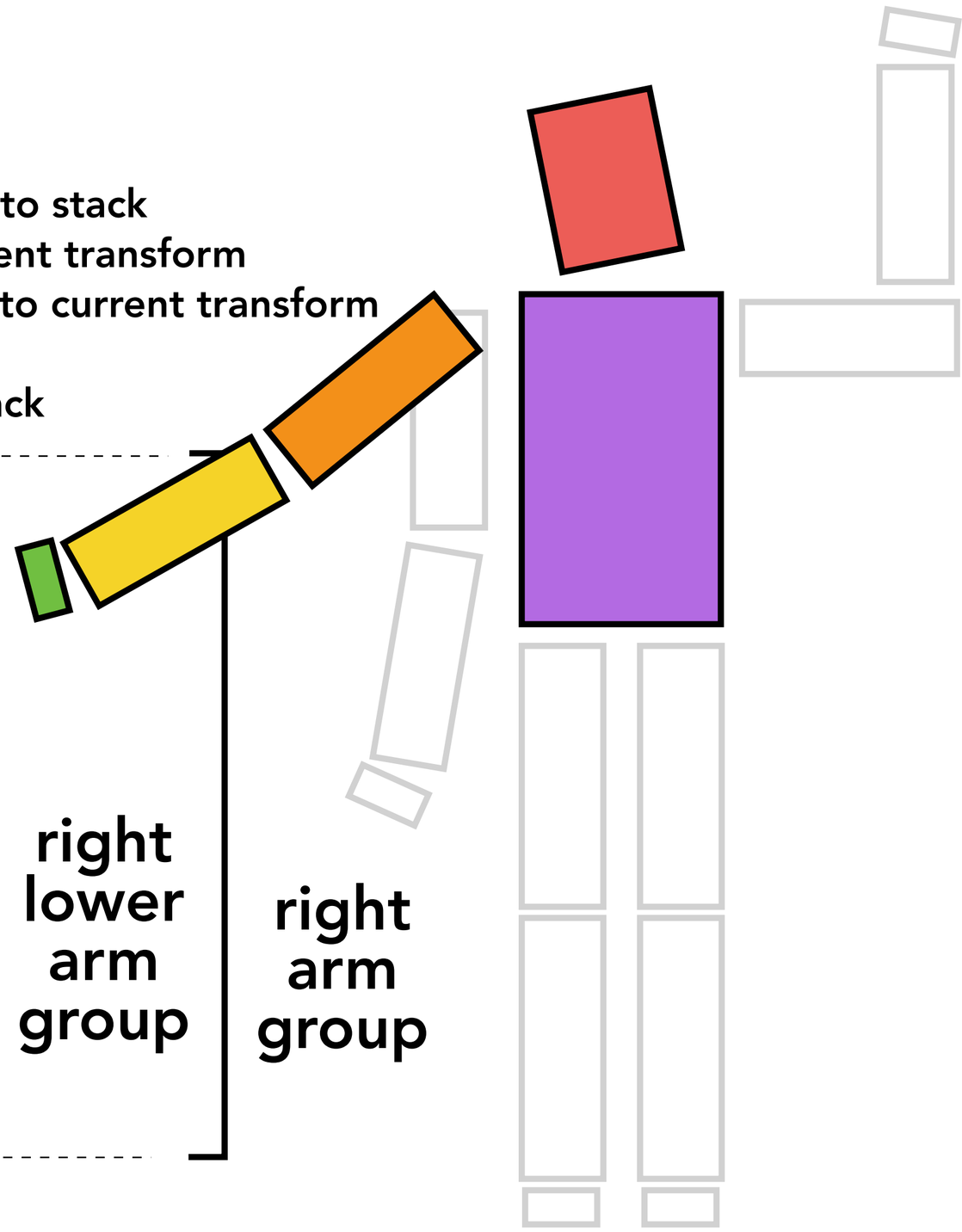
Skeleton - Hierarchical Representation

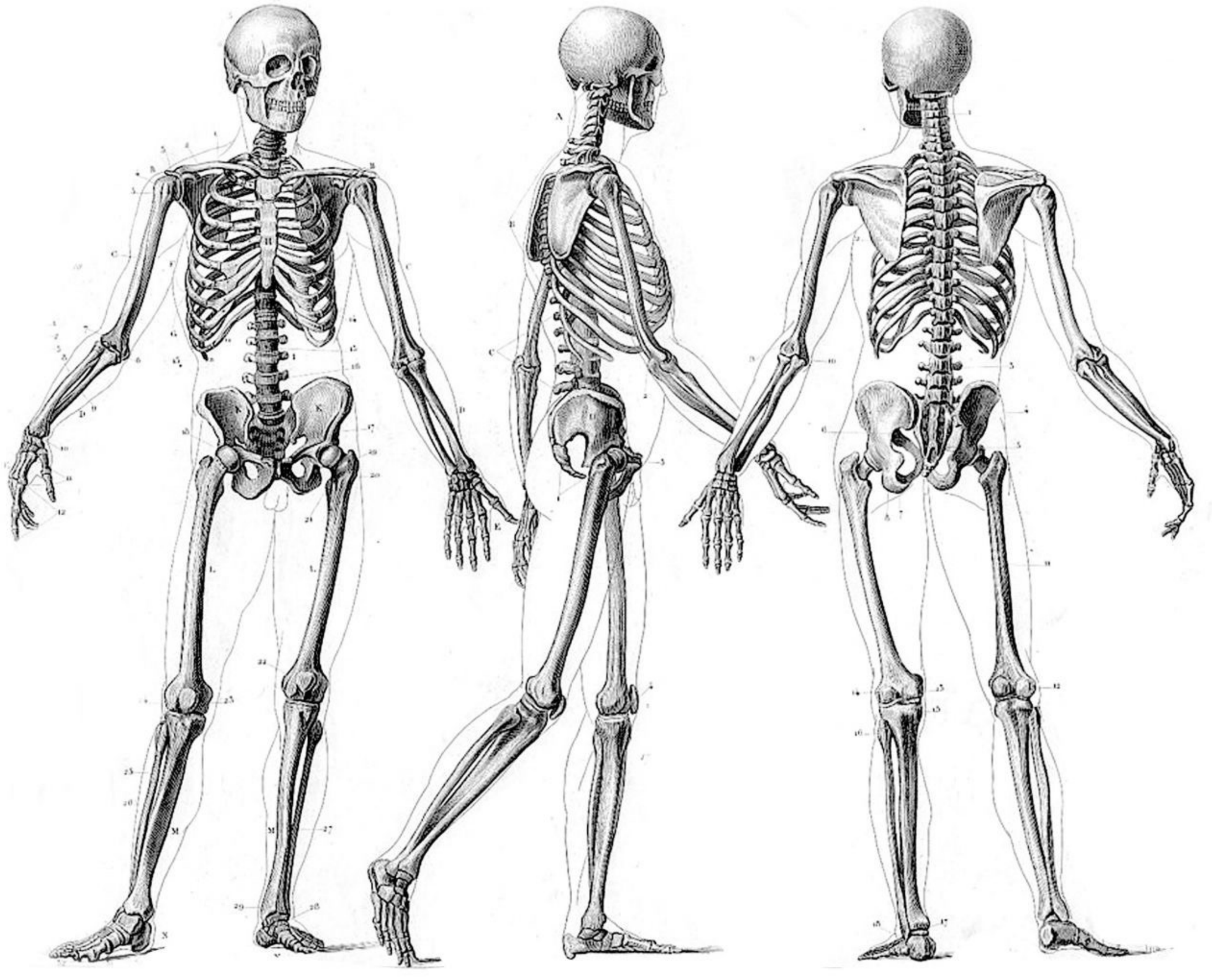
```
translate(0, 10);
    drawTorso();
        pushmatrix();  // push a copy of transform onto stack
            translate(0, 5);  // right-multiply onto current transform
            rotate(headRotation);  // right-multiply onto current transform
            drawHead();
        popmatrix();  // pop current transform off stack
        pushmatrix();
            translate(-2, 3);
            rotate(rightShoulderRotation);
            drawUpperArm();
        pushmatrix();
            translate(0, -3);
            rotate(elbowRotation);
            drawLowerArm();
            pushmatrix();
                translate(0, -3);
                rotate(wristRotation);
                drawHand();
            popmatrix();
        popmatrix();
popmatrix();
....
```

right hand

right lower arm group

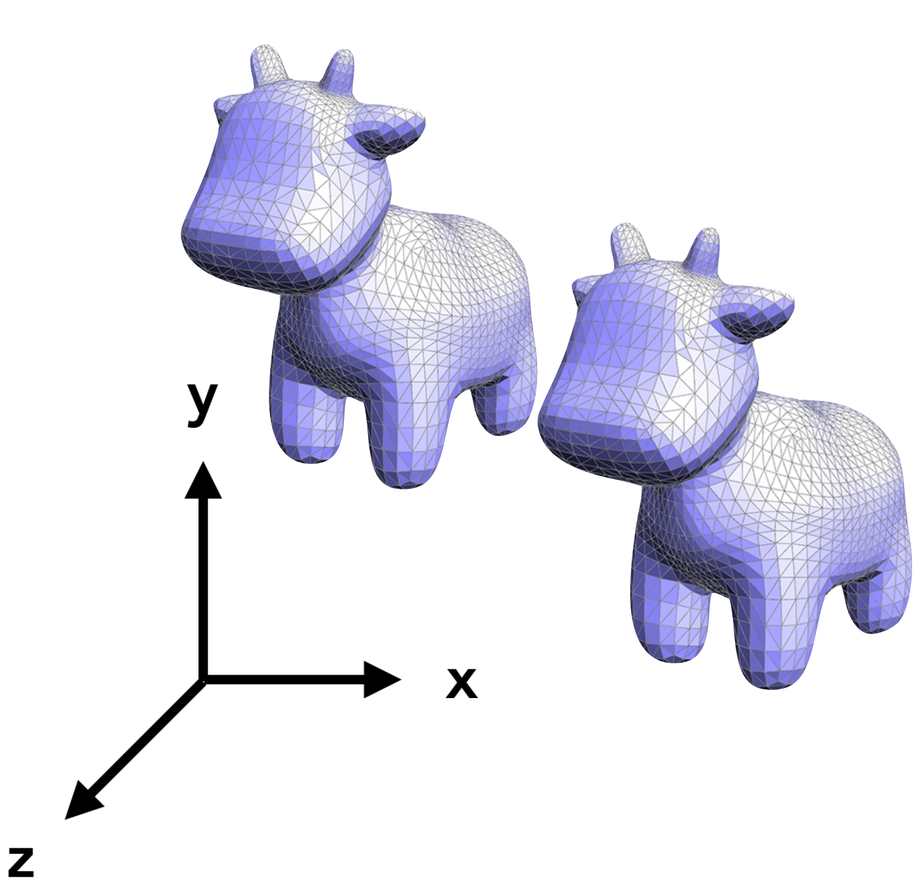
right arm group



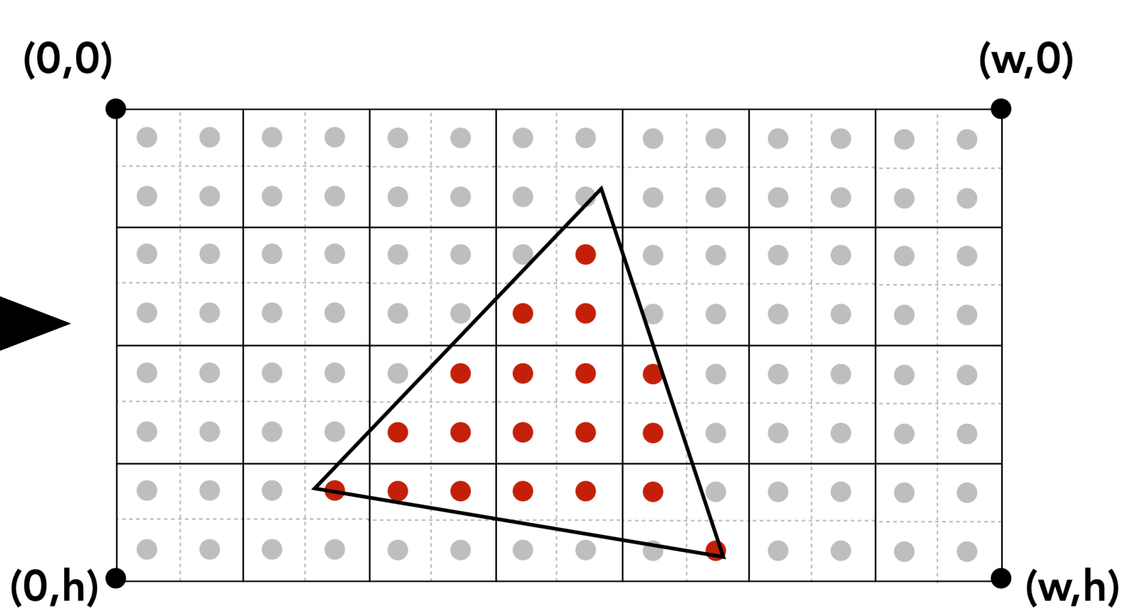


Viewing and Perspective

Viewing and Perspective Transforms



?



Scene modeling
3D world coordinates

Rasterization
2D screen coordinates

Camera Space

"Standard" Camera Space

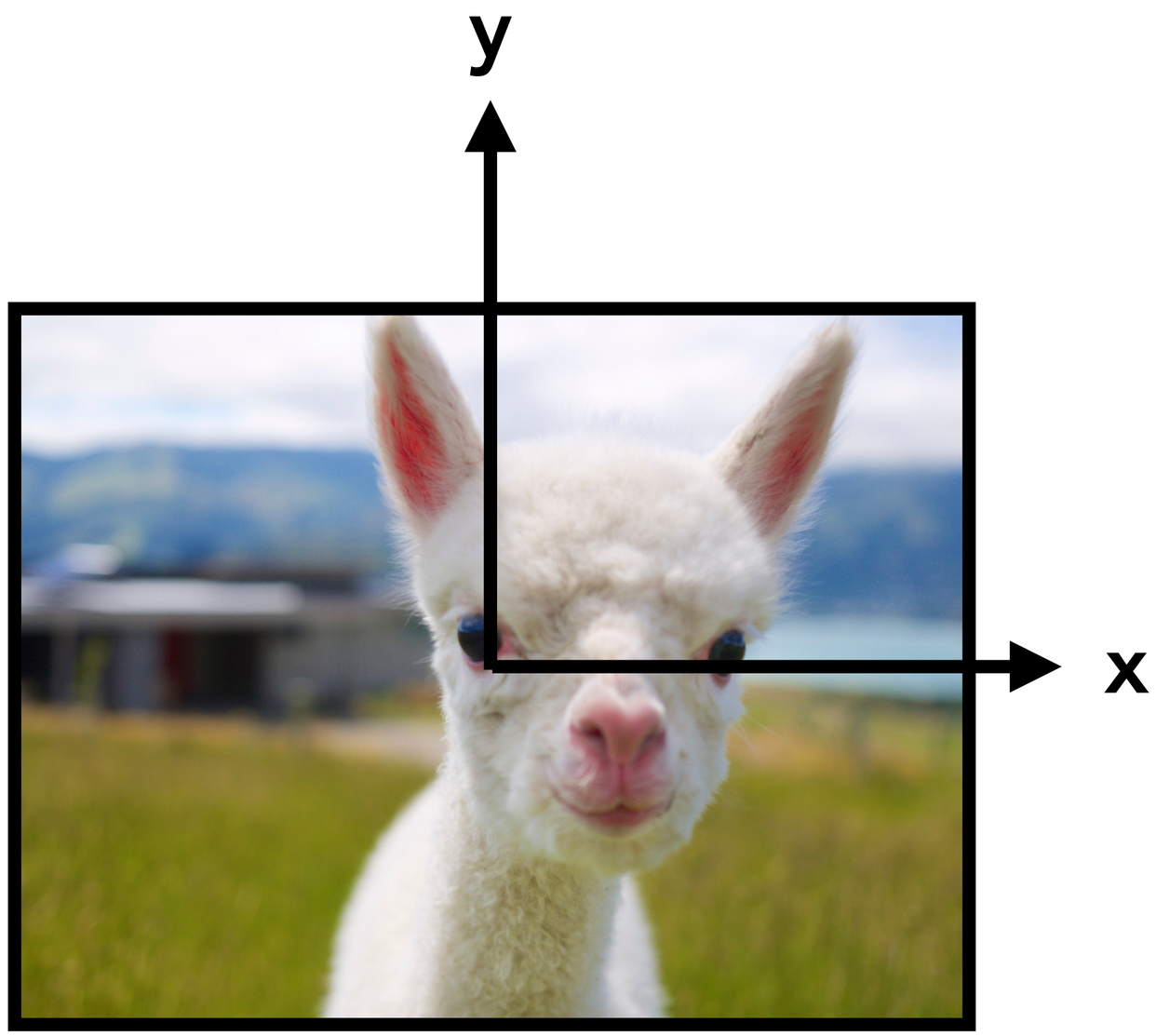
We will use this convention for "standard" camera coordinates:

- camera located at the origin
- looking down *negative* z-axis
- vertical vector is y-axis
- (x-axis) orthogonal to y & z

"Standard" Camera Coordinates

Resulting image

"Standard" Camera Coordinates



Resulting image
(z-axis pointing away from scene)

Consider A Camera Pointing in The World

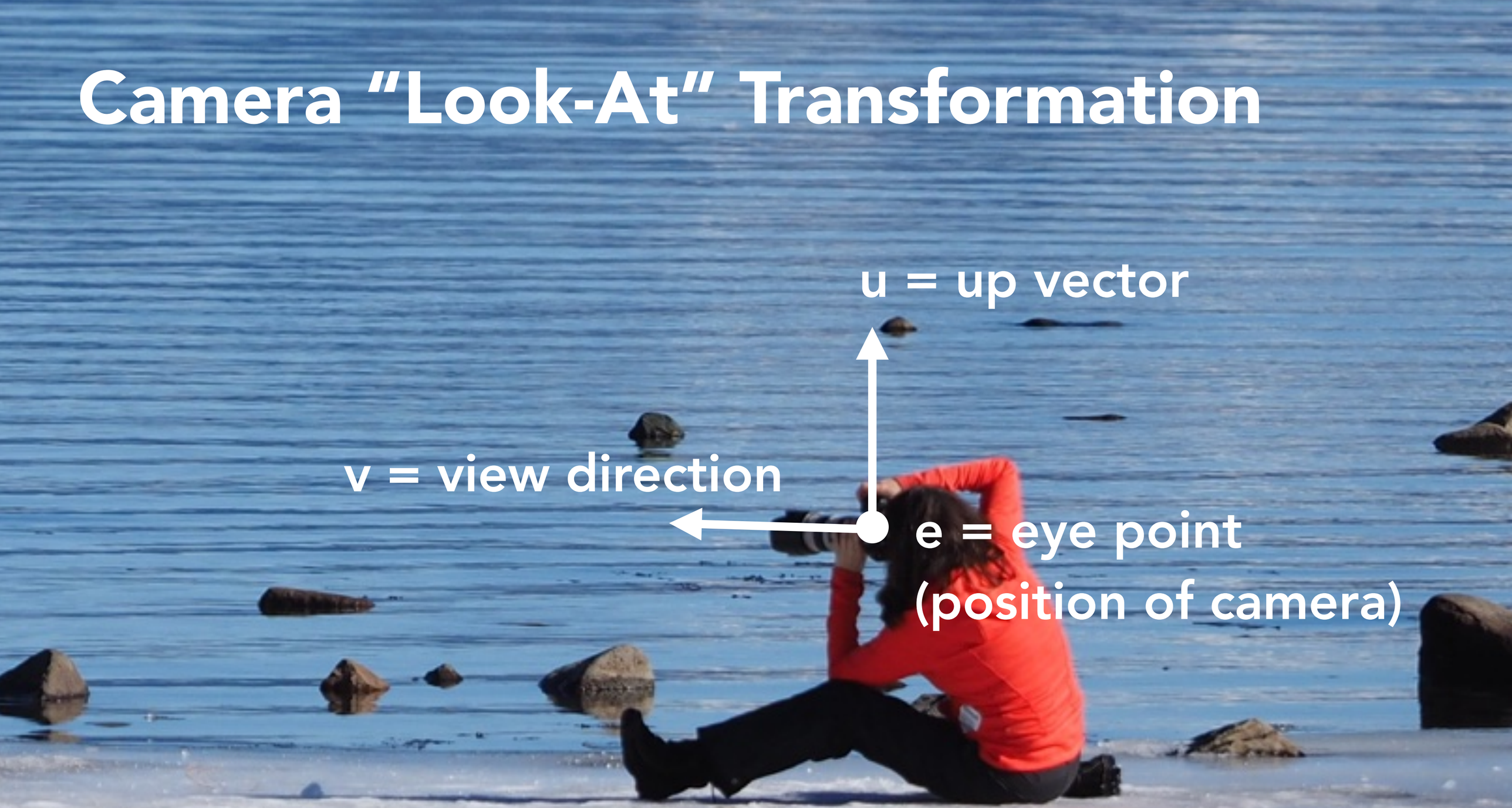
Consider A Camera Pointing in The World

u = up vector

v = view direction

e = eye point
(position of camera)

Camera “Look-At” Transformation



Input: e, u & v given in world space coordinates
Output: transform matrix from world space
to standard camera space

Camera “Look-At” Transformation

“Look-At” transformation

Camera “Look-At” Transformation

“Look-At” transformation

- Given *eye* point e, *up* vector u, *view* direction v

Camera “Look-At” Transformation

“Look-At” transformation

- Given *eye* point e, *up* vector u, *view* direction v
  - Assume u and v are orthonormal

Camera “Look-At” Transformation

“Look-At” transformation

- Given *eye* point e, *up* vector u, *view* direction v
  - Assume u and v are orthonormal
  - Find *right* vector r = v × u

Camera “Look-At” Transformation

“Look-At” transformation

- Given *eye* point e, *up* vector u, *view* direction v
  - Assume u and v are orthonormal
  - Find *right* vector r = v × u
- Transform camera (e,r,u,v) to standard camera:

Camera “Look-At” Transformation

“Look-At” transformation

- Given *eye* point e, *up* vector u, *view* direction v
  - Assume u and v are orthonormal
  - Find *right* vector r = v × u
- Transform camera (e,r,u,v) to standard camera:
  - located at the origin

Camera “Look-At” Transformation

“Look-At” transformation

- Given *eye* point e, *up* vector u, *view* direction v
  - Assume u and v are orthonormal
  - Find *right* vector r = v × u
- Transform camera (e,r,u,v) to standard camera:
  - located at the origin
  - looking down *negative* z axis

Camera "Look-At" Transformation

"Look-At" transformation

- Given *eye* point e, *up* vector u, *view* direction v
  - Assume u and v are orthonormal
  - Find *right* vector r = v × u
- Transform camera (e,r,u,v) to standard camera:
  - located at the origin
  - looking down *negative* z axis
  - up vector is y axis

Camera “Look-At” Transformation

Camera "Look-At" Transformation

Inverse: Matrix from standard camera to world space

$$\begin{pmatrix} r_x & u_x & -v_x & e_x \\ r_y & u_y & -v_y & e_y \\ r_z & u_z & -v_z & e_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Camera "Look-At" Transformation

Inverse: Matrix from standard camera to world space
(Why? This is a coordinate frame transform to (e,r,u,-v))

$$\begin{pmatrix} r_x & u_x & -v_x & e_x \\ r_y & u_y & -v_y & e_y \\ r_z & u_z & -v_z & e_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Camera “Look-At” Transformation

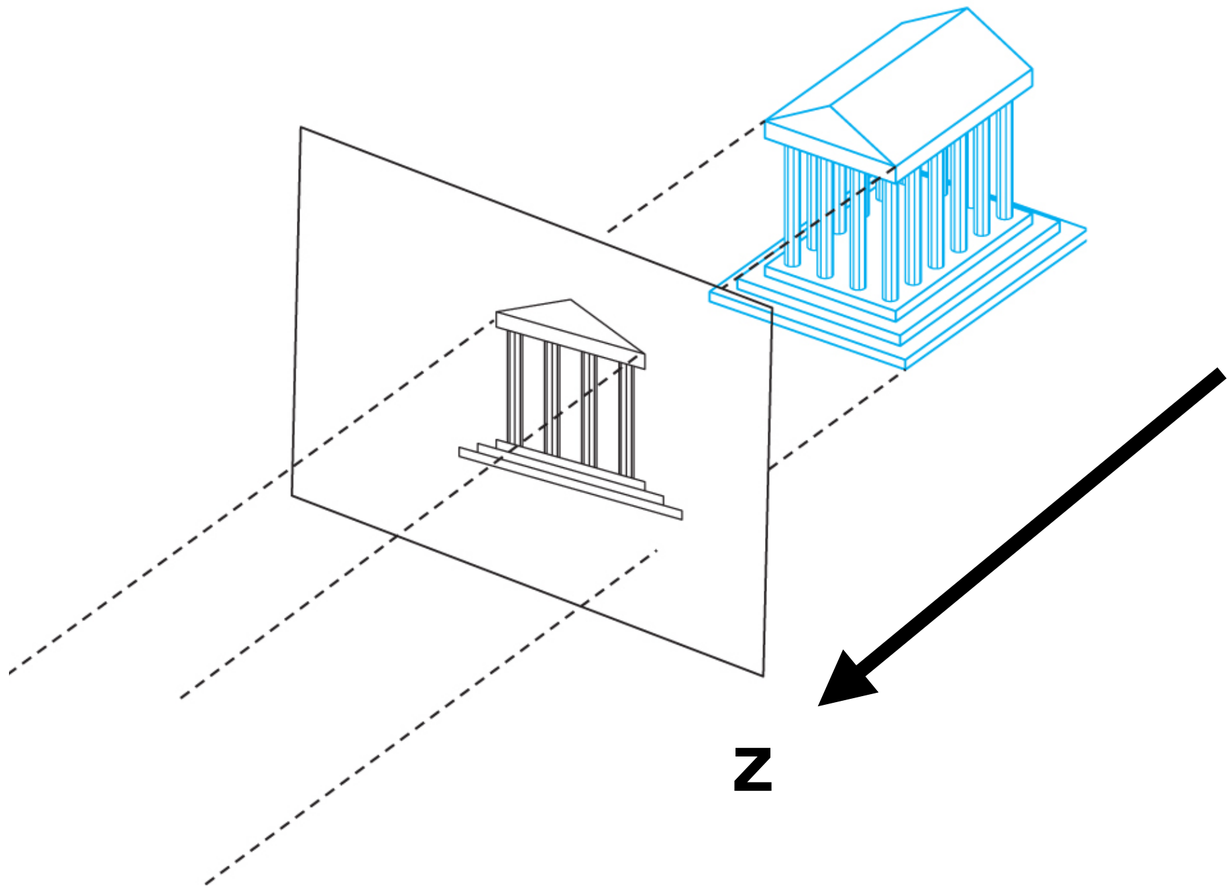
Inverse: Matrix from standard camera to world space (Why? This is a coordinate frame transform to (e,r,u,-v))

$$\begin{pmatrix} r_x & u_x & -v_x & e_x \\ r_y & u_y & -v_y & e_y \\ r_z & u_z & -v_z & e_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

“Look-at” (world->camera) transform is the inverse of above matrix:

$$\begin{pmatrix} r_x & u_x & -v_x & e_x \\ r_y & u_y & -v_y & e_y \\ r_z & u_z & -v_z & e_z \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ -v_x & -v_y & -v_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 & -e_x \\ 0 & 1 & 0 & -e_y \\ 0 & 0 & 1 & -e_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Transform Camera Space to Image Plane?



How to transform from 3D camera space to 2D image plane?

- One option: orthographic projection (just delete z)
- Useful, e.g. for engineering drawings
- But is this the whole story?

Perspective

Credit: jefflynchphoto.com

Perspective in Art

Berlinghieri 1235

Perspective in Art

Giotto 1290

Perspective in Art

Giotto 1290

Perspective in Art

Something wrong here!

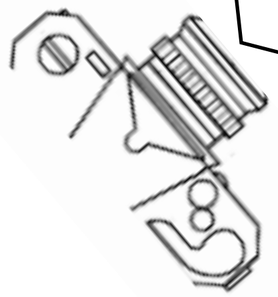
Perspective Composition = Camera Position + Angle of View

From Canon EF Lens Work III

In this sequence, angle of view decreases as distance from subject increases, to size of human subject in image.

Notice the dramatic change in background perspective.

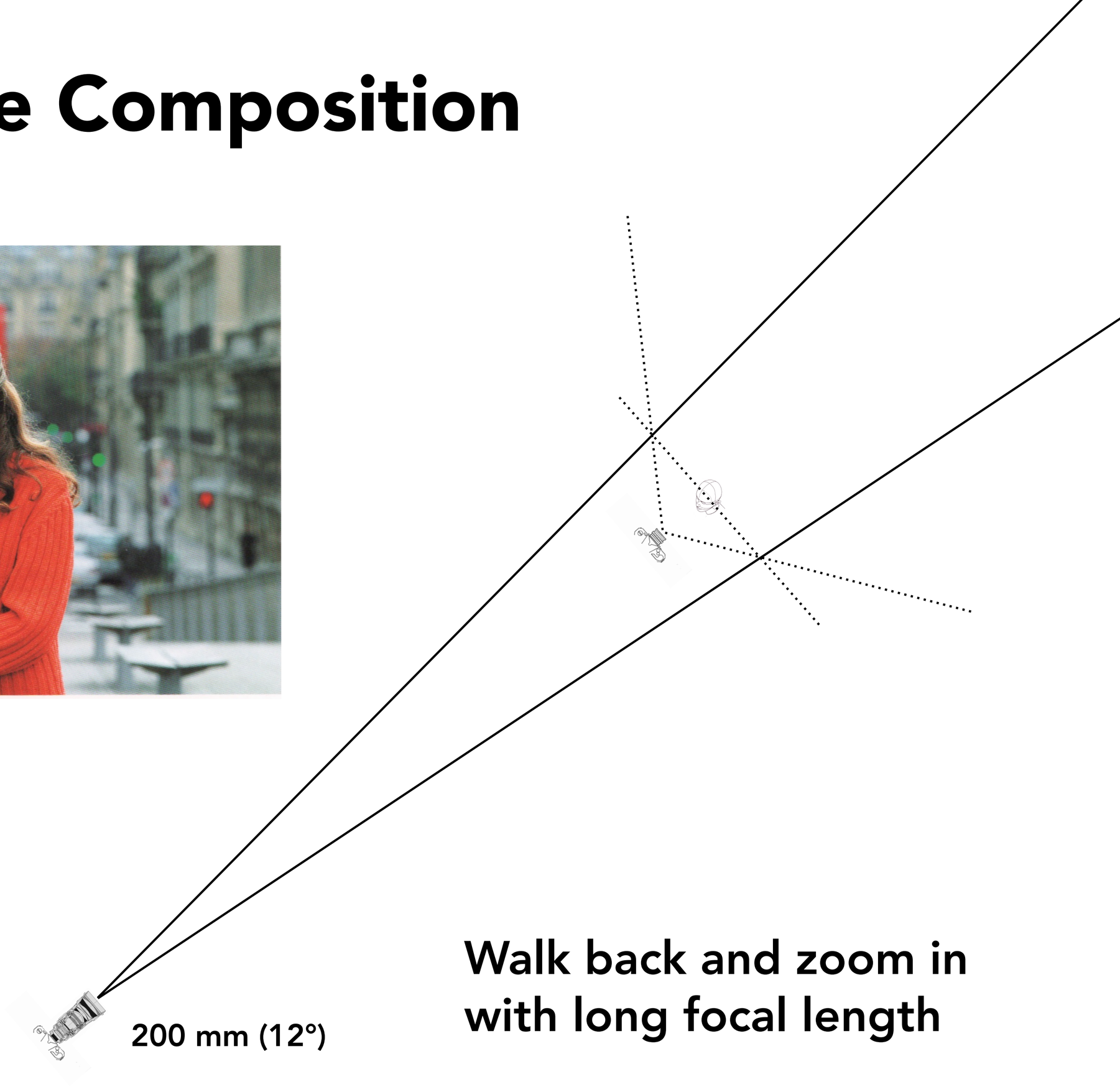
Perspective Composition



16 mm (110°)

Up close and zoomed wide with short focal length

Perspective Composition



Walk back and zoom in with long focal length

Pinhole Camera Model

Pinhole Camera

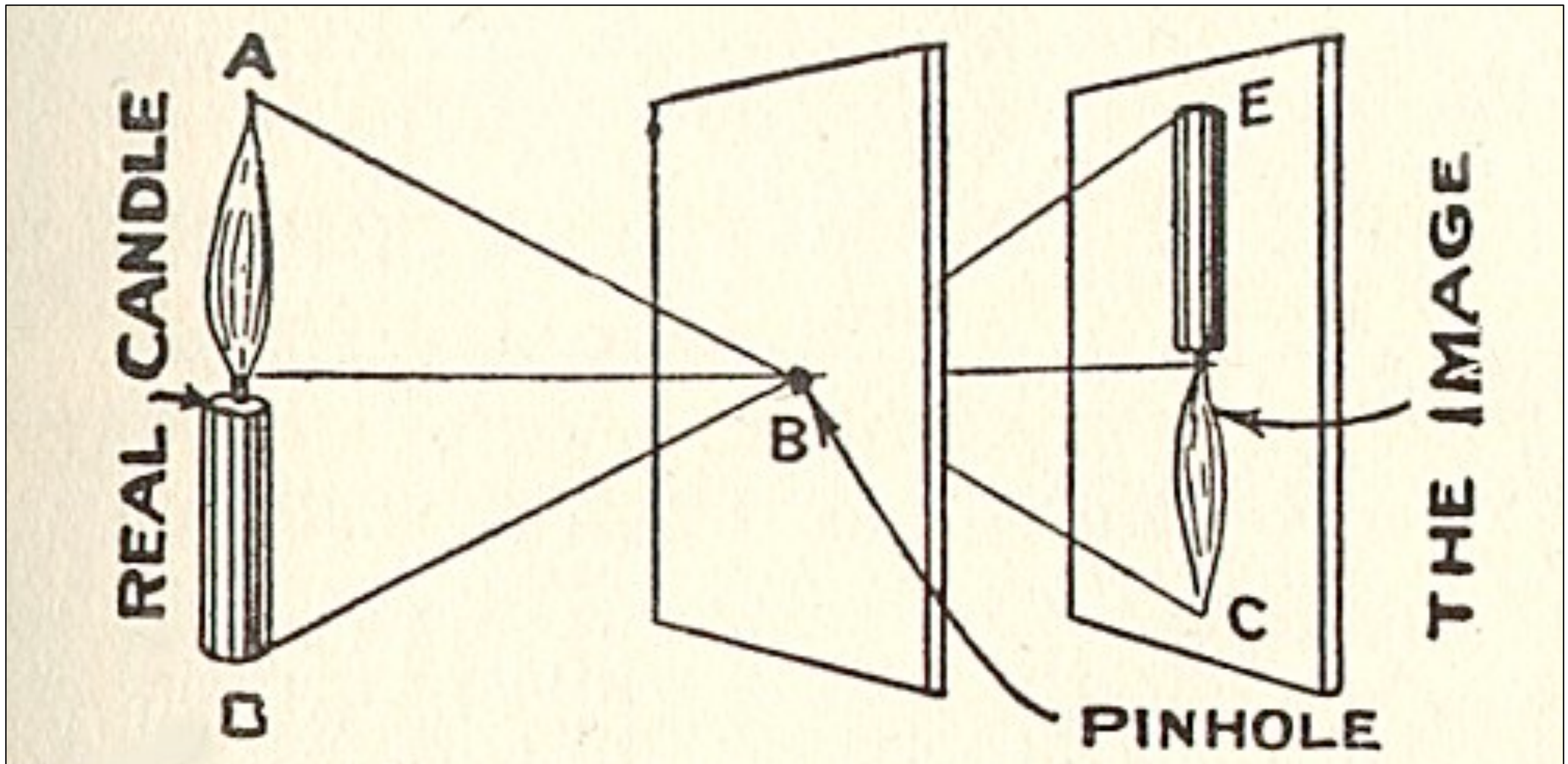


FIG. 131.—How Light and a Pinhole Form an Image.

Pinhole Camera

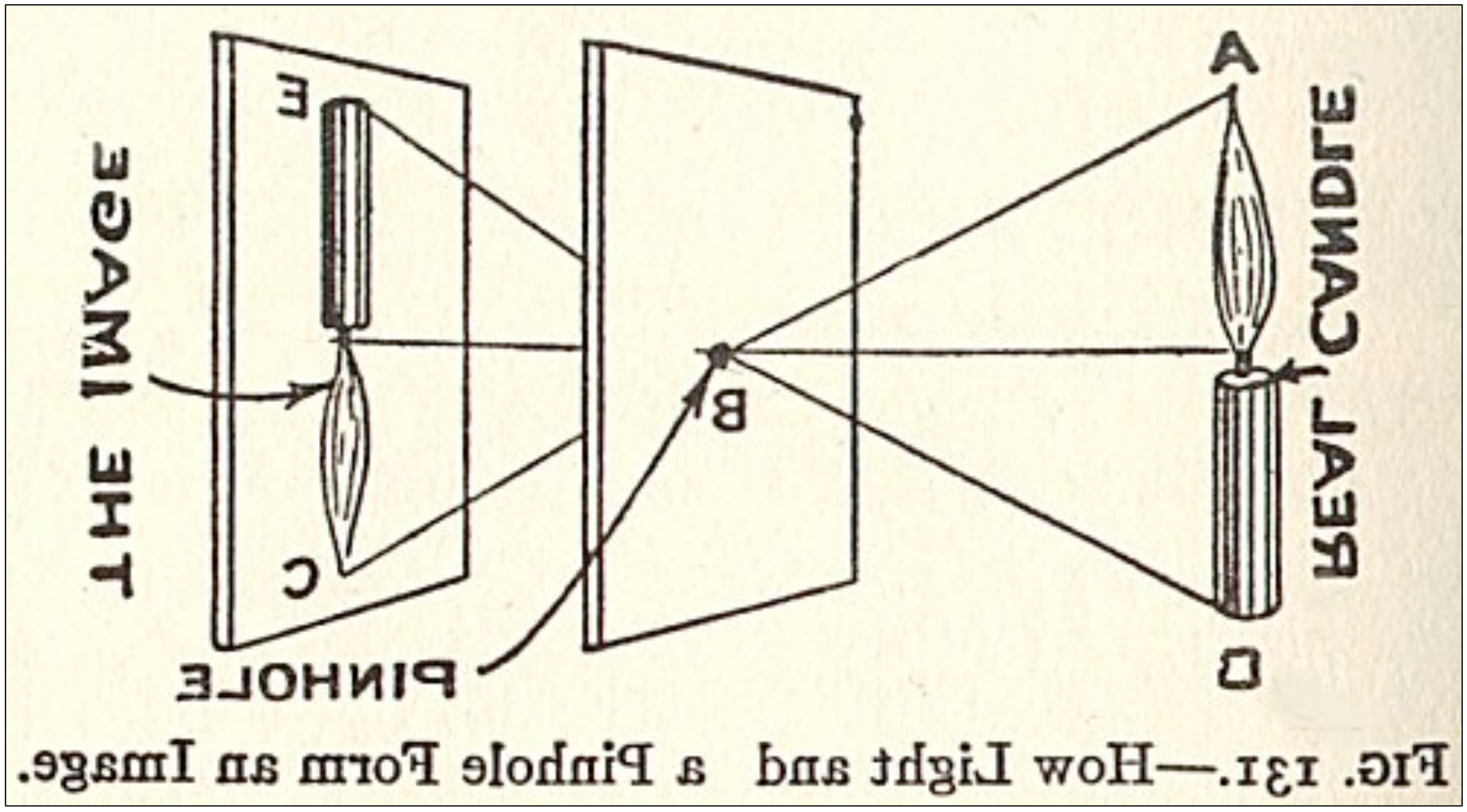
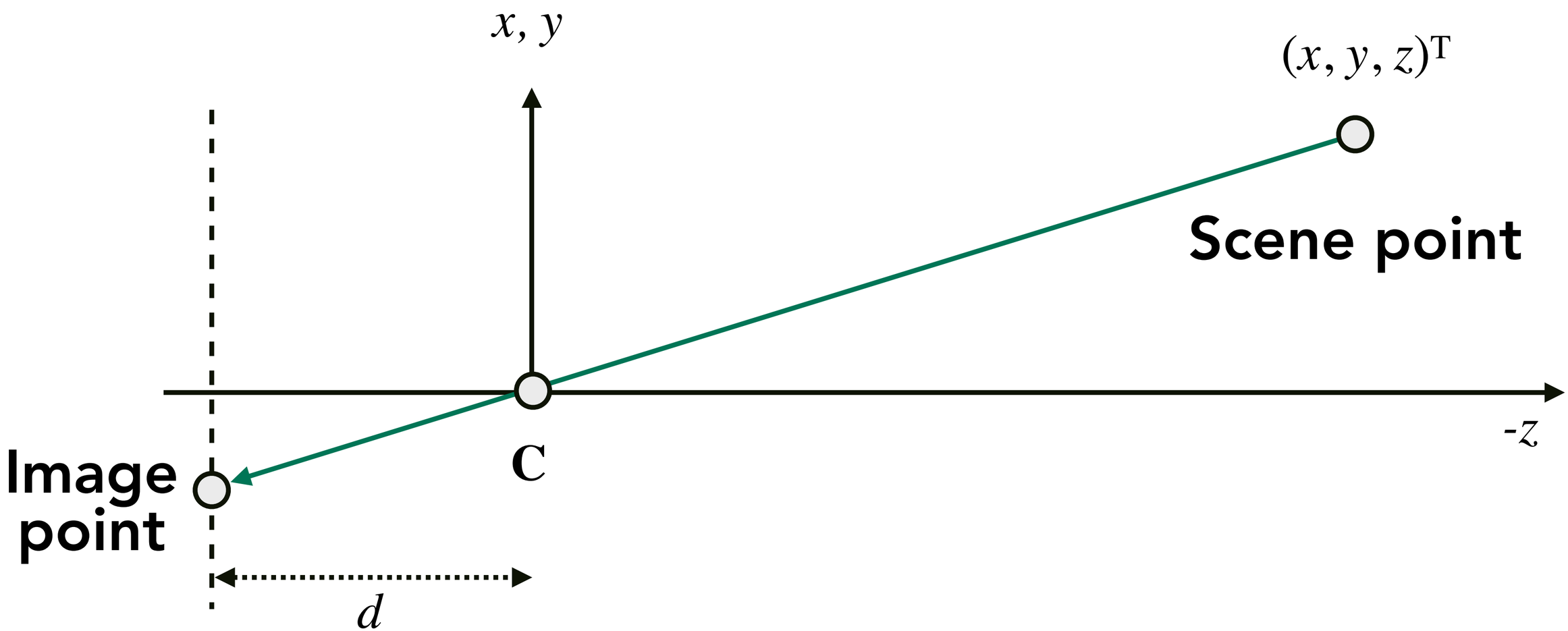


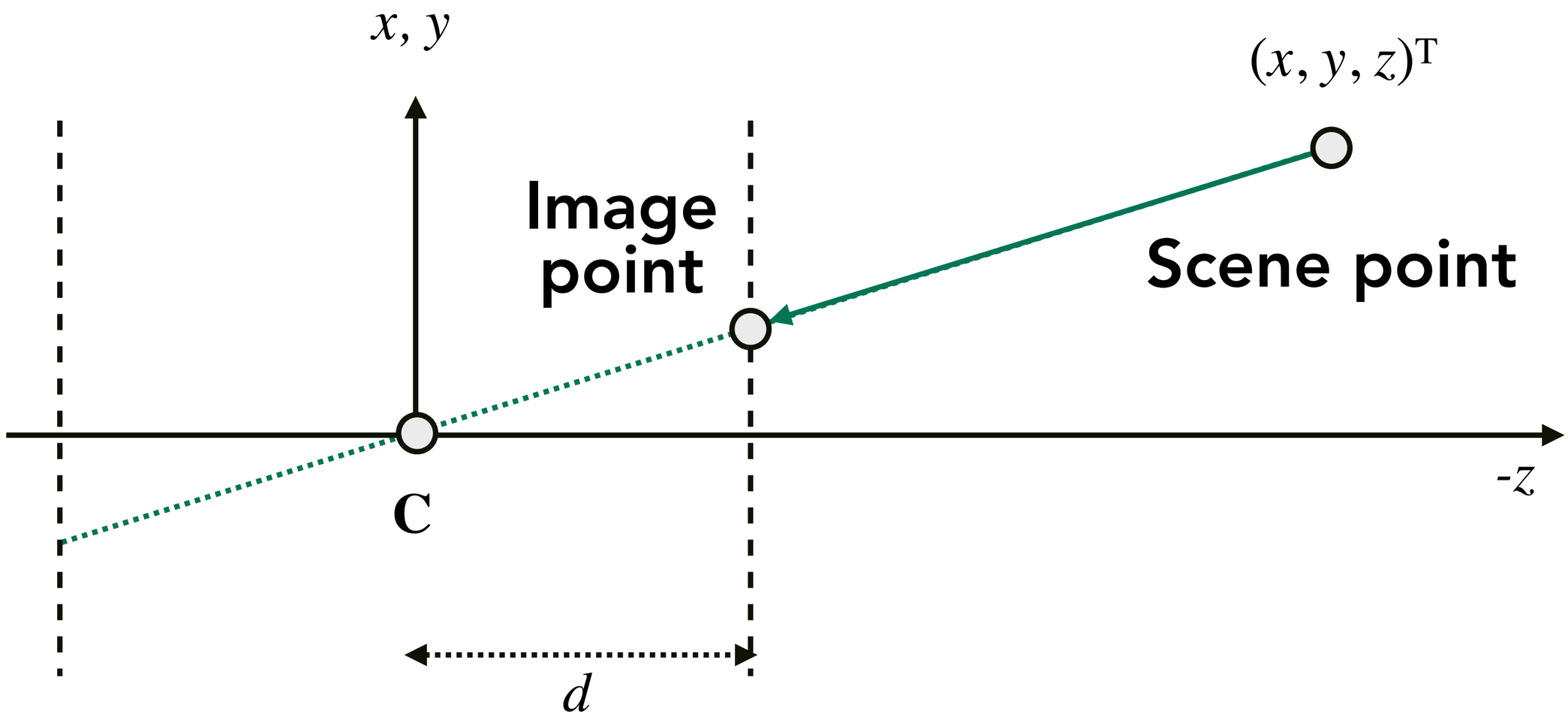
FIG. 131.—How Light and a Pinhole Form an Image.

Projective Transform



Inverted image (as in real pinhole camera)

Pinhole Camera Projective Transform

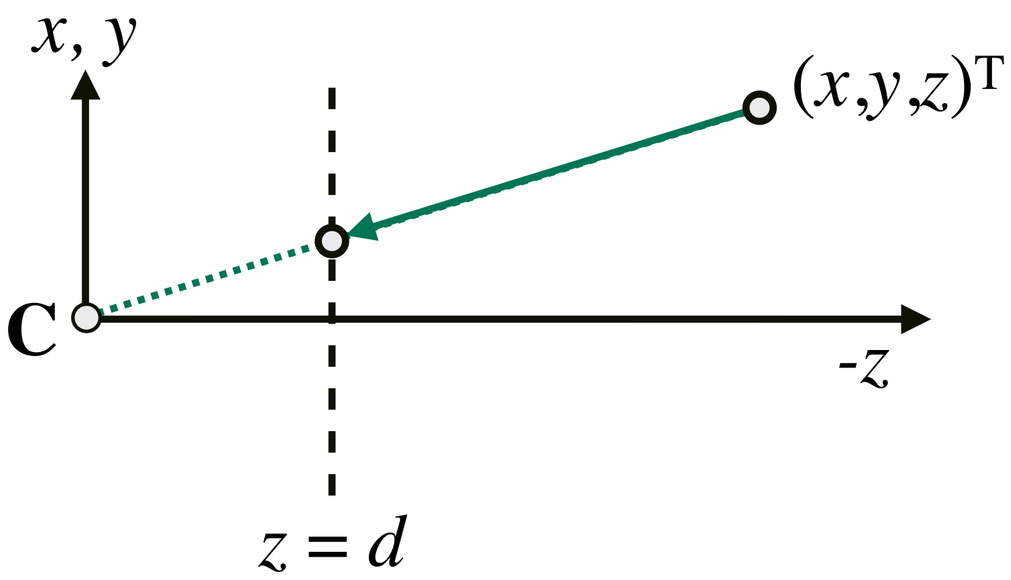


Upright image

Projective Transforms

Standard perspective projection

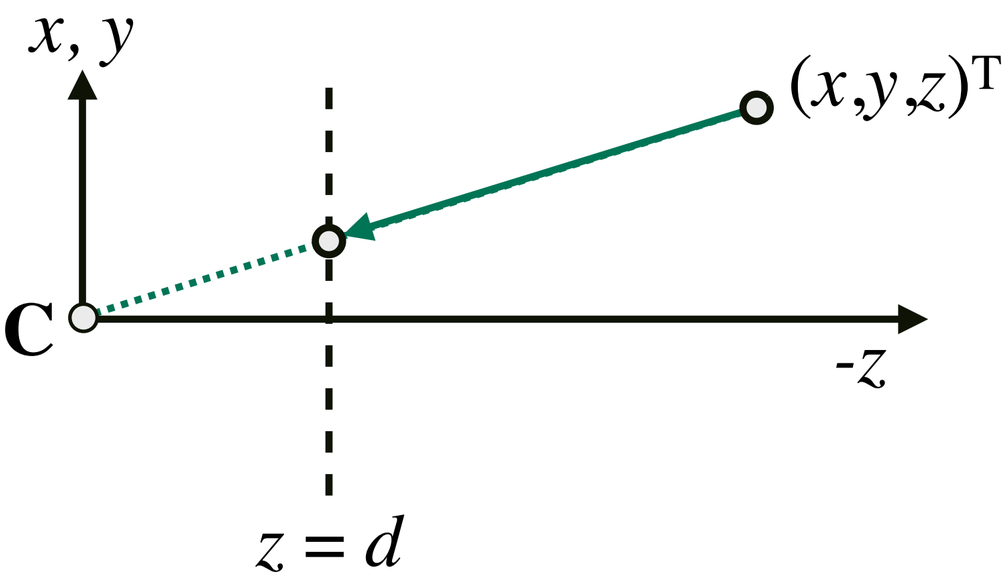
- Center of projection: $(0, 0, 0)^T$
- Image plane at $z = d$



Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$

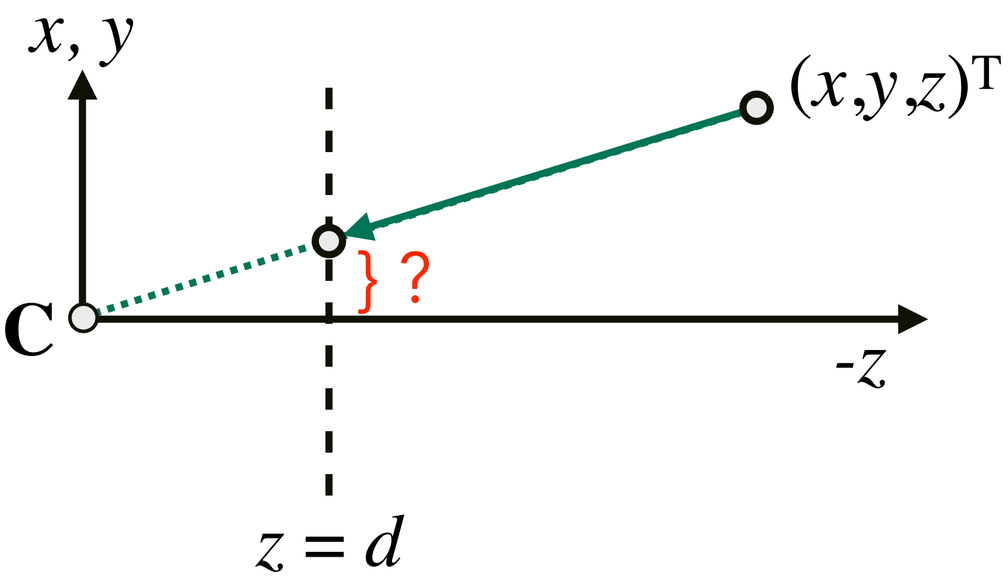


$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} \\ \\ d \end{pmatrix}$$

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$

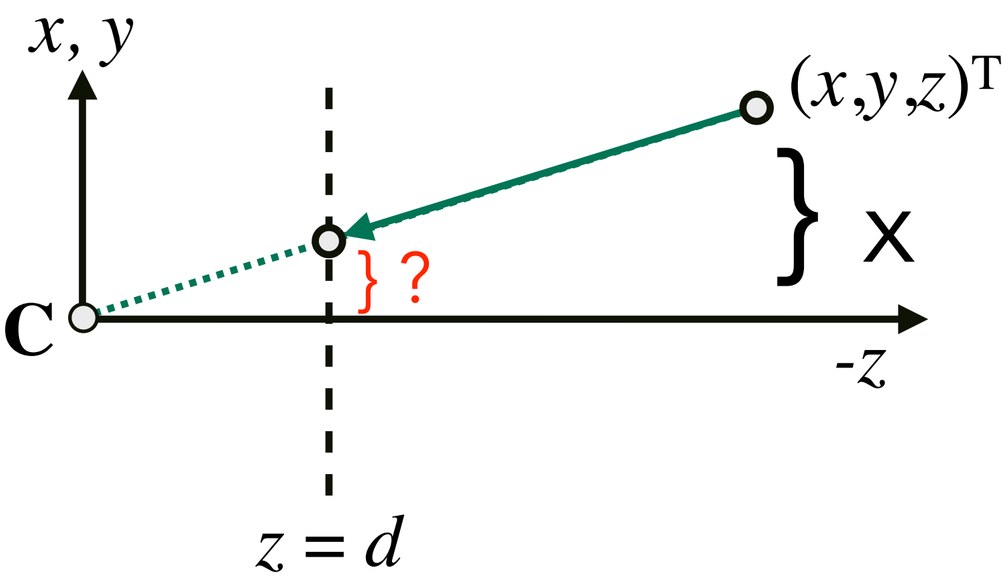


$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} \\ \\ d \end{pmatrix}$$

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$

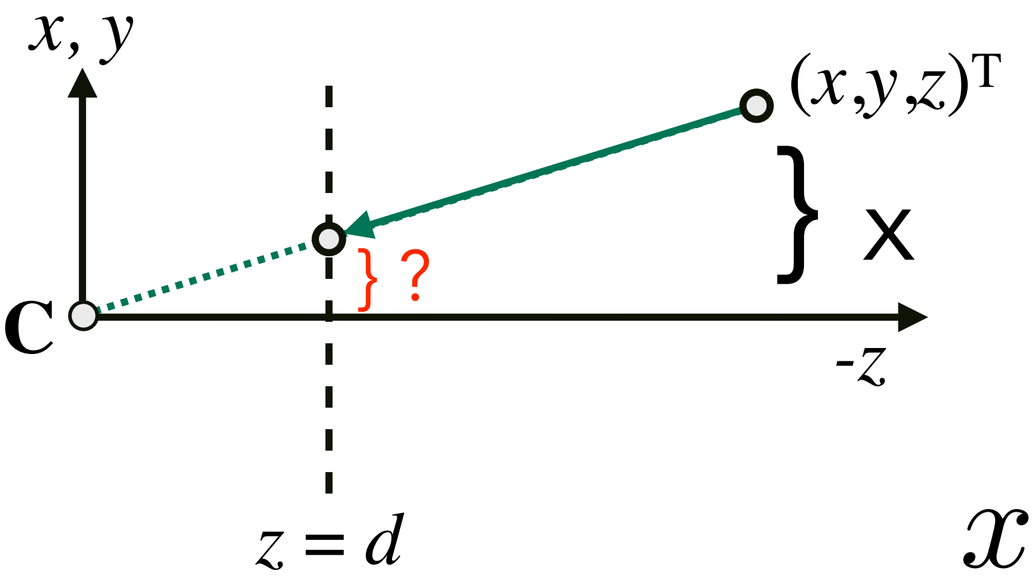


$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} \\ \\ d \end{pmatrix}$$

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$



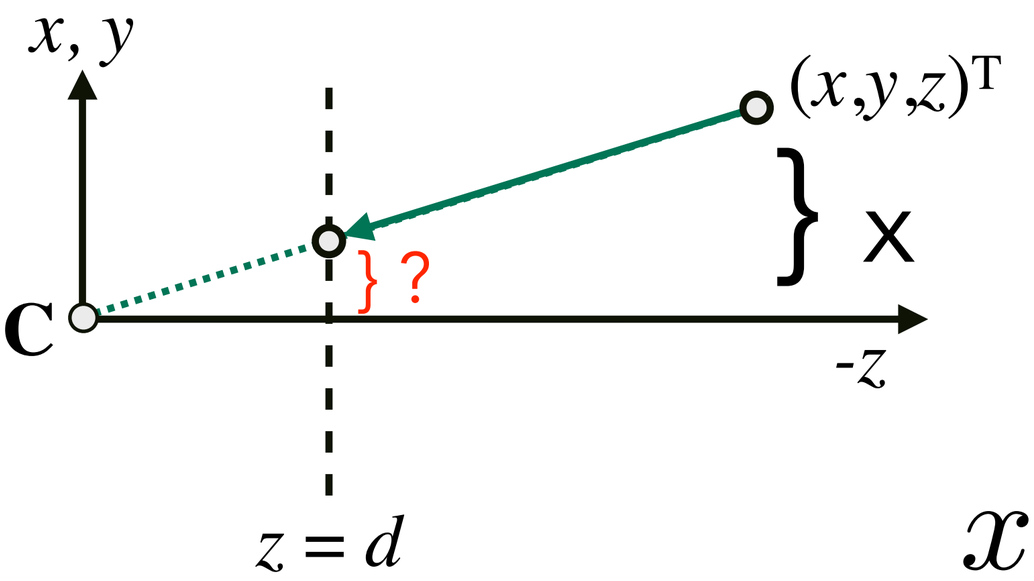
$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} \\ \\ d \end{pmatrix}$$

$$\frac{x}{z} = \frac{?}{d}$$

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$



$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} x \cdot d/z \\ y \cdot d/z \\ d \end{pmatrix}$$

$$\frac{x}{z} = \frac{?}{d}$$

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} x \cdot d/z \\ y \cdot d/z \\ d \end{pmatrix}$$

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} x \cdot d/z \\ y \cdot d/z \\ d \end{pmatrix}$$

Perspective foreshortening

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} x \cdot d/z \\ y \cdot d/z \\ d \end{pmatrix}$$

Perspective foreshortening

- Need division by z

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} x \cdot d/z \\ y \cdot d/z \\ d \end{pmatrix}$$

Perspective foreshortening

- Need division by z
- Matrix representation?

Projective Transforms

Standard perspective projection

- Center of projection: $(0, 0, 0)^{\mathrm{T}}$
- Image plane at $z = d$

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} x \cdot d/z \\ y \cdot d/z \\ d \end{pmatrix}$$

Perspective foreshortening

- Need division by z
- Matrix representation?

➡Homogeneous coordinates!

Homogenous Coordinates (3D)

$$\mathbf{p} = \begin{pmatrix} wx \\ wy \\ wz \\ w \end{pmatrix}$$

Homogenous Coordinates (3D)

$$\mathbf{p} = \begin{pmatrix} wx \\ wy \\ wz \\ w \end{pmatrix} \longleftrightarrow \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Homogenous Coordinates (3D)

$$\mathbf{p} = \begin{pmatrix} wx \\ wy \\ wz \\ w \end{pmatrix} \longleftrightarrow \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix}$$

Note non-zero term in final row.
First time we have seen this.

Homogenous Coordinates (3D)

$$\mathbf{p} = \begin{pmatrix} wx \\ wy \\ wz \\ w \end{pmatrix} \longleftrightarrow \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix}$$

Note non-zero term in final row.
First time we have seen this.

$$\mathbf{q} = \mathbf{M} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ z/d \end{pmatrix}$$

Homogenous Coordinates (3D)

$$\mathbf{p} = \begin{pmatrix} wx \\ wy \\ wz \\ w \end{pmatrix} \longleftrightarrow \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix}$$

Note non-zero term in final row.
First time we have seen this.

$$\mathbf{q} = \mathbf{M} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ z/d \end{pmatrix} \longleftrightarrow \begin{pmatrix} xd/z \\ yd/z \\ d \\ 1 \end{pmatrix}$$

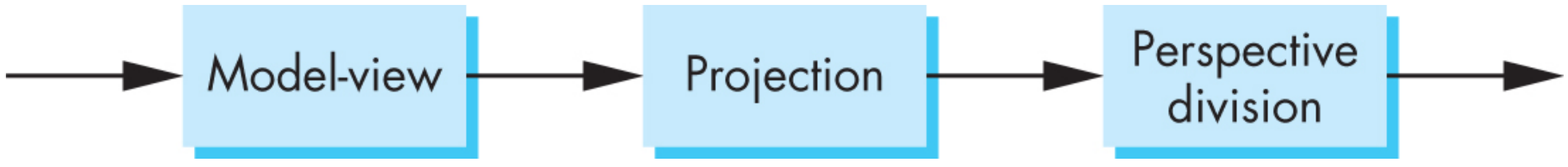
Homogenous Coordinates (3D)

$$\mathbf{p} = \begin{pmatrix} wx \\ wy \\ wz \\ w \end{pmatrix} \longleftrightarrow \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix}$$

$$\mathbf{q} = \mathbf{M} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ z/d \end{pmatrix} \longleftrightarrow \begin{pmatrix} xd/z \\ yd/z \\ d \\ 1 \end{pmatrix}$$

Note non-zero term in final row.
First time we have seen this.



Homogenous Coordinates (3D)

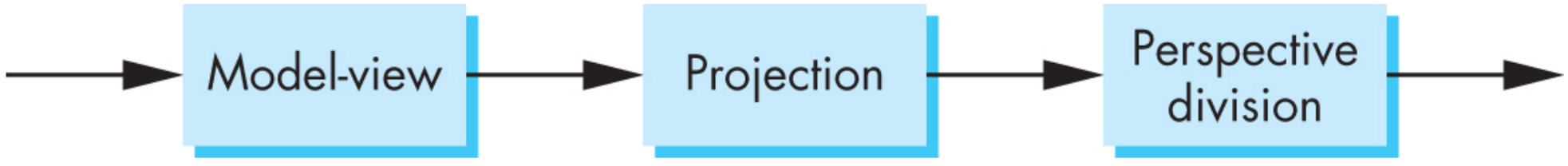
$$\mathbf{p} = \begin{pmatrix} wx \\ wy \\ wz \\ w \end{pmatrix} \longleftrightarrow \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix}$$

$$\mathbf{q} = \mathbf{M} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ z/d \end{pmatrix} \longleftrightarrow \begin{pmatrix} xd/z \\ yd/z \\ d \\ 1 \end{pmatrix}$$

Note non-zero term in final row.
First time we have seen this.

Note zeros in 4th column.
We'll change this later for hidden surface determination.



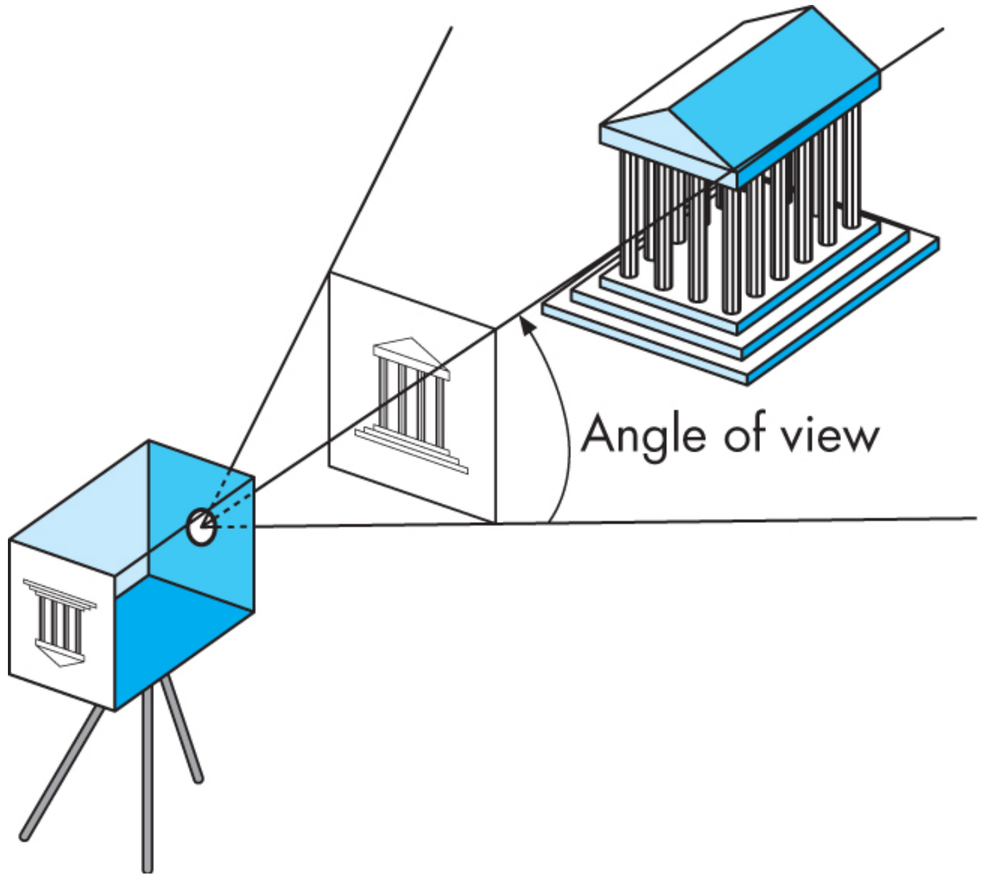
Pinhole Camera Model

This mathematical model produces all linear perspective effects!

- Converging lines + vanishing points
- Closer objects appear larger in images
- ...

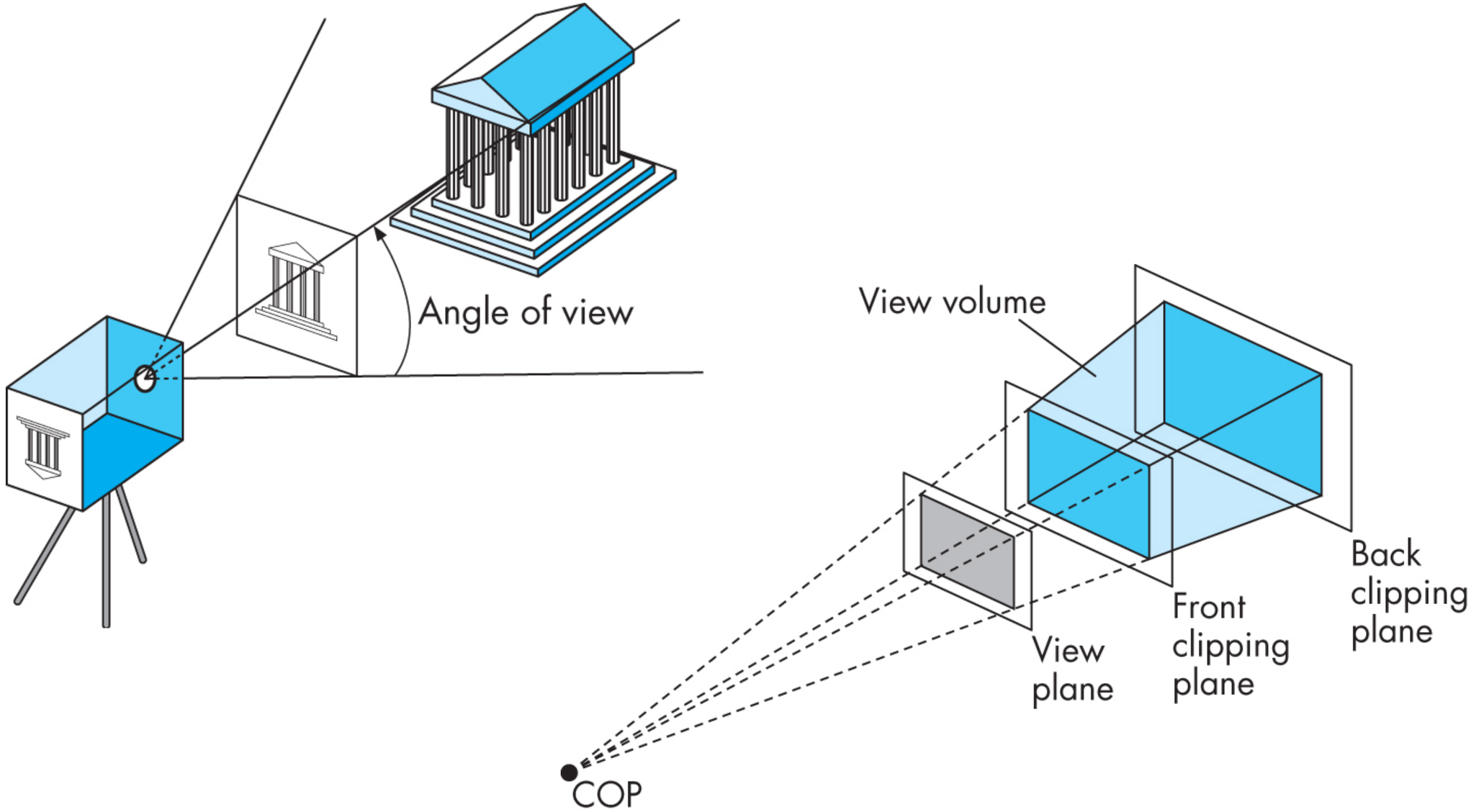
Specifying Real Camera Perspectives

Specifying Perspective Projection



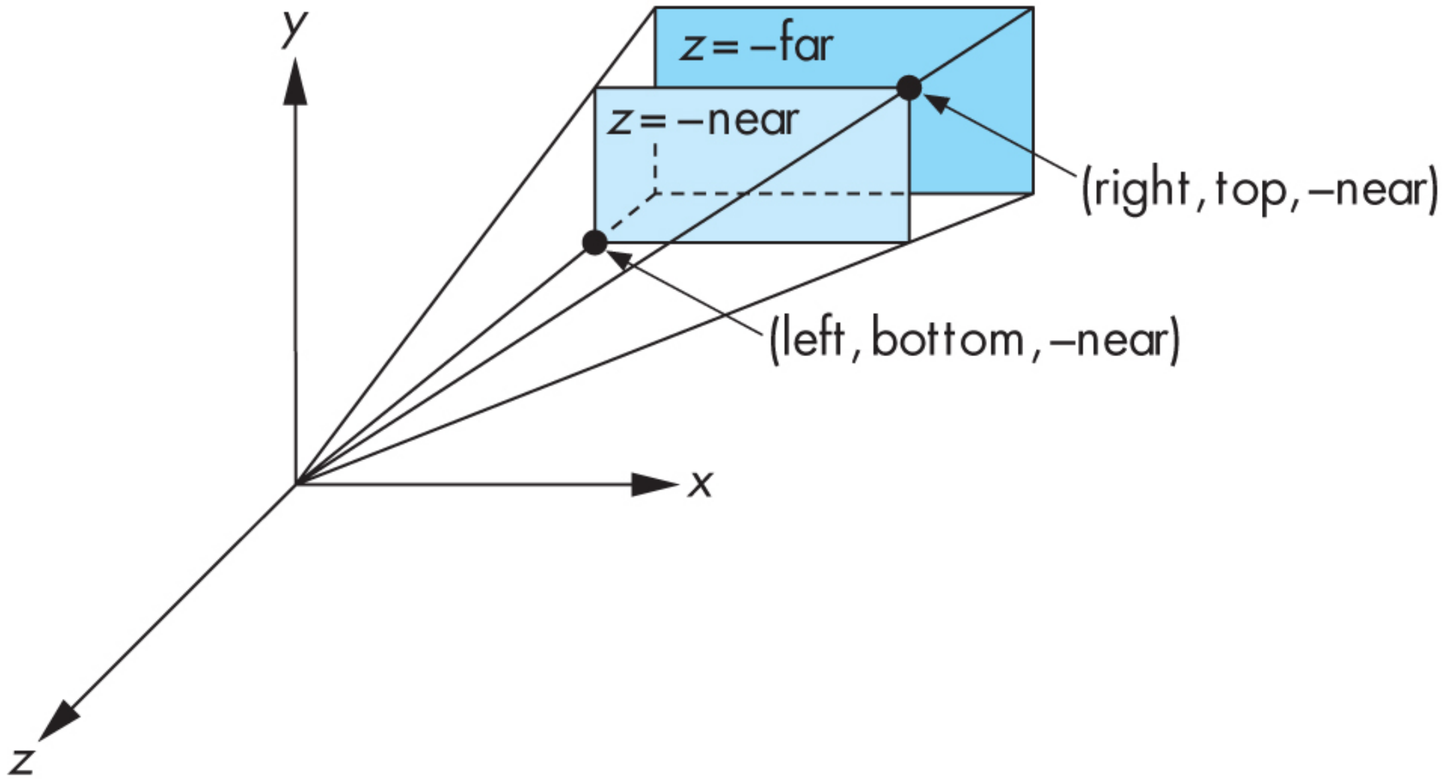
From Angel and Shreiner, Interactive Computer Graphics

Specifying Perspective Projection



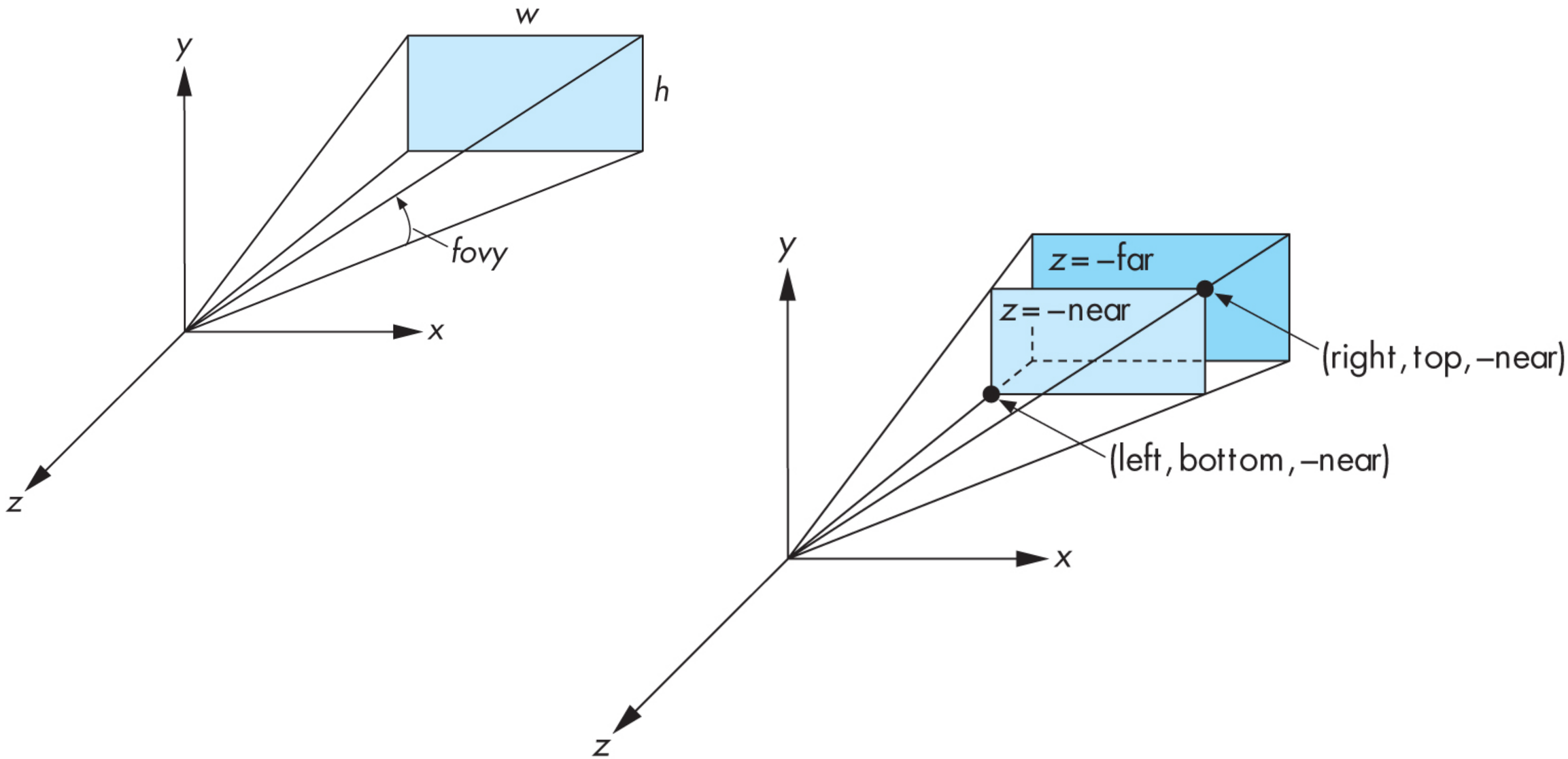
From Angel and Shreiner, Interactive Computer Graphics

Specifying Perspective Viewing Volume



From Angel and Shreiner, Interactive Computer Graphics

Specifying Perspective Viewing Volume



From Angel and Shreiner, Interactive Computer Graphics

Specifying Perspective Viewing Volume

Parameterized by

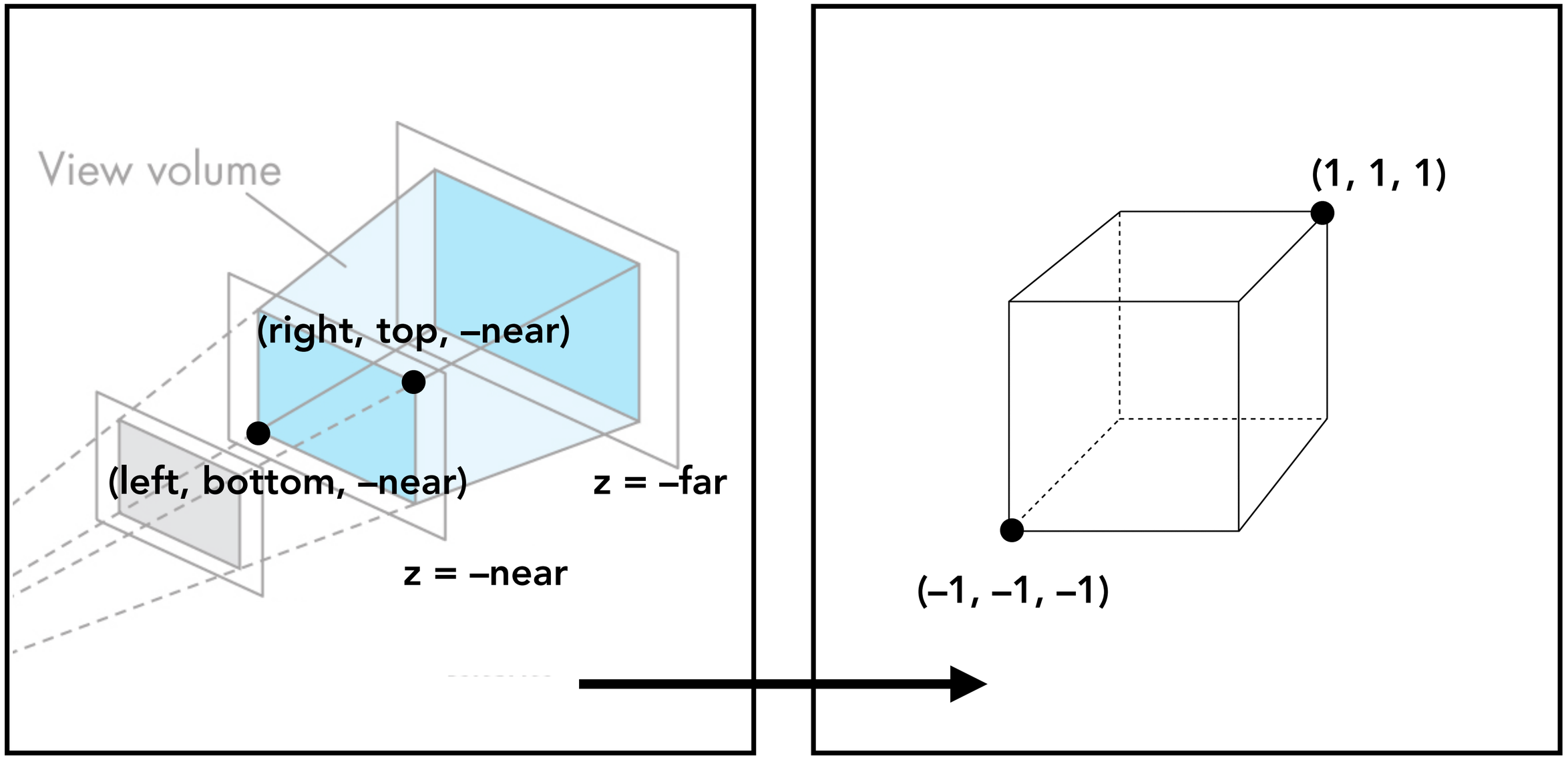
- fovy : vertical angular field of view
- aspect ratio : width / height of field of view
- near : depth of near clipping plane
- far : depth of far clipping plane

Derived quantities

- top = near * tan (fovy)
- bottom = – top
- right = top * aspect
- left = – right

Perspective Projection Implementation

Perspective Projection Transform



Camera Coordinates

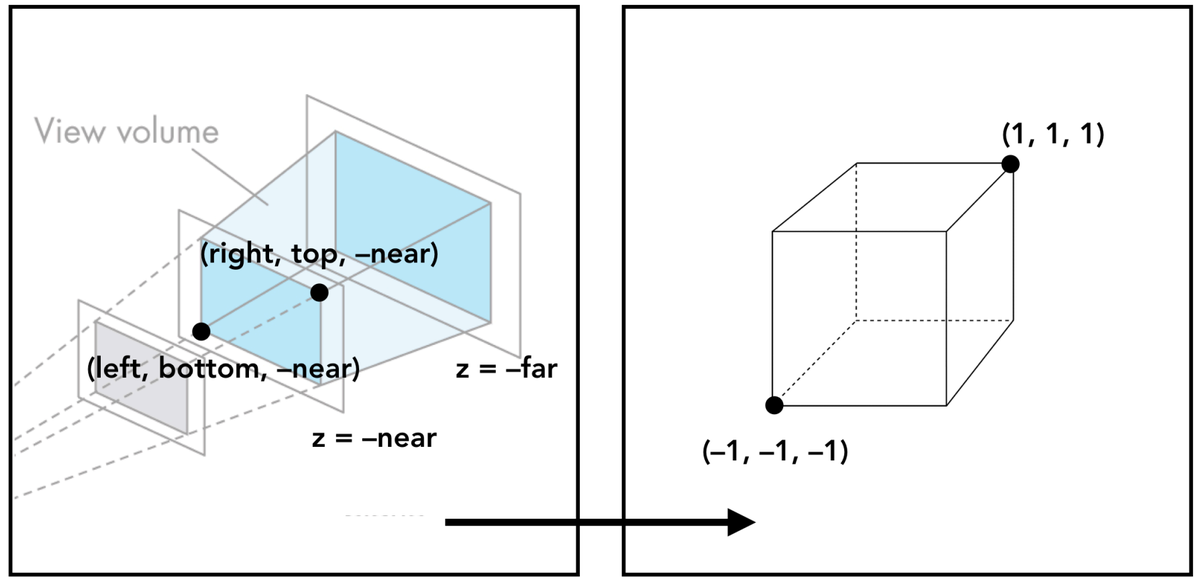
Normalized Device Coords "NDC"

Later we will "flatten and scale " NDC to get framebuffer coordinates

Perspective Projection Transform

Notes:

- *Need not be symmetric about z-axis,* but for simplicity here we assume so
- This transform will preserve depth information (ordering) in NDC



Perspective Transform Matrix

$$\mathbf{P} = \begin{bmatrix} \frac{near}{right} & 0 & 0 & 0 \\ 0 & \frac{near}{top} & 0 & 0 \\ 0 & 0 & -\frac{far+near}{far-near} & \frac{-2far*near}{far-near} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

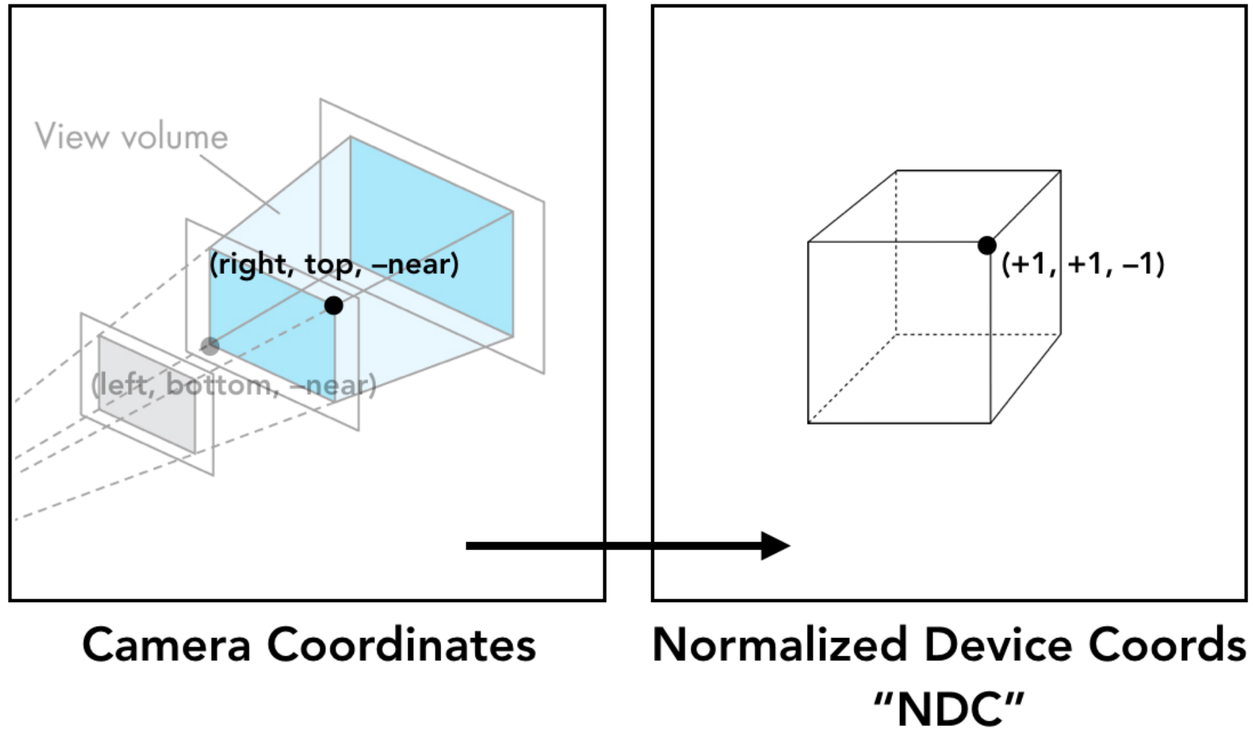
Perspective Transform Matrix

$$\mathbf{P} = \begin{bmatrix} \frac{near}{right} & 0 & 0 & 0 \\ 0 & \frac{near}{top} & 0 & 0 \\ 0 & 0 & -\frac{far+near}{far-near} & \frac{-2far*near}{far-near} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Note entry in 4th column

Perspective Transform Matrix Example

$$\mathbf{P} = \begin{bmatrix} \frac{near}{right} & 0 & 0 & 0 \\ 0 & \frac{near}{top} & 0 & 0 \\ 0 & 0 & -\frac{far+near}{far-near} & \frac{-2far*near}{far-near} \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} right \\ top \\ -near \\ 1 \end{bmatrix}$$



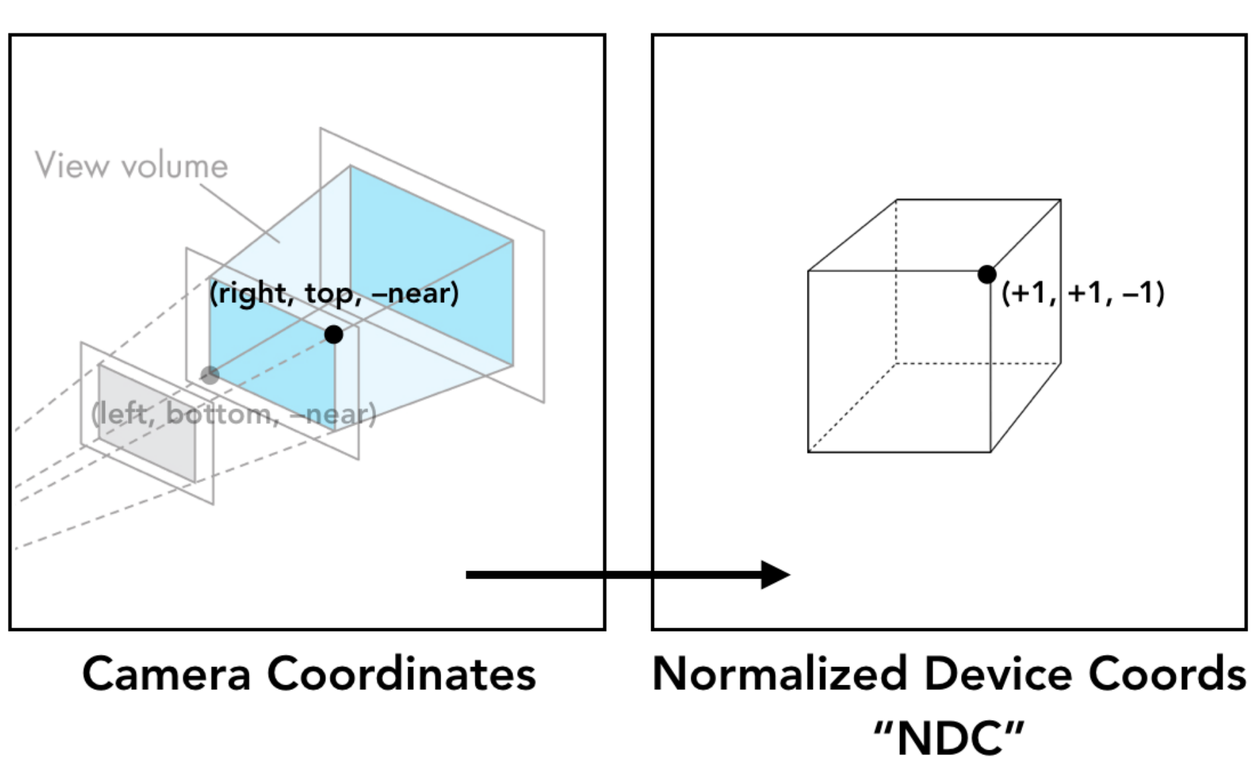
Perspective Transform Matrix Example

$$\mathbf{P} = \begin{bmatrix} \frac{near}{right} & 0 & 0 & 0 \\ 0 & \frac{near}{top} & 0 & 0 \\ 0 & 0 & -\frac{far+near}{far-near} & \frac{-2far*near}{far-near} \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} right \\ top \\ -near \\ 1 \end{bmatrix}$$

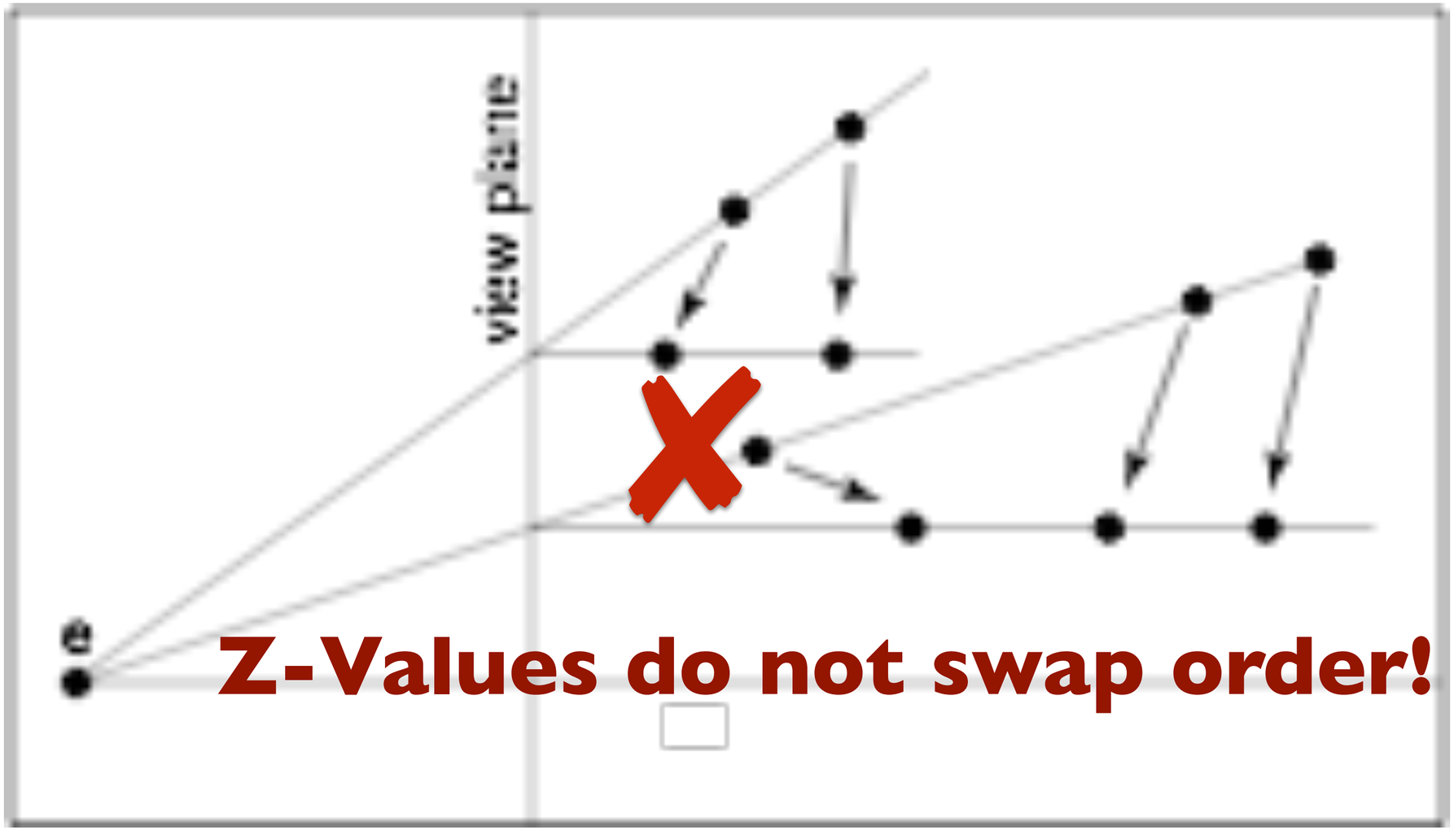
$$= \begin{bmatrix} near \\ near \\ near * \frac{far+near}{far-near} - \frac{2far*near}{far-near} \\ near \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 1 \\ \frac{-far+near}{far-near} \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 1 \\ -1 \\ 1 \end{bmatrix}$$



Perspective Projection



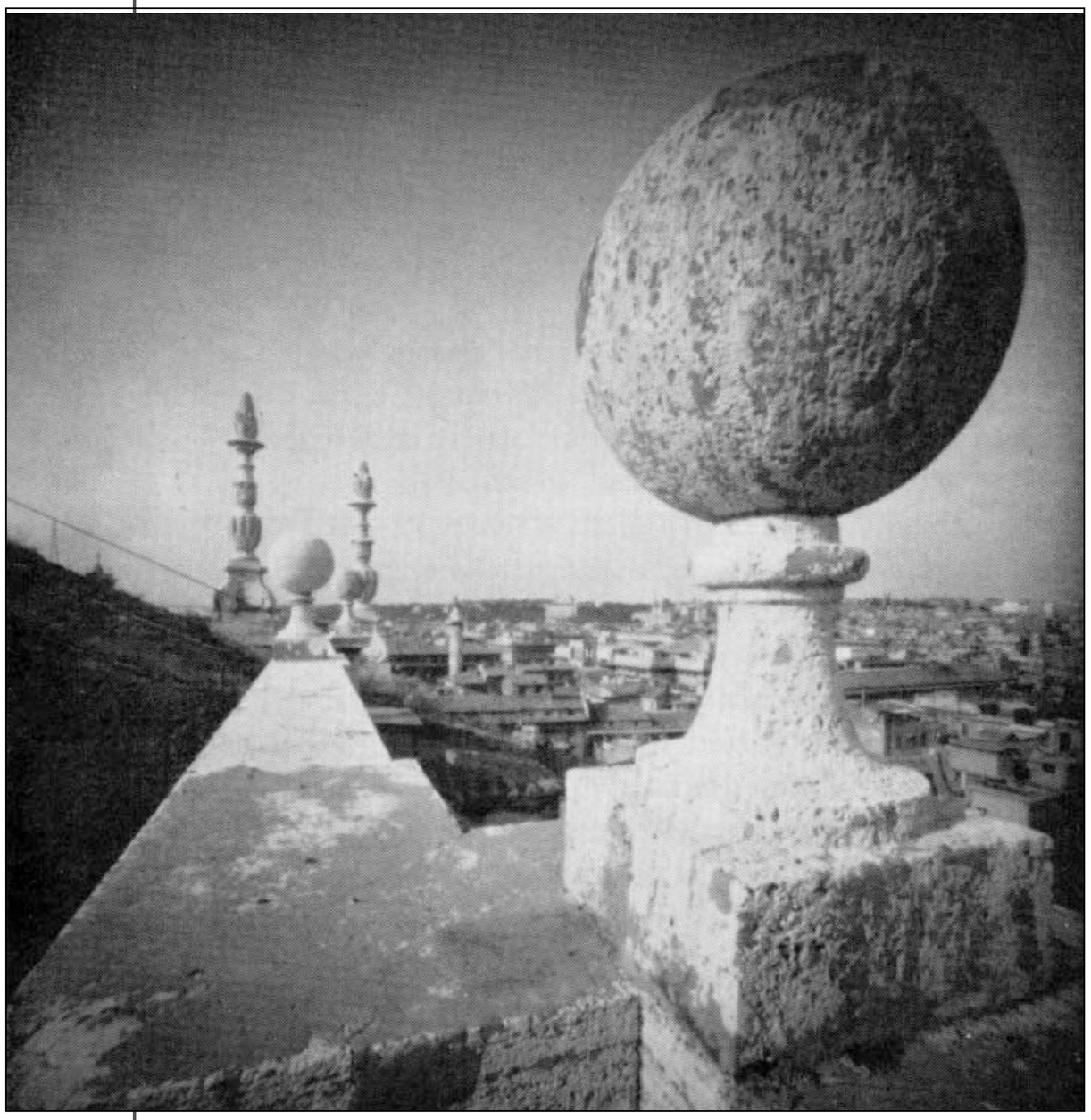
From Shirley textbook.

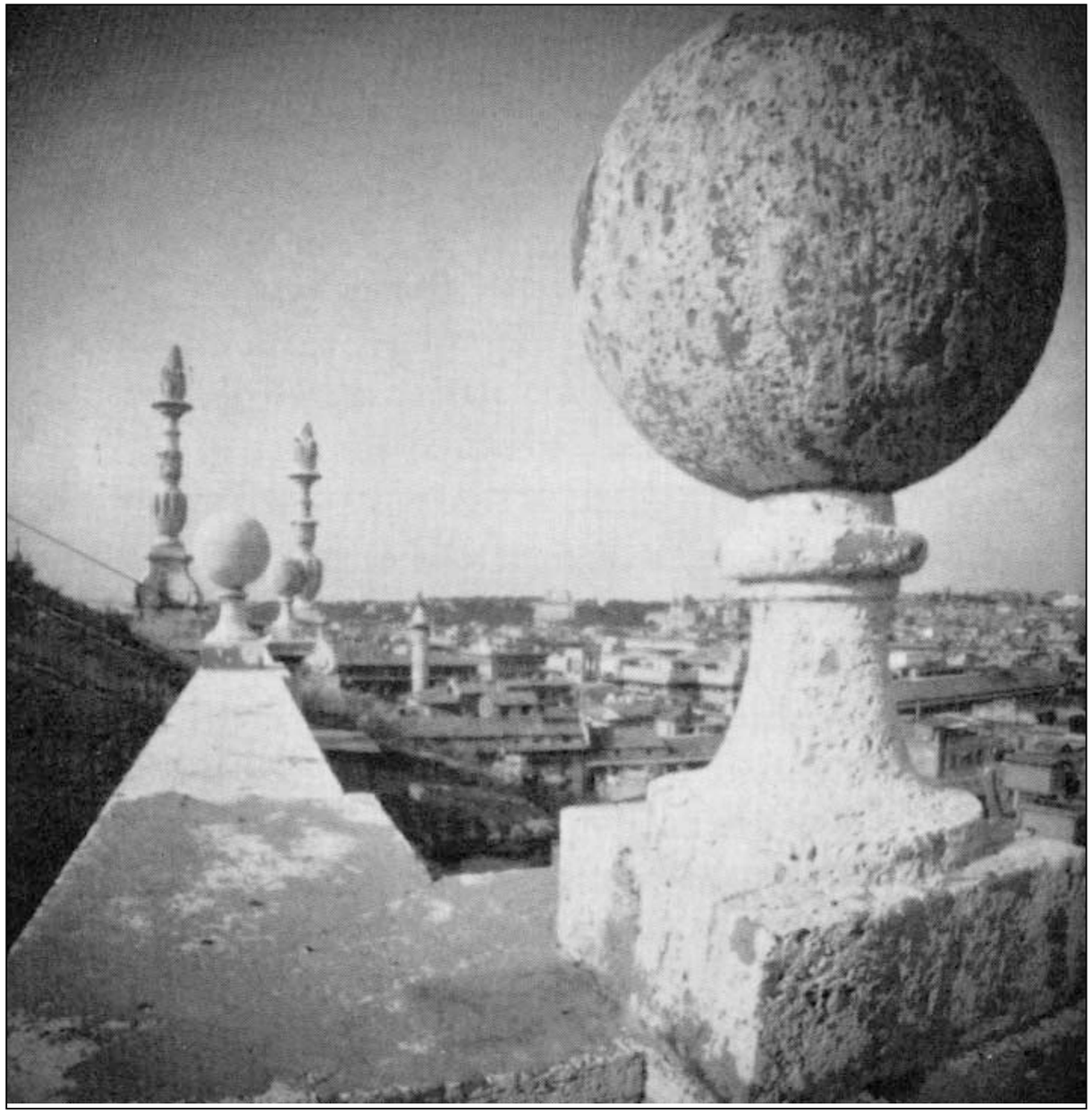
Note: Material on gray slides is optional for cs184.

Perspective Tricks

Note: Material on gray slides is optional for cs184.

Right Looks Wrong (Sometimes)

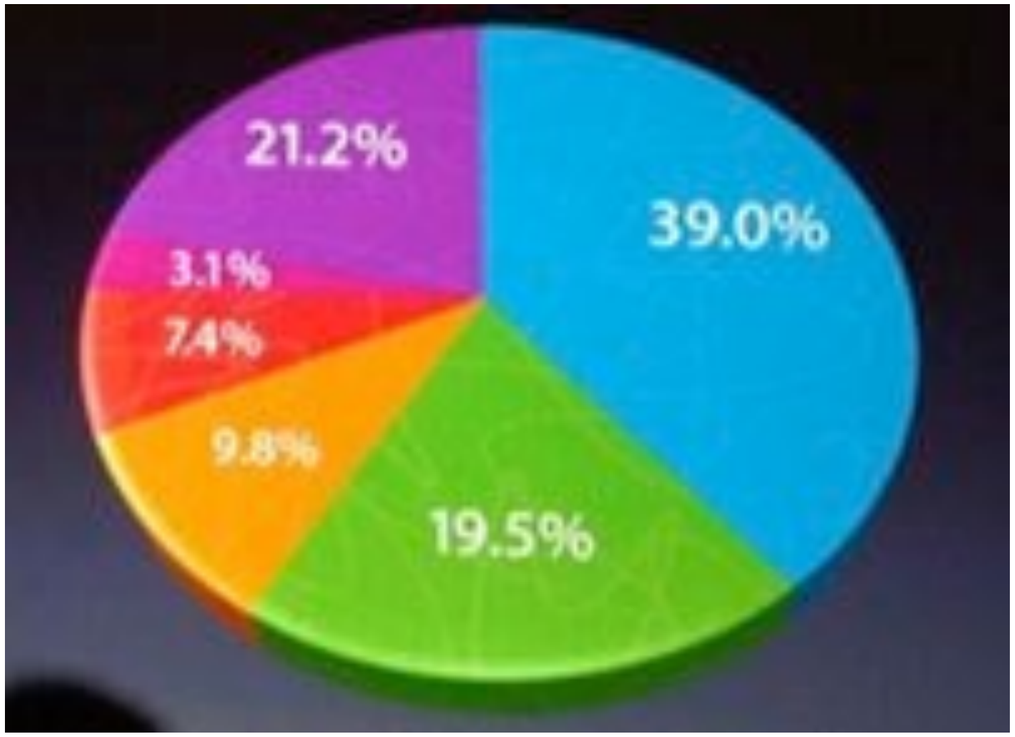


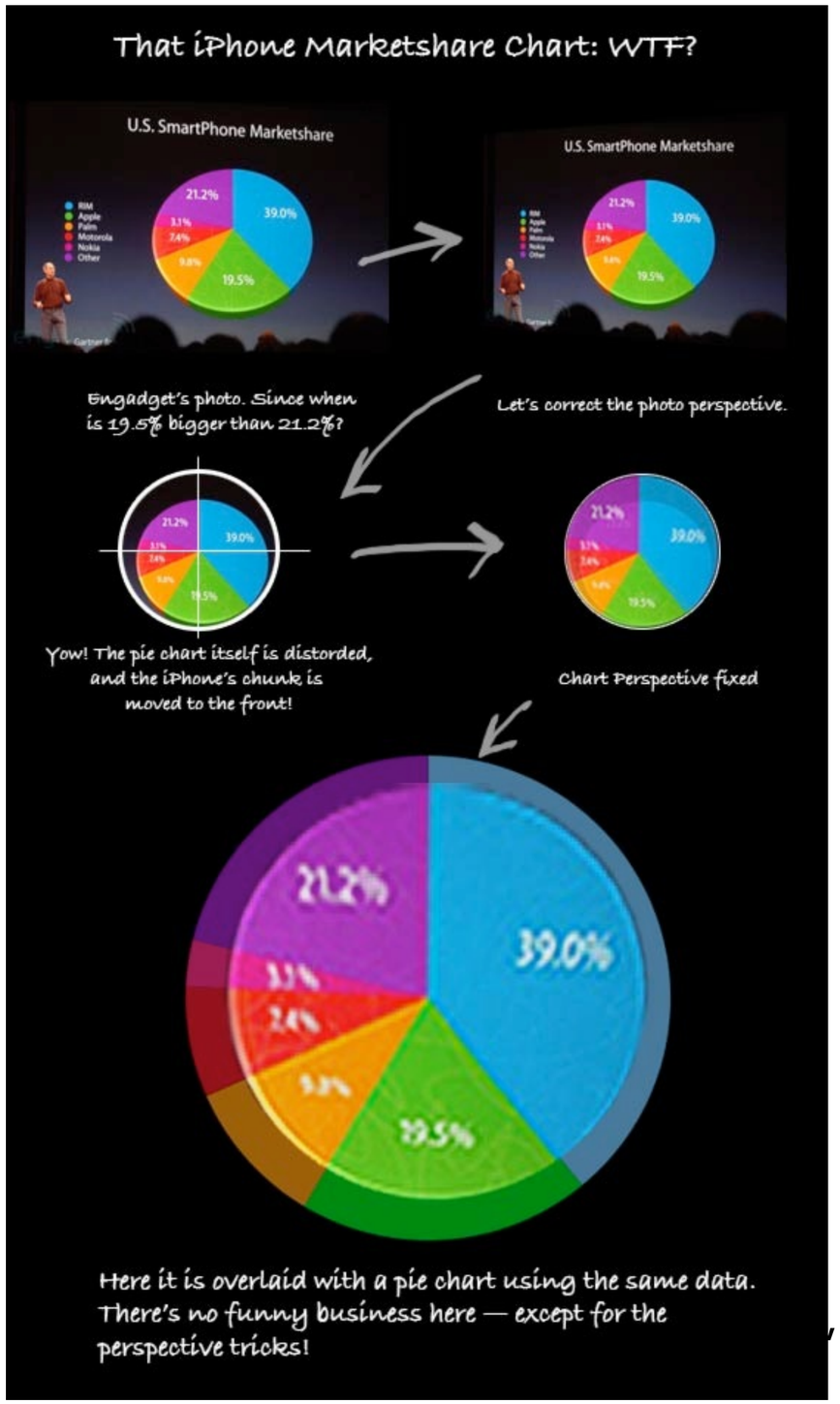


From *Correction of Geometric Perceptual Distortions in Pictures*, Zorin and Barr SIGGRAPH 1995

Note: Material on gray slides is optional for cs184.

Right Looks Wrong (Sometimes)





From WIRED Magazine

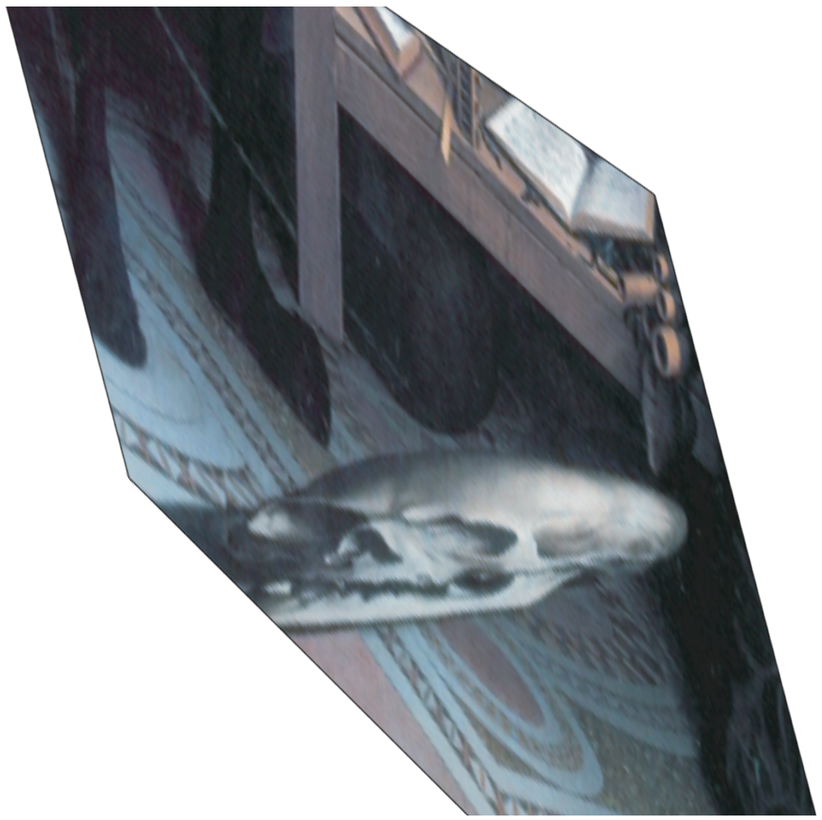
Note: Material on gray slides is optional for cs184.

Strangeness

The Ambassadors
by Hans Holbein the Younger

Note: Material on gray slides is optional for cs184.

Strangeness



The Ambassadors
by Hans Holbein the Younger

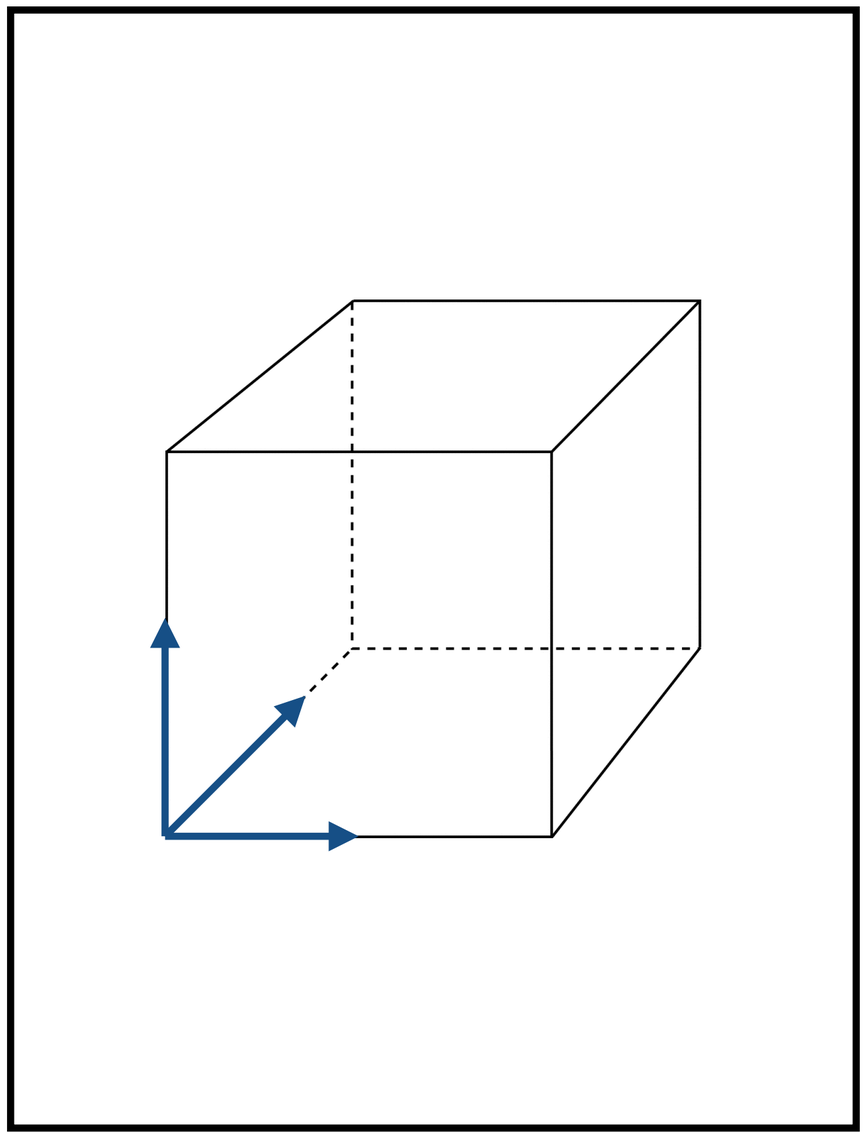
Transforms Recap

Transforms Recap

Coordinate Systems

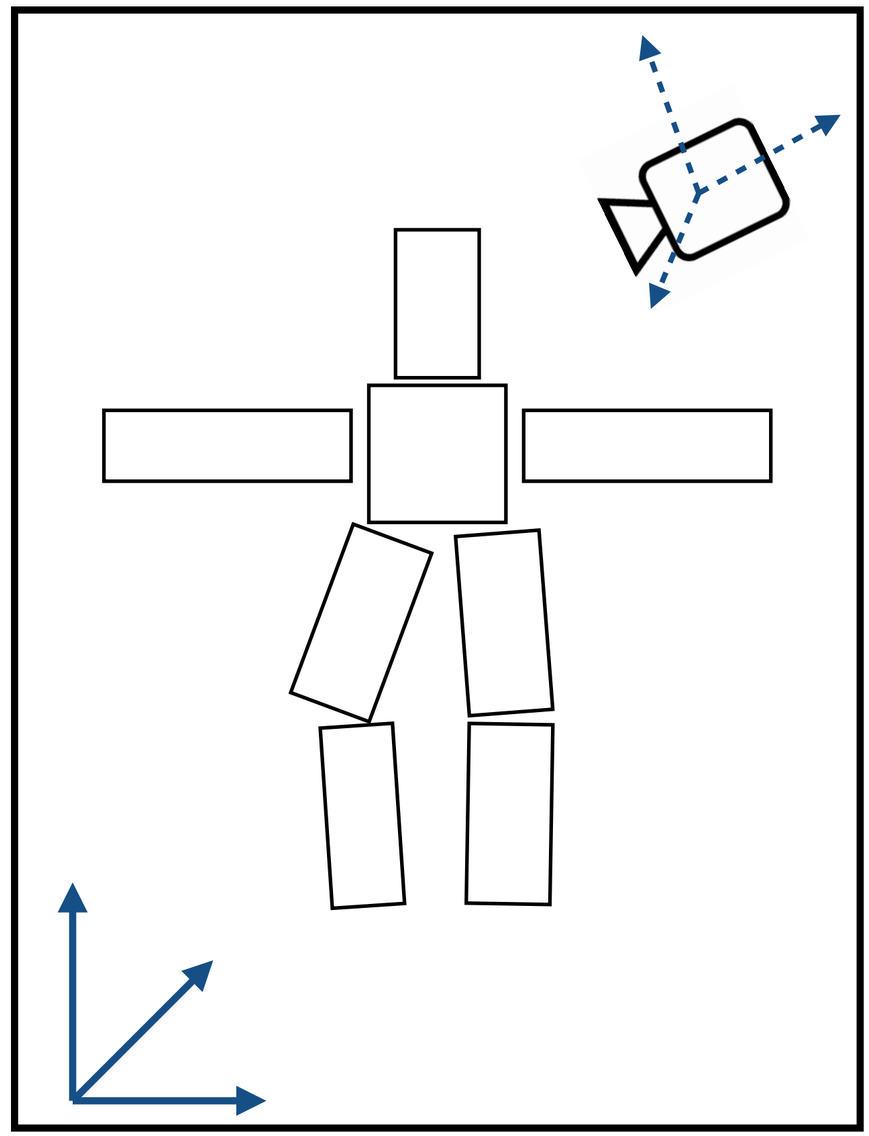
- Object coordinates
  - Apply modeling transforms…
- World (scene) coordinates
  - Apply viewing transform…
- Camera (eye) coordinates
  - Apply perspective transform + homog. divide…
- Normalized device coordinates
  - Apply 2D screen transform…
- Screen coordinates

Transforms Recap



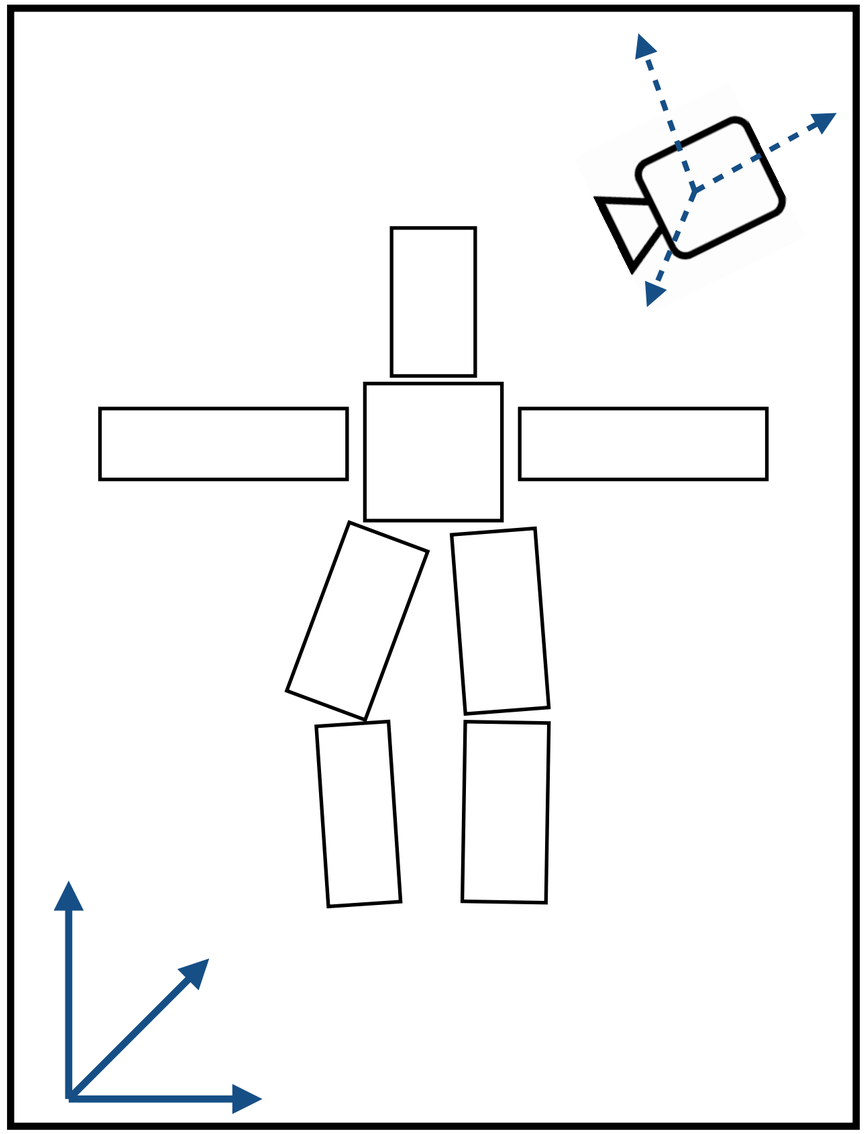
Object coords

Modeling transforms



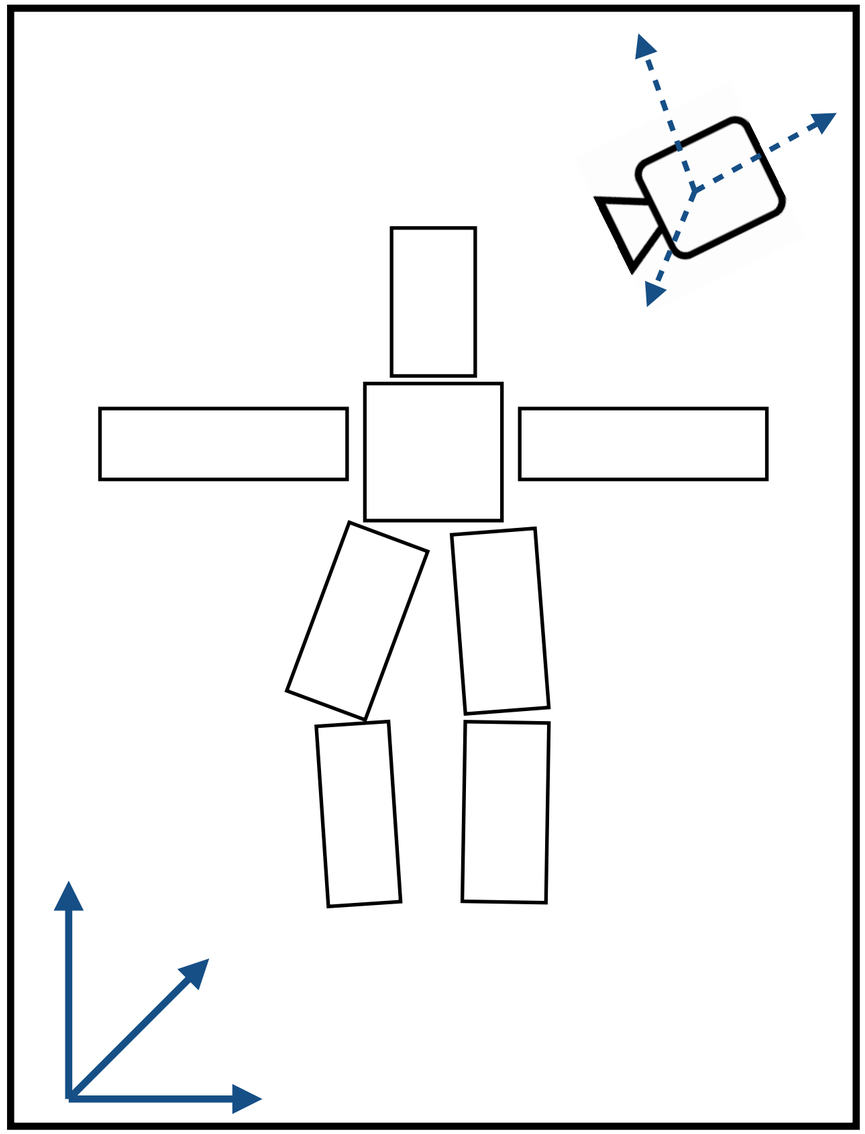
World coords

Transforms Recap



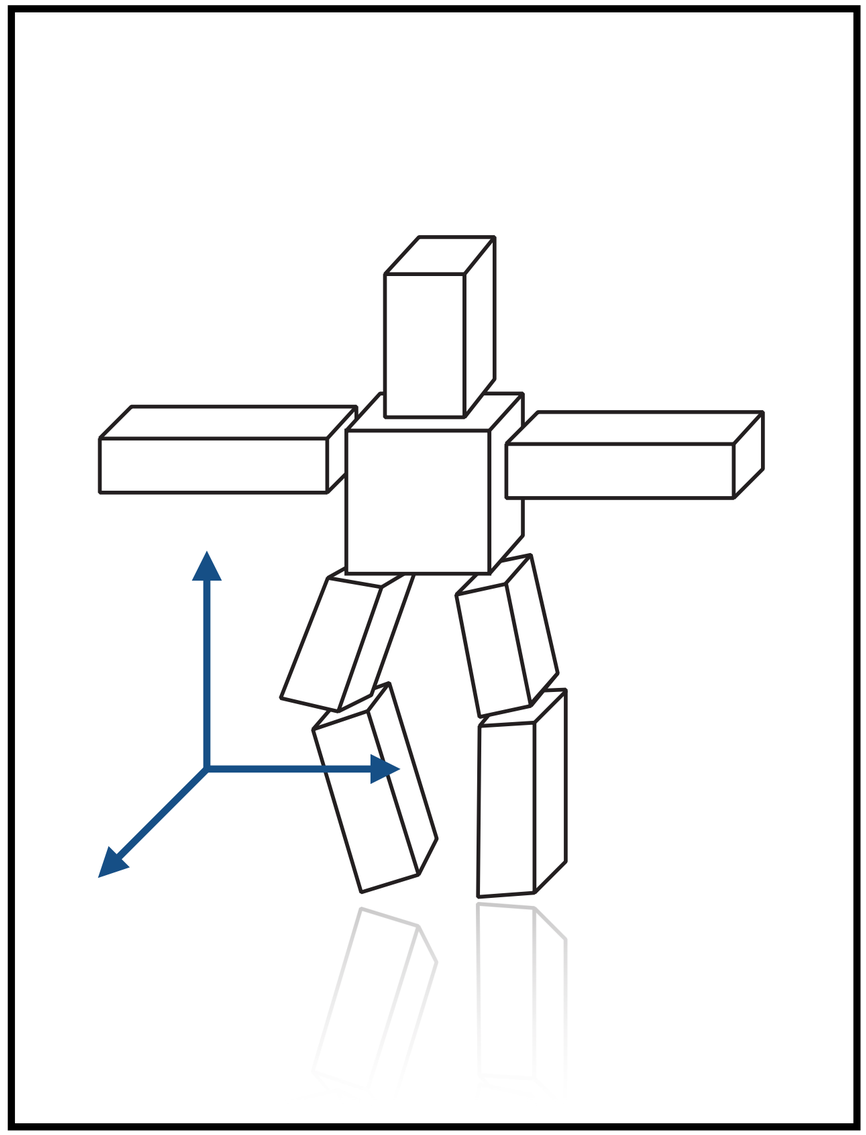
World coords

Transforms Recap



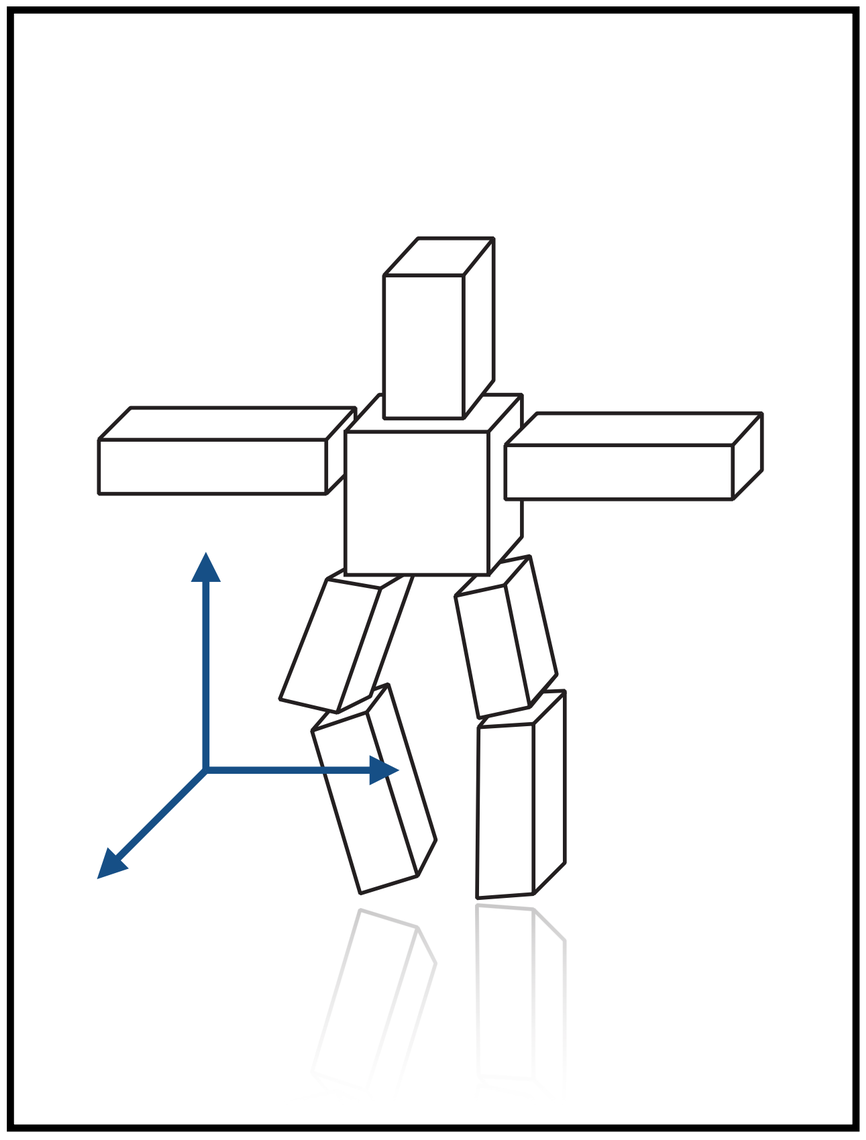
World coords

Viewing transform



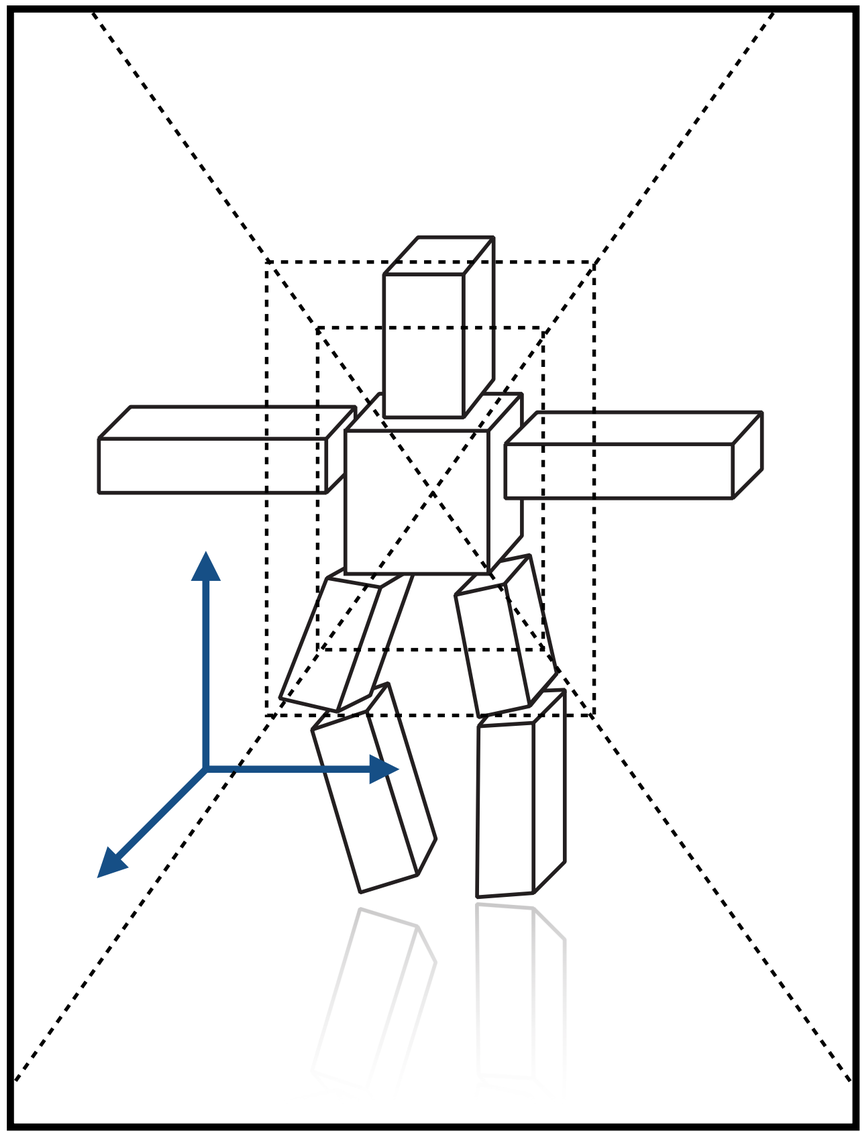
Camera coords

Transforms Recap



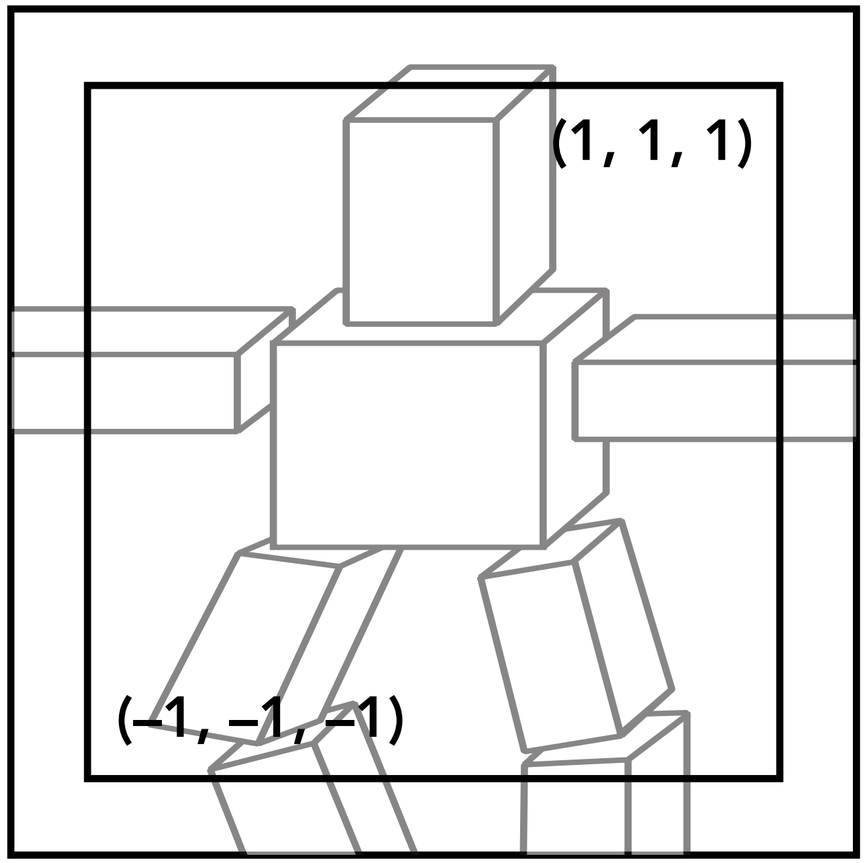
Camera coords

Transforms Recap



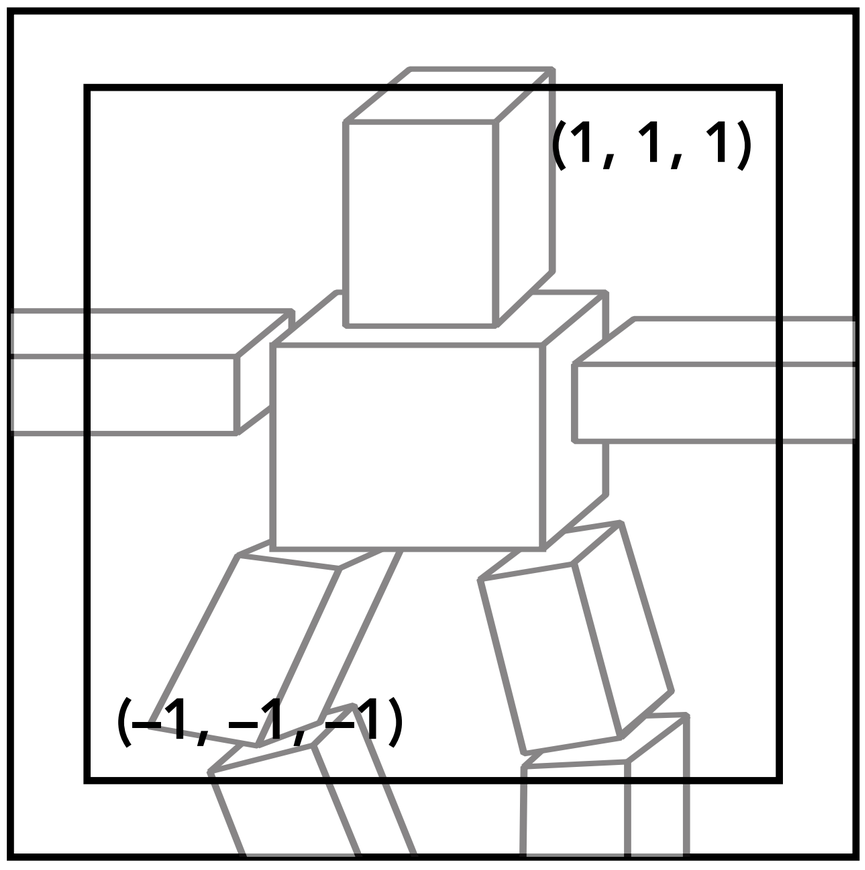
Camera coords

Perspective projection and homogeneous divide



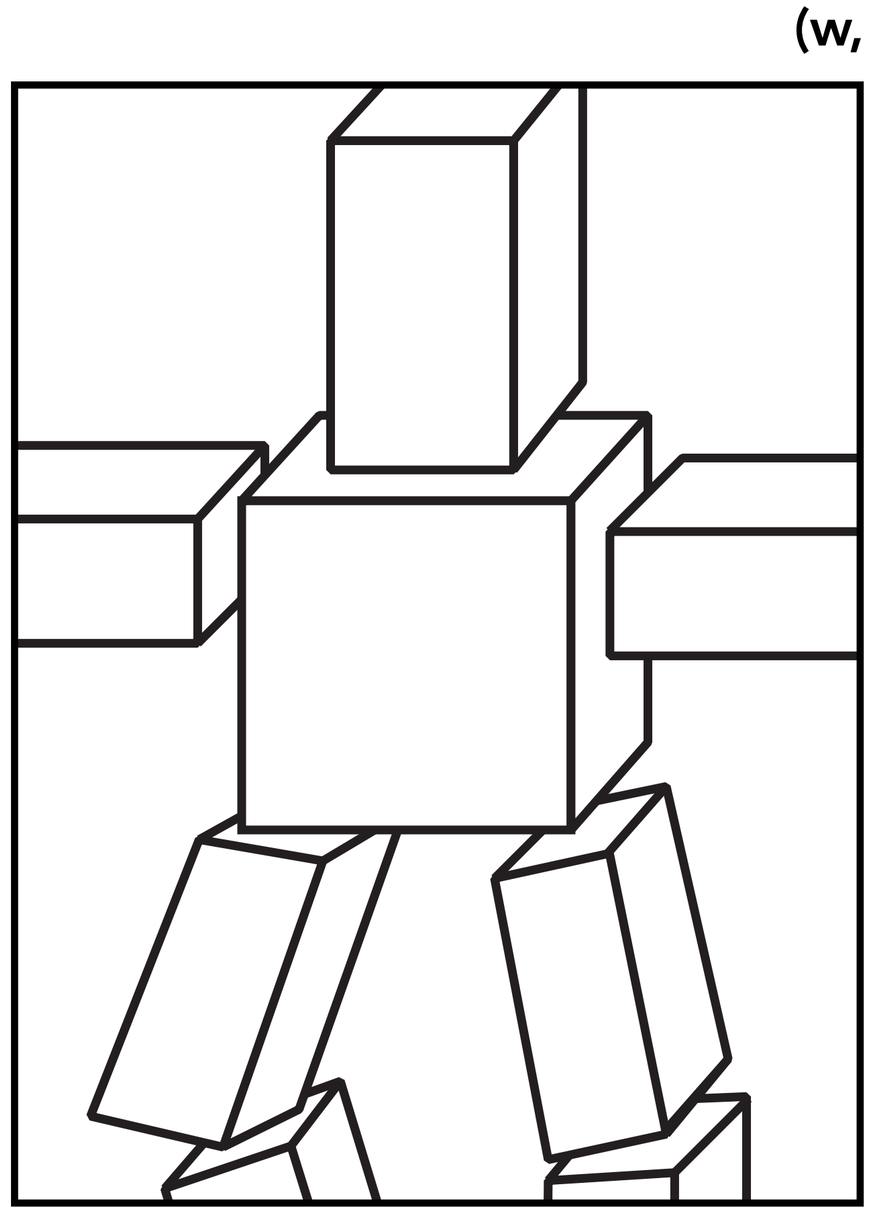
NDC

Transforms Recap



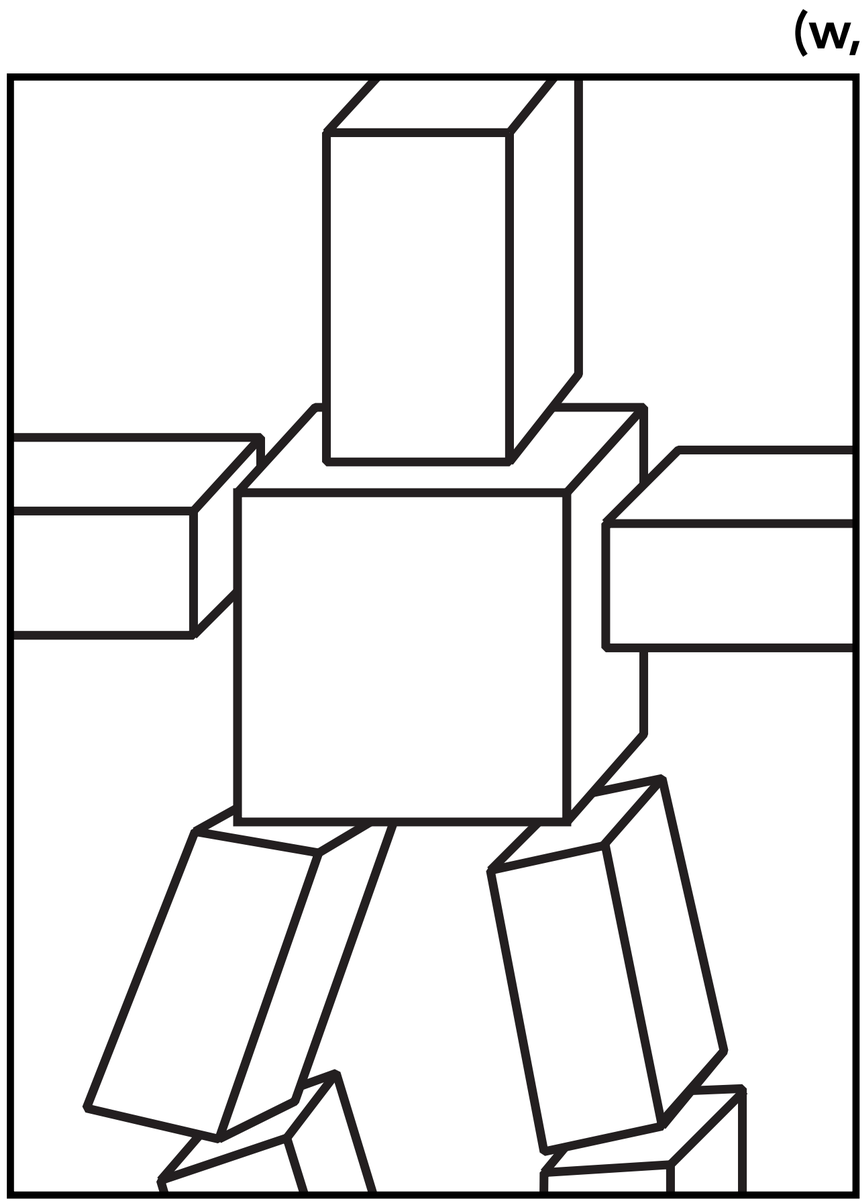
NDC

Screen transform



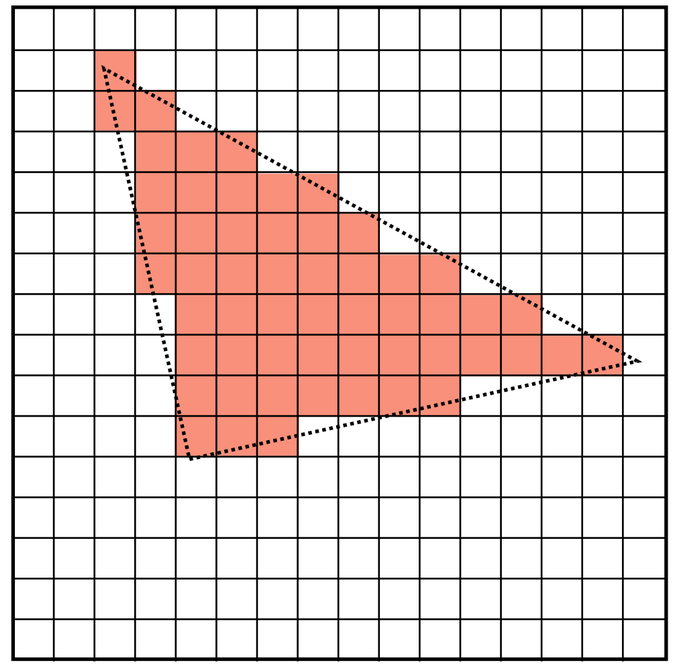
Screen coords

Transforms Recap



Screen coords

Rasterization



Things to Remember

Transform uses

- Basic transforms: rotate, scale, translate, …
- Modeling, viewing, projection, perspective
- Change in coordinate system
- Hierarchical scene descriptions by push/pop

Implementing transforms

- Linear transforms = matrices
- Transform composition = matrix multiplication
- Homogeneous coordinates for translation, projection

Acknowledgments

The CS 184 slide set was developed by:

- Ren Ng
- James O’Brien

With additional contributions from:

- Kayvon Fatahalian
- David Forsyth
- Pat Hanrahan
- Angjoo Kanazawa
- Steve Marschner
- Ben Mildenhall
- Mark Pauly
- Weilun Sun
- Lingqi Yan